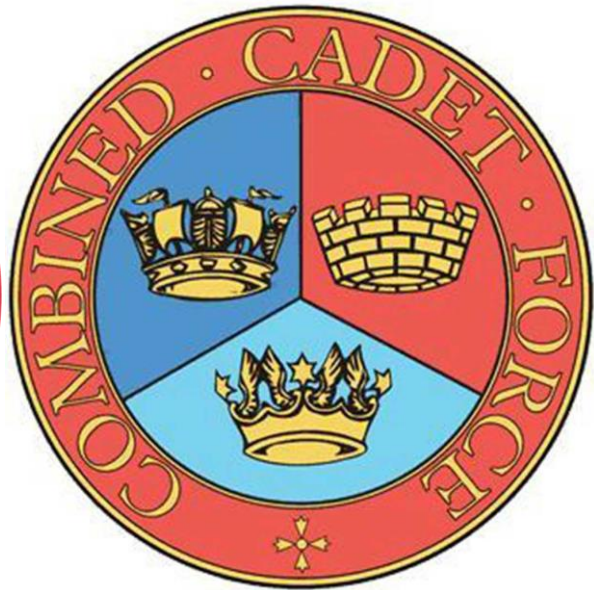


Cadet Management System



By Henry McMahon

Centre number:

Candidate number:

Link to testing video:

<https://www.youtube.com/watch?v=YLrfLXSili0>

Contents

Table of figures.....	8
Table of relevant acronyms.....	9
Analysis.....	10
Background research.....	10
Current system.....	11
Reasons for change in system.....	17
Problem identification.....	17
User identification.....	18
User limitations.....	18
Other limitations.....	19
Questionnaire for NCOs'.....	19
Responses to questionnaire from 3 NCOs' and analysis of responses.....	21
User needs.....	23
Data dictionary.....	24
Data for login page and Create/ edit/ view users page.....	24
Data for storing the equipment we have.....	25
Navigation banner (header).....	25
Data for calendar page.....	26
Data for requesting equipment functionality.....	27
Data volumes.....	27
System objectives.....	27
Categorisation of users and their abilities within the system.....	32
Admin only features.....	32
G4 features.....	33
Event owner features.....	33
Event duty cadet.....	33
All users feature.....	33
Potential solutions.....	34
Chosen solution and justification.....	34
Prototype.....	35
Login page.....	35
Calendar page for admin.....	35
Calendar page for standard user.....	36
Page to view progress over time as a cadet.....	37
Event page.....	37

Conclusion of the prototype	38
Entity relationship diagram	38
Design	39
Overall system design	39
Structure of functionality in the system	39
Flowchart for pipeline of an event in the solution	40
Definition of data requirements (normalised)	41
users table	41
events table	42
parades table	42
User_event table	42
equipment table	43
equipment_requests table	43
Input, process, storage, output	43
For login.php and logout.php	43
For calendar.php	44
For add.php	45
For event.php	46
For user input validation in JavaScript	49
For entity search	49
Grouping procedures into files categorised by their primary use cases	49
Posting of forms	50
Get requests for event.php	50
Get and post requests for events through the system	52
Get requests primarily for add.php	53
JS functions that do not make requests to the server	55
Considerations for client server communication	57
Reasons for using JSON for extra client server communication	57
Considerations for concurrent access of the database	57
Structure of webpages	57
Webpage design	58
Login page	58
Calendar page	59
<i>Calendar page for the admin (and G4)</i>	59
Event page	61
add page	63

pre-made SQL queries	64
Defining complex SQL queries	66
Selecting first names of users who can be added to event that match a search prompt (conflict management logic)	66
Likely technical solutions.....	67
Identification of server, server packages and storage media	68
Security of the system	68
Technical solutions	69
Codebase overview	70
Architectural overview.....	70
Code structure.....	70
Key security features	70
Key dependencies.....	70
Installation guide	70
login.php.....	71
uploads.php.....	74
requests\post_request_scanning.php	76
requests\get_request_scanning.php	76
logout.php	77
backup_mysql.sh	77
includes\connection.php.....	78
includes\functions.php.....	78
includes\nav.php.....	79
includes\display_parade.php	81
calendar.php.....	87
event.php.....	96
requests\post_requests.php	108
case "add_new_parade";	108
case "add_new_event";.....	108
case "add_new_user";	109
case "modify_user";.....	109
case "add_new_equipment";	110
case "modify_equipment";.....	111
case "modify_register";	112
case "modify_equipment_register";.....	112
requests\add_get_requests.php.....	113

case "search_parade_name";	113
case "search_duty";	114
case "search_first_name_user";	114
case "search_last_name_user";	115
case "search_equipment_name";	116
case "search_equipment_location";	117
case "equipment_id_info_dump";	118
case "user_id_info_dump";	118
requests\event_details_requests.php	119
"search_first_name_owner"	119
case "modify_event_details";	120
case "delete_event";	121
requests\event_get_requests.php	122
case "add_user_id";	122
case "remove_user_id";	122
case "add_equipment_id";	122
case "remove_equipment_id";	123
case "search_first_name";	123
case "search_first_name_delete";	124
case "search_first_name_other_cadet";	125
case "search_equipment_add";	126
case "search_equipment_delete";	128
case "search_equipment_other";	128
js\calendar.js.....	130
populateAdminEditEventForm().....	130
returnFeedbackHTML()	131
REGEXCheckEvent().....	132
js\event_owner_form_handeling.js	135
showResutsSearchForOwner()	135
resultHasBeenClickedOwner()	135
displayCurrentOwnerOfEvent().....	136
js\event.js	137
showResultAddCadet()	137
showResultDeleteCadet()	138
showResultSearchOtherCadet().....	139
resultHasBeenClickedAdd()	140

resultHasBeenClickedDelete()	140
showResultAddEquipment()	141
showResultDeleteEquipment()	142
showResultSearchOtherEquipment()	143
resultHasBeenClickedAddEquipment().....	144
resultHasBeenClickedDeleteEquipment().....	144
setStateOfEventApproval()	145
REGEXCheckEvent().....	146
REGEXCheckRegister().....	150
REGEXCheckEquipment()	152
js\add.js	154
showResutsSearchForUserFirstName()	154
showResutsSearchForUserLastName()	155
ShowResultsSearchForParade()	155
ResultHasBeenClickedParade().....	156
showResutsSearchForDuty()	157
resultHasBeenClickedDuty()	157
resultHasBeenClickedUser()	158
showResutsSearchForEquipmentByName().....	159
showResutsSearchForEquipmentByLocation()	160
resultHasBeenClickedEquipment().....	161
checkAreAllValuesOne().....	162
returnFeedbackHTML()	162
REGEXCheckParade().....	163
REGEXCheckEvent().....	165
REGEXCheckAddUser()	168
REGEXCheckModifyUser().....	173
REGEXCheckAddEquipment()	176
REGEXCheckModifyEquipment()	179
help.php	181
css\add-style.css	187
css\calendar-style.css	189
css\event-display-block-style.css	189
css\event-owner-form-style.css	190
css\event-style.css	190
css\login-style.css	191

css\nav-style.css	191
css\register-and-equipment-reuquests-style.css	192
Testing	194
Naming conventions for test id's	194
Screenshots for tests not included in the video	194
Definitions of tests	201
Login.php	201
Included Nav.php	202
Backup.sh	202
Calendar.php(standard user)	202
Calendar.php(an event owner)	203
Calendar.php(duty cadet)	203
Calendar.php(G4)	203
Calendar.php(admin)	204
Add.php(non-admin user)	205
Add.php(admin)	205
Event.php(standard user)	209
Event.php(event owner non-approved event)	209
Event.php(event owner approved event)	210
Event.php(G4)	212
Event.php(admin)	212
Event.php(duty cadet)	213
General tests for forms	213
Failed tests	213
Evaluation	214
Objective analysis	214
End user feedback	219
Major Jackson (admin)	219
Student 1 (logistics cadet/ G4)	220
Student 2 (event owner)	221
Student 3 (standard user)	222
End user feedback analysis	222
Future technical improvements	223
Future functionality and user experience improvements	224
Summary of the effectiveness of my solution	224

Table of figures

Figure 1 Current process of lesson planning	11
Figure 2 Pin board outside CCF office.....	12
Figure 3 White Board outside CCF office	12
Figure 4 printout lesson aid.....	13
Figure 5 screenshot of website for lesson aids.....	13
Figure 6 sign in/out of equipment sheet	17
Figure 7 screenshot of calendar page for the admin in the prototype	35
Figure 8 screenshot of calendar page for regular user in the prototype.....	36
Figure 9 screenshot of rank progress page in the prototype	37
Figure 10 screenshot of the event page in the prototype.....	37
Figure 11 ER diagram for the database	38
Figure 12 flowchart to show structure of functionality in the system	39
Figure 13 flowchart to show the lesson planning process in the new system	40
Figure 14 flowchart to show structure of webpages and their broad functionalities	57
Figure 15 screenshot of the user interface design for the login page in the new system.....	58
Figure 16 screenshot of the user interface design for the calendar page for the admin and G4 in the new system.....	59
Figure 17 screenshot of the user interface design for the calendar page for event owners and standard users in the new system	60
Figure 18 screenshot of the user interface design for the event page when the event is approved in the new system	61
Figure 19 screenshot of the user interface design for the event page when the event is not-approved in the new system	62
Figure 20 screenshot of the user interface design for the add page in the new system	63
Figure 21 contents of unzipped file 'all_code'	69
Figure 22 figure to show the routine backups of the database.....	194
Figure 23 screenshot to show the results of logging in with an inactive account	195
Figure 24 screenshot to show what happens when a malicious phrase is input into the equipment name search box	195
Figure 25 screenshot to show what happens when a malicious phrase is input into the search user by first name to modify search box	196
Figure 26 screenshot to show what happens when a malicious phrase is input into the search by first name for a different event owner	196
Figure 27 screenshot to show a 254-character input for first name of a new user	197
Figure 28 screenshot to show a 255-character input for first name of a new user	197
Figure 29 screenshot to show a 254-character input for equipment name in the add equipment form	198
Figure 30 screenshot to show a 255-character input for equipment name in the add equipment form	198
Figure 31 screenshot to show validation response of form add a parade when invalid time is input.....	199
Figure 32 screenshot to show validation response of form add an event when invalid time is input.....	199
Figure 33 screenshot to show validation response of add a user form when an email that has an invalid format is entered	200

Figure 34 screenshot to show validation response of add a user form when an email that has an invalid format is entered	200
Figure 35 screenshot to show validation response of add a user form when an email that has a valid format is entered	201

Table of relevant acronyms

Acronym	meaning
NCO	Non-commissioned officer typically referring to rank Corporal and Lance Corporal these cadets will be teaching lessons.
G4	Not necessarily an acronym but in the military, this refers to a group of individuals in charge of the logistical needs of a unit.
RSM	Military rank standing for Regimental Sergeant Major these individuals will often hold the status of duty cadet for an event
CSM	Military rank standing for Company Sergeant Major these individuals will often hold the status of duty cadet for an event
C/SGT	Military that has been shortened standing for Colour Sergeant these individuals will occasionally hold the status of duty cadet for an event
ORBAT	Stands for order of battle and is a document to show who reports to who during a camp or parade night
CCF	Combined Cadet Force (extracurricular activity sponsored by the Military)
SCC	St Columab's College (the school I attend)
SSI	Senior Staff Instructor (leadership position for an adult in the CCF)

Analysis

Background research

St Columba's College is a prep school, secondary school and 6th form with around 600 students across the secondary school and 6th form. There are around 80 Army cadets from year 8 upwards that take part in CCF(Combined Cadet Force). CCF is an extracurricular activity that takes place for two hours after school on most Fridays. On top of the Friday session after school there are also three camps in a year, with two of them being internally run on military training areas.

Presently the St Columba's CCF has a record of excellence in competitions and teaching. Recently placing first out of 18 cadet forces in the prestigious Combat Cadet Competition and having many cadets trained in teaching and planning through the SCIC (Senior Cadet Instructor Cadre) program. Major Jackson is the head of the CCF and performs many roles. Inside the CCF he is responsible for organising our camps, overseeing the storage and maintenance of all the CCF's assets (kit and equipment), approving promotions of cadets, providing ample opportunities for cadets to develop pastoral skills. Additionally Captain Hewett, the schools SSI (Senior Staff Instructor) oversees many of the day-to-day functions of the CCF which involve: overseeing the training programs for all cadets from one star (basic) up to master cadet (advanced), getting the relevant documents in place for camps i.e. main events lists and risk assessments.

Aside from these two adult leaders there are many other adults who volunteer their time to the CCF to help during Friday parade nights and camps. Many of them are ex-military and have knowledge that they pass on to cadets. Additionally, they have qualifications that aid the functioning of camps for instance, adults with range safety qualifications increase our capacity for cadets on live fire ranges.

Importantly, the cadets as an organisation promotes the practice of cadets leading other cadets in a variety of activities and tasks. This allows adults to delegate smaller tasks to senior cadets so the organisation as a whole can perform more complex activities using a hierarchical structure. Senior cadets will often further delegate tasks to the more capable cadets (some as young as 14 years old). I believe this is the greatest strength of the cadets since it allows for a wealth of personal development from a young age that would otherwise not be available to young people. Many of the skills learned here have wide reaching applications in the personal and professional lives of many cadets.

The CCF wishes to continue its success within the cadet world, for this to be achieved knowledge from the senior cadets must be passed down to the lower ranks so they may fill the roles of seniors as they leave the force and progress beyond senior school/ 6th form.

In order for this to happen, the cadets in charge of teaching lessons must adequately plan and prepare for Friday parade nights, to ensure they run smoothly. As a senior cadet, one of my roles is to ensure that this is the case. I believe that a digital platform which caters to the weekly needs of the adults and cadets in the CCF is needed. This will significantly improve the robustness and efficiency; of process we follow each week and ensure a satisfactory quality of lessons are achieved.

Current system

The CCF administrates itself through a variety of means i.e. WhatsApp groups for seniors, signup sheets on the bulletin board, emails from the Staff and conversations in the CCF office. In this day and age, a system like this is an ineffective means to store and transfer information from the adults to the lower ranks. As a result, there are often incidents of out of date/ conflicting orders which slow down the progress of parade nights.

Below I have captured the key elements of the lesson planning process in the form of a flowchart. It is important to note that each connection on this flowchart can happen through a variety of means. This highlights how difficult it is to look back and review the system for improvements.

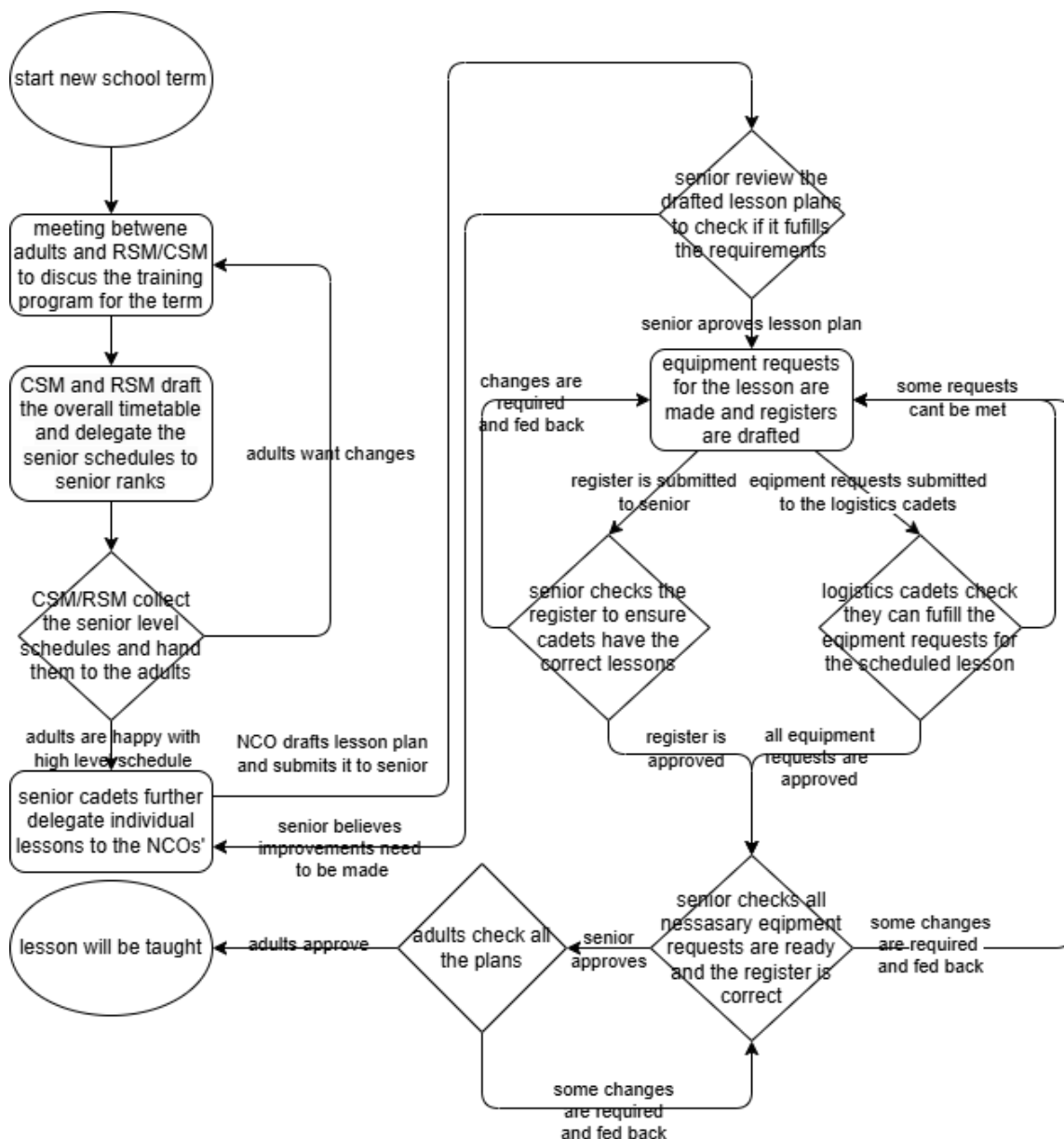


Figure 1 Current process of lesson planning

The system uses a variety of means of communication and below are some photos of notice systems. These are located in the administrative block of the school where the CCF office is situated. These are frequently populated with orders and messages (particularly the white board) from the adults. They are designed to account for all ranks of the Cadets i.e. the cadets who are in the lessons and the RSM/CSM's who must plan for everyone in the cadet force.



Figure 2 Pin board outside CCF office

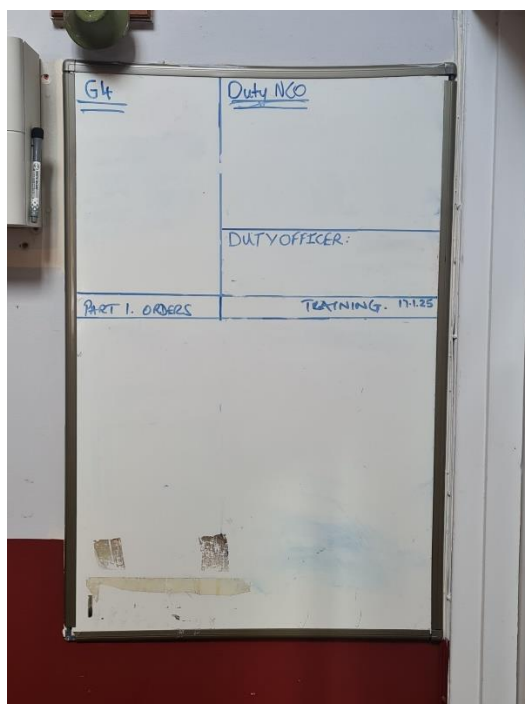


Figure 3 White Board outside CCF office

For planning lessons, it is useful to utilise pre-existing resources. For instance, diagrams of the rifle that we use. These are available online but there are also printouts for a few of the most important lessons in the cadet syllabus(drill(marching), fieldcraft and weapon handling).

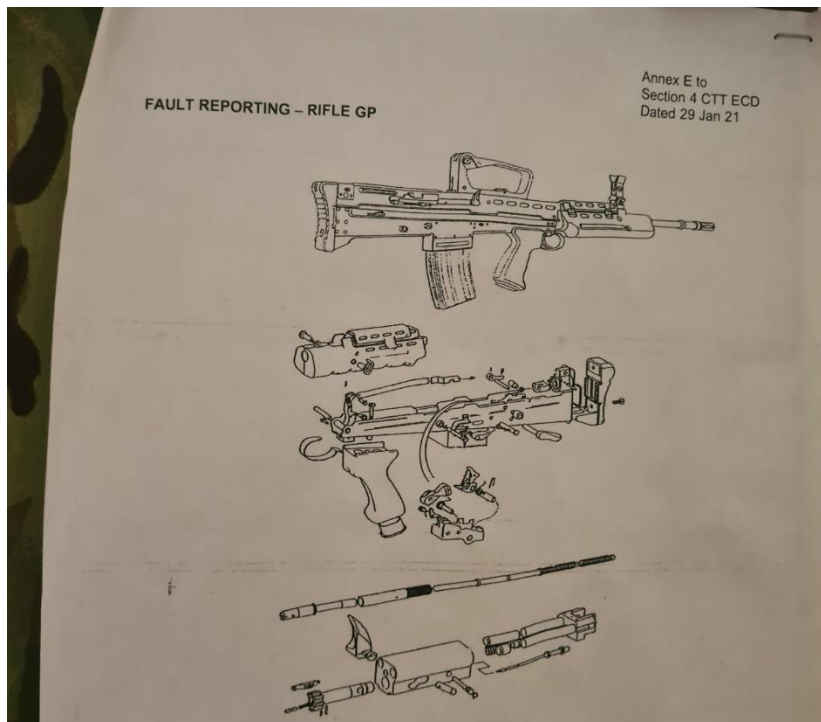


Figure 4 printout lesson aid

Below is an example of a website that offers some of the training aids. It is important to note that these websites are not always up to date. As a result, cadets often have to check a variety of other resources to verify the information in these resources is correct.

Screenshot of the website <https://www.mkbartlett.co.uk> showing Army Cadet Force Resources.

ARMY CADET FORCE RESOURCES

Turnout

Cadet APC Lessons

Star Level	LN	Lesson Title
BTS	01	Introduction & Appearance and Dress
BTS	02	Inspection of Arms & 58 Pattern Equipment
☆☆☆☆	01	(FC) Revise - 58 Pattern Equipment

Drill

Cadet APC Lessons

Star Level	LN	Lesson Title
BTS	01	Aim and Purpose of Drill
BTS	02	Attention, Stand at Ease and Stand Easy
BTS	03	Turnings at the Halt
BTS	04	Compliments
BTS	05	Saluting to the Front
BTS	06	Introduction to Marching
BTS	07	Marching and Halting
☆☆☆☆	01 - 07	Revise - Basic Training Movements
☆☆☆☆	08	Forming Up and Getting on Parade
☆☆☆☆	09	Open and Close Order
☆☆☆☆	10	Dismissing and Falling Out
☆☆☆☆	11	Wheeling in Quick Time
☆☆☆☆	12	About Turn in Quick Time
☆☆☆☆	13	Saluting, Eyes Right and Left
☆☆☆☆	01 - 11	Revise - Basic and One Star Movements
☆☆☆☆	12 - 15	Revise - One Star Movements
☆☆☆☆	16	Changing Step in Quick Time
☆☆☆☆	17	Left and Right Turns on the March
☆☆☆☆	18	Marking Time in Quick Time

The website also features a sidebar with the Royal Warrant of Appointment, a syllabus selector, filters for levels and categories, a list of subjects (Turnout, Drill, Military Knowledge, etc.), and a legal/feedback section.

Figure 5 screenshot of website for lesson aids

During the lesson planning process cadets will often have to populate a lesson plan that looks something like the one below (a lesson on camouflage and concealment I taught):

Admin/General					
Syllabus			Subject		
Lesson: Cam and concealment			Reference		
Constraints					
Time: 1500 - 1600		Location:		Class size: 7 - 8	
Logistics					
	Instructor		Students		
Dress	MTP		MTP		
Equipment	Cam Cream		Cam Cream		
Beginning - CNNFARI					
Check kit	Check everyone has Cam Cream	Number Off	N/A	NSP's	N/A
Formations	N/A Anything N/A else			Revision	N/A
Introduction	Aim	To Know how to apply cam cream and conceal yourself in the field			
	Incentive	Harder to be spotted with Cam Cream			
	Reason Why	To keep yourself hidden			
Middle					
Ser.	Learning points	Method	Formations	Time	Remarks
1	5 S' and a T	Shape, Shine, Shadow, silhouette, Sudden movement and			

		texture			
2	Cam Cream	Show them how to apply the Cam cream, Striped across the face and what is not enough and where to apply it			Don't use too much as it can start to shine
3	Webbing and breaking up your shape.	Put grass and leaves into webbing to break up your body shape			
4	Let them cam up ready to do the game	Put on Cam Cream, Conceal yourselves in the woods and section ic try to spot them have the cadets try to get to an objective			
End					
Final Confirmation	Questions (From & To)		Quiz on the 5 S's and T		
	Final practice		n/a		
Pack Kit	Cam Cream				
NSP's	N/A				
Summary	Teach 5 S's and T, show how-to put-on cam cream, get them to put it one, teach concealment				
Look Forward to	N/A				

These will typically be stored digitally and will be emailed back and forth between NCO's, seniors, the CSMs' and the adults. This frequently results in many versions of the same document being stored under multiple names which again furthers the issues already discussed of tracking progress over time.

Below is an example of a training program I put together in my earlier years in the cadets. It was designed for the incoming cadets, to bring them up to speed before they integrate with the rest of the cadets.

periods	Friday the 14th of January	Friday the 21st of January	Friday the 28th of January	Friday the 4th of February	Friday the 11th of February	Friday the 18th of February
parade (3:45-4:15)					half term	half term
1st period (4:15-5:00)	marksmanship principals	observation (air rifle indoors)	nav intro	advanced nav	half term	half term
break (5:00-5:10)					half term	half term
2nd period (5:10-5:50)	fallout and teach salute	Marching_1	Marching_2	halt	half term	half term
final debrief (5:50-6:00)					half term	half term

periods	Friday the 25th of February	Friday the 4th of March	Friday the 11th of March	Friday the 18th of March	Friday the 25th of March	2nd of April FTX
parade (3:45-4:15)						FTX
1st period (4:15-5:00)	undecided	judging distances	Patrolling + hand signals	pairs fire (fallback to cover)	harbour + duties of a sentry	FTX
break (5:00-5:10)						FTX
2nd period (5:10-5:50)	saluting to the front + catchup	types of patrols/marching order	fire control orders	pairs fire (attack a position)	all round defence +searches	FTX
final debrief (5:50-6:00)						FTX

[illegible]

Reasons for change in system

Problem identification

Multiple attempts have been made to condense all information into one area. An example of this would be the senior WhatsApp group chat however not everybody in the CCF is on this. This leads to lower ranks being behind on current schedules and goals. Furthermore, WhatsApp is not accessible on school devices so when sharing Excel charts of schedules another means of communication is needed.

An ideal system would at a low level provide cadets the ability to see what lessons they have upcoming. The NCOs' will require the ability to plan their lessons and once they are approved request equipment and add cadets to the register. The senior cadets will require the ability to view lessons that they would be responsible for in the current system. Additionally, the logistics cadets(G4) will need a facility to see equipment requests and approve/deny them. And finally, the adults will require the ability to see the bigger picture being able to 'zoom in' to the view of the NCOs' to spot any problems at the low level that potentially scale up to the higher level. Importantly, the system must take care of all the scheduling automatically i.e. prevent cadets from being in two lessons at once.

User identification

In the current system there are six different roles that people can have. Typically, a cadet or adult will fit into just one of these roles, they are as follows:

1. Adult leaders = this is primarily aimed at Major Jackson and the SSI of the contingent. They will have access to the entire system as well as special privileges to change data where required. This group will have approximately 2 to 3 users.
2. Adult volunteers = these users will require the ability to oversee larger number of cadets (/cadet leaders) and their activities. The cadets that they will have control over will be set by the adult leaders. This group will have approximately 8 to 10 users.
3. Logistics cadets = these users will require the ability to oversee the kit/ equipment requests for lessons/ events. They must also have a facility to approve/ deny equipment requests that they can facilitate. This group will approximately have 3 to 5 users.
4. Duty cadets = these cadets will be in charge of the higher-level functions of the cadet leaders. It is their role to ensure the training for their cadet group is carried out to their specifications. They will require the ability to view the progress of individual events that they have a duty to. This group will consist of ranks RSM down to Sargent and will likely have 3 to 8 users.
5. Cadet leaders = these users will also require the ability to interact with large numbers of cadets perhaps the entire contingent (for start and final parades). They will require means to plan and schedule their lessons, add cadets to lessons/ events as well as request kit and equipment for their lessons. Additionally, the ability to delegate tasks will be required. This group will have approximately 15 to 30 users.
6. Cadet = these will be the most basic users, and they will only be able to access information deemed relevant to them. This group will have approximately 60 to 90 users.

User limitations

As with any system, digital literacy will likely present a significant challenge to the adoption of the system. Some of the adult leaders/ cadets who lack experience with digital platforms, may find the system difficult to use, if the interface does not follow the norms of system design. Knowing this, it is imperative that the system's interface is intuitive to interact with.

Additionally, the initial data input for the system will likely be a significant task since cadets, equipment and parades must be manually input. Therefore, the system's benefits must be clear to potential users before the system is implemented, so they invest their time into learning it and fully utilising it's potential.

Other limitations

This system will need to be hosted on the schools internal network so that it cannot be accessed and potentially attacked by outsiders.

The school also has its own email service. While it would be ideal for this to be accessible from my system, it is unlikely to be a possibility. Due to the variety of security and authentication measures that encompass school emails. As a result, the system will likely be independent of the already existing school system. This will make functionality's such as changing password without interaction with the system admin a challenge.

Questionnaire for NCOs'

Being a senior cadet, in a typical week I would estimate I will have 2 meetings with Major Jackson and Captain Hewitt. These will be attended by other seniors and the head of G4 the logistics cadets. They are used to discuss the upcoming parade night or camp. Because of this I believe my knowledge of the issues the senior cadets, adults and logistics cadets face is up to date. However, I have not had to plan lessons on a regular basis for about two years. As a result, I believe a questionnaire to the NCO's would be an excellent way to ensure that their needs are properly considered in the development of this system. Below is the questionnaire that I will be giving to 3 randomly selected NCO's:

Section 1: Communication and Information Flow

1. How effective is the current communication system (e.g., WhatsApp, bulletin board, emails) in keeping you informed?
 - ☐ Very effective
 - ☐ Somewhat effective
 - ☐ Not effective
2. Have you experienced delays or confusion due to conflicting or outdated information?
 - ☐ Frequently
 - ☐ Occasionally
 - ☐ Rarely
 - ☐ Never
3. What is your preferred way to receive important updates (e.g., schedules, tasks, event details)?
 - ☐ WhatsApp
 - ☐ Email
 - ☐ Bulletin board
 - ☐ Other (please specify): _____

Section 2: System Organisation and Use

4. Do you feel the current system for lesson scheduling and planning is effective?
 - ☐ Yes, it's effective
 - ☐ Somewhat effective
 - ☐ No, it's not effective
5. How easy is it to access necessary information (e.g., schedules, kit lists, training plans)?
 - ☐ Very easy
 - ☐ Somewhat easy
 - ☐ Difficult
6. What specific improvements would you like to see in the current system? (e.g., a single platform for all information)

Section 3: Leadership and Delegation

7. Do you believe the delegation of tasks by senior cadets is clear and well-organised?
 - ☐ Always
 - ☐ Sometimes
 - ☐ Rarely
 - ☐ Never

Section 4: Technology and Accessibility

8. Are you comfortable using digital tools and systems for CCF-related tasks?
 - ☐ Yes, I'm comfortable
 - ☐ Somewhat comfortable
 - ☐ No, I'm not comfortable
9. Do you feel the current digital systems (e.g., email, file sharing) meet the needs of the CCF effectively?
 - ☐ Yes, they do
 - ☐ Partially
 - ☐ No, they don't

Section 5: Open Feedback

10. What are the greatest challenges you've faced due to the current system? Please explain.
11. What additional features or changes would you suggest for a new system to improve communication, organisation, and cadet retention?

Responses to questionnaire from 3 NCOs' and analysis of responses

I have asked 3 senior cadets to take part in the above questionnaire to aid my understanding of this issue. I have anonymised their responses since some questions are open response and I wanted to get the most honest feedback possible.

Q	Cadet 1	Cadet 2	Cadet 3	findings
1	Somewhat effective	Somewhat effective	Not effective	Communication systems for planning need improvement
2	Frequently	Occasionally	Frequently	Delays are a common occurrence as a result of poor and difficult to follow communication media
3	Email	WhatsApp	Convocation	Cadets use a variety of communication means and a new system must enhance the benefits of the many systems
4	No, it's not effective	Somewhat effective	Somewhat effective	Lesson planning workflows need improvement
5	Very easy	Difficult	Difficult	Access to information is challenging for most however platforms do exist that have made it easy for some cadets
6	A unified platform where the typical planning needs of cadets are easily met	A platform where the larger picture is more easily accessible to check for potential issues that may arise	A platform similar to the cadet portal but with functionality that facilitates the needs of cadets	2 user needs become clear: - a platform that streamlines planning - a platform that provides holistic oversight i.e. the bigger picture
7	Sometimes	Rarely	Always	Delegated tasks are inconsistently organised likely due to different leadership styles
8	Yes, I'm comfortable	Yes, I'm comfortable	Yes, I'm comfortable	Cadets would generally feel comfortable using digital systems to aid their planning
9	Yes, they do	No, they don't	No, they don't	Current digital systems don't meet the needs of most cadets indicating change is wanted

10	Conflicting and convoluted orders that make it difficult to understand the desires of officers.	Frequently finding out that equipment requests that have been approved by one logistics cadet can't be fulfilled due to poor oversight from the logistics cadets.	Watching cadets attendance drop over time and no action being taken to reverse this since the pattern goes unnoticed until it is too late.	The following challenges face the current system: poor organisation, lack of oversight and ineffective communication frequently result in inadequacies and a reduction in cadet engagement
11	a system with accounts that are personalised to my individual needs in planning and oversight. That allow for the senior adults to see the bigger picture and spot problems before they arise leading to more fluid parade nights that reduce the time cadets end up wasting due to lack of oversight.	A system where the logistics cadets will have a unified view of what equipment requests have been made so they can make an informed decision as to what they can accommodate during the parade night. Additionally, they should have this view on the parade night so they can prepare the requests in order.	A system where the attendance of cadets can be tracked. Leading to better oversight of which cadets should be up for promotion or demotion. A system where the delegation of lessons is easier to perform, and the officers can check on the progress of lessons to prompt senior cadets if results aren't turning out the way they want them to.	A system that is tailored and comprehensive of cadet needs is desired from all in the sample. The system should aid planning and logistics. As well as, potentially offering a bigger picture view that can be analysed for previous shortcomings from the management team.

The results from this questionnaire, have further reinforced the senior adult's and my own beliefs, on the types of issues that the CCF presently encounters. The earlier sections on the needs of the users and the identification of the users align with the views of the senior leaders in the results from this questionnaire. There is a clear need for change in the current system and, it is evident that this change requires a digital aspect. That will cater to this generation of cadets who are digitally savvy and, see the advantages of a "real time" system that is up to date. Additionally, outside of this questionnaire when speaking to NCOs'. It was highlighted to me the variety of digital media in use during the lesson planning process and in the lessons. The main examples given from this were Microsoft word (for lesson planning) and Microsoft power point (used as a lesson aid to give further details of concepts in lessons).

User needs

The user will need the following functionality's:

1. A personalised interface that is password protected
2. A dynamic calendar that facilitates the creation of lessons and parades for adult leaders allowing them to see the bigger picture all in one place
3. A dynamic calendar for the logistics cadets so they will have the ability to see all lessons and approved equipment requests
4. A dynamic calendar for the duty cadets these users will have the ability to edit and make equipment requests for lessons they have delegated
5. A dynamic calendar for the cadet leaders will have the ability to edit and make equipment requests for lessons delegated to themselves
6. A calendar for the basic cadets will have the ability to show cadets what lessons they are in hopefully increasing engagement
7. Access controls that prevent users from changing details of the system they are not permitted to change.
8. A function to request equipment from the logistics cadets for lessons they are teaching.
9. Automatic data validation to ensure all data entered adheres to the system rules and prompts users according to the validation rules that have been violated.
10. Help page to inform users of common issues and how to fix them. Additionally, this page will highlight common features to ensure the system is fully utilised.

In summary, my system that will improve the process of lesson planning, logistics and equipment management. It is intended for use by the adult leaders, senior cadets and logistics cadets respectively. As a result of this it is also appropriate this information is passed down to junior cadets who will have a capacity to view their upcoming lessons.

Data dictionary

The program will use many pieces of data that must be stored using appropriate variable names that match in the program and in the database. Exemplar data will be used in the programming in the system to avoid any accidental breaches of data protection.

The data will be stored long term in the SQL database where it can be accessed any time by a user from another device. Below will be an outline of the data being stored it will have information on why and where it will be stored.

Data for login page and Create/ edit/ view users page

Field name	Field purpose	Field type	Field length	Example data	validation
email	For contact outside the system + username	Varchar	50	18mcmahon.H@stcolumbascollege.org	Check for char before and after @
password	For authentication	Varchar	100	54072213aed8fdf963be967834b2ffa50138a69d090be0bf1419e1e6608e00fd	Check when entered is more than 8 characters
user_id	For identification	Int	6	000001	check not left blank
photoPath	Path to photo	Varchar	100	C://CCF/photos/1.jpg	Check valid path
first_name	For identification	String	20	Henry	Check not left blank
Last_name	For identification	String	20	McMahon	Check not left blank
DOB	For identification	Date	8	1/1/2000	Check not left blank
gender	For identification	String	6	Male	Must not be left blank
rank	For identification	String	20	CSM	Must fit prescribed ranks
active	For identification	Bol	1	True	Must be either true/false

Data for storing the equipment we have

Field name	Field purpose	Field type	Field length	Example data	validation
equipment_id	To identify the piece of equipment	Int	6	000001	Must be int less than 7 in length
description	Description of piece of equipment	String	100	Compass in degrees	Must not be left blank
location	Where the equipment is	String	30	Midstore navigation box	Must not be left blank

Navigation banner (header)

Field name	Field purpose	Field type	Field length	Example data	validation
user_id	For identification and getting profile photo and user attributes	Int	6	000001	If blank redirect client to login page
photoPath	Path to profile photo	Varchar	100	C://CCF/photos/1.jpg	Check valid path if not use the default photo
First_name	To be concatenated with last name and rank to display the full name	String	20	Henry	Will not be blank no validation
lastName	To display the full name of the user	String	20	McMahon	Will not be blank no validation
rank	To display the full name of the user	String	20	CSM	Will not be blank no validation
DOB	For identification	Date	8	1/1/2000	Will not be blank no validation required
gender	For identification	String	6	Male	Will not be blank no validation required
admin	If the user is an admin, they will be given the link to the admin panel page	Int	1	1	Will not be blank no validation either 1 or 0

Data for calendar page

Field name	Field purpose	Field type	Field length	Example data	validation
parade_id	Identifying what parade's details are being printed to the screen	Int	6	000001	Print message that there are no parades beyond this point in time to the user
event_id	Identifying what events details are to be printed in which parade	Int	6	000001	Will always be valid since retrieved directly from the DB
event_name	Identifies what will likely be happening during the event	Varchar	255	Turning at the halt lesson 1	Will always be valid since event form can't be submitted with an invalid name
Event_type	Identifies what kind of event the event is	Varchar	255	Drill/ fieldcraft/ weapon handling	Will always be valid since event form can't be submitted with an invalid event type
event_start	For scheduling	Datetime	12	15:45 10/10/2024	Will always be valid since form can't be submitted with invalid start time
event_end	For scheduling	Datetime	12	16:10 10/10/2024	Will always be valid since form can't be submitted with invalid end time
final_aproval	Is the event unplanned, awaiting approval or approved	Int	2	2	Only invalid state will be 3. Which will never be used in the program so this can be assumed not a possibility.
request_equipment_id	To identify where the request is from	Int	6	100340	Must be int less than 7 in length
Admin	To identify what permissions the user should have	Int	1	1	Will always be valid since form cant be submitted with invalid one
G4	To identify what the client should be able to see in terms of equipment requests	Int	1	0	Will always be valid since form cant be submitted with invalid one

Data for requesting equipment functionality

Field name	Field purpose	Field type	Field length	Example data	validation
requestEquipmentID	To identify where the request is from	Int	6	100340	Must be int less than 7 in length
equipmentID	To identify the piece of equipment	Int	6	000001	Must be int less than 7 in length
Approved	To note if it can happen	Bol	1	TRUE	Must be bol
StartTime	To identify when each item will leave	Datetime	12	15:45 10/10/2024	Must be before the end time
EndTime	To identify when each item will leave	Datetime	12	15:45 10/10/2024	Must be after the start time

Data volumes

As with any computer system, storage has to be considered. The below estimates are for a year's use where all users create profile pictures and where lesson plans are brand new. The college has limited space on local machines/ servers and therefore appropriate estimates of storage space required must be made. In order to avoid issues in the near future I think it appropriate to give the system around 2GB of storage space as this will allow concerns with storage to be realised far before they become problematic.

Rescore	Details	size
Database	7 tables MAX 2mb per table	$7 * 2 = 14\text{mb}$
File storage	100 profile pictures MAX 0.5mb	$100 * 0.5 = 50\text{mb}$
Program files	30 scripts	3mb
$3 + 50 + 14 = 67\text{ mb}$		

It is likely that after a few years of use this system will require more storage space, if the administrator wishes to keep all the records of previous cadets. At this stage more space will need to be allocated to the system and expand the size of the SQL tables.

System objectives

In summary the system will improve the process of lesson scheduling, delegating tasks, lesson planning as well as logistics and equipment management. The system is intended to be used by the entire St Columba's college cadet force.

My objectives will be broken down into the following title-groups.

Objective Area	Specific Objectives
Security of the system	1, 2, 3, 4
User experience and guidance	5, 6
Calendar page	7, 8
Administrator permissions	9, 10, 11, 12, 13, 14
User types	15, 16, 17
Lesson planning	18, 19
Register options	20, 21, 22
Input validation	23, 24
Searching	25, 26, 27, 28
Flexibility	29
Back up of database	30

To achieve this functionality the system must meet the following objective requirements:

N.O.	Objective	Likely solution
1	One way encryption of passwords	Using the inbuilt hashing function in PHP then comparing the hash to the one stored in the DB.
2	Appropriate access control on accounts so that junior cadets can't access functions intended for staff/ senior cadets.	When clients first load webpages, their user_id will be retrieved from the session information and their details will be retrieved from the DB. With the users details privilege verification can be performed and if failed the user will be redirected to a non-privileged page.
3	Users should be redirected to appropriate alternative pages in the event that they don't have the permissions required to visit a page. (i.e. Standard users visiting the add.php page)	At the start of the code for the restricted access pages the user should be checked for their permissions and if they don't meet the requirement, they should be redirected to the calendar page.
4	A navigation bar different for users of different privileges, allowing them to select the task they wish to undertake	A PHP file for the navigation bar must be included at the top of all client pages. This file must have privilege verification to offer the admin a link their page. It must also handle file paths so the profile photo of the user can be viewed.
5	Easy to use by people with a low level of IT experience	The system must be properly styled so it is intuitive to users who are used to this.

N.O.	Objective	Likely solution
6	Create a help page to guide the user through the use of the system	Create a help page that will be accessible by the navigation bar and login page.
7	Users must be able to skip forwards and backwards on the range of dates to be displayed on the calendar page	At the top bar of the calendar there should be buttons to redirect the user to the calendar but with the condition of current_date which will define the starting date of the calendar.
8	Calendar must display events in different formats according to the role of the user	Admin or G4 attributes should be checked before the events are selected. Otherwise, events are only to be selected if the client is related to the event. Once events are selected admin or G4 will always see the approval status of the event. Otherwise, if the client is duty cadet or event owner, they will also see approval status and if they are only on the register then approval status is not to be shown. Finally, admins must see events with the button modify in admin panel edit. G4 must see a log of equipment requests. Duty cadets must see duty event along the top of the event unless they are owner. Display logic will be executed when the event is being put into an html format.
9	Administrator must be able to create/modify/deactivate accounts with permissions as they see fit	Must create an admin page where the admin can search for a user then modify their details as they see fit and post them back to the server.
10	Administrator must be able to create parades and name them, so they are easy to search for	Putting a parade name attribute on the add a parade form that will be stored on the DB.
11	Administrator must be able to approve events	Admin will be able to do this on the modify event details form in the event page for the event.
12	Administrator must be able to create and modify non-approved events	Admin will have access to the event page for the event where they will be able to modify the non-approved events details and post them back to the server.
13	Administrator must be able to create/modify/delete pieces of equipment from the database	Admin page where they can create equipment and post it to the server. They must also be able to modify/ delete equipment by searching for it then populating a form with its results.
14	On the admin panel page (add.php), sensible default values must be used	When forms are written in html the drop-down boxes must have a value set to the default value of the attribute. i.e. admin will be set to non admin when the page is loaded.

N.O.	Objective	Likely solution
15	Specialised accounts for the logistics cadets (G4) to view and approve/deny equipment requests	Admin must be able to modify users to have the G4 attribute set to true. When G4 users load the calendar page they must have a page that tells them what requests there are. G4 users must be the only users able to make approval decisions in the event page and post them to the server.
16	The system must have the position of event owner. There must be one of these for every lesson, so a cadet is responsible for the bulk of the event planning.	In the modify event details form on the event page and the calendar page for the admin. There must be a search box to change the user_id of the event owner. These changes must be displayed in a way a user will understand i.e. displaying their full name somewhere on the form but not as an input field.
17	The system must allow duty cadets to view the progress on lessons	Add the attribute duty cadet to the admins page so when an event is created the duty cadet is assigned.
18	The system must allow the administrators to view lesson planning	The admin will be able to view the planning of lessons on a parade night by clicking on the admin panel edit button to view all the details of the event. i.e. if all the events have the same details, then no planning has happened but if they are all scheduled with different event owners, and they are all pending approval then planning is on schedule.
19	The system must allow lesson planners to request equipment, and that piece of equipment must be reserved for the duration of the lesson when the request is approved	The event page accessible for the event owner must have a form that displays the equipment requests for the lesson. This form must show the requests and must be modifiable only by G4. additional, event owners must be able to request equipment by searching for it.
20	The system must allow lesson planners to fill out their register at the start of their lesson. This register state must then be stored on the database for later review and use	The event page accessible for event owners must have a section that displays the register and allows the event owner to change its values, so a cadet is either present or absent. This register must then be posted back to the server when it is ready.
21	The system must allow lesson planners to populate i.e. add/remove cadets from the register for the lesson. The register must ensure that a cadet is not in 2 lessons at the same time	The event page that is accessible for event owners must have a register section that displays the register and allows for the owner to search for cadets to add to the register.
22	The register for an event must have a facility to search for a cadet to see what lesson they are in if they can't be added to the register	The event page register section will have a search box that will allow event owners to search for cadets by first name to see what lesson they are in and for what times.

N.O.	Objective	Likely solution
23	Validation rules to ensure reliable data entry and prompts to make users aware of the rules	All forms must have dedicated JS validation functions. If the form is incorrectly populated it can't be submitted (deactivate the submit button) and if the client makes an error in its population, then the client must be made aware of how to fix this through an error handling prompt in a dedicated div.
24	When a user enters an invalid input into a form then corrects it, they must be able to recover and submit the form.	Form validation JS functions must keep track of which inputs are valid and invalid so they are treated on a case-by-case basis so the form can be recovered after one invalid input is corrected. For forms that are populated by other functions I will set each 'validation state' of the input in the form to 1 by default (they will be assumed to be valid inputs). To prevent users from submitting empty forms, when a form is populated the validation function should be called so it is made aware the form is ready for submission. It must keep track of if it has been called by storing an attribute like user_id to indicate the modify user form has had its user_id hidden field populated. This attribute should be set to 0 by default since the form is not loaded populated in a valid state.
25	The system must scan all search prompts for malicious code to prevent attacks on the system.	All search prompts will come through a get request hence a function will be written to do the get request and kill the program if malicious input is detected. This function will have its own file and will be included when its needed.
26	The system must have a way to search for entities on the database i.e. cadets to add to a lesson register	The system will handle searching entities by clients submitting get requests to the server that respond with the results of the search. Results will be shown in a dedicated div and will clear when a result is clicked, or the search prompt is cleared.
27	Search fields must show a dynamic output	The JS search function should be called on every keystroke and should clear the results in the event that the prompt is empty i.e. the user pressed tab in the search box.
28	When searching for equipment or users to modify, more search options than just first name must be offered	Write search function to handle searching by last name for user modification and search by equipment location for equipment modification. There will also need to be different functions on the server side to handle these different search prompts.

N.O.	Objective	Likely solution
29	The system must allow for as wide a variety of naming conventions as possible, so the administrator may decide their own system for naming entities (this allows the system to be scalable)	The lesson names, parade names, equipment locations and ranks of users are all plaintext this allows the admin to choose to what level of description the system will have.
30	The system must perform a regular backup of the database in case of corruption or attacks	The system will have a backup shell script that is scheduled for a regular task in the early hours of the morning. This script will perform a backup and store it in a directory that www-data can't access.

Categorisation of users and their abilities within the system

Following a conversation with my client Major Jackson we have agreed that the adult leaders and adult volunteers share many characteristics. Additionally, all the adult volunteers are trusted i.e. have passed DBS checks and are responsible. As a result, we have decided that the system should merge these two groups into the admin category as this will provide the volunteers with more functionality which will ultimately reduce the workload on Major Jackson.

Following my conversations with NCOs' and seeing the value that Microsoft Word and Microsoft PowerPoint provide them. I have decided that the lesson planning process i.e. what is being taught inside the times of the lesson should stay within word. I have decided this because Word offers such a variety of formats that allow NCOs' use a variety of lesson structures. This provides NCOs' the ability to explore their teaching style and, I believe that my system would not be able to surpass the functionality that is being offered by Word. Microsoft PowerPoint is also widely used as a lesson aid, and I do not wish to 're invent the wheel' when it comes to this aspect of teaching lessons.

Since we are a Microsoft, school where students have the ability to concurrently access documents and view their progress. I believe it is in the best interest of the CCF, that this system is continued, and my platform will work alongside these established tools. In the future this could mean that the St Columba's College CCF would have its own bank of lesson aids. This would mean cadets will no longer have to rely on the previously mentioned difficult to navigate and potentially out of date resources available online.

Finally, it must be noted that a system like this is typically built by a team of developers who have sometimes decades of experience working with their programming language. I do not and instead I am interested in solving the most pressing issues that the CCF face. After a discussion with Major Jackson we have decided that my system should focus on the day to day running of the CCF. Therefore, my system should not dedicate any of its functionality to planning of camps. As a result, the signing in/ out of equipment will still be done on paper as it currently does not pose any significant bottleneck to the CCF's ability.

Admin only features

1. Add rank will allow the admin to enter a new rank for the user and if the new rank passes verification the database will be modified to show the new rank.
2. Add user will allow the administrator to enter new details for a user and if the data passes verification the user will be added to the database with the entered information.

3. View/ add/ modify/ remove equipment will allow the administrator to perform the following functions:
 - A. View, will allow the administrator to read through the register of equipment.
 - B. Add, will allow the administrator to enter details for new pieces of equipment. If the details pass verification, then the database will be updated with the new equipment.
 - C. Modify, will allow the administrator to view pieces of equipment and change their details. Which will then be updated on the database
 - D. Remove, will allow the administrator to delete pieces of equipment from the database.
4. Approve lesson will allow the administrator to change the status of a lesson on the database to approved.
5. Add lesson or event will allow the administrator to enter details for new lessons or events. If the details pass data verification, then the lesson will be added to the database.
6. Add parade night will allow the administrator to enter details for new parade nights or camp days, once the data is verified then the details will be added to the database.

G4 features

1. Approve/ deny equipment requests will only be accessible to users designated to G4(logistics) they will be able to move pending requests to approved/denied. However, the reverse can't be performed and instead requests must be deleted then re-submitted.

Event owner features

1. Add users to the register will allow the owner of a lesson to search for users and add them to their lesson. They will then also be able to work through the register in the lesson time checking off who is present and who is not.
2. Change timings of lessons will allow the owner of a lesson to enter new times for a lesson and once verified the database will be updated with the new timings.
3. Make equipment requests for lessons that they own once the equipment ID entered is verified along with the times of the request. the database will be updated with the new request and another student will fulfil it.

Event duty cadet

1. These users will have all the permissions and functionality that event owners have. However, they will have the role of overseeing the progress in planning of the events which will reduce the overall workload for the admins.

All users feature

1. View the calendar for upcoming parades so that they can plan ahead of time any equipment they will need or any arrangements they may need to make having known this.
2. Login/logout feature users will be able to login and have a token attached to their browser that allows them to access the calendar feature.

Potential solutions

N.O.	Suggested solution	Positives	Negatives
1	Apache + PHP webapp + SQLite server	--Apache and PHP well researched with SQL server --No client-side installation (plug and play system) --No simple client-side access to API keys or SQL login --I already have knowledge in HTML, CSS and JS	--PHP is an older language that is inefficient compared to modern solutions especially when scaled up --Limited debugging tools --I will have to learn PHP
2	Python tkinter + SQLite server	--Already have strong skills with python and have worked with tkinter before --Have worked with SQLite python before --Framework already exists for client-side python execution	--Will require the school to audit code and make it available to students --GUI will be basic and will be difficult to navigate --Open-source risking accidental or purposeful modification or deletion of program
3	VBS + SQLite server	--Previous experience with VBS --Efficient language that has few stages from program code to machine code	--Open source and more dangerous functionality --I will have to learn GUI programming for VBS

Chosen solution and justification

The solution I have chosen is number 1 “Apache, PHP webapp and SQLite server”. While I will have to learn PHP there are the following advantages to this kind of system:

- The school will not have to review my code that will go on the client’s machine since JS poses little to no risk to client machines
- Cadets will not have to perform an installation process
- The options for the interface are almost limitless as I will be using HTML, CSS and JS which are purpose built to offer functionality.
- The opportunities available for the student to accidentally/ purposefully modify the system in a catastrophic way are limited compared to the other options
- The server will likely run on a Unix-based system that will be more efficient compared to a free version of the windows server package
- The delay for interactions from the user will be minimal since HTML and JS will be running on the client side which will be very quick

This system will be fast and use minimal power on the client side.

Prototype

Before the system is designed it is essential that I build a prototype of the system that has the core functionality of the final system. This will verify that the methods I believe can be used to meet objectives will work and this can be used to discuss the progression of the project with the client, so major changes are made early on.

Below is a gallery of screenshots of the prototype of the system that are individually discussed.

Login page

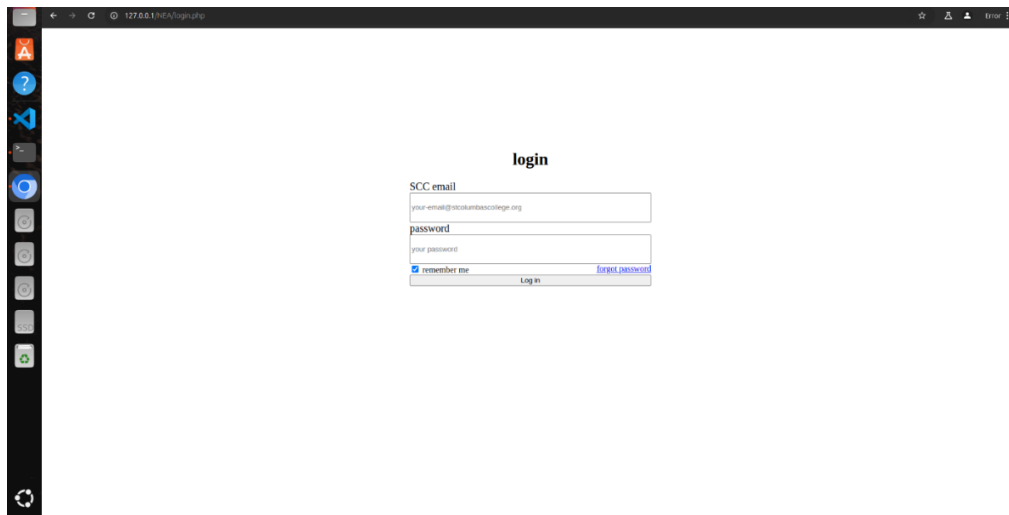


Figure e screenshot of login page in the prototype

Major Jackson my client was happy with this page. It is simple and works by posting the entered details to the server. The server will then compare the email and password hash against the database and if they match that of a user the user_id is attached to the session information of the client. Then the client is redirected to the system. If they don't match, then the login page is reloaded with a prompt telling the client why login has failed i.e. don't leave email empty.

Calendar page for admin

The calendar page was the first page that I worked on outside the login and the navigation bar it looked like this:



Figure 7 screenshot of calendar page for the admin in the prototype

The calendar page works by retrieving the current date then it selects the parade_id's of parades with a date greater or equal to that of today. Following this the parade table is used to select event_id's and the event table is then queried. The results of these queries will populate the events in the parade. this proved to be problematic since there were only 4 event slots for each parade meaning the number of events for a parade was limited which Major Jackson and I agreed would need to change in the actual system.

Originally the calendar page was not going to have a capacity to edit events however after discussion with Major Jackson it was agreed this was a necessary feature that would increase the efficiency of the system. It was also concluded that an ability to scroll the calendar would be required i.e. see parades outside the current display range.

Aside from these points the feedback from my client was positive indicating that the system and its interfaces were developing in the right direction.

Calendar page for standard user

my dashboard	my parade night's	my events	my stores	my profile	logout
< 2024-11-29 TILL 2024-12-20					
2024-11-29 15:45:00 till 18:30:00 15:45:00 till 16:10:00 start parade Henry McMahon 16:10:00 till 17:10:00 personal admin in the field Henry McMahon 17:00:00 till 18:10:00 cleaning the rifle Admin Admin 18:10:00 till 18:30:00 final parade Henry McMahon	2024-12-06 15:45:00 till 18:30:00 15:45:00 till 16:10:00 start parade Henry McMahon 16:10:00 till 17:10:00 personal admin in the field Henry McMahon 17:00:00 till 18:10:00 cleaning the rifle Admin Admin 18:10:00 till 18:30:00 final parade Henry McMahon	2024-12-13 15:45:00 till 18:30:00 15:45:00 till 16:10:00 start parade Henry McMahon 16:10:00 till 17:10:00 personal admin in the field Henry McMahon 17:00:00 till 18:10:00 cleaning the rifle Admin Admin 18:10:00 till 18:30:00 final parade Henry McMahon	2024-12-20 15:45:00 till 18:30:00 15:45:00 till 16:10:00 start parade Henry McMahon 16:10:00 till 17:10:00 personal admin in the field Henry McMahon 17:00:00 till 18:10:00 cleaning the rifle Admin Admin 18:10:00 till 18:30:00 final parade Henry McMahon	no events on this day	

Figure 8 screenshot of calendar page for regular user in the prototype

Overall feedback for the standard user calendar page was positive and cadets liked the idea of being able to see their upcoming parades and what they would be doing. After a discussion with a logistics cadet, it was brought up that if a functionality to view equipment requests was added to the calendar page, then a page dedicated to equipment requests would not be necessary. This will hopefully make the system more compact and easier to navigate.

Page to view progress over time as a cadet

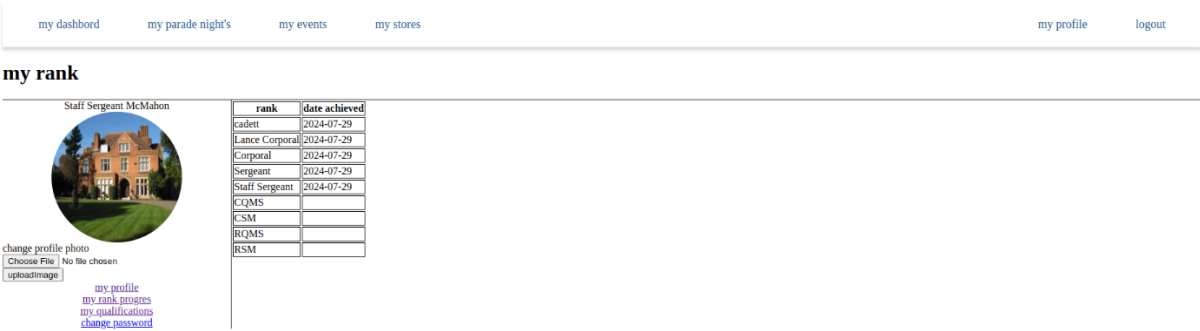


Figure 9 screenshot of rank progress page in the prototype

After discussion with Major Jackson, it was agreed that this page should be discontinued. Simply there is not enough content to warrant an entire page and the content that is already there is not a system objective. Instead, we agree that along the navigation bar there should be a drop-down menu to show the full name and rank of the user along with any other relevant information. For changing the profile photo of a user, it was agreed that this should be contained in the drop-down menu like it is on many websites. Since this page was the page users would be directed to after login. It was agreed between us that users should go straight to the calendar page since it would be where 90% of activity for all users would start. This move should hopefully make the system easier to navigate and understand.

Event page

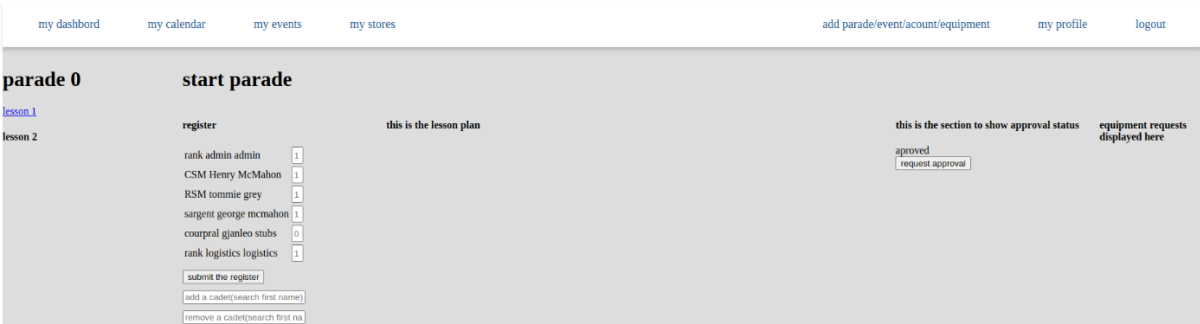


Figure 10 screenshot of the event page in the prototype

This page while basic looking was positively received by Major Jackson. He particularly liked how you would have the ability to jump to different event pages on the parade from the left menu. He believed it provided a space for the earlier mentioned objectives to be met. However, he suggested combining the approval section and requests section so more space would be given to show requests. The approval section should then be moved into the lesson plan section since it is part of the lesson planning process.

Finally, the search functions were operational and met the earlier criteria of being able to add/remove cadets from the register.

Conclusion of the prototype

Overall feedback of the prototype was positive from cadets and Major Jackson. Additionally, the following points were brought up as suggestions for the next iteration:

- Removal of the 4-event limit per parade to be completed by re-designing the relationships of the DB
- Implementation of scrolling outside the current displayed range on the calendar page
- Add in a feature for the admin to edit events on the calendar page
- A drop-down menu for the users details will be better for the UX compared to a page dedicated to the users information
- Slight re-design of the event page so equipment requests are given adequate space

All these suggestions are possible to implement, and I will treat them as system objectives.

Entity relationship diagram

In light of the feedback from the prototype. I sought to normalise the data the system would be storing. Since normalized databases are more efficient and have less data redundancy, as a result I have made the following changes to what data is being stored and how it is accessed:

- The entities that are being stored will be parades, events, users and equipment
- The calendar page will show the parades with their events
- The event page will be dedicated to events and the logistics that surround them
- The admin page will allow for creation/ modification of users and equipment it will also be the place for the creation of events and parades
- The tables user_events and equipment requests will be needed to store the register and equipment request log in an efficient manner
- The primary keys of each entity will be used in the linking tables i.e. the attribute requestEquipmentID will no longer exist because event_id can be used instead.

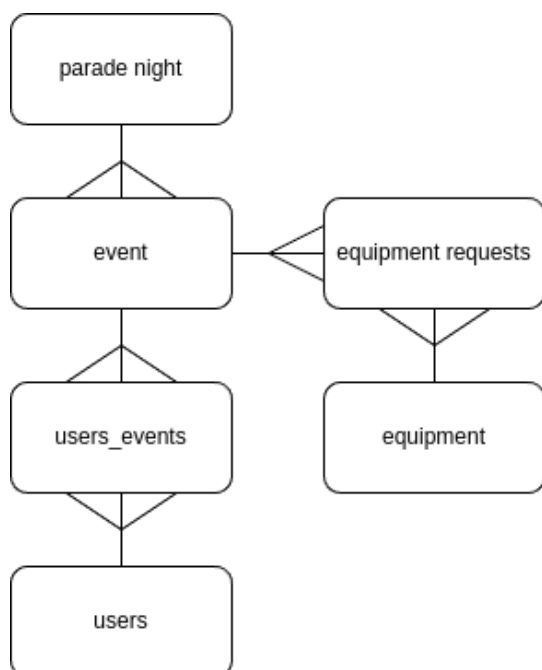


Figure 11 ER diagram for the database

Design

Overall system design

The system will have usernames and passwords for all users, so that they can access a system that is personalised to them. The system is designed as an aid to lesson planning as well as an aid for scheduling of lessons or events for parade nights or camps. After users log in, they will be directed to the calendar page. The calendar page will allow users view upcoming parades and their lessons. The user will also be able to click into individual lessons and make equipment requests for their lessons. Along the top of all pages there will be a banner giving the user access to key links such as: the logout page, calendar page and help page.

Structure of functionality in the system

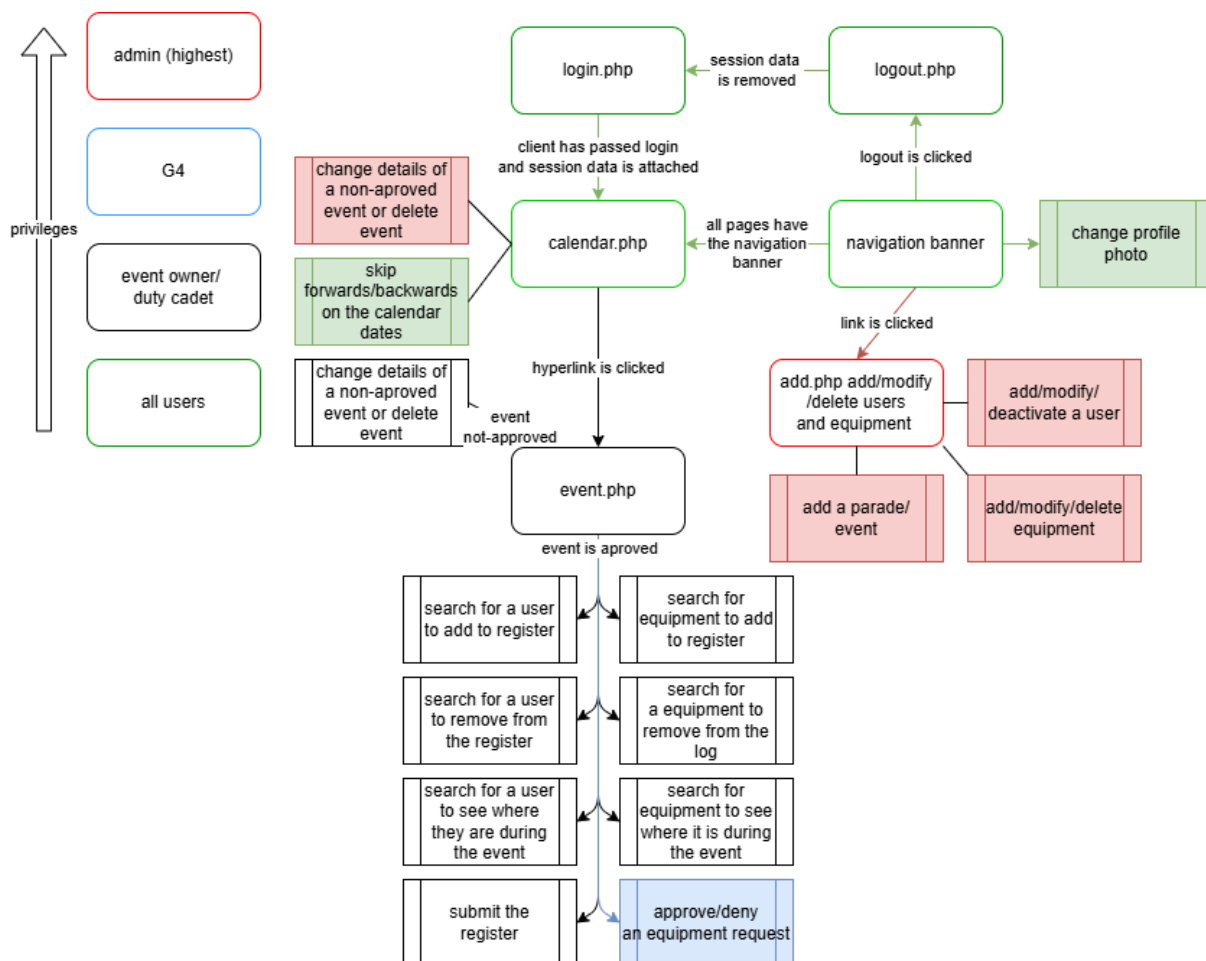


Figure 12 flowchart to show structure of functionality in the system

Flowchart for pipeline of an event in the solution

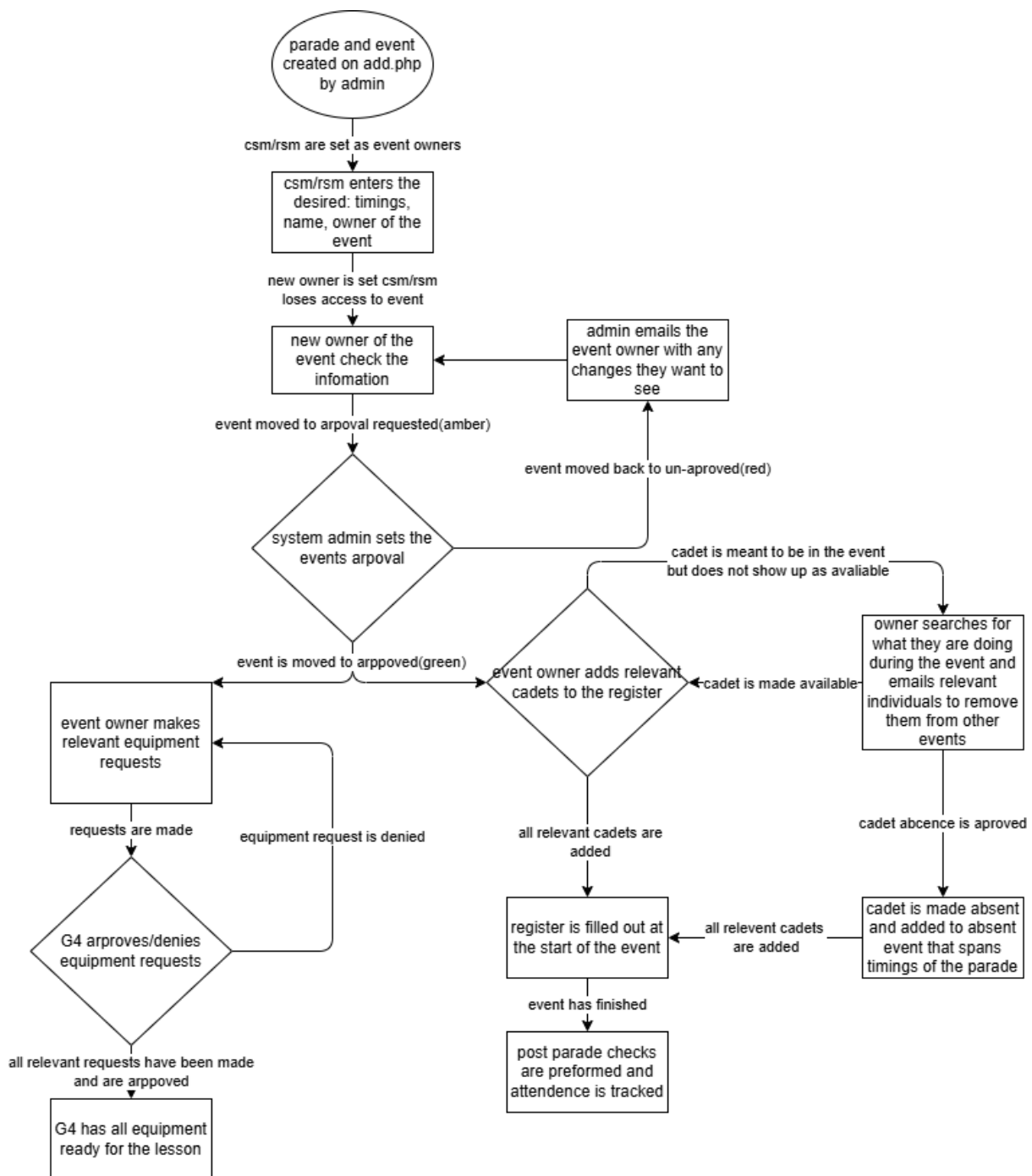


Figure 13 flowchart to show the lesson planning process in the new system

Definition of data requirements (normalised)

The data used in this program will be put into a database so that information can be stored long-term. The database will be made up of several tables and linked relationally. The tables of data have to be normalised to create efficient relationships between them. The variables in the data dictionary will be made into separate tables in an un-normalised form. Normalisation removes redundant data and creates relationships between tables. When designing a database, the data dictionary has to be split into the necessary tables to create a relational database. Below is the database that will be used in the system.

- Fields marked **bold** will be primary keys, where there are many bold fields, this indicates a composite primary key.
- Fields underlined will be foreign keys.
- Text marked **bold** in the validation section indicates the text is a regular expression that will be used to validate the data.
- Rows where length is marked N.A. indicate the length of the data is not customisable and therefore dictated by the datatype.

users table

Field	Type	Length (bytes)	Example data	Validation
User_id	Int	4	1	Must not be an id that already exists for this category
Email	Varchar	255	18mcmahon.h@stcolumbascollege.org	[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$ must fit the general form of an email Can't be empty and can't be longer than 255 characters
password	Varchar	255	one way encrypted text	Can't be longer than 255 characters in plaintext
First_name	Tinytext	255	henry	Can't be longer than 255 characters
Last_name	Tinytext	255	McMahon	Can't be longer than 255 characters
DOB	Date	N.A.	01/01/2000	\d{4}-\d{2}-\d{2}\$ / must fit the general form of a date
Gender	Tinytext	255	male	Can't be longer than 255 characters
Rank	Varchar	255	CSM	Can't be longer than 255 characters
Active	bit	1 bit	1	Must be either 1 or 0
Admin	Bit	1 bit	1	Must be either 1 or 0
G4	Bit	1bit	1	Must be either 1 or 0

events table

Field	Type	Length	Example data	validation
Event_id	Int	4	200	Must not be an id that already exists for this category
parade_id	int	4	1	Must not be an id that already exists for this category
event_type	tinytext	255	Skill at arms	Can't be longer than 255 characters
event_name	tinytext	255	Skill at arms lesson 3	Can't be longer than 255 characters
event_start	time	N.A.	15:45	\d{2}:\d{2}\$/ must fit the general format of a time
event_end	time	N.A.	16:10	\d{2}:\d{2}\$/ must fit the general format of a time
finalApproval	bit	1 bit	1	Must be either 1 or 0
owner	int	255	1	Must be a user_id
duty	Int	255	1	Must be a user_id

parades table

Field	Type	Length	Example data	Validation
user_id	Int	4	1	Must not be an id that already exists for this category
event_id	Int	4	1	Must not be an id that already exists for this category
present	bit	1 bit	1	Must be either 1 or 0

User_event table

Field	Type	Length	Example data	Validation
parade_id	Int	4	1	Must not be an id that already exists for this category
date	Date	N.A.	10/10/2024	\d{4}-\d{2}-\d{2}\$/ must fit the general form of a date
start	Time	N.A.	15:45:00	\d{2}:\d{2}\$/ must fit the general format of a time
end	time	N.A.	18:30:00	\d{2}:\d{2}\$/ must fit the general format of a time
parade_name	Tinytext	255	Lent_parade_1	Can't be longer than 255 characters

equipment table

Field	Type	Length	Example data	Validation
equipment_id	Int	4	1	Must not be an id that already exists for this category
name	tinytext	255	compass	Can't be longer than 255 characters
description	text		Always points north	Can't be longer than 255 characters
location	text		Mid_store	Can't be longer than 255 characters

equipment_requests table

Field	Type	Length	Example data	Validation
equipment_id	Int		100340	Must not be an id that already exists for this category
event_id	Int		1	Must not be an id that already exists for this category
approved	Int	2	2	Must be either 0, 1 or 2 (requested, approved or denied respectively_

Input, process, storage, output

All pages will include the nav.php document which will give users quick access to webpages they have permissions for as well as the logout button on the right-hand drop-down menu.

For login.php and logout.php

Since login.php will look the same for all users, it will be written primarily in HTML with some PHP to handle the post request with the users login information.

Input	Process	Storage	Possible outputs
Login Username, password	Authenticate login credentials, attach user ID token to \$userData variable	Credentials stored in temporary variable and comparison happens on server against the database's records	Credentials correct redirect to calendar Credentials incorrect redirect to login page
Selecting logout box Mouse click of box	Detach user id token from \$userData	None	User is logged out redirect to login page

For calendar.php

Calendar.php will be the most complex PHP page. It will be primarily written in PHP since there are few assumptions that can be made as to the structure of the page the user will receive. There will be many functions that produce HTML outputs given inputs regarding the user the page is currently serving. This code will primarily be written as functions since there are many elements on the calendar page that are repeatable. Additionally, this will lead to a cleaner easier to read code.

The CSS that styles the elements will be modular. For instance, the admin edit form will have its own style document to negate the need for users to download it when they don't have access to the admin edit form, this will decrease load times for non-admin users. However, there will be a calendar-style.css document that account for the majority of styling of the calendar.php page. This will style the calendar into a grid format with individual events having different colour codes to represent the stag they are currently in regarding their approval status. I have chosen the grid styling method since this system is intended to be used on the student's laptops and not mobile phones (which would require a different styling format).

Finally, there will be some JS to handle the population of the admin edit tab, the validation of the admin edit form. For these JS functions to be properly utilized the PHP code will generate the inputs for the JS function calls. For instance, the populate admin edit form call will have all the event details prewritten into the function call.

Input	Process	Storage	Possible outputs
Selecting calendar option	Retrieve 5 parades with date >= todays date. Retrieve events with parade id matching.	Calendar data stored on the server then output as html code.	No upcoming parades, parades upcoming without events, parades with events upcoming
Clicking skip forward/backwards on calendar	this redirects user to new page with \$_GET("current_date") set to the date of the arrow the user has selected.	Current_date is stored on the server.	Valid date and null. Null will be interpreted as todays date
Clicking the link to events.php for an event on the calendar page	User redirected to events.php with \$_GET("parade_id") and \$_GET("event_id") set to their respective values.	Parade_id and event_id stored in \$user_data dictionary.	Event not approved /pending approval so event plan input field loaded, event approved load in register and equipment requests. Both cases the list of events during the parade is loaded on the left

Click for admin panel edit is clicked	JS function called to populate the admin panel with the correct information and the submit/delete event buttons are activated.	None	Field is populated
Submit is clicked for an admin panel edit on calendar.php	Post request sent to event_details_request.php.	Values of form fields are temporarily stored on server until they are input into an SQL query and stored permanently in the events table.	The form is valid and committed to the database
Delete event is clicked for an admin panel edit on calendar.php	Post request sent to server with_event id and parade_id. Record is then deleted in the events table.	Parade_id and event_id temporarily stored on server until event is deleted and user is redirected to calendar page.	Data is valid and the event is deleted.

For add.php

Many assumptions can be made as to the structure of the add.php page since it will always be accessed by the admin and the forms will never need to be pre-populated for the user. For styling there will be one styling document that is not used elsewhere. Finally, there will be JS scripting unique to add.php this will have 2 roles: validating the forms that will be posted to the server, performing get requests back to the server and populating hidden input fields according to the options the user has selected.

Input	Process	Storage	Possible outputs
Add a parade is filled out on add.php and submit is clicked	Post request with values of form fields is sent to post_requests.php. the maximum parade id is selected and incremented by one. Data is inserted into parades table.	Date, start_time, finish_time, parade_name are temporarily stored until data is added to database.	The form is validly filled out and the user is redirected to add.php once process completed
Add an event is filled out on add.php and submit is clicked.	Post request with values of form fields is sent to post_requests.php, The max event_id is selected from events table and incremented. Data is inserted into events.php.	Parade_id, owner_id, duty_id, event_type, event_name, event_start, event_end are all stored temporarily on the server until the data is inserted.	The form is filled out validly the process is successful, and the user is redirected back to add.php
Add a user is filled out on add.php and submit is clicked	Post request with values of form fields is sent to post_requests.php, The max user_id is selected from users table and incremented (new user's user_id). Data is inserted into users table.	Email, password (in plaintext), first_name, last_name, DOB, gender, rank, active, admin, G4 are all temporarily stored until data is inserted.	The form is filled out validly the process is successful, and the user is redirected back to add.php

Modify a user is filled out on add.php and submit is clicked	Post request with values of form fields is sent to post_requests.php. The password field is hashed (if it exists), data is inserted into users table where the user_id = the pre-selected user_id.	Email, password (in plaintext), first_name, last_name, DOB, gendar, rank, active, admin, G4 are all temporarily stored until data is inserted.	The form is filled out validly the process is successful, and the user is redirected back to add.php. If the password field is left blank, then a different query is used that does not change the password.
Add equipment form on add.php is submitted	Post request with values of form fields is sent to post_requests.php, the values are then inserted into the equipment table.	Name, description and location are all stored as temporary variables until its entered to equipment table.	Data is inserted and user is redirected to add.php.
Modify equipment form on add.php is submitted	Post request with values of form fields is sent to post_requests.php, the action fields value is checked if it is set to modify then, the record with equipment id equal to the equipment_id that was posted is updated with the new data. If it was set to deleted, then the record with equipment_id = equipment_id that was posted is deleted.	Equipment_id, name, description and location are all stored as temporary variables until query is executed.	Record is updated or deleted, and user is redirected to add.php.

For event.php

For event.php more assumptions can be made as to its structure since this page will only be accessed with a valid event_id and parade_id. As a result, some of event.php will be written in HTML this will decrease load times since the server will not have to perform any logic for some of the output HTML. Some functions must also be written to produce the populated form that contains the events details and the list to show the other events happening on that parade night.

The CSS that will style this document will be primarily provided by an event-style.css document those styles unique elements not seen elsewhere, such as the register and request equipment form. However, some styling will be inherited from the event-style.css document used in the calendar page since this will style the column that shows the other events happening on the parade night.

Finally, there will be JS to handle the validation of the edit event form, this will be the same script used in calendar.php (to decrease load times). However, there will be unique JS scripts used here that are not used elsewhere such as the script to handle the get requests to see which users/equipment are available to add/remove from the register/equipment request log these requests will go to a PHP requests document that will

be purpose built just for the event.php form since no other document will need access to this page. Furthermore, there will be validation for the register/equipment log forms for instance the register must not contain anything other than 0's or 1's. This will prevent the user from being redirected back to the form page when the form fails validation on the server side (since this will delete the inputs the user had provided).

Input	Process	Storage	Possible outputs
modify an event's details is filled out on add.php and submit is clicked	Post request with values of form fields is sent to post_requests.php, values of the form are then copied into the query. Update query is executed, and user is redirected back to appropriate page.	Parade_id, owner_id, event_tye, event_name, event_start, event_end are all stored temporarily on the server until the data is inserted.	The form is filled out validly the process is successful, and the user is redirected back to event.php if they are admin or owner of the event. If not, then user is redirected back to the calendar.
Delete event is clicked on modify event details form.	Post request sent to server with event_id and parade_id. Record is then deleted in the events table along with any records in the equipment_requests table and user_events table.	Parade_id and event_id temporarily stored on server until event is deleted and user is redirected to calendar page.	Data is valid and the event is deleted.
Submit the register is clicked	Post request sent to post_request.php. the flag is removed, and the event_id and parade_id are also removed from the post dictionary. The post dictionary is then looped through with user_id as the key and present (1 or 0) is the value and records are updated. The user is then redirected back to the event.php page.	Parade_id, event id are temporarily stored on the server until the register is finished and the user is redirected back to the event.php page.	Data is valid and the register is updated.
Submit the equipment request log is clicked	Post request sent to post_requests.php and the flag, event_id, parade_id are all removed from the dictionary, the post dictionary is then looped through with keys being the equipment_id and the values being the state of the equipment request this is sent to the database in the form of update commands.	Parade_id, event_id are temporarily stored until the register is finished submitting to the database then the user is redirected back to the event.php page.	Data is valid and the equipment request log is updated.

Search first name add a cadet to the register	Get request is sent to event_get_requests.php which will first compile a list of cadets that can't make it to the event (because they have a conflict, or they are in the lesson already) then this list is subtracted from the list of all users and the users search prompt further removes users from this list. Finally, the results are output in html format and returned to the JS function where they are printed to the users' screen.	The users search prompt, and the event_id are stored on the server until the query has finished executing and the output is returned to the user.	Results are found and they are put into html format then returned. No results are found, and a html message will be returned reflecting this. The input is deemed malicious, and the program will die returning a HTML error code of 500 to the JS function.
Search first name remove a cadet from the register	Get request is sent to event_get_requests.php which will compile a list of users which are in the lesson then some users are removed by the search prompt and the list is returned in html format to the JS function where it is then printed to the users' screen.	The users search prompt, and the event_id are stored on the server until the query has finished executing and the output is returned to the user.	Results are found and they are put into html format then returned. No results are found, and a html message will be returned reflecting this. The input is deemed malicious, and the program will die returning a HTML error code of 500 to the JS function.
Search first name for another cadet's event	Get request is sent to event_get_requests.php which will compile a list of user_id's and event_id's of cadets in events during the current events time. This list is then put into html format and returned to the JS function where it is then printed to the users' screen.	The users search prompt, and the event_id are stored on the server until the query has finished executing and the output is returned to the user.	Results are found and they are put into html format then returned. No results are found, and a html message will be returned reflecting this. The input is deemed malicious, and the program will die returning a HTML error code of 500 to the JS function.

For user input validation in JavaScript

Input	Process	Storage	Possible outputs
Onkeyup JS function called to validate input field	The user input and input to check variables are input. Input_to_check dictates what rules the user input must adhere to. The input is checked then form_array and form_feedback arrays are changed to reflect the current state of the system. The arrays are then checked to see if the submit button should be activated if it isn't then the form_feedback array is used to return an appropriate prompt to the user.	The form_feedback, form_array, input_to_check and user_input are all stored locally on the users' machine.	The input is valid and form_array for the current field is set to 1 and form_feedback for the current field is emptied. The input is invalid, and form_array is set to 0 and form_feedback is set to the correct output.

For entity search

Input	Process	Storage	Possible outputs
Appropriate JS function is called to handle the search for the current form	The JS function will then make a get request to the server. The server will then perform a search of the appropriate table for the user's prompt. An appropriate response will then be sent back to the original JS function (either HTML or JSON). The function will then print the results of the search to the user.	The users prompt will be temporarily stored on the server until the search is finished and results are returned.	Results are found and they are put into html format then returned. No results are found, and a html message will be returned reflecting this. The input is deemed malicious, and the program will die returning a HTML error code of 500.

Grouping procedures into files categorised by their primary use cases

In order to ensure the software is efficient in its use of server resources. I created an Excel sheet of all the procedures/function I have on the client side and the server side. I then grouped functions together into files so 1 file on the server side will only handle requests from 1 part of the website. This ensures that my server is not unnecessarily retrieving code from secondary storage. Additionally, this will contribute to the readability of my code and will help with optimisation in the future.

Cell colour	Purple	Blue	Green	Orange
Associated page making the request	Add page	Event page	Calendar page	Various pages

Posting of forms

type of request	post_requess.php	flag	inputs	used by
POST	add new user to users	add_new_user	email, password, first_name, last_name, DOB, gender, rank, active, admin	add.php
POST	add new equipment to equipment	add_new_equipment	name, description, location	add.php
POST	add new parade to parades	add_new_parade	date, start, end, parade_name	add.php
POST	add new event to events	add_new_event	parade_id, event_type, event_start, event_end, owner_id	add.php
POST	modify user in users	modify_user	same as add_new_user but with modify_ tags infront of each get tag	add.php
POST	modify equipment in equipment	modify_equipment	same as add_new_equipment but with modify_ tags infront	add.php
POST	modify register	modify_register	array of all user_id's on register, event_id, parade_id	event.php
POST	modify equipment register	modify_equipment_register	array of all equipment_id's on register, event_id, parade_id	event.php

Get requests for event.php

type of request	event_get_request .php	flag	inputs	return format	request made from	function name	outputs	inputs to js function	host of function user
POST	add to register	add_user_id	user_id, event_id	n/a	event.js	resultHasBeenClickedAdd	n/a, reloads the page	user_id, event_id	Event .php
POST	remove from register	remove_user_id	remove_user_id, event_id	n/a	event.js	resultHasBeenClickedDelete	n/a, reloads the page	user_id, event_id	Event .php

type of request	event_get_request .php	flag	inputs	return format	request made from	function name	outputs	inputs to js function	host of function user
POST	add to equipment for event	add_equipment_id	add_equipment_id, event_id	n/a	event.js	resultHasBeenClickedAddEquipment	n/a, reloads the page	Equipment_id, event_id	Event .php
POST	remove equipment from event	remove_equipment_id	remove_equipment_id, event_id	n/a	event.js	resultHasBeenClickedDeleteEquipment	n/a, reloads the page	Equipment_id, event_id	Event .php
GET	search first name add	search_first_name	prompt, event_id	HTML	event.js	showResultAddCadet	search results in div	str, search_for, event_id	Event .php
GET	search first name remove	search_first_name_delete	prompt, event_id	HTML	event.js	showResultDeleteCadet	search results in div	str, search_for, event_id	Event .php
GET	search first name cadet in other activity	search_first_name_other_cadet	prompt, event_id	HTML	event.js	showResultSearchOtherCadet	search results in div	str, event_id	Event .php
GET	search equipment name add	search_equipment_add	prompt, event_id	HTML	event.js	showResultAddEquipment	search results in div	str, event_id	Event .php
GET	search equipment name remove	search_equipment_delete	prompt, event_id	HTML	event.js	showResultDeleteEquipment	search results in div	str, event_id	Event .php
GET	search equipment name in other activity	search_equipment_other	prompt, event_id	HTML	event.js	showResultSearchOtherEquipment	search results in div	str, event_id	Event .php

Get and post requests for events thought the system

type of request	event_details_requests.php	flag	inputs	return format	request made from	function name	outputs	inputs to js function	host of function user
GET	search by first name for new owner of an event	search_first_name_owner	prompt	HTML	event_owner_form_handling.js	showResutsSearchForOwne r	search results in div	prompt	Calendar .php, event.php, add.php
POST	modify event details	modify_event_details	parade_id, event_id, event_type, event_name, event_start, event_end, owner_id, calendar_flag	n/a	calendar.php, event.php	calendar.php or event.php	n/a redirects the user to page they were on	see inputs for request	Calendar .php, event.php
POST	delete event	delete_event	delete_parade_id, delete_event_id	n/a	calendar.php, event.php	post request from calendar.php or event.php	n/a redirects the user to calendar page	see inputs for request	Calendar .php, event.php

Get requests primarily for add.php

type of request	add_get_requests.php	value of flag	inputs	return format	request made from	function name	output	inputs to js function	host of function user
GET	search parade name	search_parade_name	prompt	HTML	add.js	ShowResultsSearchForParade	search results in div	prompt	add.php
GET	search user first name for event duty cadet	search_duty	prompt	HTML	add.js	showResutsSearchForDuty	search results in div	prompt	add.php
GET	search first name modify user	search_first_name_user	prompt	HTML	add.js	showResutsSearchForUserFirstName	search results in div	prompt	add.php
GET	search last name modify user	search_last_name_user	prompt	HTML	add.js	showResutsSearchForUserLastName	search results in div	prompt	add.php
GET	search equipment name modify equipment	search_equipment_name	prompt	HTML	add.js	showResutsSearchForEquipmentByName	search results in div	prompt	add.php
GET	search equipment location modify equipment	search_equipment_location	prompt	HTML	add.js	showResutsSearchForEquipmentByLocation	search results in div	prompt	add.php
GET	get all attributes of equipment with id	equipment_id_info_dump	equipment_id	JSON	add.js	resultHasBeenClickedEquipment	populate the modify equipment form	equipment_id	add.php
GET	get all attributes of user with user_id	user_id_info_dump	user_id	JSON	add.js	resultHasBeenClickedUser	populate the modify user form	user_id	add.php

type of request	add_get_requests.php	value of flag	inputs	return format	request made from	function name	output	inputs to js function	host of function user
GET	get all attributes of user with user_id	user_id_info_dump	user_id	JSON	event_owner_form_handeling.js	displayCurrentOwnerOfEvent	display the full name of event owner when it changes	user_id	Calendar.php, event.php
GET	get all attributes of user with user_id	user_id_info_dump	user_id	JSON	add.js	resultHasBeenClickedDuty	display the full name of event duty cadet when it changes	user_id	add.php
GET	get all attributes of user with user_id	user_id_info_dump	user_id	JSON	event.js	REGEXCheckRegister	displaying the name of the cadet whose present value is invalid	user_id	Event.php

JS functions that do not make requests to the server.

These function do not make http requests and primarily are used for populating/validating forms.

user of function	location of function	name of function	function purpose	inputs to function
calendar.php	calendar.js	populateAdminEditEventForm	populate the admin panel edit form	parade_id, event_id, event_type, event_name, event_start, event_end, owner_id, final_aproval
calendar.php	calendar.js	checkAreAllValuesOne	check all values in the event_array are set to 1	obj
calendar.php	calendar.js	returnFeedbackHTML	return a string of error handling prompts from the error handling prompt dictionary in html	obj
calendar.php	calendar.js	REGEXCheckEvent	validate the admin panel edit form	str, input to check
calendar.php	event_owner_form_handling.js	resultHasBeenClickedOwner	populates the hidden field modify_owner_id	user_id
event.php	event.js	setStateOfEventApproval	sets the drop down box for approval state to its value (since html doesnt automatically do this)	state
event.php	event.js	checkAreAllValuesOne	check all values in the event_array, register_array or equipment_array are set to 1	obj
event.php	event.js	returnFeedbackHTML	return a string of error handling prompts from the error handling prompt dictionary in html	obj
event.php	event.js	REGEXCheckEvent	validates the modify event form when the event is not approved	str, input_to_check, admin

user of function	location of function	name of function	function purpose	inputs to function
event.php	event.js	REGEXCheckEquipment	validates the equipment request log form (only lets G4 users change states)	str, equipment_id, original_approval, G4
event.php	event.js	REGEXCheckRegister	validates the register form i.e. only allows 1 or 0 to be entered	str, user_id
event.php	add.js	checkAreAllValuesOne	checks all the values in the validation arrays are set to 1 i.e. form is ready for submission	obj
event.php	add.js	returnFeedbackHTML	return a string of error handling prompts from the error handling prompt dictionary in html	obj
add.php	add.js	REGEXCheckParade	validates the add a parade form	str, input_to_check
add.php	add.js	REGEXCheckEvent	validates the add a event form	str, input_to_check
add.php	add.js	REGEXCheckAddUser	validates the add a user form	str, input_to_check
add.php	add.js	REGEXCheckModifyUser	validates the modify a user form	str, input_to_check
add.php	add.js	REGEXCheckAddEquipment	validates the add equipment form	str, input_to_check
add.php	add.js	REGEXCheckModifyEquipment	validates the modify equipment form	str, input_to_check
add.php	add.js	ResultHasBeenClickedParade	sets the hidden parade_id field to its value in the add an event form and updates the validation state for parade id	parade_id, parade_date, parade_name
add.php	event_owner_form_handling.js	resultHasBeenClickedOwner	poulates the hidden field modify_owner_id	user_id

Considerations for client server communication

Reasons for using JSON for extra client server communication

When a client has selected a search result, they must populate the form they are working in with the information of this search result. Instead of sending all the needed information to populate the form in each search result, I have decided that it will be better for the client to request further information once a result has been selected. For this to work a get request must be made and the information put into a form the client can interpret. This leaves two obvious candidates JSON and XML of which I have chosen JSON for the following reasons: it is more character efficient, there are inbuilt tools in JS and PHP for working with JSON and it is newer/ the new industry standard.

Considerations for concurrent access of the database

All clients of the system will be using the same reduced permission MySQL user account with username apache_user. Concurrent access will be managed by timestamp ordering which will be automatically performed by the MySQL server package, I have installed on the ubuntu server.

Managing concurrent access for this system is not a major concern since the majority of changes to the database will occur from admin accounts on the system. These changes will likely happen from the hours of 8am till 4pm while the admins are in school. These changes could affect to any entity on the database.

From the hours of 4pm till 7pm the senior cadets and lower ranks will have left school, and they will likely be changing entities that only relate to them i.e. event that they are owner of. Since only a small handful of non-admin users will be able to do this to 1 event the chances of.

Considering the above facts about the likely usage habits of users. It is safe to assume that frequent and reoccurring issues as a result of concurrent access will be minimal.

Structure of webpages

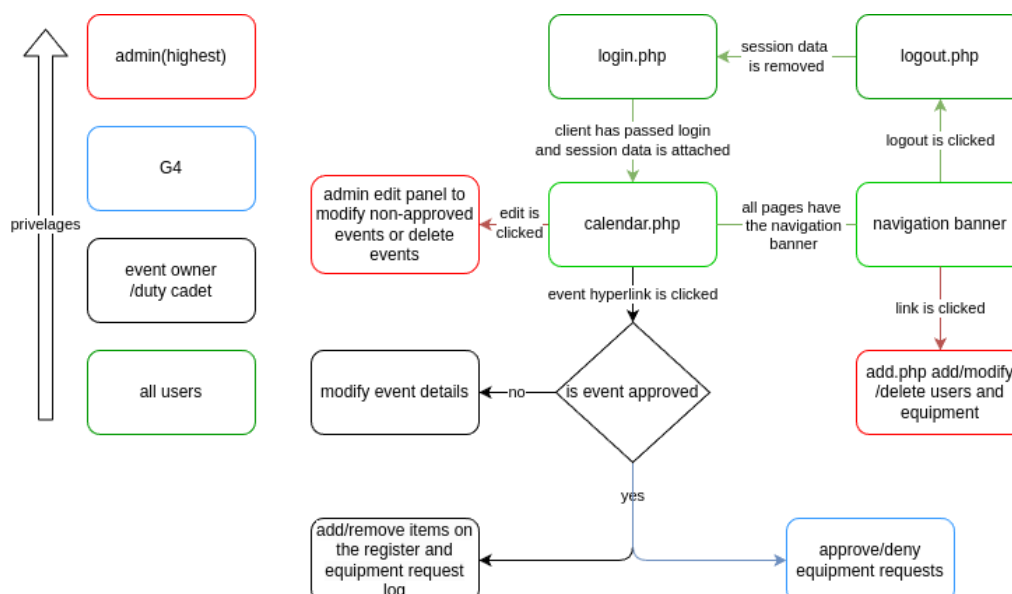


Figure 14 flowchart to show structure of webpages and their broad functionalities

Webpage design

Below, I have written html webpages that give an outline of how I envisage a typical webpage to look.

Login page

127.0.0.1/NEA/login.php

login

SCC email

your-email@stcolumbascollege.org

password

your password

☒ remember me [forgot password](#)

Log in

Figure 15 screenshot of the user interface design for the login page in the new system

Calendar page

Calendar page for the admin (and G4)

Admins will have access to the admin panel seen on the left

G4 users will see the log of approved equipment requests when there are approved equipment requests

The screenshot shows a web browser window displaying a calendar application. The address bar shows the URL `127.0.0.1/NEA/calendar.php`. The page has a top navigation bar with links for "my calendar" and "add parade/event/acount/equipment". Below this is a date range selector showing "2025-01-24 till 2025-02-28".

On the left side, there is an "admin panel" with the following fields:

- event type:
- event name:
- event start:
- event end:
- approved:
- event owner:

Below these fields are two buttons: "submit" and "delete event".

The main calendar area displays a grid of events for the dates 2025-01-24, 2025-01-31, 2025-02-07, and 2025-02-28. Each date has a "parade" event. The events are color-coded: green for parades, yellow for training, and pink for lessons. Each event card includes the time, title, organizer, and a "click for admin panel edit" button.

At the bottom left, there is a "log of approved equipment requests" table:

name	location	start time	finish time	lesson name
compas_4	mid store	16:00:00	16:15:00	start parade
compas_5	mid store	16:00:00	16:15:00	start parade
compas_1	mid store	18:00:00	18:15:00	final parade

Below this table is another "log of approved equipment requests" section with columns for name, location, start, finish, and lesson.

Figure 16 screenshot of the user interface design for the calendar page for the admin and G4 in the new system

Calendar page for other users

The events with a link indicate the user that is logged in owns the event hence they can click the link to go to the event.php page

Events that are just grey indicate the user is only on the register for the event

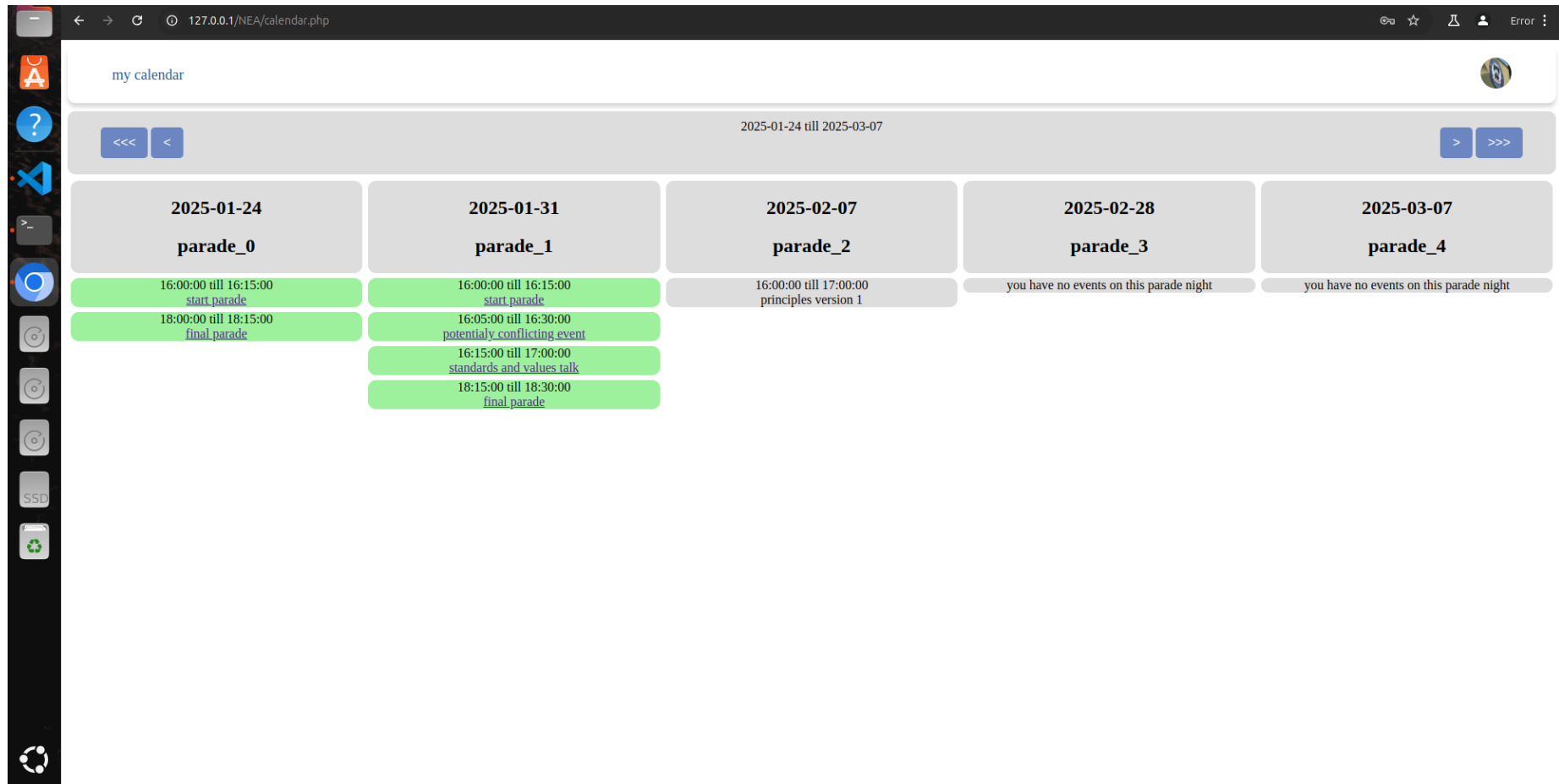


Figure 17 screenshot of the user interface design for the calendar page for event owners and standard users in the new system

Event page

The screenshot below is of an event that has been approved

my calendar [add parade/event/acount/equipment](#)

parade_0 **start parade**

2025-01-24

register		event aproved, lesson plan not modifiable	
16:00:00 till 16:15:00 start parade	rank admin admin	1	
17:00:00 till 18:00:00 overview of training	CSM Henry McMahon	1	
18:00:00 till 18:15:00 final parade	RSM tommie grey	1	
	sargent george mcmahon	0	
	courpral gianleo stubs	0	
	rank logistics logistics	1	

[submit the register](#)

equipment requests displayed here

compas_4

compas_5

compas_6

[submit the equipment request log](#)

Figure 18 screenshot of the user interface design for the event page when the event is approved in the new system

The screenshot below is of an event that has not yet been approved, hence there is access to the modification of event details

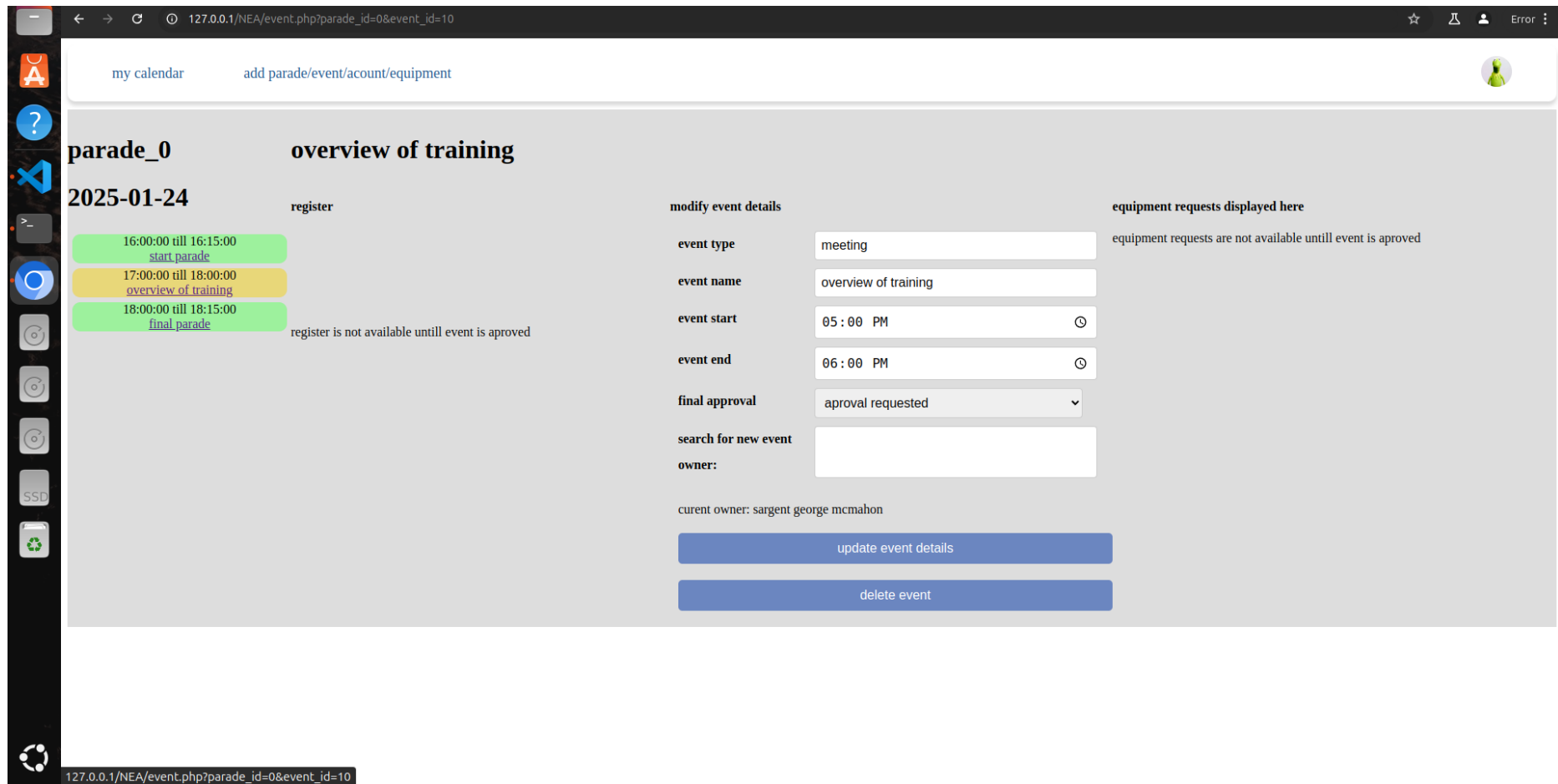


Figure 19 screenshot of the user interface design for the event page when the event is not-approved in the new system

add page

my calendar add parade/event/account/equipment

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-04-25 the next parade should be: 2025-05-02

date:

start time:

finish time:

parade name:

add an event

event type:

event name:

event start:

event end:

event owner:

current owner: none selected

parade:

current parade: none selected

users

add a user

email:

password:

first name:

last name:

date of birth:

gender:

rank:

active:

admin:

G4:

modify a user

leave the password field blank to leave it unchanged

search by first name:

search by last name:

email:

password:

first name:

last name:

date of birth:

gender:

rank:

active:

admin:

G4:

equipment

add equipment

equipment name:

equipment description:

equipment location:

update equipment

search for equipment by name:

search for equipment by location:

equipment name:

equipment description:

equipment location:

action:

Figure 20 screenshot of the user interface design for the add page in the new system

pre-made SQL queries

Table	Create table
users	<pre>CREATE TABLE `users` (`user_id` int unsigned NOT NULL, `email` varchar(255) NOT NULL, `password` varchar(255) NOT NULL, `first_name` tinytext NOT NULL, `last_name` tinytext NOT NULL, `DOB` date NOT NULL, `gender` tinytext NOT NULL, `rank` varchar(255) NOT NULL, `active` bit(1) NOT NULL COMMENT '1 = active, 0 = retired', `admin` bit(1) NOT NULL COMMENT '1 = admin, 0 = non-admin', `G4` bit(1) NOT NULL COMMENT '1 = g4, 0 = not-g4', PRIMARY KEY (`user_id`))</pre>
events	<pre>CREATE TABLE `events` (`event_id` int unsigned NOT NULL, `parade_id` int unsigned NOT NULL, `event_type` tinytext NOT NULL, `event_name` tinytext NOT NULL, `event_start` time NOT NULL, `event_end` time NOT NULL, `final_aproval` tinyint NOT NULL COMMENT '0 = not aproved, 1 = aproved, 2 = pending aproval', `owner` int unsigned NOT NULL, `duty` int unsigned NOT NULL, PRIMARY KEY (`event_id`, `parade_id`), KEY `parade_id` (`parade_id`), KEY `owner` (`owner`), CONSTRAINT `events_ibfk_1` FOREIGN KEY (`parade_id`) REFERENCES `parades` (`parade_id`), CONSTRAINT `events_lbfk_1` FOREIGN KEY (`duty`) REFERENCES `users` (`user_id`), CONSTRAINT `events_ibfk_2` FOREIGN KEY (`owner`) REFERENCES `users` (`user_id`))</pre>

parades	<pre>CREATE TABLE `parades` (`parade_id` int unsigned NOT NULL, `date` date NOT NULL, `start` time NOT NULL, `end` time NOT NULL, `parade_name` tinytext NOT NULL, PRIMARY KEY (`parade_id`))</pre>
User_event	<pre>CREATE TABLE `user_event` (`user_id` int unsigned NOT NULL, `event_id` int unsigned NOT NULL, `present` bit(1) NOT NULL COMMENT '0 = absent, 1 = present', PRIMARY KEY (`user_id`, `event_id`), KEY `event_id` (`event_id`), CONSTRAINT `user_event_ibfk_1` FOREIGN KEY (`event_id`) REFERENCES `events` (`event_id`), CONSTRAINT `user_event_ibfk_2` FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`))</pre>
equipment	<pre>CREATE TABLE `equipment` (`equipment_id` int unsigned NOT NULL, `name` tinytext NOT NULL, `description` text NOT NULL, `location` text NOT NULL, PRIMARY KEY (`equipment_id`))</pre>

Equipment_requests	<pre> CREATE TABLE `equipment_requests` (`event_id` int unsigned NOT NULL, `equipment_id` int unsigned NOT NULL, `aproved` int NOT NULL COMMENT '0: requested, 1: aproved, 2:denied', PRIMARY KEY (`event_id`, `equipment_id`), KEY `equipment_id` (`equipment_id`), CONSTRAINT `equipment_requests_ibfk_1` FOREIGN KEY (`equipment_id`) REFERENCES `equipment` (`equipment_id`), CONSTRAINT `equipment_requests_ibfk_3` FOREIGN KEY (`event_id`) REFERENCES `events` (`event_id`) ON DELETE RESTRICT ON UPDATE RESTRICT) </pre>
--------------------	--

Defining complex SQL queries

Selecting first names of users who can be added to event that match a search prompt (conflict management logic)

For this query I will first select all the information about the current event this will be stored in a dictionary called \$event

\$query = "SELECT users.first_name, users.last_name, users.rank, users.user_id FROM users defining what attributes from the users table are being selected

WHERE users.first_name REGEXP "" . str_replace("", "", \$search_first_name) . "" only selecting users with a first name that fit the regular expression with the users search prompt

AND users.user_id NOT IN only selecting users that do not appear in the next query (which will select users who can't make it to the event)

(SELECT user_event.user_id defining what attributes from the user_event table are being selected

FROM events, user_event defining what table the attributes will be selected from

WHERE user_event.event_id = events.event_id linking the tables user_event and events

AND events.parade_id = "" . \$event["parade_id"] . "" only selecting events on the parade night

AND ((events.event_start <= "" . \$event["event_start"] . "" AND events.event_end > "" . \$event["event_start"] . "")) selecting events that start before the clients event and finish after the start time

OR (events.event_start < "" . \$event["event_end"] . "" AND events.event_end >= "" . \$event["event_end"] . "") selecting events that finish after the clients event and start before the end time

OR (events.event_start <= "" . \$event["event_start"] . "" AND events.event_end >= "" . \$event["event_end"] . "") selecting events that run outside of the clients event

OR (events.event_start >= "" . \$event["event_start"] . "" AND events.event_end <= "" . \$event["event_end"] . "") selecting events that run inside of the clients event

OR (events.event_id = "" . \$event["event_id"] . "")); checking we are not looking at the current event

Likely technical solutions

Technique	Likely implementation	Place of implementation
Cross-table parameterised SQL	Use in the scheduling system that ensures a user is not in 2 events at the same time	Event.php page
Aggregate SQL functions	Retrieving max id numbers of records so the next record can be entered with a unique id	Add.php posts the users details one of the request files will perform the algorithm
User/CASE-generated DDL script	Storing a DLL script on the repository that stores the servers code so when the repo is cloned a database can be created for the server to interact with	In the repository storing all server-side code
Hashing	For storage and verification of passwords	Login.php and add.php
Complex user-defined algorithms (e.g. optimisation, minimisation, scheduling, pattern matching) or equivalent difficulty	The scheduling system that ensures a user is not in 2 events at the same time and a piece of equipment is not in 2 events at the same time	Client makes search for cadet or equipment on event.php and one of the requests pages will perform the scheduling SQL algorithm then return the results
Server-side scripting using request and response objects and server-side extensions for a complex client-server model	PHP on the server side that handles GET and POST requests for a variety of purposes i.e. populating forms, getting results of searches, updating the database	All pages
Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model	GET requests that retrieve records from the database and put them into JSON format for the client to	Add.php for population of forms when a record has been searched then clicked

Identification of server, server packages and storage media

For this system the server will use a version of Ubuntu LST(long term support) since these are typically supported for 5 years following the publishing date which will provide a foreseeably long life for the system.

I will use the web server package Apache 2 running on port 80(HTTP), this will host the web pages and interpret the PHP. I will also have to install the current version of PHP as well as MySQL-server. The database service will never be connected to directly by clients, instead the webserver will connect to it through the localhost. As a result, no port forwarding rules will need to be configured for the MySQL server. All packages will be installed using the APT package manager since this will take care of routine updates and security patches as long as the server administrator follows good practice and routinely performs an apt-get update then apt-get upgrade command.

All storage will be on the server. This server will be connected to the internet and will host both the database server and Apache server.

Possible storage media could be one of the following:

Media	Advantages	Disadvantages
HDD	Cheap and reliable	Slower read write speeds compared to other media
SSD	Fastest read write speeds compared to the other options	Limited number of read write cycles
Optical disc	Cheap and easily portable/removable for long term backup	Limited number of read write cycles

Considering all the options available I have chosen HDD storage to be most appropriate for the following reasons: it has the lowest cost per unit storage, read write speeds are not key to the operation of this system since queries will be relatively simple using an SSD will only negligibly boost the systems performance.

Security of the system

In order to protect the functionality of this system a few measures must be taken to prevent malicious SQL or PHP injections into the server. Firstly, all inputs whether get or post requests must be scanned for malicious code. This will be performed by a regular expression that will check for common malicious codes such as the semicolon and the backslash character. If one of these is detected, then the current process will die to prevent an attack if it has already occurred. Additionally, the MySQL login that users will have will not have the permissions to interact with the non-main database or drop/ create tables in the database. Furthermore, the MySQL database can only be interacted with on the local network so it can never be attacked directly from the internet.

To protect the system from users who have limited access to the system and wish to escalate this. I will attach session information to the user which will be the user_id and the hashed password of the user. On every page this information will be checked against the database to prevent users from changing their sessions user_id and gaining access to higher level permissions of another user.

Finally, a 24-hour backup that will delete after 7 days will be performed by an automated shell script process that will be running on the server administrators account this will use a shell script that copies the table into a directory that the users will not be able to access (this will happen at 2AM GMT).

Technical solutions

During the development process of this system, it was agreed by Major Jackson and I that to support the event owners a status should be added to the system that is of equal privilege to the event owner. The presence of this user on the system would improve the overall planning workflow of the system. This is because, in the current solution the cadet above the event owner in the rank structure doesn't have a capacity to view the progress of an event and therefore scheduling tasks would fall upon the admins instead of the senior cadets. This would likely increase the workload on the admin.

As a result, I have made the following changes (some of which can't be seen in the technical solutions section). Changes that can't be seen in technical solutions will also have a note at the top of their section of code which will be in **bold**:

1. Added duty cadet search box to add.php and added div to display the full name of the duty cadet.
2. Added the function showResultsSearchForDuty() to add.js to handle the searching for the duty cadet
3. In the file add_get_requests.php I created the case for searching for the duty cadet
4. Added the function resultHasBeenClickedDuty() to add.js to handle the input of the duty cadets id to the add an event form on add.php
5. Added the attribute duty_id to event_array in the function REGEXCheckEvent() on add.js this attribute will have a value of 1 when the duty cadet has been populated i.e. the duty cadet id is ready for submission
6. In post_requests.php I amended the query to handle the posting of details of the new event so the user_id of the duty cadet for the event is stored on the database
7. I update event.php so when a user visits an event page, and they are the duty cadet of the event they are not redirected to the calendar page.
8. I updated the function generate_html_for_lesson_plan() in event.php so when the html is generated the full name of the duty cadet is displayed like how the full name of the owner of the event is displayed
9. I updated the function html_for_parade() in the file includes\display_parade.php so users would see events that they are duty cadet for. Additionally, these events would be displayed with the words duty event along the top of them to make the user aware they are not the event owner and are instead the duty cadet.
10. I updated event_details_requests.php so users who are the duty cadet of an event are redirected to the event page for the event when the event details form is posted from event.php

I did not feel it necessary to include these changes in the technical solutions since they add little to no more complexity to the system. In simple terms the duty cadet is just a second event owner, but they are assigned when the event is first created and can't be changed. This additional access to the event page will aid delegation of tasks ensuring that the workload for the admins is kept manageable. It will also help event owners receive advice/guidance that is more personalised to them, since duty cadets will often assign the same event owners each week for different lessons.

All these changes have been fully noted in the testing section and are displayed in the testing video.

Please note, the indentation of code in the technical solutions section is inconsistent. This likely happened when the code was copied into the word document because I was using word online (on the ubuntu server). This means that some lines of code in the technical solutions contain 2 lines of code in reality. To see the real indentation of the code view the zip folder attached to this document. It should contain the bellow files and folders:

Name	Status	Date modified	Type	Size
css	●	03/04/2025 21:16	File Folder	
includes	●	03/04/2025 21:16	File Folder	
js	●	03/04/2025 21:16	File Folder	
profile-photos	●	03/04/2025 21:16	File Folder	
requests	●	03/04/2025 21:16	File Folder	
add	●	01/04/2025 21:39	PHP Source File	14 KB
backup_mysql	●	01/04/2025 21:39	SH Source File	1 KB
calendar	●	01/04/2025 21:39	PHP Source File	12 KB
event	●	01/04/2025 21:39	PHP Source File	17 KB
help	●	01/04/2025 21:39	PHP Source File	11 KB
login	●	01/04/2025 21:39	PHP Source File	3 KB
logout	●	01/04/2025 21:39	PHP Source File	1 KB
sql	●	01/04/2025 21:39	Text Document	4 KB
uploads	●	01/04/2025 21:39	PHP Source File	2 KB

Figure 21 contents of unzipped file 'all_code'

Codebase overview

Architectural overview

The frontend of the codebase uses the following languages

- HTML for structuring webpages.
- CSS for displaying the webpages in a more user-friendly way .
- JS for validation of forms, population of forms, getting data from the server and posting data to the server.

The backend of the codebase uses the following languages

- PHP for handling get and post requests (the API).
- PHP for creating the webpages for the user.
- PHP for handling files being uploaded and storing/deleting files on the server.
- Shell script for performing routine tasks i.e. routine backup of the database.
- MySQL for querying the database.

Code structure

- Webpage files are all .php files and are stored in the / directory.
- API files are all .php files and are stored in the directory /requests/.
- Client-side scripting files are all .js files and are stored in the directory /js/.
- Webpage styling files are all .css files and are stored in the directory /css/.
- PHP functions used throughout the codebase are all stored in .php files in the directory /includes/.
- All profile photos are stored in the /profile-photos/ directory. the file types are varied, and their will only ever be 1 profile photo per user.
- The file sql.txt contains a script that will initialise a database so the system can be run.

Key security features

- All user passwords are stored as hashed versions on the database using the algorithm SHA256.
- All search prompts will pass through a regular expression filter that will scan for common malicious inputs. This filter is stored in get_requests.php and the function get_request() is used to retrieve data sent by a get request. A similar process is followed for data that is posted to the server.
- If this system were to be implemented, it would be strongly advised to use the HTTPS protocol since passwords are posted in plaintext to the server. Users will be vulnerable if the HTTP protocol is used.

Key dependencies

- Apache 2 has been used as the webserver software this can be installed using **sudo apt install apache2**
- MySQL-server has been used as the software to handle queries to the database this can be installed using the command **sudo apt install mysql-server**
- Since PHP is the primary scripting language of this project it must be installed using the command **sudo apt install php libapache2-mod-php**

Installation guide

1. Install all the key dependencies of the project and restart your ubuntu environment.
2. Using a software such as phpMyAdmin or the CLI, set up a MySQL user with username apache_user and password Cadet2025 (these can be changed in /includes/connection.php). Be sure to give this MySQL user the minimum sufficient permissions on the server.
3. Set up a database called main_v2 and run the script in sql.txt to create the tables. From here a user must be added to the users table, ensure to hash the users password (sha256) and set their admin attribute to 1 from here the system will be self sufficient with no more installation required.

login.php

1	<code><?php</code>	
2		
3	<code>session_start();</code>	Initialising the session data
4	<code>include_once("includes/connection.php");</code>	Including the mysql connection
5	<code>include("includes/functions.php");</code>	Including the check_login() function
6	<code>\$error_msg = "";</code>	Initialising the error message as an empty string
7	<code>if(\$_SERVER['REQUEST_METHOD'] == "POST"){</code>	The block of code will run when login details are posted to login.php
8	<code>include_once("requests/post_request_scanning.php");</code>	Including the post_request() function
9	<code>\$email = post_request("email");</code>	Setting \$email
10	<code>\$password = post_request("password");</code>	Setting \$password
11	<code>if(!empty(\$email) && !empty(\$password)){</code>	Checking the submission was real
12	<code>\$query = "SELECT * FROM users WHERE email = '" . \$email . "' LIMIT 1;";</code>	Query to find the details of a user with the email posted
13	<code>\$result = mysqli_query(\$con, \$query);</code>	
14	<code>if(\$result && mysqli_num_rows(\$result) > 0){</code>	Checking if the user exists in the database
15	<code>\$user_data = mysqli_fetch_assoc(\$result);</code>	If it does then put all the users data into a dictionary
16	<code>if(\$user_data["password"] == hash("sha256", \$password)){</code>	Checking if the passwords match
17	<code>if(\$user_data["active"] == 1){</code>	Checking if the user is active
18	<code>\$_SESSION["user_id"] = \$user_data["user_id"];</code>	If they are active attach the user_id to the session
19	<code>\$_SESSION["password"] = \$user_data["password"];</code>	Attaching the hashed password to the session
20	<code>header("Location: calendar.php");</code>	Now they have their session information set they can be redirected to the calendar page
21	<code>die;</code>	

22	<code>}else{</code>	
23	<code>\$error_msg .= "Your account is inactive.
 Speak to the system admin if this is an error";</code>	the record that contained the posted email exists but is set to inactive so prompt the user accordingly
24	<code>}</code>	
25	<code>}else{</code>	
26	<code>\$error_msg .= "wrong password!";</code>	The hashed passwords did not match so prompt the user accordingly
27	<code>}</code>	
28	<code>}else{</code>	
29	<code>\$error_msg .= "wrong email!";</code>	A record containing the posted email could not be found prompt the user accordingly
30	<code>}</code>	
31	<code>}else{</code>	
32	<code>\$error_msg .= "Please fill in all the fields";</code>	One or both of the fields were empty prompt the user accordingly
33	<code>}</code>	
34	<code>}</code>	
35		
36	<code>?></code>	
37	<code><!DOCTYPE html></code>	
38	<code><head></code>	
39	<code><title>login</title></code>	
40	<code><link rel="stylesheet" href="css/login-style.css"></code>	Including the style document in the html
41	<code></head></code>	
42	<code><body></code>	
43	<code><div class="container"></code>	
44	<code><div class="title"></code>	
45	<code><H1>login</H1></code>	
46	<code></div></code>	
47	<code><form class="form" method="post"></code>	Defining the submission method for the form that will automatically be submitted to login.php
48	<code><div></code>	
49	<code><label class="input-heading" for="username">SCC email</label>
</code>	

50	<code><input style="width: 450px; height: 50px;" type="text" placeholder="your-email@stcolumbascollege.org" id="username" name="email"></code>	Defining the email (text type) input box and setting its name to email which we used earlier to set \$email
51	<code></div></code>	
52	<code><div></code>	
53	<code><label class="input-heading" for="password">password</label>
</code>	
54	<code><input style="width: 450px; height: 50px;" type="password" placeholder="your password" id="password" name="password"></code>	Defining the password (password type) input box and setting its name to password which we used earlier to set \$password
55	<code></div></code>	
56	<code><div class="minor-container"></code>	
57	<code><div class="remember-me"></code>	
58	<code><input class="checkbox" type="checkbox" checked="checked"/></code>	Including a remember me checkbox to allow for future functionality of attaching cookies to authenticate users
59	<code><label class="remember-me-tag">remember me</label></code>	
60	<code></div></code>	
61	<code>forgot password</code>	Including a forgot password link for future functionality of user restable passwords
62	<code></div></code>	
63	<code><input type="submit" value="Log in" class="login-button"></code>	
64	<code><?php echo "\$error_msg";?></code>	Printing the \$error_msg variable set to an empty string when the page is initially loaded
65	<code></form></code>	
66	<code></div></code>	
67	<code></body></code>	

uploads.php

```
1 <a href="calendar.php">click here to go back to the
  calendar page</a><br>
2 <?php
3 session_start();
4 include_once("includes/connection.php");
5 include("includes/functions.php");
6 $user_data = check_login($con);
7
8 $target_dir = "profile-photos/";
9 $target_file = $target_dir .
  basename($_FILES["fileToUpload"]["name"]);
10 $uploadOk = 1;
11 $imageFileType =
  strtolower(pathinfo($target_file,PATHINFO_EXTENSION));
12
13 if(isset($_POST["submit"])) {
14 $check =
  getimagesize($_FILES["fileToUpload"]["tmp_name"]);
15 if($check !== false) {
16 $uploadOk = 1;
17 } else {
18 $uploadOk = 0;
19 }
20 }
21
22 if ($_FILES["fileToUpload"]["size"] > 500000) {
23 ?>your image is to large try cropping it</br><?php
24 $uploadOk = 0;
25 }
26 if($imageFileType != "jpg" && $imageFileType != "png" &&
  $imageFileType != "jpeg") {
27 ?>your image was not one of the following alowed types:
  jpg, jpeg or png</br><?php
28 $uploadOk = 0;
29 die;
30 }
31
32 $target_file = $target_dir . $user_data["user_id"] .
  ".jpg";
33 if (file_exists($target_file) == true) {
34 unlink($target_file);
35 }
```

1) If the upload has failed the user can go back to the calendar page

3) initialising the session data so the \$user_data can be retrieved
4-5) including the SQL connection and check_login() function

8) setting the upload target directory
9) setting the location of the upload so it can be accessed later in the program

10) creating an upload ok flag

11) retrieving the file type of the submission for validation later

13) only upload if a post request was sent
14) retrieving the size of the image submission
15) Checking the submission was real
16) when submission real set flag to true state

17) if submission not real set flag to false state

22) checking the image size is smaller than 500Kib
23-24) if submission to large then prompt user and set flag to false state

26) only allow certain file types for profile photos since some types i.e. GIF can be used maliciously
27-29) if the file type is incorrect then prompt the user, set the flag to false then kill the program

32) the submission has passed validation, and the previous profile photo must be deleted (jpg)
33) checking if the jpg exists
34) if it does delete previous profile photo.

36		
37	<code>\$target_file = \$target_dir . \$user_data["user_id"] .</code>	37-40) doing the same for jpeg
	<code>".jpeg";</code>	
38	<code>if (file_exists(\$target_file) == true) {</code>	
39	<code>unlink(\$target_file);</code>	
40	<code>}</code>	
41		
42	<code>\$target_file = \$target_dir . \$user_data["user_id"] .</code>	42-45) doing the same for png
	<code>".png";</code>	
43	<code>if (file_exists(\$target_file) == true) {</code>	
44	<code>unlink(\$target_file);</code>	
45	<code>}</code>	
46		
47	<code>\$target_file = \$target_dir . \$user_data["user_id"] . "."</code>	47) setting the file path of the
	<code>. \$imageFileType;</code>	submitted profile photo
48		
49		
50	<code>if (\$uploadOk == 0) {</code>	50) checking the profile photo has
51	<code>echo "Sorry, your file was not uploaded.";</code>	passed validation from the flag
52	<code>} else {</code>	52) if validation is failed prompt the
53	<code>if</code>	user
	<code>(move_uploaded_file(\$_FILES["fileToUpload"]["tmp_name"],</code>	53) moving the submission to its new
	<code>\$target_file)) {</code>	file path
54	<code>echo "The file ". htmlspecialchars(basename(</code>	54) prompting the user their profile
	<code>\$_FILES["fileToUpload"]["name"])). " has been</code>	photo has been uploaded
	<code>uploaded.";</code>	
55	<code>header("Location: calendar.php");</code>	55) redirecting the user back to
56	<code>die;</code>	calendar.php
57	<code>} else {</code>	
58	<code>echo "Sorry, there was an error uploading your file.";</code>	57-59) if the submission failed prompt
59	<code>die;</code>	the user and kill the program
60	<code>}</code>	
61	<code>}</code>	
62	<code>?></code>	

requests\post_request_scanning.php

```
1 <?php
2
3 function post_request($key){
4 if(isset($_POST[$key])){
5 $requested = $_POST[$key];
6 $pattern =
7 "/(UNION.*SELECT|SELECT.*FROM|DROP.*TABLE|INSERT.*INTO|--
8 |\#|\\\/\*|\\*\\/|\\bOR\\b|\\bAND\\b|\\bXOR\\b|\\bLIKE\\b|\\bBETWEEN\\b)/i";
9 if(preg_match($pattern, $requested)){
10 die("search failed");
11 }
12 }else{
13 $requested = null;
14 }
15 return $requested;
16 }
```

3) defining the post_request function
4) checking the submission is not null
5) retrieving the post request
6) defining the regular expression to check the submission
7) matching the regular expression the value of the submission
8) if they match kill the program with the message search failed
10-12) if the post submission failed set the return value to null
13) return the requested value since it is not malicious

requests\get_request_scanning.php

```
1 <?php
2 function get_request($key){
3 if(isset($_GET[$key])){
4 $requested = $_GET[$key];
5 $pattern =
6 "/(UNION.*SELECT|SELECT.*FROM|DROP.*TABLE|INSERT.*INTO|--
7 |\#|\\\/\*|\\*\\/|\\bOR\\b|\\bAND\\b|\\bXOR\\b|\\bLIKE\\b|\\bBETWEEN\\b)/i";
8 if(preg_match($pattern, $requested)){
9 die("search failed");
10 }
11 }else{
12 $requested = null;
13 }
14 return $requested;
15 }
```

2) defining the get_request function
3) checking the submission is not null
4) retrieving the get request
5) defining the regular expression to check the submission
6) matching the regular expression the value of the submission
7) if they match kill the program with the message search failed
9-11) if the post submission failed set the return value to null
12) return the requested value since it is not malicious

logout.php <pre> 1 <?php 2 3 session_start(); 4 5 if(isset(\$_SESSION['user_id'])) 6 { 7 unset(\$_SESSION['user_id']); 8 unset(\$_SESSION['password']); 9 } 10 11 header("Location: login.php"); 12 die;</pre>	3) initialising the session data 5) checking the session has the user_id attribute 7-8) removing the user_id and password attributes from the session information 11) redirecting the user to the login.php page
---	--

backup_mysql.sh <pre> 1 USER="artermes27" 2 PASSWORD="Ubuntu2025" 3 Database="main_v2" 4 5 BACKUP_DIR="/home/artermes27/backups" 6 DATE=\$(date +%F) 7 BACKUP_FILE="\${BACKUP_DIR}/\${DATABASE}_\${DATE}.sql" 8 9 mkdir -p \$BACKUP_DIR 10 11 mysqldump -u\$USER -p\$PASSWORD > \$BACKUP_FILE 12 13 find \$BACKUP_DIR -type f -name "*.sql" -mtime +7 - exec rm{} \; 14</pre>	1-3) defining the database login information 5) defining the backup directory 6) retrieving the current date 7) defining the path of the backup directory 9) creating the backup directory if it doesn't already exist 11) logging in to the database, backing it up then putting the output of the backup into the backup file 13) removing the backup from 7 days ago since the latest backup was successful. exec is used because there are no more commands to run after the removal. Hence the current shell script process is killed and is replaced by the removal process decreasing overall CPU time of the backup.
--	---

includes\connection.php

1	<?php	
2		
3	<code>\$dbhost = "localhost";</code>	Storing the hostname of the MySQL server
4	<code>\$dbuser = "apache_user";</code>	Storing the username of the MySQL user
5	<code>\$dbpass = "Cadet2025";</code>	Storing the password of the MySQL user
6	<code>\$dbname = "main_v2";</code>	Stating the database for the MySQL connection
7		
8	<code>if(!\$con = mysqli_connect(\$dbhost,\$dbuser,\$dbpass,\$dbname)){</code>	If opening the MySQL connection fails
9	<code>die("failed to connect!");</code>	Kill the current process with the error message failed to connect
10	<code>}</code>	

includes\functions.php

1	<?php	
2		
3	<code>function check_login(\$con){</code>	Declaring the function check_login to verify the client has logged in and taking the MySQL connection as a parameter
4	<code>if(isset(\$_SESSION['user_id']) && isset(\$_SESSION['password'])){</code>	Checking that the user_id and password are attached to the clients session information
5	<code>\$id = \$_SESSION['user_id'];</code>	Storing the user_id
6	<code>\$password = \$_SESSION['password'];</code>	Storing the password in hashed form
7	<code>\$query = "SELECT * FROM users Where `user_id` = '\$id' AND `password` = '\$password' limit 1;";</code>	Query to select a record from the users table where the user_id of the record is known and checking the hashed forms of the passwords match
8	<code>\$result = mysqli_query(\$con,\$query);</code>	
9	<code>if(\$result && mysqli_num_rows(\$result) > 0){</code>	Checking that there were results
10	<code>\$user_data = mysqli_fetch_assoc(\$result);</code>	Storing the results in a dictionary format to return to the program that called the function
11	<code>if (isset(\$_GET["parade_id"])){</code>	Checking if a parade_id was sent by a get request
12	<code>\$user_data["parade_id"] = \$_GET["parade_id"];</code>	If a parade_id was sent append it to the \$user_data dictionary
13	<code>}if (isset(\$_GET["event_id"])){</code>	Checking if an event_id was sent by a get request
14	<code>\$user_data["event_id"] = \$_GET["event_id"];</code>	If an event_id was sent append it to the \$user_data dictionary

15	}	
16	return <code>\$user_data</code> ;	The client has passed the login check so return the <code>\$user_data</code> dictionary
17	}	
18	}	
19	header ("Location: logout.php");	The client has failed the login check so redirect them to logout.php so their session information will be removed if they have any before going to the login page
20	die ;	And kill the current process
21		
22	}	
23	<code>?></code>	

includes\nav.php

1	<code><?php</code>	
2	include_once ("includes/connection.php");	Including the MySQL connection
3		
4	<code>\$user_data = check_login(\$con);</code>	Storing the <code>\$user_data</code> dictionary. The <code>check_login()</code> function is already included since functions.php is included in all files that call nav.php hence nav.php has access to these inclusions.
5		
6	function <code>html_to_display_profile_photo(\$user_id){</code>	Function to produce html for displaying the profile photo with html code taking the <code>user_id</code> as a parameter to find the profile photo
7	<code>\$path = "profile-photos/" . \$user_id;</code>	Initialising the file path variable but not including the file format since this is unknown
8	if (file_exists ("profile-photos/" . <code>\$user_id</code> . ".jpg") == TRUE){	Checking if the profile photo is stored as a jpg file type
9	<code>\$path = "profile-photos/" . \$user_id . ".jpg";</code>	If it is then append a .jpg file type to the file path for the profile photo
10	}	
11	elseif (file_exists ("profile-photos/" . <code>\$user_id</code> . ".jpeg") == TRUE){	If the file type is not jpg, then check if it is jpeg
12	<code>\$path = "profile-photos/" . \$user_id . ".jpeg";</code>	If it is jpeg then append jpeg to the file path for the profile photo
13	}	

14	<code>elseif (file_exists("profile-photos/" . \$user_id . ".png") == TRUE){</code>	If the file type is not jpg or jpeg then check if it is a png
15	<code>\$path = "profile-photos/" . \$user_id . ".png";</code>	If it is png then append .png to the file path for the profile photo
16	<code>}</code>	
17	<code>else{</code>	If there was no profile photo for the user
18	<code>\$path = "profile-photos/default.png";</code>	Set the file path for the profile photo to the default profile photo
19	<code>}</code>	
20	<code>return "";</code>	Return an img html element that will serve as the hover button to activate a dropdown box for the user. Set the location of the profile photo file path to the one decided by the earlier logic
21	<code>}</code>	
22	<code>?></code>	
23		
24	<code><link rel="stylesheet" href="css/nav-style.css"></code>	Including the navigation style sheet
25	<code><nav></code>	
26	<code><div class="left"></code>	
27	<code>my calendar</code>	Highlightable button to take the user to the calendar page
28	<code><?php</code>	
29	<code>if(\$user_data["admin"] == "1") {</code>	If the user is an admin
30	<code>echo("add parade/event/acount/equipment");</code>	Highlightable button to take the admin to the add.php page
31	<code>}?></code>	
32	<code></div></code>	
33	<code><div class="right"></code>	
34	<code><div class="dropdown-container"></code>	
35	<code><?php echo(html_to_display_profile_photo(\$user_data["user_id"]));?></code>	Printing the html for the profile photo to the page
36	<code><div class="dropdown-content"></code>	
37	<code><a>user: <?php echo(\$user_data["rank"] . " " . \$user_data["first_name"] . " " . \$user_data["last_name"]);?></code>	Displaying the users full name
38	<code><a>email: <?php echo(\$user_data["email"]);?></code>	Displaying the users email
39	<code><a>dob: <?php echo(\$user_data["DOB"]);?></code>	Displaying the users date of birth
40	<code><a>gender: <?php echo(\$user_data["gender"]);?></code>	Displaying the users gender
41	<code><form action="uploads.php" method="post" enctype="multipart/form-data"></code>	Opening a form that posts to uploads.php for changing the profile photo
42	<code><input type="submit" name="submit">change profile photo</input></code>	

18	<pre> \$return_html = "<div class=\"\" . \$style_class . "\"><a>\" . \$event["event_start"] . " till " . \$event["event_end"] . "
\n\" . \$event["event_name"] . "
\n<a>\" . \$owner["rank"] . " " . \$owner["first_name"] . " " . \$owner["last_name"] . "
\n<button onclick=\"populateAdminEditEventForm('\" . \$event["parade_id"] . '\", '\" . \$event["event_id"] . '\", '\" . \$event["event_type"] . '\", '\" . \$event["event_name"] . '\", '\" . \$event["event_start"] . '\", '\" . \$event["event_end"] . '\", '\" . \$event["owner"] . '\", '\" . \$event["final_aproval"] . '\")\">click for admin panel edit</button></div>\n"; </pre>	Define the style class of the event to indicate its approval status (red, amber or green). Display the event start time, end time, name, full name of the owner Create the link to the events.php page for this event. Create a button that when clicked will populated the admin edit form on the left of the page and input all relevant information into this JavaScript function call for the event. The title of this button will be click for admin panel edit.
19	<pre> }elseif(\$event_display_type == "admin_event"){//if the user is an admin and they are on the event page, the admin panel edit button is not needed </pre>	If the format to display the event is for the admin on the event page then
20	<pre> \$query = "SELECT `first_name`, `last_name`, `rank` FROM users WHERE user_id = \" . \$event["owner"] . "\";"; </pre>	
21	<pre> \$result = mysqli_query(\$con, \$query); </pre>	
22	<pre> \$owner = mysqli_fetch_assoc(\$result); </pre>	
23	<pre> \$return_html = "<div class=\"\" . \$style_class . "\"><a>\" . \$event["event_start"] . " till " . \$event["event_end"] . "
\n\" . \$event["event_name"] . "
\n<a>\" . \$owner["rank"] . " " . \$owner["first_name"] . " " . \$owner["last_name"] . "
\n</div>\n"; </pre>	Define the style class of the event to indicate its approval status (red, amber or green). Display the event start time, end time, name, full name of the owner Create the link to the events.php page for this event so the admin can navigate to the event pages for many events on the parade
24	<pre> }elseif(\$event_display_type == "G4"){//if in future events were to be displayed differently they can be easily changed here </pre>	If the user is G4 then
25	<pre> \$return_html = "<div class=\"\" . \$style_class . "\"><a>\" . \$event["event_start"] . " till " . \$event["event_end"] . "
\n\" . \$event["event_name"] . "</div>\n"; </pre>	Define the style class of the event to indicate its approval status (red, amber or green). Display the event start time, end time, name Create the link to the events.php page for this event so the user can navigate to the event pages for many events on the parade
26	<pre> }elseif(\$event_display_type == "standard"){ </pre>	If the display type is a standard user
27	<pre> if (\$event["owner"] == \$user_id){//event owners must have the event.php link and the aproval status displayed </pre>	If the user is the owner of the event

28	<pre> \$return_html = "<div class=\"\" . \$style_class . "\"><a>\" . \$event["event_start"] . " till " . \$event["event_end"] . "
\n\" . \$event["event_name"] . "</div>\n"; </pre>	Define the style class of the event to indicate its approval status(red, amber or green). Display the event start time, end time, name Create the link to the events.php page for this event so the user can navigate to the event pages for many events on the parade
29	<pre> }elseif(\$event["duty"] == \$user_id){//if the user is not the owner and instead they are the duty cadet then duty event must be displayed as well as the lik to the event page </pre>	If the user is the duty cadet of the event and not the event owner
30	<pre> \$return_html = "<div class=\"\" . \$style_class . "\"><a>duty event
<a>\" . \$event["event_start"] . " till " . \$event["event_end"] . "
\n\" . \$event["event_name"] . "</div>\n"; </pre>	Define the style class of the event to indicate its approval status(red, amber or green). Display the event start time, end time, name and display the words duty event at the top. Create the link to the events.php page for this event so the user can navigate to the event pages for many events on the parade
31	<pre> }else{//the user is just on the register so only display times of the event and the name. do not indicate the aproval status of the event </pre>	If the user is just a standard user then
32	<pre> \$return_html = "<div class=\"event\"><a>\" . \$event["event_start"] . " till " . \$event["event_end"] . "
\n<a>\" . \$event["event_name"] . "</div>\n"; </pre>	Display the event as a standard event listing its start time, finish time and name. use the style class of event so the events approval status is not indicated
33	<pre> } </pre>	
34	<pre> } </pre>	
35	<pre> return \$return_html; </pre>	Return the html for the event
36	<pre> } </pre>	
37		
38	<pre> function html_for_equipment_r- equest_log(\$equipment_reuquest_log){//function to display and equipment request log given an array of equipment requests </pre>	Function to display the equipment request log at the bottom of a parade for G4 users
39	<pre> \$log = "<div class=\"event\">\n"; </pre>	Opening a div to display the log and creating a table with its headings
40	<pre> \$log .= "<h3 style=\"text-decoration: underline;\">log of aproved equipment requests</h3>"; </pre>	
41	<pre> \$log .= "<table>"; </pre>	
42	<pre> \$log .= "<tr>\n"; </pre>	
43	<pre> \$log .= "<th>name</th>"; </pre>	
44	<pre> \$log .= "<th>location</th>"; </pre>	
45	<pre> \$log .= "<th>start time</th>"; </pre>	

46	<code>\$log .= "<th>finish time</th>";</code>	
47	<code>\$log .= "<th>lesson name</th>";</code>	
48	<code>\$log .= "</tr>";</code>	
49	<code>foreach (\$equipment_reuquest_log as \$request) {</code> //looping through each equipment request	Looping through the dictionary equipment request log and retrieving the values for each item which will be pasted into the table as a complete row
50	<code>\$log .= "<tr>";</code>	
51	<code>\$log .= "<td>" . \$request["name"] . "</td>";</code>	
52	<code>\$log .= "<td>" . \$request["location"] . "</td>";</code>	
53	<code>\$log .= "<td>" . \$request["event_start"] . "</td>";</code>	
54	<code>\$log .= "<td>" . \$request["event_end"] . "</td>";</code>	
55	<code>\$log .= "<td>" . \$request["event_name"] . "</td>";</code>	
56	<code>\$log .= "</tr>";</code>	
57	<code>}</code>	
58	<code>\$log .= "</table>";</code>	Closing the table
59	<code>\$log .= "</div>\n";</code>	
60	<code>return \$log;</code>	Returning the html
61	<code>}</code>	
62		
63	<code>//fuction to generate all the html for a parade with a known date</code>	
64	<code>function html_for_parade(\$con, \$parade_date, \$user_data, \$output_format){</code>	Function to generate all the html for a parade on either the calendar or event page
65	<code>//selecting the parade_id</code>	
66	<code>\$query = "SELECT parade_id, parade_name FROM parades WHERE date = '\$parade_date'";</code>	Query to select the relevant details of the parade from the database
67	<code>\$result = mysqli_query(\$con, \$query);</code>	
68	<code>\$parade = mysqli_fetch_assoc(\$result);</code>	
69	<code>//setting the display type of the user i.e. should they see an equipment request log or should they have the admin panel edit button</code>	
70	<code>if(\$user_data["admin"] == 1 and \$output_format == "calendar"){</code> //if the user is admin or G4 then display all events for the parade	If the user is an admin and the output format is for the calendar page
71	<code>\$query = "SELECT events.* FROM events WHERE parade_id = " . \$parade["parade_id"] . " ORDER BY events.event_start;";</code>	Query to select all events on the parade because the user is an admin
72	<code>\$event_display_type = "admin_calendar";</code>	Setting the event_display_type to "admin_calendar" for the function html_for_displaying_an_event() so it knows how events should be displayed

73	<pre>}elseif(\$user_data["admin"] == 1 and \$output_format == "event"){//if the user is admin or G4 then display all events for the parade</pre>	If the user is an admin and the output format is the event page.
74	<pre>\$query = "SELECT events.* FROM events WHERE parade_id = " . \$parade["parade_id"] . " ORDER BY events.event_start;";</pre>	Query to select all events on the parade because the user is an admin
75	<pre>\$event_display_type = "admin_event";</pre>	Setting the event_display_type to "admin_event" for the function html_for_displaying_an_event() so it knows how events should be displayed
76	<pre>}elseif(\$user_data["G4"] == 1){</pre>	If the user is G4 (the output for an event is the same on calendar and event pages) then
77	<pre>\$query = "SELECT events.* FROM events WHERE parade_id = " . \$parade["parade_id"] . " ORDER BY events.event_start;";</pre>	Query to select all events on the parade because the user is G4
78	<pre>\$event_display_type = "G4";</pre>	Setting the event_display_type to "G4" for the function html_for_displaying_an_event() so it knows how events should be displayed
79	<pre>}else{//if the user is not admin or G4 then they should only see events they are related to</pre>	
80	<pre>\$query = "SELECT DISTINCT events.* FROM events LEFT JOIN user_event ON events.event_id = user_event.event_id WHERE (parade_id = " . \$parade["parade_id"] . " AND user_event.user_id = " . \$user_data["user_id"] . ") OR ((events.duty = " . \$user_data["user_id"] . " OR events.owner = " . \$user_data["user_id"] . ") AND parade_id = " . \$parade["parade_id"] . ") ORDER BY events.event_start;";</pre>	if the user is not admin or G4 then they should only see events they are related to. Query to select events where the parade id is that of the parade which html is currently being generated. Only select events where the user is either duty cadet, owner or on the register and order by the start times of the event
81	<pre>\$event_display_type = "standard";</pre>	Setting the event_display_type to "G4" for the function html_for_displaying_an_event() so it knows how events should be displayed i.e. assume the user is only on the register unless they show up as the owner or duty cadet.
82	<pre>}</pre>	
83	<pre>\$result = mysqli_query(\$con, \$query);</pre>	Executing the query decided by the above logic
84	<pre>\$events = mysqli_fetch_all(\$result, MYSQLI_ASSOC);</pre>	Putting the results into a dictionary format
85	<pre>if(\$output_format == "calendar"){//the headings of the parade are different formats for the calendar and event pages respectively</pre>	If the output is for the calendar page then
86	<pre>\$parade_html = "<div class=\"event\"><h2>" . \$parade_date . "</h2><h2>" . \$parade["parade_name"] . "</h2>";</pre>	Display the headings as h2 and put them inside an event div.
87	<pre>\$parade_html .= "</div>\n";</pre>	

88	<code>}if(\$output_format == "event"){</code>	If the output is for the event page then
89	<code>\$parade_html = "<h1>" . \$parade_date . "</h1><h1>" . \$parade["parade_name"] . "</h1>";</code>	Display the headings as h1 and do not put any div around them
90	<code>}</code>	
91	<code>\$event_count = 0;</code>	Setting the counter for what events have been converted to html to 0
92	<code>if(count(\$events) == 0){</code>	If the number of events to display is 0
93	<code>\$parade_html .= "<div class=\"event\"><a>you have no events on this parade night</div>";</code>	Display a message making the user aware of this
94	<code>}else{</code>	If there are a non-zero number of events to display
95	<code>if (\$user_data["G4"] == 1 and \$output_format == "calendar"){//if the user is G4 then an equipment request log must be produced</code>	If the user is G4 and the output is intended for the calendar page
96	<code>\$equipment_reuquest_log = [];//initialising the equipment request log</code>	
97	<code>while(\$event_count < count(\$events)){//loop to produce the html for each event and append the equipment requests to the array of requests</code>	while loop to produce html for each event on the parade the G4 user should see
98	<code>\$parade_html .= html_for_displaying_an_event(\$con, \$events[\$event_count]["event_id"], \$event_display_type, \$user_data["user_id"]);</code>	Call the function to display the event in the desired html format and append the html for the event to the \$parade_html variable.
99	<code>//query to select approved equipment requests for an event which are appended to the array equipment_request_log</code>	
100	<code>\$query = "SELECT equipment.name, equipment.location, events.event_name, events.event_start, events.event_end, users.rank, users.first_name, users.last_name FROM equipment, events, users, equipment_requests WHERE events.event_id = " . \$events[\$event_count]["event_id"] . " AND users.user_id = events.owner AND equipment_requests.event_id = events.event_id AND equipment.equipment_id = equipment_requests.equipment_id AND equipment_requests.aproved = 1";</code>	Query to select the details of all approved equipment requests for the event.
101	<code>\$result = mysqli_query(\$con, \$query);</code>	Executing the query and storing its result
102	<code>while(\$request = mysqli_fetch_assoc(\$result)){//appending to the equipment request log</code>	While loop to append the equipment requests for the event to a master log of equipment requests
103	<code>\$equipment_reuquest_log[] = \$request;</code>	Appending the equipment request currently being handled by the while loop
104	<code>}</code>	
105	<code>\$event_count = \$event_count + 1;</code>	Incrementing the event counter so the next iteration will handle the next event in the parade

106	}	All events have been put into an html format and the master log of equipment requests has been populated
107	if (count (\$equipment_reuquest_log) > 0){//outputing an equipment request log if their are requests	If the master log of equipment requests is not empty i.e. approved equipment requests have been made
108	\$parade_html .= html_for_equipment_request_log (\$equipment_reuquest_log);	Calling the function to convert the master log of equipment requests into html and appending the html to the html for the whole parade
109	}	
110	}else{	If the user is not G4 and on the calendar page i.e. no equipment requests log needs to be produced
111	while (\$event_count < count (\$events)){//loop to produce the html for each event	while loop to produce html for each event on the parade the user should see
112	\$parade_html .= html_for_displaying_an_event (\$con, \$events[\$event_count]["event_id"], \$event_display_type, \$user_data["user_id"]);	Call the function to display the event in the desired html format and append the html for the event to the \$parade_html variable.
113	\$event_count = \$event_count + 1;	Incrementing the event counter so the next iteration will handle the next event in the parade
114	}	
115	}	
116	}	
117	return \$parade_html ;	Return the html for displaying the parade
118	}	
119	?>	

calendar.php

1	<?php	
2	session_start ();	
3		
4	include_once ("includes/connection.php");	
5	include ("includes/functions.php");	
6		
7	\$user_data = check_login (\$con);	initialising the \$user_data dictionary
8		
9	//function to get the dates the skip forwards/backwards buttons should redirect the user to	

10	function date_skip_method (\$con, \$current_date, \$skip){	function to retrieve dates to skip to for the move forwards/backwards on the calendar if a date is not found then the error code null is returned.
11	if (\$skip == "-1"){	skip by -1 will retrieve the parade before \$current_date
12	\$query = "SELECT date FROM parades WHERE date < '\$current_date' ORDER BY date DESC limit 1;";	selecting dates from parades that are smaller than \$current_date ordering by descending and setting a limit so the maximum date is retrieved
13	\$result = mysqli_query (\$con, \$query);	
14	\$dates = mysqli_fetch_all (\$result, MYSQLI_ASSOC);	putting the results into an associative array
15	if (count (\$dates) == 0){	If there are no dates found return the null error code
16	return "null";	
17	} else {	Otherwise return the only date in the results of the query
18	return \$dates[0]["date"];	
19	}	
20	}if (\$skip == "+1"){	date skip forward by one method
21	\$query = "SELECT date FROM parades WHERE date > '\$current_date' ORDER BY date ASC limit 1;";	query to select date from parades where date is greater than the \$current_date order by ascending and limit 1 to select the date 1 greater than the \$current_date
22	\$result = mysqli_query (\$con, \$query);	
23	\$dates = mysqli_fetch_all (\$result, MYSQLI_ASSOC);	
24	if (count (\$dates) == 0){	if no results returned return the null error code
25	return "null";	
26	} else {	Otherwise return the only date in the results of the query
27	return \$dates[0]["date"];	
28	}	
29	}if (\$skip == "-5"){	date skip back by five method.
30	\$query = "SELECT date FROM parades WHERE date < '\$current_date' ORDER BY date DESC limit 5;";	selecting dates from parades where the date is smaller than \$current_date order by date descending and limit 5 so the top result will be the wanted date
31	\$result = mysqli_query (\$con, \$query);	
32	\$dates = mysqli_fetch_all (\$result, MYSQLI_ASSOC);	
33	if (mysqli_num_rows (\$result) == 0){	if no results returned return the null error code
34	return "null";	
35	} else {	If results are returned, then

36	<code>if(count(\$dates) >= 5){</code>	If the number of results is equal to 5 then
37	<code>return \$dates[4]["date"];</code>	Return the fifth date in the array
38	<code>}else{</code>	If the number of results is less than 5 (these 2 cases are distinct because in future the system may tell the user that there are less than 5 dates in the future before the click the skip button)
39	<code>return \$dates[count(\$dates) - 1]["date"];</code>	Otherwise return the smallest date in the array
40	<code>}</code>	
41	<code>}</code>	
42	<code>}if(\$skip == "+5"){</code>	date skip forward by 5
43	<code>\$query = "SELECT date FROM parades WHERE date > '\$current_date' ORDER BY date ASC limit 5;";</code>	selecting date from parades where the date is greater than \$current_date order by ascending and limit 5 to have the fifth result the fifth date in the future
44	<code>\$result = mysqli_query(\$con, \$query);</code>	
45	<code>\$dates = mysqli_fetch_all(\$result, MYSQLI_ASSOC);</code>	
46	<code>if(mysqli_num_rows(\$result) == 0){</code>	if no results returned return the null error code
47	<code>return "null";</code>	
48	<code>} else {</code>	Otherwise return the date further in the future
49	<code>return \$dates[count(\$dates)-1]["date"];</code>	
50	<code>}</code>	
51	<code>}if(\$skip == "-4"){</code>	works similar to the skip forward and back by 5 but this does it by four instead. this is used by the admin to allow space for the admin panel edit column which takes up a column in the calendar hence they only see 4 parades instead of 5
52	<code>\$query = "SELECT date FROM parades WHERE date < '\$current_date' ORDER BY date DESC limit 4;";</code>	
53	<code>\$result = mysqli_query(\$con, \$query);</code>	
54	<code>\$dates = mysqli_fetch_all(\$result, MYSQLI_ASSOC);</code>	
55	<code>if(mysqli_num_rows(\$result) == 0){</code>	
56	<code>return "null";</code>	
57	<code>} else {</code>	
58	<code>if(count(\$dates) >= 4){</code>	
59	<code>return \$dates[3]["date"];</code>	
60	<code>}else{</code>	
61	<code>return \$dates[count(\$dates) - 1]["date"];</code>	
62	<code>}</code>	
63	<code>}</code>	

64	<code>}if(\$skip == "+4"){</code>	works similar to the skip forward and back by 5 but this does it by four instead. this is used by the admin to allow space for the admin panel edit collum which takes up a column in the calendar hence they only see 4 parades instead of 5
65	<code>\$query = "SELECT date FROM parades WHERE date > '\$current_date' ORDER BY date ASC limit 4;";</code>	
66	<code>\$result = mysqli_query(\$con, \$query);</code>	
67	<code>\$dates = mysqli_fetch_all(\$result, MYSQLI_ASSOC);</code>	
68	<code>if(mysqli_num_rows(\$result) == 0){</code>	
69	<code>return "null";</code>	
70	<code>} else {</code>	
71	<code>return \$dates[count(\$dates)-1]["date"];</code>	
72	<code>}</code>	
73	<code>}</code>	
74	<code>}</code>	
75		
76	<code>//function to return an array of 5 dates and the final valid date in the array.</code>	
77	<code>function get_parade_date_range(\$con, \$current_date, \$admin) {</code>	function to return an array of 5 dates and the final valid date in the array
78	<code>\$query = "SELECT date FROM parades WHERE date >= '\$current_date' ORDER BY date ASC limit 5;";</code>	
79	<code>\$result = mysqli_query(\$con, \$query);</code>	
80	<code>\$dates = mysqli_fetch_all(\$result, MYSQLI_ASSOC);</code>	
81	<code>if (count(\$dates) < 5) { //if their are less than 5 dates then the array is filled with an error message</code>	
82	<code>\$end_date = \$dates[count(\$dates)-1]["date"];</code>	
83	<code>while(count(\$dates) < 5) {</code>	
84	<code>\$dates[] = ["date" => "null"];</code>	
85	<code>}</code>	
86	<code>} else { //the admin will have one less date displayed since they have the admin edit panel displayed</code>	
87	<code>if(\$admin == 1){</code>	
88	<code>\$end_date = \$dates[count(\$dates)-2]["date"];</code>	
89	<code>}else{</code>	
90	<code>\$end_date = \$dates[count(\$dates)-1]["date"];</code>	
91	<code>}</code>	
92	<code>}</code>	
93	<code>return [\$dates, \$end_date];</code>	
94	<code>}</code>	
95		

96	<code>function html_for_admin_page_on_callandar(){//function to output the html needed for the admin panel edit form</code>	
97	<code>\$html = "<div class=\"event\">";</code>	
98	<code>\$html .= "<h2>admin panel</h2>\n";</code>	
99	<code>\$html .= "<link rel=\"stylesheet\" href=\"css/event-owner- form-style.css\">\n";</code>	
100	<code>\$html .= "<form class=\"modify_lesson_details\" action=\"requests/event_details_requests.php\" method=\"POST\">\n";</code>	
101	<code>\$html .= "<input hidden value=\"modify_event_details\" type=\"text\" name=\"flag\" id=\"flag\">\n";</code>	flagging the request came from the calendar to redirect the user correctly once the form is submitted
102	<code>\$html .= "<input hidden value=\"1\" type=\"text\" name=\"calendar_flag\" id=\"calendar_flag\">\n";</code>	Flag to identify the form came from the calendar page so the user is correctly redirected
103	<code>\$html .= "<input hidden value=\"\" type=\"text\" name=\"parade_id\" id=\"parade_id\">\n";</code>	hidden information populated by the JS function that the user can't edit
104	<code>\$html .= "<input hidden value=\"\" type=\"text\" name=\"event_id\" id=\"event_id\">\n";</code>	
105	<code>\$html .= "<input hidden value=\"\" type=\"text\" name=\"owner_id\" id=\"owner_id\">\n";</code>	
106	<code>\$html .= "<input hidden value=\"\" type=\"text\" name=\"original_aproval\" id=\"original_aproval\">\n";</code>	
107	<code>\$html .= "<label>event type</label>\n";</code>	input elements with labels for the admin to edit when the form is populated all of which have validation
108	<code>\$html .= "<input type=\"text\" name=\"event_type\" id=\"event_type\" onkeyup=\"REGEXCheckEvent(this.value, 'event_type')\">\n";</code>	
109	<code>\$html .= "<label>event name</label>\n";</code>	
110	<code>\$html .= "<input type=\"text\" name=\"event_name\" id=\"event_name\" onkeyup=\"REGEXCheckEvent(this.value, 'event_name')\">\n";</code>	
111	<code>\$html .= "<label>event start</label>\n";</code>	
112	<code>\$html .= "<input type=\"time\" name=\"event_start\" id=\"event_start\" onkeyup=\"REGEXCheckEvent(this.value, 'event_start')\">\n";</code>	
113	<code>\$html .= "<label>event end</label>\n";</code>	
114	<code>\$html .= "<input type=\"time\" name=\"event_end\" id=\"event_end\" onkeyup=\"REGEXCheckEvent(this.value, 'event_end')\">\n";</code>	
115	<code>\$html .= "<label>approved</label>\n";</code>	
116	<code>\$html .= "<select id=\"final_aproval\" name=\"final_aproval\" onclick=\"REGEXCheckEvent(this.value, 'final_aproval')\">\n";</code>	

117	<code>\$html .= "<option value=\"0\">not-aproved</option>\n";</code>	
118	<code>\$html .= "<option value=\"1\">aproved</option>\n";</code>	
119	<code>\$html .= "<option value=\"2\">aproval requested</option>\n";</code>	
120	<code>\$html .= "</select>\n";</code>	
121	<code>\$html .= "<label>event owner</label>\n";</code>	
122	<code>\$html .= "<input type=\"text\" name=\"event_owner_search_box\" id=\"event_owner_search_box\" onkeyup=\"showResutsSearchForOwner(this.value)\">\n";</code>	input box for searching for a new event owner
123	<code>\$html .= "<div class=\"input_handeling\" id=\"livesearch_owner\"></div>\n";</code>	div for displaying the results of the search
124	<code>\$html .= "<div class=\"input_handeling\" id=\"event-input-handeling\"></div>\n";</code>	div for displaying any error handling prompts
125	<code>\$html .= "<div class=\"input_handeling\" id=\"display_current_owner\"></div>\n";</code>	div to display the full name of the event owner
126	<code>\$html .= "<button class=\"input_handeling\" id=\"add-event-submit\" disabled>submit</button>\n";</code>	
127	<code>\$html .= "</form>\n";</code>	
128	<code>\$html .= "<form class=\"modify_lesson_details\" action=\"requests/event_details_requests.php\" method=\"POST\">\n";</code>	hidden input to identify the form, event_id, parade_id and the event that is to be deleted
129	<code>\$html .= "<input hidden value=\"delete_event\" type=\"text\" name=\"flag\" id=\"flag\">\n";</code>	
130	<code>\$html .= "<input hidden value=\"\" type=\"text\" name=\"delete_parade_id\" id=\"delete_parade_id\">\n";</code>	
131	<code>\$html .= "<input hidden value=\"\" type=\"text\" name=\"delete_event_id\" id=\"delete_event_id\">\n";</code>	
132	<code>\$html .= "<button class=\"input_handeling\" id=\"delete-event-submit\" style=\"width: 100%;\" disabled>delete event</button>\n";</code>	Button to delete the event
133	<code>\$html .= "</form>\n";</code>	
134	<code>\$html .= "</div>\n";</code>	
135	<code>return \$html;</code>	returning the html for the admin panel edit forms'
136	<code>}</code>	
137		
138	<code>include("requests/get_request_scanning.php");</code>	Including the function get_request for retrieving the get request and scanning it
139	<code>if(isset(\$_GET["current_date"]) and get_request("current_date") != "null") {</code> <code>//retreiving the starting date for the calendar page</code>	If the current date variable exists and its value is not the null error code
140	<code>\$current_date = get_request("current_date");</code>	Set the current date to the one parametrised in the link to the webpage
141	<code>} else {</code> <code>//no starting date set so use todays date</code>	if the current_date was invalid or was not set then

142	<code>\$current_date = str_replace("/", "-", date("Y/m/d"));</code>	Set the current date to todays date
143	<code>}</code>	
144	<code>//getting the parade date range and the end date storing in variable temp</code>	
145	<code>\$temp = get_parade_date_range(\$con, \$current_date, \$user_data["admin"]);</code>	getting the parade date range and the end date storing in variable temp
146	<code>\$parade_dates = \$temp[0];</code>	Storing the array of parade dates
147	<code>\$end_date = \$temp[1];</code>	Storing the end date
148	<code>\$temp = null;</code>	Clearing the memory allocated for the temp variable
149		
150	<code>\$min_add = date_skip_method(\$con, \$current_date, "+1");</code>	Setting the min add/subtract variables which will define what dates the minor skip forward/backwards buttons will redirect the user to
151	<code>\$min_subtract = date_skip_method(\$con, \$current_date, "-1");</code>	
152	<code>if(\$user_data["admin"] == 0) {</code> <code>//if the user doesnt have the admin edit panel then they use all 5 slots on the calendar page so they skip forward/back by 5 parades</code>	If the user isn't an admin i.e. their page has 5 parade slots. Set the max skip forward/backwards buttons to 5 dates away
153	<code>\$max_add = date_skip_method(\$con, \$current_date, "+5");</code>	
154	<code>\$max_subtract = date_skip_method(\$con, \$current_date, "-5");</code>	
155	<code>}</code> <code>else {</code> <code>//admins have the admin panel edit so they only skip by 4 parades</code>	The user is an admin i.e. their page has 4 parade slots. Set the max skip forward/backwards buttons to 4 dates away
156	<code>\$max_add = date_skip_method(\$con, \$current_date, "+4");</code>	
157	<code>\$max_subtract = date_skip_method(\$con, \$current_date, "-4");</code>	
158	<code>}</code>	
159	<code>\$output_count = 0;</code>	Setting the counter for tracking which parade is to be printed to 0
160		
161	<code>include("includes/display_parade.php");</code>	Including the functions to produce the html for displaying a parade in the calendar page for all types of users
162	<code>?></code>	
163		
164	<code><!DOCTYPE html></code>	
165	<code><html></code>	
166	<code><head></code>	
167	<code><title>my Dashbord</title></code>	
168	<code><link rel="stylesheet" href="css/calendar-style.css"></code>	Including the general styling for the calendar
169	<code><link rel="stylesheet" href="css/event-display-block-style.css"></code>	Including the styling for the different event approval statuses

170	<script src="js/calendar.js"></script>	Including the JS for the general operation of the calendar page
171	<script src="js/event_owner_form_handeling.js"></script>	Including the functions for changing the owner of the event i.e. searching, clicking results and displaying the full name of the owner
172	</head>	
173	<body>	
174	<?php include("includes/nav.php");?>	Including the navigation bar
175	<div>	
176	<div class="banner">	
177	<input hidden value=<?php echo(\$current_date);?> type="date" name="current-date" id="current-date">	
178	<div class="prev">	
179	<button onclick="window.location.href='calendar.php?current_date=<?php echo(\$max_subtract)?>';"><<<</button>	Setting the skip backwards button dates
180	<button onclick="window.location.href='calendar.php?current_date=<?php echo(\$min_subtract)?>';"><</button>	
181	</div>	
182	<div class="display-dates-overview">	
183	<?php echo(\$current_date . "till " . \$end_date);?>	Displaying the date range the calendar will be displaying
184	</div>	
185	<div class="next">	
186	<button onclick="window.location.href='calendar.php?current_date=<?php echo(\$min_add)?>';">>>>/button>	Setting the skip forwards button dates
187	<button onclick="window.location.href='calendar.php?current_date=<?php echo(\$max_add)?>';">>>>>>/button>	
188	</div>	
189	</div>	
190	<div class="calendar">	
191	<div class="parade1">	
192	<?php	
193	if(\$user_data["admin"] == 1) {	If the user is an admin then
194	echo(html_for_admin_page_on_callandar());	Display the admin panel edit and do not increment the output count
195	}else {	
196	//echo(html_for_parade_on_callendar(\$con, \$parade_dates[\$output_count]["date"], \$user_data));	

197	<code>echo(html_for_parade(\$con, \$parade_dates[\$output_count]["date"], \$user_data, "calendar"));</code>	Calling the function for printing the html for the parade and setting the display type to calendar so the headings for the parade are properly displayed. Then printing the results of this function
198	<code>\$output_count = \$output_count + 1;</code>	Incrementing the output count so the next parade is printed
199	<code>}</code>	
200	<code>?></code>	
201	<code></div></code>	
202	<code><div class="parade2"></code>	
203	<code><?php</code>	
204	<code>if (\$parade_dates[\$output_count]["date"] == "null"){</code>	If the second parade is an invalid date then
205	<code>echo("<div class=\"event\"><h2>no parades beyond this point</h2></div>");</code>	Make the user aware of this with a message
206	<code>}else{</code>	If the parade is valid
207	<code>echo(html_for_parade(\$con, \$parade_dates[\$output_count]["date"], \$user_data, "calendar"));</code>	Print the html for the page
208	<code>}</code>	
209	<code>\$output_count = \$output_count + 1;</code>	Increment the output counter
210	<code>?></code>	
211	<code></div></code>	
212	<code><div class="parade3"></code>	
213	<code><?php</code>	Works identically to parade 2
214	<code>if (\$parade_dates[\$output_count]["date"] == "null"){</code>	
215	<code>echo("<div class=\"event\"><h2>no parades beyond this point</h2></div>");</code>	
216	<code>}else{</code>	
217	<code>echo(html_for_parade(\$con, \$parade_dates[\$output_count]["date"], \$user_data, "calendar"));</code>	
218	<code>}</code>	
219	<code>\$output_count = \$output_count + 1;</code>	
220	<code>?></code>	
221	<code></div></code>	
222	<code><div class="parade4"></code>	
223	<code><?php</code>	Works identically to parade 2
224	<code>if (\$parade_dates[\$output_count]["date"] == "null"){</code>	
225	<code>echo("<div class=\"event\"><h2>no parades beyond this point</h2></div>");</code>	
226	<code>}else{</code>	

227	<code>echo(html_for_parade(\$con, \$parade_dates[\$output_count]["date"], \$user_data, "calendar"));</code>	
228	<code>}</code>	
229	<code>\$output_count = \$output_count + 1;</code>	
230	<code>?></code>	
231	<code></div></code>	
232	<code><div class="parade5"></code>	
233	<code><?php</code>	Works identically to parade 2
234	<code>if (\$parade_dates[\$output_count]["date"] == "null"){</code>	
235	<code>echo("<div class=\"event\"><h2>no parades beyond this point</h2></div>");</code>	
236	<code>}else{</code>	
237	<code>echo(html_for_parade(\$con, \$parade_dates[\$output_count]["date"], \$user_data, "calendar"));</code>	
238	<code>}</code>	
239	<code>\$output_count = \$output_count + 1;</code>	
240	<code>?></code>	
241	<code></div></code>	
242	<code></div></code>	
243	<code></div></code>	
244	<code></body></code>	
245	<code></html></code>	
246	<code><?php mysqli_close(\$con)?></code>	Closing the MySQL connection

event.php

1	<code><?php</code>	
2	<code>session_start();</code>	Initialising the session data
3		
4	<code>include_once("includes/connection.php");</code>	Including the MySQL connection \$con
5	<code>include("includes/functions.php");</code>	Including the function check_login
6		
7	<code>\$user_data = check_login(\$con);</code>	Initialising the \$user_data dictionary
8	<code>\$query = "SELECT `owner`, `duty`, `event_name`, `final_aproval`, `date` FROM events, parades WHERE event_id = " . \$user_data["event_id"] . " AND events.parade_id = parades.parade_id;";</code>	Query to select the user_id of the owner and duty cadet also selecting the name of the event its approval status and the date it is happening. Joining the tables parades and events to retrieve the date the event is happening
9	<code>\$result = mysqli_query(\$con, \$query);</code>	
10	<code>\$event = mysqli_fetch_assoc(\$result);</code>	Storing the results of the query in a dictionary called event

11	//checking the user has permission to be on the event page	
12	<code>if(!(\$user_data["admin"] == 1 or \$user_data["G4"] == 1 or \$user_data["user_id"] == \$event["owner"] or \$user_data["user_id"] == \$event["duty"])){</code>	If the client is not either (G4, admin, event owner or duty cadet) i.e. they don't have permission to visit the page
13	<code>header("location: calendar.php");</code>	Redirect them to the calendar page
14	<code>}</code>	
15		
16	<code>function generate_html_for_lesson_plan(\$con, \$event_id, \$admin){</code>	Function to generate the html for the lesson plan
17	<code>\$query = "SELECT `parade_id`, `event_type`, `event_name`, `event_start`, `event_end`, `owner`, `final_aproval`, `duty` FROM events WHERE event_id = \$event_id;";</code>	Query to select all the details of the event that are modifiable/ are required to identify the event
18	<code>\$result = mysqli_query(\$con, \$query);</code>	
19	<code>\$event = mysqli_fetch_assoc(\$result);</code>	Storing the details of the event in a dictionary called event
20	<code>\$query = "SELECT `rank`, `first_name`, `last_name` FROM users WHERE user_id = " . \$event["owner"] . ";";</code>	Query to select the full name of the event owner
21	<code>\$result = mysqli_query(\$con, \$query);</code>	
22	<code>\$owner_info_dump = mysqli_fetch_assoc(\$result);</code>	Storing the details of the event owner in a dictionary
23	<code>\$query = "SELECT `rank`, `first_name`, `last_name` FROM users WHERE user_id = " . \$event["duty"] . ";";</code>	Query to select the full name of the duty cadet
24	<code>\$result = mysqli_query(\$con, \$query);</code>	
25	<code>\$duty_info_dump = mysqli_fetch_assoc(\$result);</code>	Storing the details of the duty cadet in a dictionary
26	<code>\$lesson_plan_html = "";</code>	
27	<code>\$lesson_plan_html .= "<h4>modify event details</h4>\n";</code>	
28	<code>\$lesson_plan_html .= "<link rel=\"stylesheet\" href=\"css/event-owner-form-style.css\">\n";</code>	
29	<code>\$lesson_plan_html .= "<form class=\"modify_lesson_details\" id=\"modify_event_details\" action=\"requests/event_details_requests.php\" method=\"POST\">\n";</code>	Creating the form for modifying the details of the event
30	<code>\$lesson_plan_html .= "<input hidden value=\"modify_event_details\" type=\"text\" name=\"flag\" id=\"flag\">\n";</code>	Hidden field to identify the form
31	<code>\$lesson_plan_html .= "<input hidden value=\"0\" type=\"text\" name=\"calendar_flag\" id=\"calendar_flag\">\n";</code>	Hidden field to identify where the user should be redirected to when the event is submitted
32	<code>\$lesson_plan_html .= "<input hidden value=\"\" . \$event[\"owner\"] . \"\" type=\"text\" name=\"user_id\" id=\"user_id\">\n"; \$lesson_plan_html .= "<input hidden value=\"\" . \$event_id . \"\" type=\"text\" name=\"event_id\" id=\"event_id\">\n";</code>	

33	<pre>\$lesson_plan_html .= "<input hidden value=\"\" . \$event["owner"] . "\" type=\"text\" name=\"owner_id\" id=\"owner_id\">\n";</pre>	
34	<pre>\$lesson_plan_html .= "<input hidden value=\"\" . \$event["parade_id"] . "\" type=\"text\" name=\"parade_id\" id=\"parade_id\">\n";</pre>	
35	<pre>\$lesson_plan_html .= "<input hidden value=\"\" . \$event["duty"] . "\" type=\"text\" name=\"duty_id\" id=\"duty_id\">\n";</pre>	
36	<pre>\$lesson_plan_html .= "<input hidden value=\"\" . \$event["final_aproval"] . "\" type=\"text\" name=\"original_aproval\" id=\"original_aproval\">\n";</pre>	
37	<pre>\$lesson_plan_html .= "<label>event type</label>\n";</pre>	Non-hidden input fields that will allow the user to modify the details of the event
38	<pre>\$lesson_plan_html .= "<input value=\"\" . \$event["event_type"] . "\" type=\"text\" name=\"event_type\" id=\"event_type\" onkeyup=\"REGEXCheckEvent(this.value, 'event_type', '\" . \$admin . '\")\">\n";</pre>	All showing input fields will call their validation functions when clicked. The parameter admin will be passed in to the validation function to ensure only the admin is able to modify the details of the event and the form is invalid if a standard user modifies the approval status of the event from this form.
39	<pre>\$lesson_plan_html .= "<label>event name</label>\n";</pre>	
40	<pre>\$lesson_plan_html .= "<input value=\"\" . \$event["event_name"] . "\" type=\"text\" name=\"event_name\" id=\"event_name\" onkeyup=\"REGEXCheckEvent(this.value, 'event_name', '\" . \$admin . '\")\">\n";</pre>	
41	<pre>\$lesson_plan_html .= "<label>event start</label>\n";</pre>	
42	<pre>\$lesson_plan_html .= "<input value=\"\" . \$event["event_start"] . "\" type=\"time\" name=\"event_start\" id=\"event_start\" onkeyup=\"REGEXCheckEvent(this.value, 'event_start', '\" . \$admin . '\")\">\n";</pre>	
43	<pre>\$lesson_plan_html .= "<label>event end</label>\n";</pre>	
44	<pre>\$lesson_plan_html .= "<input value=\"\" . \$event["event_end"] . "\" type=\"time\" name=\"event_end\" id=\"event_end\" onkeyup=\"REGEXCheckEvent(this.value, 'event_end', '\" . \$admin . '\")\">\n";</pre>	
45	<pre>\$lesson_plan_html .= "<label>final aproval</label>\n";</pre>	
46	<pre>\$lesson_plan_html .= "<select value=\"\" . \$event["final_aproval"] . "\" id=\"final_aproval\" name=\"final_aproval\" onclick=\"REGEXCheckEvent(this.value, 'final_aproval', '\" . \$admin . '\")\">\n";</pre>	
47	<pre>\$lesson_plan_html .= "<option value=\"0\">not- approved</option>\n";</pre>	

48	<code>\$lesson_plan_html .= "<option value=\"1\">approved</option>\n";</code>	
49	<code>\$lesson_plan_html .= "<option value=\"2\">approval requested</option>\n";</code>	
50	<code>\$lesson_plan_html .= "</select>\n";</code>	
51	<code>\$lesson_plan_html .= "<label>search for new event owner:</label>\n";</code>	
52	<code>\$lesson_plan_html .= "<input type=\"text\" name=\"event_owner_search_box\" id=\"event_owner_search_b- ox\" onkeyup=\"showResultsSearchForOwner(this.value)\">\n";</code>	Search for new owner box
53	<code>\$lesson_plan_html .= "<div class=\"livesearch\" id=\"livesearch_owner\"></div>\n";</code>	Div to display results of a search for a new owner
54	<code>\$lesson_plan_html .= "<div class=\"input_error_handeling\" id=\"event-input-handeling\"></div>\n";</code>	Div to display error handling prompts for the user when the form is incorrectly populated
55	<code>\$lesson_plan_html .= "<div class=\"display_current_owner\" id=\"display_current_owner\"><a>curent owner: " . \$owner_info_dump["rank"] . " " . \$owner_info_dump["first_name"] . " " . \$owner_info_dump["last_name"] . "</div>\n";</code>	Div to display the current event owner which will be populated with the owners full name when the page is first loaded
56	<code>\$lesson_plan_html .= "<div class=\"display_current_owner\" id=\"display_current_duty\"><a>duty cadet: " . \$duty_info_dump["rank"] . " " . \$duty_info_dump["first_name"] . " " . \$duty_info_dump["last_name"] . "</div>\n";</code>	Div to display the full name of the duty cadet of the event
57	<code>\$lesson_plan_html .= "<button class=\"submit_button\" id=\"add-event-submit\">update event details</button>\n";</code>	
58	<code>\$lesson_plan_html .= "</form>\n";</code>	
59	<code>\$lesson_plan_html .= "<form class=\"modify_lesson_details\" action=\"requests/event_details_requests.php\" method=\"POST\">\n";</code>	Form to delete the event
60	<code>\$lesson_plan_html .= "<input hidden value=\"delete_event\" type=\"text\" name=\"flag\" id=\"flag\">\n";</code>	Hidden flag to identify the form
61	<code>\$lesson_plan_html .= "<input hidden value=\"\" . \$event["parade_id"]. "\" type=\"text\" name=\"delete_parade_id\" id=\"delete_parade_id\">\n";</code>	Hidden details to identify the event
62	<code>\$lesson_plan_html .= "<input hidden value=\"\" . \$event_id . \"\" type=\"text\" name=\"delete_event_id\" id=\"delete_event_id\">\n";</code>	Hidden details to identify the event
63	<code>\$lesson_plan_html .= "<button class=\"input_handeling\" id=\"delete-event-submit\" style=\"width: 100%;\">delete event</button>\n";</code>	
64	<code>\$lesson_plan_html .= "</form>\n";</code>	
65	<code>return \$lesson_plan_html;</code>	Return all the html for the modify the event details form
66	<code>}</code>	

67		
68	function html_for_register(\$con, \$event_id, \$parade_id){	Function to generate the register for the event
69	\$query = "SELECT user_event.present, users.user_id, users.rank, users.first_name, users.last_name FROM user_event, users WHERE user_event.event_id = " . \$event_id . " and users.user_id = user_event.user_id";	Query to select all the details of the cadets who are on the register for the event
70	\$result = mysqli_query(\$con, \$query);	
71	\$output_html = "";	
72	if(mysqli_num_rows(\$result) > 0) {	If there are users on the register
73	\$output_html .= "<div class=\"register-main\"><table>\n";	Open the register
74	\$output_html .= "<form method=\"post\" action=\"requests/post_requests.php\">\n";	
75	\$output_html .= "<input hidden value=\"modify_register\" type=\"text\" name=\"flag\" id=\"flag\">\n";	
76	\$output_html .= "<input hidden value=\"\" . \$event_id . "\" type=\"text\" name=\"event_id\" id=\"event_id\">\n";	
77	\$output_html .= "<input hidden value=\"\" . \$parade_id . "\" type=\"text\" name=\"parade_id\" id=\"parade_id\">\n";	
78	while(\$cadet = mysqli_fetch_assoc(\$result)) {	While loop to loop through all the cadets on the register and store them as dictionaries in the variable \$cadet
79	\$output_html .= "<tr>\n";	Displaying the cadet as a table row with their full name
80	\$output_html .= "<td><label>" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "</label></td>\n";	
81	\$output_html .= "<td><input style=\"width: 10px; height: 20px;\" type=\"text\" value=\"\" . \$cadet[\"present\"] . \"\" placeholder=\"\" . \$cadet[\"present\"] . \"\" name=\"\" . \$cadet[\"user_id\"] . \"\" onkeyup=\"REGEXCheckRegister(this.value, \" . \$cadet[\"user_id\"] . \")\"></td>\n";	Displaying the present status of the cadet in the value so the form is valid when the user loads it. displaying it in the placeholder as well so the user can see if the cadet was present/absent when the form is first loaded.
82	\$output_html .= "</tr>\n";	
83	}	
84	\$output_html .= "</table></div>\n";	
85	\$output_html .= "<div class=\"input_error_handling\" id=\"register-input-handeling\"></div>\n";	Div for displaying error handling prompts when the user is populating the form
86	\$output_html .= "<div style=\"padding-top:10px\" class=\"submit-the-register\"><input type=\"submit\" value=\"submit the register\" id= \"register-submit\" class=\"register-button\">\n";	
87	\$output_html .= "</form></div>\n";	
88	} else {	If no cadets were returned from the query then

89	<code>\$output_html .= "no cadets will be present\n";</code>	Display a message to the user
90	<code>}</code>	
91	<code>return \$output_html;</code>	Return the html for the register for the event
92	<code>}</code>	
93		
94	<code>function html_for_amending_the_register(\$event_id){</code>	Function to generate the html for adding a cadet to the register, removing a cadet from the register and viewing where a cadet is if they cant be added to the register
95	<code>\$all_html = "";</code>	
96	<code>\$all_html .= "<div class=\"search-box-all\">\n";</code>	
97	<code>\$all_html .= "<input placeholder=\"add a cadet (search first name)\" type=\"text\" size=\"30\" id=\"search_first_name\" value=\"\" onkeyup=\"showResultAddCadet(this.value, 'search_first_name', '\" . \$event_id . '\")\">
\n";</code>	
98	<code>\$all_html .= "<div id=\"livesearch\"></div>";</code>	
99	<code>\$all_html .= "</div>\n";</code>	
100	<code>\$all_html .= "<div class=\"search-box-all\">\n";</code>	
101	<code>\$all_html .= "<input placeholder=\"remove a cadet (search first name)\" type=\"text\" size=\"30\" id=\"search_first_name_delete\" value=\"\" onkeyup=\"showResultDeleteCadet(this.value, 'search_first_name_delete', '\" . \$event_id . '\")\">
\n";</code>	
102	<code>\$all_html .= "<div id=\"livesearch_delete\"></div>";</code>	
103	<code>\$all_html .= "</div>\n";</code>	
104	<code>\$all_html .= "<div class=\"search-box-all\">\n";</code>	
105	<code>\$all_html .= "<input placeholder=\"search for another cadet's event\" type=\"text\" size=\"30\" id=\"search_first_name_other\" value=\"\" onkeyup=\"showResultSearchOtherCadet(this.value, '\" . \$event_id . '\")\">
\n";</code>	
106	<code>\$all_html .= "<div id=\"livesearch_other_cadet\"></div>";</code>	
107	<code>\$all_html .= "</div>\n";</code>	
108	<code>return \$all_html;</code>	Returning the html
109	<code>}</code>	
110		
111	<code>function html_for_equipment(\$con, \$event_id, \$parade_id, \$G4){</code>	Function to generate the html that will display the equipment request log

112	<pre>\$query = "SELECT equipment_requests.equipment_id, equipment_requests.aproved, equipment.name FROM equipment_requests, equipment WHERE event_id = " . \$event_id . " AND equipment_requests.equipment_id = equipment.equipment_id";</pre>	Query to select the relevant details of an equipment request. Linking the tables equipment_requests and equipment to achieve this and only selecting records that apply to the event.
113	<pre>\$result = mysqli_query(\$con, \$query);</pre>	
114	<pre>\$all_html = "";</pre>	Initialising the \$all_html variable
115	<pre>if(mysqli_num_rows(\$result) > 0){</pre>	If there are results
116	<pre>\$all_html .= "<div class=\"equipemnt-display-register- main\">\n";</pre>	Html to open the equipment request form and store the hidden details of the log in hidden fields
117	<pre>\$all_html .= "<table>\n";</pre>	
118	<pre>\$all_html .= "<form method=\"post\" action=\"requests/post_requests.php\">\n";</pre>	
119	<pre>\$all_html .= "<input hidden value=\"modify_equipemnt_reg- ister\" type=\"text\" name=\"flag\" id=\"flag\">\n";</pre>	
120	<pre>\$all_html .= "<input hidden value=\"\" . \$event_id . \"\" type=\"text\" name=\"event_id\" id=\"event_id\">\n";</pre>	
121	<pre>\$all_html .= "<input hidden value=\"\" . \$parade_id . \"\" type=\"text\" name=\"parade_id\" id=\"parade_id\">\n";</pre>	
122	<pre>while(\$equipment_request = mysqli_fetch_assoc(\$result)){</pre>	While loop to go through all the returned results and put them in dictionary format in the variable \$equipment_request
123	<pre>\$all_html .= "<tr>\n";</pre>	Opening the table row for the equipment request
124	<pre>\$all_html .= "<td><label>\" . \$equipment_request[\"name\"] . \"</label></td>\n";</pre>	Displaying the details of the equipment request in html table row format
125	<pre>\$all_html .= "<td><select value=\"\" . \$equipment_request[\"aproved\"] . \"\" id=\"\" . \$equipment_request[\"equipment_id\"] . \"\" name=\"\" . \$equipment_request[\"equipment_id\"] . \"\" onclick=\"REGEXCheckEquipment(this.value, \"\" . \$equipment_request[\"equipment_id\"] . \"\", \"\" . \$equipment_request[\"aproved\"] . \"\", \"\" . \$G4 . \"\")\">\n";</pre>	Including the validation function for onclick of the drop down box.
126	<pre>\$all_html .= "<option value=\"0\">requested</option>\n";</pre>	
127	<pre>\$all_html .= "<option value=\"1\">approved</option>\n";</pre>	
128	<pre>\$all_html .= "<option value=\"2\">denied</option>\n";</pre>	
129	<pre>\$all_html .= "</select></td>\n";</pre>	
130	<pre>\$all_html .= "</tr>\n";</pre>	
131	<pre>}</pre>	All results have been put into table rows so
132	<pre>\$all_html .= "</table><div style=\"padding-top:10px\" class=\"submit-the-equipment\"><input type=\"submit\" value=\"submit the equipment request log\" class=\"register-button\" id=\"equipment-submit\">\n";</pre>	Close the table and print out the submit button

133	<code>\$all_html .= "</form></div>\n";</code>	Close the form
134	<code>\$all_html .= "<div class=\"input_error_handeling\" id=\"equipment-input-handeling\"></div>\n";</code>	Div to display the input validation correction messages
135	<code>\$all_html .= "</div>\n";</code>	
136	<code>} else{</code>	If their were no results found
137	<code>\$all_html .= "<a>no equipment requests\n";</code>	Print a message to tell the user this
138	<code>}</code>	
139	<code>return \$all_html;</code>	Return the html
140	<code>}</code>	
141		
142	<code>function html_for_amending_the_equipment(\$event_id){</code>	Function to produce the html for the add, remove and other search boxes for equipment requests
143	<code>\$all_html = "";</code>	
144	<code>\$all_html .= "<div class=\"search-box-all\">\n";</code>	
145	<code>\$all_html .= "<input placeholder=\"add equipment request (search name)\" type=\"text\" size=\"30\" id=\"search_equipment_name_add\" name=\"search_equipment_name_add\" value=\"\" onkeyup=\"showResultAddEquipment(this.value, '\" . \$event_id . '\")\">
\n";</code>	
146	<code>\$all_html .= "<div id=\"livesearch_equipment_add\"></div>";</code>	
147	<code>\$all_html .= "</div>\n";</code>	
148	<code>\$all_html .= "<div class=\"search-box-all\">\n";</code>	
149	<code>\$all_html .= "<input placeholder=\"remove equipment request (search name)\" type=\"text\" size=\"30\" id=\"search_equipment_name_delete\" value=\"\" onkeyup=\"showResultDeleteEquipment(this.value, '\" . \$event_id . '\")\">
\n";</code>	
150	<code>\$all_html .= "<div id=\"livesearch_delete_equipement\"></div>";</code>	
151	<code>\$all_html .= "</div>\n";</code>	
152	<code>\$all_html .= "<div class=\"search-box-all\">\n";</code>	
153	<code>\$all_html .= "<input placeholder=\"search for another equipment request\" type=\"text\" size=\"30\" id=\"search_equipment_name_other\" value=\"\" onkeyup=\"showResultSearchOtherEquipment(this.value, '\" . \$event_id . '\")\">
\n";</code>	
154	<code>\$all_html .= "<div id=\"livesearch_other_equipement\"></div>";</code>	
155	<code>\$all_html .= "</div>\n";</code>	
156	<code>return \$all_html;</code>	
157	<code>}</code>	Returning the html
158		

159	<pre>function javascript_for onload(\$con, \$event_id, \$event_aproval){</pre>	Function to generate the JavaScript when the page loads so the following things are correct: the approval status of the event in the modify event details form, the arrays for keeping track of the validation status of the cadets on the register, the arrays for keeping track of the validation status of the equipment requests While it is not entirely necessary to initialise the dictionaries it does allow for more client side processing in future that will minimise the number of times the page needs to be reloaded
160	<pre>\$return_js = "";</pre>	Initializing the variable to store the JS to be returned
161	<pre>if(\$event_aproval != 1){</pre>	If the approval status of the event is not 1 i.e. the form for modifying the event details will be shown
162	<pre>\$return_js .= "setStateOfEventApproval(" . \$event_aproval . ");";</pre>	Calling the JS function for setting the approval status of the event and inputting the approval status.
163	<pre>}else{</pre>	If the event is approved i.e. the registers and the equipment request log could be populated.
164	<pre>\$initialise_array = "register_array = {";</pre>	Initialise the dictionary for keeping track of which cadets have a valid present status i.e. 1 or 0
165	<pre>\$initialise_feedback = "register_feedback = {";</pre>	Initialise the dictionary for keeping track of the feedback messages for each cadet if their present status is incorrectly populated
166	<pre>\$query = "SELECT user_id FROM user_event WHERE event_id = " . \$event_id . " . ";";</pre>	Query to select the user id's of cadets who are on the register
167	<pre>\$result = mysqli_query(\$con, \$query);</pre>	
168	<pre>while(\$cadet = mysqli_fetch_assoc(\$result)){</pre>	While loop to loop through the user_id's of cadets on the register
169	<pre>\$initialise_array .= \$cadet["user_id"] . ": 1,";</pre>	Appending the user_id to the register_array dictionary setting the state to 1 since the present status of the cadet will be valid when the page is loaded
170	<pre>\$initialise_feedback .= \$cadet["user_id"] . ": \"\",";</pre>	Appending the user_id to the register_feedback dictionary setting the feedback as empty since the present status will be in a valid state when the page is loaded
171	<pre>}</pre>	
172	<pre>\$initialise_array .= "};equipment_array = {";</pre>	Closing the register_array dictionary and opening the equipment_array dictionary

173	<code>\$initialise_feedback .= "};equipment_feedback = {";</code>	Closing the register_feedback dictionary and opening the equipment_feedback dictionary
174	<code>\$query = "SELECT equipment_id, aproved FROM equipment_requests WHERE event_id = " . \$event_id . ";;"</code>	Query to select the equipment requests for the event
175	<code>\$result = mysqli_query(\$con, \$query);</code>	
176	<code>while(\$equipment_request = mysqli_fetch_assoc(\$result)){</code>	Looping through the equipment requests
177	<code>\$return_js .= "setStateOfEquipmentRequest(" . \$equipment_request["aproved"] . ", " . \$equipment_request["equipment_id"] . ");";</code>	Appending the equipment request and calling the function to set the drop-down boxes for if the equipment request has been approved/denied
178	<code>\$initialise_array .= \$equipment_request["equipment_id"] . ": 1,";</code>	Appending the equipment request to the equipment_array dictionary and setting the state to 1 (request is in a valid state when the page is loaded)
179	<code>\$initialise_feedback .= \$equipment_request["equipment_id"] . ": \"\",";</code>	Appending the equipment request to the equipment_feedback dictionary and leaving the feedback blank since the request will be in a valid state
180	<code>}</code>	
181	<code>\$initialise_array .= "};";</code>	Closing the equipment_array dictionary
182	<code>\$initialise_feedback .= "};";</code>	Closing the equipment_feedback dictionary
183	<code>\$return_js .= \$initialise_array . \$initialise_feedback;</code>	Appending the initialisation code for the _array dictionaries and the _feedback dictionaries to the main JavaScript code
184	<code>}</code>	
185	<code>return \$return_js;</code>	Returning the initialisation JavaScript
186	<code>}</code>	
187	<code>?></code>	
188		
189	<code><!DOCTYPE html></code>	
190	<code><html></code>	
191	<code><head></code>	
192	<code><script src="js/event.js"></script></code>	Including the JS functions for modifying and validating the register/equipment request log and validating the modify event details form
193	<code><script src="js/event_owner_form_handeling.js"></script></code>	Including the JS functions for changing the event owner and displaying the new event owner
194	<code><title>my Dashbord</title></code>	
195	<code><link rel="stylesheet" href="css/event-style.css"></code>	Including the styling for the event page in general

196	<code><link rel="stylesheet" href="css/event-display-block-style.css"></code>	Including the styling for the events on the left of the page that show the other events for the parade
197	<code><link rel="stylesheet" href="css/register-and-equipment-reuquests-style.css"></code>	Including the styling for the event owner form
198	<code></head></code>	
199	<code><body onload='<?php echo(javascript_for onload(\$con, \$user_data["event_id"], \$event["final_aproval"]));?>'></code>	Calling the function to produce the JavaScript for initialising the JavaScript environment for the page when the page loads. Then setting this to be executed when the page loads
200	<code><?php include("includes/nav.php");?></code>	Including the navigation bar
201	<code><div class="grid-container"></code>	
202	<code><div class="left-side"></code>	
203	<code><?php</code>	
204	<code>include("includes/display_parade.php");</code>	Including the PHP functions to produce the list of events the client can see for the parade
205	<code>echo(html_for_parade(\$con, \$event["date"], \$user_data, "event"));</code>	Calling the function to produce the list of events and setting the output style for the function to event so: the headings of the parade are correct, admins don't have the admin panel edit button and G4 users don't see the log of equipment requests for the whole parade
206	<code>?></code>	
207	<code></div></code>	
208	<code><div class="right-side"></code>	
209	<code><h1><?php echo(\$event["event_name"]);?></h1><h1></h1><h1></h1></code>	Displaying the name of the event
210	<code><div class="register-box-all"></code>	
211	<code><h4>register</h4></code>	
212	<code><?php</code>	
213	<code>if(\$event["final_aproval"] == 1){</code>	If the final approval of the event is 1 i.e. the event is approved. Then
214	<code>echo(html_for_register(\$con, \$user_data["event_id"], \$user_data["parade_id"]));</code>	Call the function to display the register for the event and print its results
215	<code>echo(html_for_amending_the_register(\$user_data["event_id"]));</code>	And call the function to produce the html for adding/removing cadets from the register
216	<code>} else {</code>	If the event is not approved, then
217	<code>echo("<a>register is not available untill event is aproved");</code>	Display a message to the user to tell them this
218	<code>}</code>	
219	<code>?></code>	
220	<code></div></code>	
221	<code><div class="lesson-plan"></code>	

222	<?php	
223	if(\$event["final_aproval"] == 1){	If the final approval of the event is 1 i.e. the event is approved. Then
224	echo(" event aproved, lesson plan not modifiable ");	Display a message to tell the user the lesson plan is not modifiable since the event is approved
225	} else {	If the event is not approved, then
226	echo(generate_html_for_lesson_plan(\$con, \$user_data["event_id"], \$user_data["admin"]));	Call the function to display the html for the lesson plan modification form. Pass in the parameter admin to make sure the validation function for this form knows if the user is allowed to modify the approval status of the event.
227	?>	
228	</div>	
229	<div class="equipment-requests">	
230	equipment requests displayed here	
231	<?php	
232	if(\$event["final_aproval"] == 1){	If the event has an approval status of 1 i.e. the event is approved. Then
233	echo(html_for_equipment(\$con, \$user_data["event_id"], \$user_data["parade_id"], \$user_data["G4"]));	Call the function to display the log of equipment request and print its results to the page
234	echo(html_for_amending_the_equipment(\$user_data["event_id"]));	Call the function for producing the html for adding/removing equipment requests from the log
235	} else {	If the event is not approved, then
236	echo("<a>equipment requests are not available untill event is aproved");	Display a message to let the users know equipment requests for the event can't be added until its approved
237	}	
238	?>	
239	</div>	
240	</div>	
241	</div>	
242	</body>	
243	</html>	
244	<?php mysqli_close(\$con)?>	Closing the MySQL connection

requests\post_requests.php

1	<?php	
2		
3	switch (\$_POST["flag"]){	Opening a switch statement for the posted value pair of flag
4	case "add_new_parade"; case "add_new_parade";	Case handling the adding of a new parade
5	include_once ("post_request_scanning.php");	Including the post_request() function to scan for malicious inputs
6	\$date = post_request ("date");	Storing the posted values
7	\$start = post_request ("start");	
8	\$end = post_request ("end");	
9	\$parade_name = post_request ("parade_name");	
10	include_once ("../includes/connection.php");	Including the MySQL connection variable \$con
11	\$query = "SELECT MAX(parade_id) FROM parades;";	Selecting the maximum parade_id in the table parades
12	\$result = mysqli_query (\$con, \$query);	
13	\$parade_id = mysqli_fetch_assoc (\$result)["MAX(parade_id)"] + 1;	Setting the parade_id of the parade we are adding to the system to be one greater than the maximum currently in the table
14	\$query = "INSERT INTO `parades` (`parade_id`, `date`, `start`, `end`, `parade_name`) VALUES ('" . \$parade_id . "', '" . \$date . "', '" . \$start . "', '" . \$end . "', '" . \$parade_name . "')";	Query to insert the posted values of the parade into the table with the incremented parade_id
15	mysqli_query (\$con, \$query);	Executing the query
16	header ("Location: ../add.php");	Re-directing the user back to the add.php page
17	case "add_new_event"; case "add_new_event";	Case for adding a new event to the events table
18	include_once ("post_request_scanning.php");	Including the post_request() function to scan for malicious inputs
19	\$parade_id = post_request ("parade_id");	Storing the posted values
20	\$event_type = post_request ("event_type");	
21	\$event_name = post_request ("event_name");	
22	\$event_start = post_request ("event_start");	
23	\$event_end = post_request ("event_end");	
24	\$owner = post_request ("owner_id");	
25	\$duty = post_request ("duty_id");	
26	include_once ("../includes/connection.php");	Including the MySQL connection variable \$con
27	\$query = "SELECT MAX(event_id) FROM events;";	Query to select the largest event_id from events table
28	\$result = mysqli_query (\$con, \$query);	

29	<code>\$event_id = mysqli_fetch_assoc(\$result)["MAX(event_id)"] + 1;</code>	Incrementing the largest event_id by 1 and storing it to be used for the event being added to the table
30	<code>\$query = "INSERT INTO `events` (`event_id`, `parade_id`, `event_type`, `event_name`, `event_start`, `event_end`, `owner`, `final_aproval`, `duty`) VALUES ('" . \$event_id . "', '" . \$parade_id . "', '" . \$event_type . "', '" . \$event_name . "', '" . \$event_start . "', '" . \$event_end . "', '" . \$owner . "', '0', '" . \$duty . "');";</code>	Query to insert the event posted into the database using the incremented event_id and setting the approval status to 0 by default i.e. the event will be not-approved
31	<code>mysqli_query(\$con, \$query);</code>	Executing the query
32	<code>header("Location: ../add.php");</code>	Re-directing the user back to the add.php page
33	<code>case "add_new_user"; case "add_new_user";</code>	Case to add a new user to the users table posted from the add.php page
34	<code>include_once("post_request_scanning.php");</code>	Including the post_request() function to scan for malicious inputs
35	<code>\$email = post_request("email");</code>	Storing the posted values
36	<code>\$password = post_request("password");</code>	
37	<code>\$first_name = post_request("first_name");</code>	
38	<code>\$last_name = post_request("last_name");</code>	
39	<code>\$DOB = post_request("DOB");</code>	
40	<code>\$gender = post_request("gender");</code>	
41	<code>\$rank = post_request("rank");</code>	
42	<code>\$active = post_request("active");</code>	
43	<code>\$admin = post_request("admin");</code>	
44	<code>\$G4 = post_request("G4");</code>	
45	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection variable \$con
46	<code>\$query = "SELECT MAX(user_id) FROM users;";</code>	Query to select the maximum user_id in the user table
47	<code>\$result = mysqli_query(\$con, \$query);</code>	
48	<code>\$user_id = mysqli_fetch_assoc(\$result)["MAX(user_id)"] + 1;</code>	Incrementing the maximum user_id in the table by 1 and storing it to be used of the user being inserted into the table
49	<code>\$query = "INSERT INTO `users` (`user_id`, `email`, `password`, `first_name`, `last_name`, `DOB`, `gender`, `rank`, `active`, `admin`, `G4`) VALUES ('" . \$user_id . "', '" . \$email . "', '" . hash("sha256", \$password) . "', '" . \$first_name . "', '" . \$last_name . "', '" . \$DOB . "', '" . \$gender . "', '" . \$rank . "', '" . \$active . "', '" . \$admin . "', '" . \$G4 . "');";</code>	Query to insert the new user into the users table using the incremented user_id and hashing the password with the sha256 algorithm before it is inserted into the table.
50	<code>\$result = mysqli_query(\$con, \$query);</code>	
51	<code>header("Location: ../add.php");</code>	Re-directing the user back to the add.php page
52	<code>case "modify_user"; case "modify_user";</code>	Case for modifying the details of a user already in the users table

53	<code>include_once("post_request_scanning.php");</code>	Including the post_request() function to scan for malicious inputs
54	<code>\$user_id = post_request("modify_user_id");</code>	Storing the posted values
55	<code>\$email = post_request("modify_email");</code>	
56	<code>\$password = post_request("modify_password");</code>	
57	<code>\$first_name = post_request("modify_first_name");</code>	
58	<code>\$last_name = post_request("modify_last_name");</code>	
59	<code>\$DOB = post_request("modify_DOB");</code>	
60	<code>\$gender = post_request("modify_gender");</code>	
61	<code>\$rank = post_request("modify_rank");</code>	
62	<code>\$active = post_request("modify_active");</code>	
63	<code>\$admin = post_request("modify_admin");</code>	
64	<code>\$G4 = post_request("modify_G4");</code>	
65	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection variable \$con
66	<code>if(\$password == ""){</code>	If the password is blank then the password of the user it not to be updates so
67	<code>\$query = "UPDATE `users` SET `email` = '" . \$email . "', `first_name` = '" . \$first_name . "', `last_name` = '" . \$last_name . "', `DOB` = '" . \$DOB . "', `gender` = '" . \$gender . "', `rank` = '" . \$rank . "', `active` = '" . \$active . "', `admin` = '" . \$admin . "', `G4` = '" . \$G4 . "' WHERE `users`.`user_id` = '" . \$user_id . "';";</code>	Defining a query that will update the attributes of the record with the matching user_id. The attribute password will remain unchanged
68	<code>mysqli_query(\$con, \$query);</code>	Executing the query
69	<code>}else{</code>	If the password is populated i.e. the password is meant to be changed
70	<code>\$query = "UPDATE `users` SET `email` = '" . \$email . "', `password` = '" . hash("sha256", \$password) . "', `first_name` = '" . \$first_name . "', `last_name` = '" . \$last_name . "', `DOB` = '" . \$DOB . "', `gender` = '" . \$gender . "', `rank` = '" . \$rank . "', `active` = '" . \$active . "', `admin` = '" . \$admin . "', `G4` = '" . \$G4 . "' WHERE `users`.`user_id` = '" . \$user_id . "';";</code>	Defining a query that will update the attributes of the record with the matching user_id. The attribute password will be changed and set to the hashed value of the password submitted by the form.
71	<code>mysqli_query(\$con, \$query);</code>	Executing the query
72	<code>}</code>	
73	<code>header("Location: ../add.php");</code>	Re-directing the user back to the add.php page
74	<code>case "add_new_equipment";</code> <code>case "add_new_equipment";</code>	Case for adding equipment to the equipment table. From the add.php page add equipment form
75	<code>include_once("post_request_scanning.php");</code>	Including the post_request() function to scan for malicious inputs
76	<code>\$equipment_id = post_request("equipment_id");</code>	Storing the posted values
77	<code>\$name = post_request("name");</code>	
78	<code>\$description = post_request("description");</code>	
79	<code>\$location = post_request("location");</code>	
80	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection variable \$con

81	<code>\$query = "SELECT MAX(equipment_id) FROM equipment;";</code>	Selecting the maximum equipment_id from the equipment table
82	<code>\$result = mysqli_query(\$con, \$query);</code>	
83	<code>\$equipment_id = mysqli_fetch_assoc(\$result)["MAX(equipment_id)"] + 1;</code>	Incrementing the maximum equipment_id and storing it to be used as the equipment_id of the equipment that was just posted
84	<code>\$query = "INSERT INTO `equipment` (`equipment_id`, `name`, `description`, `location`) VALUES ('" . \$equipment_id . "', '" . \$name . "', '" . \$description . "', '" . \$location . "')";</code>	Query that will insert the details of the posted equipment into the equipment table
85	<code>mysqli_query(\$con, \$query);</code>	Executing the query
86	<code>header("Location: ../add.php");</code>	Re-directing the user back to the add.php page
87	<code>case "modify_equipment"; case "modify_equipment";</code>	Case to modify the details of a piece of equipment in the equipment table. Details posted from the add.php page modify equipment form
88	<code>if(\$_POST["operation"] == "delete"){</code>	If the operation to perform on the piece of equipment is a delete then
89	<code>include_once("post_request_scanning.php");</code>	Including the post_request() function to scan for malicious inputs
90	<code>\$equipment_id = post_request("modify_equipment_id");</code>	Retrieve the equipment_id of the piece of equipment that is to be deleted
91	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection variable \$con
92	<code>\$query = "DELETE FROM `equipment` WHERE `equipment`.`equipment_id` = " . \$equipment_id . " ";</code>	Query that will delete the record of that piece of equipment from the equipment table. Identifying the record by its equipment_id.
93	<code>mysqli_query(\$con, \$query);</code>	Executing the query
94	<code>header("Location: ../add.php");</code>	Re-directing the user back to the add.php page
95	<code>}else{</code>	If the equipment is not to be deleted then its details are to be modified
96	<code>include_once("post_request_scanning.php");</code>	Including the post_request() function to scan for malicious inputs
97	<code>\$equipment_id = post_request("modify_equipment_id");</code>	Storing the posted values
98	<code>\$name = post_request("modify_equipment_name");</code>	
99	<code>\$description = post_request("modify_equipment_description");</code>	
100	<code>\$location = post_request("modify_equipment_location");</code>	
101	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection variable \$con
102	<code>\$query = "UPDATE `equipment` SET `name` = '" . \$name . "', `description` = '" . \$description . "', `location` = '" . \$location . "' WHERE `equipment`.`equipment_id` = " . \$equipment_id . " ";</code>	Defining a query to update the details of the piece of equipment in the equipment table. Identifying the record by its equipment_id
103	<code>mysqli_query(\$con, \$query);</code>	Executing the query

104	header ("Location: ../add.php");	Re-directing the user back to the add.php page
105	}	
106	case "modify_register"; case "modify_register";	Case to modify the register submitted from the event.php page register form. This will update the 'present' status of cadets to show if they have/ have-not been in attendance of the event.
107	include_once ("post_request_scanning.php");	Including the post_request() function to scan for malicious inputs
108	\$event_id = post_request ("event_id");	Storing the event_id of the event to be updated and using post_request function to ensure the value isn't malicious
109	unset (\$_POST["event_id"]);	Removing the event_id attribute from the \$_POST dictionary
110	\$parade_id = post_request ("parade_id");	Storing the parade_id of the event that is to be updated and using post_request function to ensure the value isn't malicious
111	unset (\$_POST["parade_id"]);	Removing the parade_id attribute from the \$_POST dictionary
112	unset (\$_POST["flag"]);	Removing the flag attribute from the \$_POST dictionary
113	include_once ("../includes/connection.php");	Including the MySQL connection variable \$con
114	foreach (\$_POST as \$key => \$value) {	For each loop to loop through each key value pair in the \$_POST dictionary
115	\$query = "UPDATE user_event SET present = " . \$value . " WHERE user_id = " . \$key . " AND event_id = " . \$event_id . "";	For each key value pair define a query to update the present status of the cadet
116	//updating the present status of the cadet	
117	mysqli_query (\$con, \$query);	Executing the query
118	}	When each cadet's present status has been updated
119	header ("location: ../event.php?parade_id=" . \$parade_id . "&event_id=" . \$event_id);	Redirect the user back to the event.php page for the event and use the parade_id and event_id that was stored
120	case "modify_equipment_register"; case "modify_equipment_register";	Case to update the approval status of the equipment request log for an event. From the event.php page equipment request log form.
121	include_once ("post_request_scanning.php");	Including the post_request() function to scan for malicious inputs
122	\$event_id = post_request ("event_id");	Storing the event_id of the event to be updated and using post_request function to ensure the value isn't malicious
123	unset (\$_POST["event_id"]);	Removing the event_id attribute from the \$_POST dictionary
124	\$parade_id = post_request ("parade_id");	Storing the parade_id of the event that is to be updated and using post_request function to ensure the value isn't malicious

125	<code>unset(\$_POST["parade_id"]);</code>	Removing the parade_id attribute from the \$_POST dictionary
126	<code>unset(\$_POST["flag"]);</code>	Removing the flag attribute from the \$_POST dictionary
127	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection variable \$con
128	<code>foreach (\$_POST as \$key => \$value) {</code>	For each loop to loop through each key value pair in the \$_POST dictionary
129	<code> \$query = "UPDATE equipment_requests SET aproved = " . \$value . " WHERE equipment_id = " . \$key . " AND event_id = " . \$event_id . "";</code>	Query that will update the approval status of the equipment request using the equipment_id and event_id to identify which record to update
130	<code>mysqli_query(\$con, \$query);</code>	Executing the query
131	<code>}</code>	When each equipment requests approval status has been updated
132	<code>header("location: ../event.php?parade_id=" . \$parade_id . "&event_id=" . \$event_id);</code>	Redirect the user back to the event page using the parade_id and event_id that was stored earlier in the case
133	<code>}</code>	Closing the switch statement
134	<code>mysqli_close(\$con)?></code>	Closing the MySQL connection

requests\add_get_requests.php

1	<code><?php</code>	
2	<code>if (isset(\$_GET["flag"])){</code>	testing for the flag variable to identify the form
3	<code>include_once("get_request_scanning.php");</code>	Including the get_request function
4	<code>switch (get_request("flag")){</code>	Opening a switch statement to handle the different cases
5	<code>case "search_parade_name";</code> <code>case "search_parade_name";</code>	Search for a parade by its name case
6	<code>\$parade_name = get_request("prompt");</code>	Storing the search prompt
7	<code>include_once("../includes/connection.php");</code>	Including the mysql connection
8	<code>\$query = "SELECT parade_id, date, parade_name FROM parades WHERE parade_name REGEXP '" . str_replace("'", "", \$parade_name) . "'";</code>	Selecting the relevant details of the parade where the parade name matches the search prompt as a regular expression and removing the double quote marks that surround the \$parade_name
9	<code>\$result = mysqli_query(\$con, \$query);</code>	
10	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking if there are results
11	<code>\$output = "";</code>	Initialising the output variable
12	<code>while(\$parade = mysqli_fetch_assoc(\$result)){</code>	While loop to loop through all the results of the search
13	<code>\$output .= "<a style=\"background-color:#ddd;\" onclick=\"ResultHasBeenClickedParade('" . \$parade["parade_id"] . "', '" . \$parade["date"] . "', '"</code>	Putting the results of the search into readable html format with an onclick handler to populate the form with the details of the selected option of the search

	<code>. \$parade["parade_name"] . "'\">" . \$parade["parade_name"] . "
;</code>	
14	<code>}</code>	
15	<code>echo \$output;</code>	Returning the output of the get request
16	<code>} else{</code>	
17	<code>echo "<a>no names match your prompt;</code>	Returning a prompt to the user that no results for their search have been found
18	<code>}</code>	
19	<code>break;</code>	Breaking out of the switch statement
20	<code>case "search_duty"; case "search_duty";</code>	Case for searching for a new duty cadet of an event from add.php when adding an event to a parade
21	<code>\$first_name = get_request("prompt");</code>	Temporarily storing the first name prompt for the user
22	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection \$con
23	<code>\$query = "SELECT users.user_id, users.rank, users.first_name, users.last_name FROM users WHERE first_name REGEXP '" . str_replace("'", "", \$first_name) . "' ORDER BY first_name ASC;";</code>	Query to select the necessary attributes for a user to be shown in the search results. The users first name must match a regular expression of the search prompt
24	<code>\$result = mysqli_query(\$con, \$query);</code>	Executing the query
25	<code>if (mysqli_num_rows(\$result) > 0){</code>	If the number of results is non-zero
26	<code>\$output = "";</code>	Initialise the output variable
27	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	While loop to loop through each results and store them in the \$cadet variable with the result in a dictionary format
28	<code>\$output .= "" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
;</code>	Appending the result in html format to the output and populating the full name of the user and the function to call when the result is clicked which requires the user_id of the user when called
29	<code>}</code>	
30	<code>}else{</code>	If there are no results, then
31	<code>\$output = "<a>no names match your prompt;</code>	Set the outputs to a prompt to make the user aware that no results were found
32	<code>}</code>	
33	<code>echo \$output;</code>	Returning the output to the JS function that requested it.
34	<code>break;</code>	Breaking from the switch statement
35	<code>case "search_first_name_user"; case "search_first_name_user";</code>	Case for searching the first name of a user for user modification on add.php
36	<code>\$name = get_request("prompt");</code>	Setting the \$name variable to the search prompt for use in the sql query
37	<code>include_once("../includes/connection.php");</code>	Including the mysql connection

38	<code>\$query = "SELECT users.user_id, users.rank, users.first_name, users.last_name FROM users WHERE first_name REGEXP '" . str_replace("'", "", \$name) . "' ORDER BY first_name ASC;";</code>	Selecting the relevant details of the user where their name matches a regular expression of the search prompt and ordering the names alphabetically by first name
39	<code>\$result = mysqli_query(\$con, \$query);</code>	
40	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Testing if there were results to the query
41	<code>\$output = "";</code>	Initialising the output variable
42	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	Looping through the results of the search and putting them into readable html format
43	<code>if(\$output == ""){</code>	If the string is empty
44	<code>\$output = "" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
";</code>	Then the string can be set to the first search result in html format Add an onclick handler that passes in the user's user_id so their details can be retrieved to populate the modify user form
45	<code>} else {</code>	If output already contains search results
46	<code>\$output = \$output . "" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
";</code>	The next result must be concatenated to output instead of reassigning output to the search result. Adding an onclick handler that passes in the user's user_id so their details can be retrieved to populate the modify user form
47	<code>}</code>	
48	<code>}</code>	
49	<code>echo \$output;</code>	Returning the results of the search to the JS function that requested it for it to be displayed to the admin
50	<code>} else{</code>	
51	<code>echo "<a>no names match your prompt";</code>	Returning a prompt to tell the user no results were found
52	<code>}</code>	
53	<code>break;</code>	Breaking out of the switch statement
54	<code>case "search_last_name_user"; case "search_last_name_user";</code>	Case for searching for user to modify by their last name on add.php
55	<code>\$name = get_request("prompt");</code>	Setting \$name to the prompt the user provided
56	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
57	<code>\$query = "SELECT users.user_id, users.rank, users.first_name, users.last_name FROM users WHERE last_name REGEXP '" . str_replace("'", "", \$name) . "' ORDER BY last_name ASC;";</code>	Query to select the relevant details of a user. Similar to the previous case but using the last_name attribute instead of first_name
58	<code>\$result = mysqli_query(\$con, \$query);</code>	
59	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking if there were results to the query

60	<code>\$output = "";</code>	Initialising the output variable
61	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	Looping through the results of the search
62	<code>if(\$output == ""){</code>	If output is empty
63	<code>\$output = " " . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
";</code>	Reassign output to the first result of the search in html format. Adding an onclick handler that passes in the user's user_id so their details can be retrieved to populate the modify user form
64	<code>} else {</code>	If the string already contains results
65	<code>\$output = \$output . " " . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
";</code>	Concatenate the string of results with the current result displaying the users rank, first and last name. Adding an onclick handler that passes in the user's user_id so their details can be retrieved to populate the modify user form
66	<code>}</code>	
67	<code>}</code>	
68	<code>echo \$output;</code>	Returning the results of the search to the js function that made the get request
69	<code>} else{</code>	
70	<code>echo "<a>no names match your prompt";</code>	If no users are found, then prompt the admin accordingly
71	<code>}</code>	
72	<code>break;</code>	Breaking out of the switch statement
73	<code>case "search_equipment_name"; case "search_equipment_name";</code>	Search equipment by name used by add.php for the modify equipment form
74	<code>\$name = get_request("prompt");</code>	Setting \$name to the prompt for the search the user provided
75	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
76	<code>\$query = "SELECT * FROM equipment WHERE name REGEXP '" . str_replace("'", "", \$name) . "'";</code>	Selecting all attributes of a piece of equipment with a name that matches a regular expression containing the users prompt
77	<code>\$result = mysqli_query(\$con, \$query);</code>	
78	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking there were results to the search
79	<code>\$output = "";</code>	Initialising the output variable
80	<code>while(\$equipment = mysqli_fetch_assoc(\$result)){</code>	Looping through the results of the search
81	<code>if(\$output == ""){</code>	If the output variable is empty, then reassign output to the first result of the search
82	<code>\$output = "name:" .</code>	Putting the result of the search into html format with an onclick handler that uses the equipment_id to populate the modify equipment form

	<code>\$equipment["name"] . " location:" . \$equipment["location"] . "
;</code>	
83	<code>} else {</code>	The output variable is not empty so the current result must be concatenated to the previous results
84	<code>\$output = \$output . "name:" . \$equipment["name"] . " location:" . \$equipment["location"] . "
;</code>	Putting the result of the search into html format with an onclick handler that uses the equipment_id to populate the modify equipment form
85	<code>}</code>	
86	<code>}</code>	
87	<code>echo \$output;</code>	Returning the results of the search to the JS function that requested them
88	<code>} else{</code>	
89	<code>echo("<a>no names match your prompt");</code>	If not, results were found then prompt the user accordingly
90	<code>}</code>	
91	<code>break;</code>	Breaking out of the switch statement
92	<code>case "search_equipment_location"; case "search_equipment_location";</code>	Case for searching for equipment by its location for the modify equipment form on add.php
93	<code>\$location = get_request("prompt");</code>	Setting the location variable to the prompt the user provided
94	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
95	<code>\$query = "SELECT * FROM equipment WHERE location REGEXP '" . str_replace("'", "", \$location) . "'";</code>	Similar to previous case but for location
96	<code>\$result = mysqli_query(\$con, \$query);</code>	
97	<code>if(mysqli_num_rows(\$result) > 0) {</code>	If the number of results is greater than 0 i.e. the piece of equipment is in the database
98	<code>\$output = "";</code>	set \$equipment to a dictionary of the result
99	<code>while(\$equipment = mysqli_fetch_assoc(\$result)){</code>	Encode the piece of equipment in JSON format and return it to the
100	<code>if(\$output == ""){</code>	no equipment was found so
101	<code>\$output = "name:" . \$equipment["name"] . " location:" . \$equipment["location"] . "
;</code>	Return a JSON message stating no results were found back to the function
102	<code>} else {</code>	
103	<code>\$output = \$output . "<a style=\"background-color:#ddd;\" onclick='resultHasBeenClickedEquipment(" . \$equipment["equipment_id"] . ")' name=" .</code>	Exit from the switch statement

	<code>\$equipment["equipment_id"] . ">name:" . \$equipment["name"] . " location:" . \$equipment["location"] . "
;</code>	
104	<code>}</code>	Case for retrieving all the attributes of a user in JSON format
105	<code>}</code>	
106	<code>echo \$output;</code>	Returning the results of the users search to the JS function that requested it
107	<code>} else{</code>	
108	<code>echo("<a>no locations match your prompt");</code>	No results found prompt the user accordingly
109	<code>}</code>	
110	<code>break;</code>	Breaking out of the switch statement
111	<code>case "equipment_id_info_dump"; case "equipment_id_info_dump";</code>	Case for getting all attributes about a piece of equipment used by JS function resultHasBeenClickedEquipment() to populate the modify equipment form on add.php
112	<code>\$equipment_id = get_request("prompt");</code>	Setting the equipment_id to the prompt the JS function provided
113	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
114	<code>\$query = "SELECT * FROM equipment WHERE equipment_id = " . \$equipment_id . ";;"</code>	Selecting all attributes of the equipment with known equipment_id
115	<code>\$result = mysqli_query(\$con, \$query);</code>	
116	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking the equipment_id was valid
117	<code>\$equipment = mysqli_fetch_assoc(\$result);</code>	Putting the result into a dictionary format
118	<code>echo json_encode(\$equipment);</code>	Encoding the dictionary format of the piece of equipment into a JSON format and returning it to the JS function to populate the form
119	<code>} else {</code>	
120	<code>echo json_encode(["message" => "no equipment found"]);</code>	If no result was found for the requested equipment_id prompt the function accordingly
121	<code>}</code>	
122	<code>break;</code>	Breaking out of the switch statement
123	<code>case "user_id_info_dump"; case "user_id_info_dump";</code>	Case for getting all attributes of a user used by JS function resultHasBeenClickedUser() to populate the modify user form on add.php
124	<code>\$user_id = get_request("prompt");</code>	Setting the user_id to the prompt the JS function provided
125	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
126	<code>\$query = "SELECT * FROM users WHERE user_id = " \$user_id . ";;"</code>	Selecting all attributes of the user with known user_id
127	<code>\$result = mysqli_query(\$con, \$query);</code>	

128	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking the user_id was real
129	<code>\$cadet = mysqli_fetch_assoc(\$result);</code>	Setting \$cadet to a dictionary of the result
130	<code>\$cadet["password"] = "";</code>	Removing the password from the dictionary since it is hashed (and therefore practically useless to display and potentially dangerous to send over an un-encrypted connection)
131	<code>echo json_encode(\$cadet);</code>	Encoding the dictionary into a JSON format for the JS function that made the request to use
132	<code>} else {</code>	
133	<code>return json_encode(["message" => "no names match your prompt"]);</code>	If the user_id was not real, then prompt the JS function accordingly
134	<code>}</code>	
135	<code>break;</code>	
136	<code>}</code>	
137	<code>mysqli_close(\$con)</code>	Since all cases have been handled close the MySQL connection as it is good practice
138	<code>}</code>	
139	<code>?></code>	

requests\event_details_requests.php

1	<code><?php</code>	This file has since changed so the case modify_event_details will now redirect duty cadets back to the event page
2	<code>include_once("get_request_scanning.php");</code>	Including the get_request() function
3	<code>"search_first_name_owner"</code> <code>if(isset(\$_GET["flag"]) and get_request("flag") == "search_first_name_owner"){</code>	Testing if the flag variable in the \$_GET dictionary exists. If it does, then the value of this variable indicates this case should be executed.
4	<code>\$name = get_request("prompt");</code>	Setting \$name to the value of the search prompt
5	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
6	<code>\$query = "SELECT users.user_id, users.rank, users.first_name, users.last_name FROM users WHERE first_name REGEXP '" . str_replace("'", "", \$name) . "'";</code>	Selecting the relevant details to display the results of the search to the user and only selecting users that have a first name that matches the search prompt as a regular expression
7	<code>\$result = mysqli_query(\$con, \$query);</code>	
8	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking results do exists
9	<code>\$output = "";</code>	Initialising the output variable
10	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	Looping through the results of the search
11	<code>if(\$output == ""){</code>	If \$output is empty, then
12	<code>\$output = " " . \$cadet["rank"] .</code>	Assign \$output with the results of the first search and create an onclick handler with the user's user_id so when a result is clicked the

	<code>" " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
;</code>	modify event form can be correctly populated with the new user_id of the owner
13	<code>} else {</code>	The \$output variable already contains results
14	<code>\$output = \$output . "" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
;</code>	Concatenate \$output with the current result and create an onclick handler with the user's user_id so when a result is clicked the modify event form can be correctly populated with the new user_id of the owner
15	<code>}</code>	
16	<code>}</code>	
17	<code>echo \$output;</code>	Return the results of the search to the JS function that requested it
18	<code>} else{</code>	
19	<code>echo "<a>no names match your prompt;</code>	No results were found so prompt the user accordingly
20	<code>}</code>	
21	<code>}</code>	
22		
23	<code>if (isset(\$_POST["flag"])){</code>	Testing if a post request was made with the flag attribute
24	<code>include_once("post_request_scanning.php");</code>	Including the post_request() function
25	<code>switch (post_request("flag")){</code>	Opening the switch statement to handle the different values of flag indicating the different forms
26	<code>case "modify_event_details";</code> <code>case "modify_event_details";</code>	Case for modifying the events details form from calendar.php and event.php
27	<code>\$parade_id = post_request("parade_id");</code>	setting the variables for the SQL query
28	<code>\$event_id = post_request("event_id");</code>	
29	<code>\$event_type = post_request("event_type");</code>	
30	<code>\$event_name = post_request("event_name");</code>	
31	<code>\$event_start = post_request("event_start");</code>	
32	<code>\$event_end = post_request("event_end");</code>	
33	<code>\$owner = post_request("owner_id");</code>	
34	<code>\$final_aproval = post_request("final_aproval");</code>	
35	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
36	<code>\$query = "UPDATE events SET event_type = '" . \$event_type . "', event_name = '" . \$event_name . "', event_start = '" . \$event_start . "', event_end = '" . \$event_end . "', owner = '" . \$owner . "', final_aproval = '" . \$final_aproval . " WHERE event_id = '" . \$event_id;</code>	Query to update the attributes of an event with a known event_id
37	<code>\$result = mysqli_query(\$con, \$query);</code>	
38	<code>session_start();</code>	Including the \$_session dictionary to access the user's user_id and hashed password

39	<code>include_once("../includes/functions.php");</code>	Including the check_login() function
40	<code>\$user_data = check_login(\$con);</code>	Assigning the user's data including the admin and G4 attributes
41	<code>if((post_request("user_id") == \$owner or \$user_data["admin"] == 1 or \$user_data["G4"] == 1) and \$_POST["calendar_flag"] == 0){</code>	Checking if the user is the owner of the event or if they are admin or if they are G4(this means they can access the event.php page for the event) additionally we check that they have not accessed this form through the calendar by the calendar flag being set to 0.
42	<code>header("location: ../event.php?parade_id=" . \$parade_id . "&event_id=" . \$event_id);</code>	If the user has met the above conditions, then they should be redirected back to the event.php page
43	<code>}else{</code>	
44	<code>header("location: ../calendar.php");</code>	The user has not met the conditions so should be redirected to the calendar.php page
45	<code>}</code>	Since the string that identifies this case is not a subset of the string that identifies other cases in this switch statement a break command is not needed
46	<code>case "delete_event";</code> <code>case "delete_event";</code>	Case to handle del the deletion of an event
47	<code>\$parade_id = post_request("delete_parade_id");</code>	Setting the parade_id of the event
48	<code>\$event_id = post_request("delete_event_id");</code>	Setting the event_id of the event
49	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
50	<code>\$query = "DELETE FROM user_event WHERE event_id = " . \$event_id;</code>	Query to delete the cadets who may have been put on the register for this event. This ensures the forging key rules for the database are adhered to
51	<code>mysqli_query(\$con, \$query);</code>	Executing the query
52	<code>\$query = "DELETE FROM equipment_requests WHERE event_id = " . \$event_id;</code>	Query to delete the equipment requests that may have been put on for this event. This also ensures that foreign key rules for the database are adhered to
53	<code>mysqli_query(\$con, \$query);</code>	Executing the query
54	<code>\$query = "DELETE FROM events WHERE event_id = " . \$event_id;</code>	Query to delete the event from the events table.
55	<code>mysqli_query(\$con, \$query);</code>	Executing the query
56	<code>header("location: ../calendar.php");</code>	Redirecting the user to the calendar page since the event.php page for the event will no longer exist
57	<code>}</code>	
58	<code>}</code>	
59	<code>mysqli_close(\$con)</code>	Closing the MySQL connection for all cases

60 ?>

requests\event_get_requests.php

1	<?php	
2	if (isset(\$_GET["flag"])){	Checking if a get request has been made and if the flag attribute in the dictionary \$_GET exists
3	include_once("get_request_scanning.php");	Including the get_request() function
4	switch (get_request("flag")){	Opening a switch statement to handle the different values of the flag attribute
5	case "add_user_id";	Case for adding a user to an event from event.php
	case "add_user_id";	
6	include_once("../includes/connection.php");	Including the MySQL connection
7	\$query = "INSERT INTO user_event (user_id, event_id, present) VALUES (" . get_request("add_user_id") . ", " . get_request("event_id") . ",0);";	Query to insert the user_id and event_id into the user_event table. By default, the user will be marked absent
8	mysqli_query(\$con, \$query);	Executing the query
9	case "remove_user_id";	Case to handle the removal of a user from an event's register of cadets from event.php
	case "remove_user_id";	
10	include_once("../includes/connection.php");	Including the MySQL connection
11	\$query = "DELETE FROM user_event WHERE user_id=" . get_request("remove_user_id") . " AND event_id=" . get_request("event_id") . ";";	Query to delete the user from the register where the user_id and event_id match the desired values
12	mysqli_query(\$con, \$query);	Executing the query
13	case "add_equipment_id";	Case to add a piece of equipment to the log of equipment requests for an event from event.php
	case "add_equipment_id";	
14	include_once("../includes/connection.php");	Including the MySQL connection
15	\$query = "SELECT equipment_id FROM equipment_requests WHERE equipment_id=" . get_request("add_equipment_id") . " AND event_id=" . get_request("event_id") . ";";	Query to check if the equipment request already exists
16	\$result = mysqli_query(\$con, \$query);	Executing the query and storing the result
17	if(mysqli_num_rows(\$result) > 0){	If the request has already been made
18	\$query = "UPDATE equipment_requests SET aproved=0 WHERE equipment_id=" . get_request("add_equipment_id") . " AND event_id=" . get_request("event_id") . ";";	Update the equipment request so its approval is set to 0(approval requested)
19	} else {	If the request has not yet been made
20	\$query = "INSERT INTO equipment_requests (equipment_id, event_id, aproved) VALUES (" . get_request("add_equipment_id") . ", " . get_request("event_id") . ",0);";	Inset the equipment request into equipment_requests and set approval to 0(approval requested)
21	}	

22	<code>mysqli_query(\$con, \$query);</code>	Execute the query decided by the number of rows returned from the first query
23	<code>case "remove_equipment_id";</code> <code>case "remove_equipment_id";</code>	Case to handle the removal of equipment requests from the equipment_request log
24	<code>include_once("../includes/connection.php");</code>	Including the MySQL login
25	<code>\$query = "DELETE FROM equipment_requests WHERE equipment_id=" . get_request("remove_equipment_id") . " AND event_id=" . get_request("event_id") . " ;";</code>	Query to delete equipment requests from equipment_requests
26	<code>mysqli_query(\$con, \$query);</code>	Executing the query
27	<code>case "search_first_name";</code> <code>case "search_first_name";</code>	Case to search for the first name of a cadet to be added to the register used by event.php
28	<code>\$search_first_name = get_request("prompt");</code>	Storing the search prompt
29	<code>\$event_id = get_request("event_id");</code>	Storing the event_id of the event that the cadet should be added to(the clients event)
30	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
31	<code>\$query = "SELECT event_id, parade_id, event_start, event_end FROM events WHERE event_id = " . \$event_id . " ;";</code>	Selecting the relevant information for later checks if an event clashes with other events on that parade
32	<code>\$result = mysqli_query(\$con, \$query);</code>	
33	<code>\$event = mysqli_fetch_assoc(\$result);</code>	Storing the results in a dictionary format
34	<code>\$query = "SELECT users.first_name, users.last_name, users.rank, users.user_id</code>	<u>LARGE QUERY</u> selecting the relevant details of users who could be added to the event
35	<code>FROM users</code>	Selecting from the users table
36	<code>WHERE users.first_name REGEXP '"' . str_replace('"' , "'", \$search_first_name) . "'</code>	Excluding users who have a first name that does not fit a regular expression of the clients search prompt
37	<code>AND users.user_id NOT IN</code>	Only selecting user_id of users who do not apere in the next query. The next query will select user_id's of users who are in an event that clashes with the clients event
38	<code>(SELECT user_event.user_id</code>	Selecting user_ids from the user_event table (that stores the relation between users and an event)
39	<code>FROM events, user_event</code>	Listing the tables being accessed
40	<code>WHERE user_event.event_id = events.event_id</code>	Linking the tables together
41	<code>AND events.parade_id = " . \$event["parade_id"] . "</code>	Only selecting events happening on the parade of the clients event.
42	<code>AND ((events.event_start <= '"' . \$event["event_start"] . "' AND events.event_end > '"' . \$event["event_start"] . "''))</code>	Selecting events that start before the clients event and finish after the beginning of the clients event
43	<code>OR (events.event_start < '"' . \$event["event_end"] . "' AND events.event_end >= '"' . \$event["event_end"] . "''))</code>	Selecting events that start before the end of the clients event and finish after the clients event

44	<code>OR (events.event_start <= '' . \$event["event_start"] . '' AND events.event_end >= '' . \$event["event_end"] . '')</code>	Selecting events that start and finish inside of the clients event
45	<code>OR (events.event_start >= '' . \$event["event_start"] . '' AND events.event_end <= '' . \$event["event_end"] . '')</code>	Selecting events that start and finish outside of the clients event
46	<code>OR (events.event_id = '' . \$event["event_id"] . ")););";</code>	Selecting the clients event since cadets already on the register fit into this category hence they can't be added to the clients event.
47	<code>\$result = mysqli_query(\$con, \$query);</code>	Storing the result
48	<code>if(mysqli_num_rows(\$result) > 0) {</code>	If there are results, then cadets can make it to the event that fit the clients search prompt
49	<code>\$output_html = "";</code>	Initialising the output_html variable
50	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	Looping through the list of users that can be added to the clients event
51	<code>\$output_html .= "" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
";</code>	Appending html to the output to show a user can make it to the event. Adding an onclick handler so when the user is clicked by the client the user_id and event_id are sent back to the server and added to the register
52	<code>}</code>	
53	<code>echo \$output_html;</code>	Returning the results of the search to the js function that requested it
54	<code>} else{</code>	
55	<code>echo "<a>no names match your prompt";</code>	Returning a prompt to the user telling them no cadets that fit their description could be added to the event register
56	<code>}</code>	
57	<code>break;</code>	Breaking from the switch statement since no other cases should be run if this one is executed
58	<code>case "search_first_name_delete"; case "search_first_name_delete";</code>	Case to search for users who can be removed from the register used by event.php
59	<code>\$first_name = get_request("prompt");</code>	Setting the prompt for the search for first name
60	<code>\$event_id = get_request("event_id");</code>	Setting the event_id of the event of interest
61	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
62	<code>\$query = "SELECT users.first_name, users.last_name, users.rank, users.user_id FROM users WHERE users.user_id IN (SELECT user_event.user_id FROM user_event WHERE user_event.event_id = '' . \$event_id . ") AND users.first_name REGEXP '' . str_replace(' ', '', \$first_name) . '";</code>	Selecting the relevant details of a user from the users table where the first name of the user fits a regular expression of the prompt provided and the user_id is in. A list of records from the user_event table where the event_id is the desired one

		indicating the user with said user_id is on the register for the event hence they can be removed from the register for this event
63	<code>\$result = mysqli_query(\$con, \$query);</code>	
64	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking if there were results
65	<code>\$output_html = "";</code>	Initialising \$output_html
66	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	Looping through each cadet that fits the prescribed conditions
67	<code>\$output_html .= "" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . "
";</code>	Putting the results into a readable html format with an onclick handler that calls a JS function to delete the record from the database where the user_id and event_id are known.
68	<code>}</code>	
69	<code>echo \$output_html;</code>	Returning the results to the JS function that requested them
70	<code>} else{</code>	
71	<code>echo "<a>no names match your prompt";</code>	No records were found that met the desired conditions hence we prompt the user accordingly
72	<code>}</code>	
73	<code>break;</code>	Breaking from the switch statement
74	<code>case "search_first_name_other_cadet";</code> <code>case "search_first_name_other_cadet";</code>	Case for, searching for what cadets are doing during the event the client it currently looking at on the event.php page
75	<code>\$first_name = get_request("prompt");</code>	Storing the prompt the client provided
76	<code>\$event_id = get_request("event_id");</code>	Storing the event_id that the client is presently looking at on event.php
77	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
78	<code>\$query = "SELECT events.event_start, events.event_end, events.parade_id FROM events WHERE events.event_id = " . \$event_id . " . ";";</code>	Query to select the parade_id, start/end times of the event the client is currently looking at
79	<code>\$result = mysqli_query(\$con, \$query);</code>	
80	<code>\$event = mysqli_fetch_assoc(\$result);</code>	Storing the results of the query
81	<code>\$query = "SELECT users.first_name, users.last_name, users.rank, events.event_start, events.event_end, events.event_name, events.owner</code>	LARGE QUERY selecting the relevant details of the user and event that the user is in
82	<code>FROM events, user_event, users</code>	Defining the tables to select from
83	<code>WHERE user_event.event_id = events.event_id</code>	Linking the user_event and events table
84	<code>AND users.first_name REGEXP '"' . str_replace('\'', "'", \$first_name) . "'</code>	Condition to select records that fit the search prompt the client provided
85	<code>AND user_event.user_id = users.user_id</code>	Linking the user_event and users table
86	<code>AND events.event_id != " . \$event_id . "</code>	only selecting events where the event_id is different to the event_id of the event the client is currently looking at

87	<code>AND events.parade_id = " . \$event["parade_id"] . "</code>	Only selecting events that are on the same parade as the client's event
88	<code>AND ((events.event_start <= ' ' . \$event["event_start"] . ' ' AND events.event_end > ' ' . \$event["event_start"] . ' '))</code>	Selecting events that start before the clients event and finish after the beginning of the clients event
89	<code>OR (events.event_start < ' ' . \$event["event_end"] . ' ' AND events.event_end >= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start before the end of the clients event and finish after the clients event
90	<code>OR (events.event_start <= ' ' . \$event["event_start"] . ' ' AND events.event_end >= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start and finish inside of the clients event
91	<code>OR (events.event_start >= ' ' . \$event["event_start"] . ' ' AND events.event_end <= ' ' . \$event["event_end"] . ' '))";</code>	Selecting events that start and finish outside of the clients event
92	<code>\$result = mysqli_query(\$con, \$query);</code>	
93	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking if there are results
94	<code>\$output_html = "";</code>	There are results so initialise the \$output_html variable
95	<code>while(\$cadet = mysqli_fetch_assoc(\$result)){</code>	Looping through the results
96	<code>\$output_html .= "<a>" . \$cadet["rank"] . " " . \$cadet["first_name"] . " " . \$cadet["last_name"] . " " . \$cadet["event_name"] . " " . \$cadet["event_start"] . " " . \$cadet["event_end"] . "
;</code>	Putting the record into a readable html format that displays the cadets full name, event name and details of the event start and end times
97	<code>}</code>	
98	<code>echo \$output_html;</code>	Returning the results to the JS function that requested them
99	<code>} else{</code>	
100	<code>echo "<a>no names match your prompt;</code>	No results were found so prompt the user accordingly
101	<code>}</code>	
102	<code>break;</code>	Breaking from the switch statement
103	<code>case "search_equipment_add"; case "search_equipment_add";</code>	Case for searching equipment by name to add to the equipment request log for an event
104	<code>\$equipment_name = get_request("prompt");</code>	Storing the prompt the client provided
105	<code>\$event_id = get_request("event_id");</code>	Storing the event_id of the event the client is currently looking at
106	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
107	<code>\$query = "SELECT event_id, parade_id, event_start, event_end FROM events WHERE event_id = " . \$event_id . ";";</code>	Query to select the parade_id, start/end times of the event the client is currently looking at
108	<code>\$result = mysqli_query(\$con, \$query);</code>	
109	<code>\$event = mysqli_fetch_assoc(\$result);</code>	Storing the details of the event as a dictionary
110	<code>\$query = "SELECT equipment.equipment_id, equipment.name</code>	Selecting the relevant details of the available equipment

111	<code>FROM equipment</code>	Defining the table
112	<code>WHERE equipment.equipment_id NOT IN</code>	Only selecting equipment_id's that are not in the list of equipment_id's in other events
113	<code>(SELECT equipment_requests.equipment_id</code>	Inner query to select equipment_id's of equipment not possible to add to the equipment request log for the clients event
114	<code>FROM equipment_requests, events</code>	Defining the tables
115	<code>WHERE events.event_id = equipment_requests.event_id</code>	Linking the events and equipment_requests tables
116	<code>AND events.parade_id = " . \$event["parade_id"] . "</code>	Only selecting events happening on the same parade as the clients event
117	<code>AND equipment_requests.aproved != 2</code>	Selecting pending or approved equipment requests not denied ones
118	<code>AND ((events.event_start <= ' ' . \$event["event_start"] . ' ' AND events.event_end > ' ' . \$event["event_start"] . ' '))</code>	Selecting events that start before the clients event and finish after the beginning of the clients event
119	<code>OR (events.event_start < ' ' . \$event["event_end"] . ' ' AND events.event_end >= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start before the end of the clients event and finish after the clients event
120	<code>OR (events.event_start <= ' ' . \$event["event_start"] . ' ' AND events.event_end >= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start and finish inside of the clients event
121	<code>OR (events.event_start >= ' ' . \$event["event_start"] . ' ' AND events.event_end <= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start and finish outside of the clients event
122	<code>OR (events.event_id = " . \$event["event_id"] . ")))</code>	Selecting the event with event_id of the clients event
123	<code>AND equipment.name REGEXP ' ' . str_replace(' ', "", \$equipment_name) . ' ';</code>	Only selecting results that have equipment name that fits a regular expression of the users search prompt
124	<code>\$result = mysqli_query(\$con, \$query);</code>	
125	<code>if(mysqli_num_rows(\$result) > 0){</code>	Checking to see if there were results
126	<code>\$output_html = "";</code>	Initialising the output_html
127	<code>while(\$equipment = mysqli_fetch_assoc(\$result)){</code>	Looping through the list of results
128	<code>\$output_html .= ">" . \$equipment[\"name\"] . "
";</code>	Putting the results in a readable html format with an onclick handler to add the equipment request to the log when clicked
129	<code>}</code>	
130	<code>echo \$output_html;</code>	Returning the results of the search to the js function that requested it
131	<code>}else{</code>	
132	<code>echo "<a>no equipment matches your prompt";</code>	Prompting the user that no results that met their conditions were found
133	<code>}</code>	
134	<code>break;</code>	Breaking from the switch statement

135	case "search_equipment_delete"; case "search_equipment_delete";	Case to search for pieces of equipment to remove from the equipment request log
136	<code>\$equipment_name = get_request("prompt");</code>	Storing the clients search prompt
137	<code>\$event_id = get_request("event_id");</code>	Storing the event_id of the event the client is currently looking at on event.php
138	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
139	<code>\$query = "SELECT equipment.equipment_id, equipment.name FROM equipment WHERE equipment.equipment_id IN (SELECT equipment_requests.equipment_id FROM equipment_requests WHERE equipment_requests.event_id = " . \$event_id . " AND equipment_requests.aproved != 2) AND equipment.name REGEXP '" . str_replace("'", "", \$equipment_name) . "';";</code>	Selecting the relevant details of equipment in the equipment table with name that fits a regular expression of the clients search prompt. Only selecting equipment that is in the list of pending or approved equipment requests for the event.
140	<code>\$result = mysqli_query(\$con, \$query);</code>	
141	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking for query results
142	<code>\$output_html = "";</code>	Initialising the output_html
143	<code>while(\$equipment = mysqli_fetch_assoc(\$result)) {</code>	Looping through the results of the query
144	<code>\$output_html .= "" . \$equipment["name"] . "
";</code>	Putting the results into a readable html format and adding an onclick handler to remove the equipment requests from the equipment requests log of the event.
145	<code>}</code>	
146	<code>echo \$output_html;</code>	Returning the results of the search to the JS function that requested them
147	<code>} else {</code>	
148	<code>echo "<a>no equipment matches your prompt";</code>	No results were found so prompt the user accordingly
149	<code>}</code>	
150	<code>break;</code>	Breaking from the switch statement
151	case "search_equipment_other"; case "search_equipment_other";	Case to search for where a piece of equipment is during the event that the client is currently looking at on event.php
152	<code>\$equipment_name = get_request("prompt");</code>	Storing the search prompt
153	<code>\$event_id = get_request("event_id");</code>	Storing the event_id contained in the request
154	<code>include_once("../includes/connection.php");</code>	Including the MySQL connection
155	<code>\$query = "SELECT events.event_start, events.event_end, events.parade_id FROM events WHERE events.event_id = " . \$event_id . ";";</code>	Query to select the parade_id, start/end times of the event the client is currently looking at
156	<code>\$result = mysqli_query(\$con, \$query);</code>	
157	<code>\$event = mysqli_fetch_assoc(\$result);</code>	<u>Large query</u> Storing the details of the event the client is currently looking at
158	<code>\$query = "SELECT equipment.equipment_id, equipment.name, events.event_start, events.event_end, events.event_name</code>	Selecting the relevant details of the equipment and where it is during the client's event

159	<code>FROM equipment, equipment_requests, events</code>	Defining the tables
160	<code>WHERE equipment.equipment_id = equipment_requests.equipment_id</code>	Linking the equipment and equipment_requests table
161	<code>AND equipment_requests.event_id = events.event_id</code>	Linking the equipment_requests and events table
162	<code>AND events.parade_id = " . \$event["parade_id"] . "</code>	Only selecting events to be conflicting if they are on the same parade as the client's event
163	<code>AND events.event_id != ' ' . \$event_id . ' '</code>	only selecting events if they have an event_id different to the client's event
164	<code>AND equipment_requests.aproved != 2</code>	Only selecting pending or approved equipment requests
165	<code>AND equipment.name REGEXP ' ' . str_replace(' ', "", \$equipment_name) . ' '</code>	Only selecting equipment with a name that matches a regular expression of the clients search prompt
166	<code>AND ((events.event_start <= ' ' . \$event["event_start"] . ' ' AND events.event_end > ' ' . \$event["event_start"] . ' '))</code>	Selecting events that start before the client's event and finish after the beginning of the clients event
167	<code>OR (events.event_start < ' ' . \$event["event_end"] . ' ' AND events.event_end >= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start before the end of the client's event and finish after the clients event
168	<code>OR (events.event_start <= ' ' . \$event["event_start"] . ' ' AND events.event_end >= ' ' . \$event["event_end"] . ' '))</code>	Selecting events that start and finish inside of the client's event
169	<code>OR (events.event_start >= ' ' . \$event["event_start"] . ' ' AND events.event_end <= ' ' . \$event["event_end"] . ' '))));</code>	Selecting events that start and finish outside of the client's event
170	<code>\$result = mysqli_query(\$con, \$query);</code>	
171	<code>if(mysqli_num_rows(\$result) > 0) {</code>	Checking if there were results to the query
172	<code>\$output_html = "";</code>	Initialising the output_html variable
173	<code>while(\$equipment = mysqli_fetch_assoc(\$result)) {</code>	Looping through the results of the search
174	<code>\$output_html .= "<a>" . \$equipment["name"] . " " . \$equipment["event_name"] . " " . \$equipment["event_start"] . " " . \$equipment["event_end"] . "
;</code>	Putting the results into a readable html format that displays where the piece of equipment is
175	<code>}</code>	
176	<code>echo \$output_html;</code>	Returning the results of the search to the JS function that requested them
177	<code>} else {</code>	
178	<code>echo "<a>no equipment matches your prompt;</code>	No results were found so prompt the client accordingly
179	<code>}</code>	
180	<code>break;</code>	Breaking from the switch statement
181	<code>}</code>	
182	<code>}</code>	
183	<code>mysqli_close(\$con);</code>	Closing the MySQL connection

js\calendar.js

1	populateAdminEditEventForm() function populateAdminEditEventForm (parade_id, event_id, event_type, event_name, event_start, event_end, owner_id, final_aproval) {	Function for populating the admin edit an event form on the left of the calendar.php for admins
2	console.log(owner_id);	
3	document.getElementById("parade_id").value = parade_id;	Setting the hidden field parade_id
4	document.getElementById("event_id").value = event_id;	Setting the hidden field event_id
5	document.getElementById("event_type").value = event_type;	Populating the event_type input box
6	document.getElementById("event_name").value = event_name;	Populating the event_name input box
7	document.getElementById("event_start").value = event_start;	Populating the event_start input box
8	document.getElementById("event_end").value = event_end;	Populating the event_end input box
9	document.getElementById("owner_id").value = owner_id;	Setting the hidden field owner_id
10	document.getElementById("final_aproval").value = final_aproval;	Setting the final approval drop down box
11	document.getElementById("original_aproval").value = final_aproval;	Setting the original approval input box for form validation purposes (preventing an approved event from being moved back to un-approved or approval requested
12	document.getElementById("event-input-handeling").innerHTML = "";	Clearing the div for prompting the user with error messages when the form is filled out incorrectly
13	document.getElementById("delete_event_id").value = event_id;	Setting the event_id for the delete an event form
14	document.getElementById("delete_parade_id").value = parade_id;	Setting the parade_id for the delete an event form
15	document.getElementById("add-event-submit").disabled = false;	Enabling the submit button for the event edit form since all fields have been populated validly
16	document.getElementById("delete-event-submit").disabled = false;	Enabling the submit button for the delete event button since all necessary fields have been populated validly
17	displayCurrentOwnerOfEvent (owner_id);	Calling the function to populate the div to display the full name of the current owner of the event
18	event_array = undefined;	Wiping the event_array dictionary so they are set to their original state where all fields are filled out validly

19	<code>event_feedback = undefined;</code>	Wiping the event_feedback dictionary so no error messages are stored if the user has previously filled out the form incorrectly
20	<code>REGEXCheckEvent(owner_id, "event_id");</code>	Setting the event_id validation state for the form to 1 indicating it is populated correctly
21	<code>document.getElementById("event-input-hendeling").innerHTML = "";</code>	Clearing the error handling prompts from any previous selection of an event
22	<code>return;</code>	
23	<code>}</code>	
24		
25	<code>function checkAreAllValuesOne(obj) {</code>	Function to check all the values in a dictionary are set to one indicating a form has been filled out correctly. Takes the dictionary as a parameter.
26	<code>for (let key in obj) {</code>	For loop to loop through every key in the dictionary
27	<code>if (obj.hasOwnProperty(key) && obj[key] !== 1) {</code>	If the value of the key in the dictionary is set to anything other than one, then return false since this indicates the form has been filled out incorrectly
28	<code>return false;</code>	Return false since the form would not pass validation
29	<code>}</code>	
30	<code>}</code>	
31	<code>return true;</code>	The for loop has run and the function has not been killed so all values are one hence return true to reflect this
32	<code>}</code>	
33		
34	<code>returnFeedbackHTML()</code> <code>function returnFeedbackHTML(obj){</code>	Function to return all error messages in a feedback dictionary. Takes the dictionary as a parameter.
35	<code>returnHTML = ""</code>	Initialising the feedback string
36	<code>for (let key in obj) {</code>	For loop to loop through every key in the dictionary
37	<code>if (obj.hasOwnProperty(key) && obj[key] !== "") {</code>	If the value contains anything other than an empty string
38	<code>returnHTML = returnHTML + obj[key];</code>	Append the value to the error message string so it can be displayed in the error handling div
39	<code>}</code>	
40	<code>}</code>	
41	<code>return returnHTML;</code>	Return the error messages
42	<code>}</code>	
43		

44	REGEXCheckEvent() function REGEXCheckEvent (str, input_to_check){	Function to validate the admin edit an event form on the left of the calendar.php page for admins
45	console.log (str);	
46	if (typeof event_array === 'undefined'){	If the dictionary to track what fields are filled out validly does not exist
47	 event_array = {event_type: 1, event_name: 1, event_start: 1, event_end: 1, final_aproval: 1, event_id: 0};	Initialise it assuming all fields are filled out correctly since they will be initially populated by the button. event_id will be set to 0 to ensure the form is initially populated by the button and not the user to prevent invalid submissions (without an event id)
48	}	
49	 if (typeof event_feedback === 'undefined'){	If the dictionary to store what feedback needs to be returned to the client does not exist
50	event_feedback = {event_type: "", event_name: "", event_start: "", event_end: "", final_aproval: ""};	Initialise it assuming no feedback is required
51	}	
52	if (document.getElementById('original_aproval').value === "1"){	If the original approval hidden input field is set to 1 indicating the event was approved before the form was populated, then
53	event_feedback["final_aproval"] = "<a>this event has already been aproved so cant be edited<a> <a>click another event to continue editing<a> ";	The form is not to be populated hence add a feedback prompt to the feedback dictionary to make the user aware of this issue
54	event_array["final_aproval"] = 0;	Set the approval state of final_aproval to 0 indicating the form is not ready for submission due to its final approval state.
55	else {	If the event is not yet approved (it can be edited)
56	if (input_to_check === "event_id") {	If the input to validate is the event id then
57	 event_array["event_id"] = 1;	Set the validation state of event id to 1 indicating the event id is ready for submission. No validation caried out here since this process is only ever called by the form population function
58	else if (input_to_check === "event_type") {	If the input to validate is the event type
59	 if (str.match("^.{255,\$}")){ event_feedback["event_type"] = "<a>event type is to long<a> ";	If the input for event type matches a regular expression that only accepts strings with 255 or more wildcard characters. Add a feedback prompt to the feedback dictionary to alert the user to this issue.
60	 event_array["event_type"] = 0;	Set the validation state of the event type to 0 indicating the event type is not ready for submission.

61	<code>} else if(str === ""){</code>	if the input for event type is an empty string, then
62	<code>event_feedback["event_type"] = "<a>event type cant empty<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue
63	<code>event_array["event_type"] = 0;</code>	Set the validation state of the event type to 0 indicating the event type is not ready for submission.
64	<code>} else{</code>	If the input for event type has passed the above 2 validation steps
65	<code>event_feedback["event_type"] = "";</code>	Clear the feedback prompts for the event type
66	<code>event_array["event_type"] = 1;</code>	Set the validation state of the event type to 1 to indicate the event type is ready for submission
67	<code>}</code>	
68	<code>} else if (input_to_check === "event_name"){</code>	If the input to validate is the event name
69	<code>if (str.match("^.{255,}\$")){ event_feedback["event_name"] = "<a>event name is to long<a>
";</code>	If the event name input doesn't match a regular expression that only accepts strings of wildcards with more than 255 characters. Add a feedback prompt for the event name to make the user aware of this issue.
70	<code>event_array["event_name"] = 0;</code>	Set the state of the event_array dictionary to reflect the fact that the event name is not ready to submit
71	<code>} else if(str === ""){ event_feedback["event_name"] = "<a>event name cant empty<a>
";</code>	If the event name is an empty string add a feedback prompt to make the user aware of this issue
72	<code>event_array["event_name"] = 0;</code>	Set the state of the event_array dictionary to reflect the fact that the event name is not ready to submit.
73	<code>} else{</code>	If the input has passed the above 2 validation conditions
74	<code>event_feedback["event_name"] = "";</code>	Clear the feedback for the event name
75	<code>event_array["event_name"] = 1</code>	Update the validation state of the event name to show the event name is ready to submit
76	<code>}</code>	
77	<code>} else if (input_to_check === "event_start"){</code>	If the input to check is the start time of the event.
78	<code>if (str === ""){</code>	If the start time is an empty string
79	<code>event_feedback["event_start"] = "<a>event start can't be empty<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue

80	<code>event_array["event_start"] = 0;</code>	Set the validation state of the event start to 0 indicating the event start is not ready for submission
81	<code>} else if (!str.match(/^\\d{2}:\\d{2}\$/)) { event_feedback["event_start"] = "<a>event start must be in the format HH:MM<a>
";</code>	If the start time does not fit a regular expression of a time format. If it doesn't then add in a prompt for the user to the feedback dictionary for start time.
82	<code>event_array["event_start"] = 0;</code>	Change the validation state of the event start time field to 0 indicating the start time field is not yet ready to submit
83	<code>} else {</code>	If the start time field has passed validation
84	<code>event_feedback["event_start"] = "";</code>	Empty the event start time feedback
85	<code>event_array["event_start"] = 1;</code>	Set the valid state of the event start time field to 1 indicating the start time field is ready to submit
86	<code>}</code>	
87	<code>} else if (input_to_check === "event_end"){</code>	If the input to check is the event finish time. This works almost identically to the start time but with slightly different prompts and variable names.
88	<code>if (str === ""){</code>	
89	<code>event_feedback["event_end"] = "<a>event end can't be empty<a>
";</code>	
90	<code>event_array["event_end"] = 0;</code>	
91	<code>} else if (!str.match(/^\\d{2}:\\d{2}\$/)) { event_feedback["event_end"] = "<a>event end must be in the format HH:MM<a>
";</code>	
92	<code>event_array["event_end"] = 0;</code>	
93	<code>} else {</code>	
94	<code>event_feedback["event_end"] = "";</code>	
95	<code>event_array["event_end"] = 1;</code>	
96	<code>}</code>	
97	<code>}</code>	
98	<code>}</code>	
99	<code>if (checkAreAllValuesOne(event_array) === true){</code>	If all the values in the event_array dictionary are set to 1 indicating all fields pass validation
100	<code>document.getElementById("add-event-submit").disabled = false;</code>	Enable the submit button
101	<code>document.getElementById("event-input-handeling").innerHTML = "";</code>	Clear the feedback div
102	<code>}else{</code>	If one or more fields are not yet validly populated
	<code>document.getElementById("add-event-submit").disabled = true;</code>	Disable the submit button

	<code>document.getElementById("event-input-handeling").innerHTML = returnFeedbackHTML(event_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
	<code>}</code>	
	<code>}</code>	

js\event_owner_form_handling.js

1	<code>showResultsSearchForOwner() function showResultsSearchForOwner(str) { console.log(str);</code>	Function for searching for an owner of an event called onkeyup by the search for owner first name
2	<code>if (str.length==0) {</code>	If the search prompt is an empty string then
3	<code>document.getElementById("livesearch_owner").innerHTML="" ;</code>	Clear the div that displays the results of the search for an owner
4	<code>document.getElementById("livesearch_owner").style.border ="0px";</code>	Set the border of the search results div to 0 so it is no longer visible
5	<code>return;</code>	Exit from the function
6	<code>}</code>	
7	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialise a http request
8	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener
9	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the status is 200 (the http response is ok).
10	<code>document.getElementById("livesearch_owner").innerHTML=this.responseText;</code>	Populate the div with the results of the search.
11	<code>document.getElementById("livesearch_owner").style.border ="1px solid #A5ACB2";</code>	Set the border of the search results div to 1px to differentiate it from the rest of the page
12	<code>}</code>	
13	<code>}</code>	
14	<code>xmlhttp.open("GET","requests/event_details_requests.php?flag=search_first_name_owner&prompt="+str,true);</code>	Point the request to the correct php page and add in the search prompt as a get request
15	<code>xmlhttp.send();</code>	Send the request
16	<code>}</code>	
17		
18	<code>resultHasBeenClickedOwner() function resultHasBeenClickedOwner(user_id) { console.log(user_id);</code>	Function to be called onclick of a search result takes user_id as a parameter to populate the owner id hidden input field for the event form
19	<code>document.getElementById("livesearch_owner").innerHTML="" ;</code>	Clear the search results div
20	<code>document.getElementById("livesearch_owner").style.border ="0px";</code>	Set the border of the search results div to 0 to hide it

21	<code>document.getElementById("owner_id").value = user_id;</code>	Set the owner id hidden input filed to the user_id of the owner that the client has clicked on
22	<code>displayCurrentOwnerOfEvent(user_id);</code>	Call the function to display the owner of the event in a readable format for the client
23	<code>document.getElementById("event_owner_search_box").value = "";</code>	Clear the search prompt from the search input field
24	<code>try {</code>	Try block for setting the validation state of the owner_id since
25	<code>REGEXCheckEvent(user_id, "owner_id");</code>	REGEXCheckEvent() is in a different JS script and varies in functionality. For the forms on the calendar and event page this is not necessary however on the add.php page it is.
26	<code>} catch{</code>	
27	<code>//do nothing</code>	If the function call has failed, then it is implied that the owner_id does not need to be validated
28	<code>}</code>	
29	<code>return;</code>	
30	<code>}</code>	
31		
32	<code>displayCurrentOwnerOfEvent()</code> <code>function displayCurrentOwnerOfEvent(owner_id) {</code>	Function to display the full name of an event owner since the owner_id is the only stored attribute of the owner in the event edit form
33	<code>if (owner_id.length==0) {</code>	If the owner_id is empty (on the add.php page or the calendar admin event edit page when the form is not populated)
34	<code>document.getElementById("display_current_owner").innerHTML = "";</code>	Clear the current contents of the display the owner div
35	<code>document.getElementById("display_current_owner").innerHTML="<a>curent owner: none selected<a>";</code>	Populate the div with html to alert the user that no owner has been selected for the event
36	<code>return;</code>	Exit from the function
37	<code>}</code>	It is now known the owner_id does exist hence details of the owner must be obtained
38	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialise a http request
39	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener
40	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the status is 200 (the http response is ok).

41	<code>let owner_info = JSON.parse(this.responseText);</code>	Convert the response to a JSON object so individual attributes of the user can be accessed
42	<code>document.getElementById("display_current_owner").innerHTML=<a>curent owner: " + owner_info.rank + " + owner_info.first_name + " " + owner_info.last_name + "<a>;</code>	Populate the div for displaying the current owner of an event with an a tag element with the full name of the current event owner
43	<code>}</code>	
44	<code>}</code>	
45	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=owner_id_info_dump&prompt="+owner_id,true);</code>	Pointing the http request to the PHP file to handle the request
46	<code>xmlhttp.send();</code>	Sending the request
47	<code>}</code>	

js\event.js

1	<code>showResultAddCadet()</code>	Function to request search results for adding a cadet to the event's register
2	<code>function showResultAddCadet(str, search_for, event_id) {</code>	
3	<code>if (str.length==0) {</code>	If the search prompt is an empty string
4	<code>document.getElementById("livesearch").innerHTML="";</code>	Clear the div that displays the results of the search
5	<code>document.getElementById("livesearch").style.border="0px";</code>	Set the border of the search results div to 0 so it is hidden from the rest of the page
6	<code>return;</code>	Exit from the function
7	<code>}</code>	
8	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialise a http request
9	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener
10	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the status is 200 (the http response is ok).
11	<code>document.getElementById("livesearch").innerHTML=this.responseText;</code>	Populate the div with the results of the search.
12	<code>document.getElementById("livesearch").style.border="1px solid #A5ACB2";</code>	Set the border of the search results div to 1px to differentiate it from the rest of the page
13	<code>}</code>	
14	<code>}</code>	

14	<pre>if (search_for == "search_first_name") {</pre>	If the search request is for the first name of the cadet, then (This allows for the function to be expanded in the future to include rank searches or last name searches)
15	<pre>xmlhttp.open("GET","requests/event_get_requests.php?flag=search_first_name&prompt="+str+"&event_id=" + event_id,true);</pre>	Point the request to the correct pup page and add in the search prompt as a get request as well as the event id for conflict management purposes
16	<pre>xmlhttp.send();</pre>	Send the request
17	<pre>}</pre>	
18	<pre>}</pre>	
19		
20	<pre>showResultDeleteCadet() function showResultDeleteCadet(str, search_for, event_id) {</pre>	Function for searching for cadets to remove from the register for the event. Called onkeyup for the delete a cadet search box. This function works almost identically to the add a search for adding a cadet. However, the destination of the search is different since a different query must be performed additionally the names of the divs' are different.
21	<pre>if (str.length==0) {</pre>	
22	<pre>document.getElementById("livesearch_delete").innerHTML="";</pre>	
23	<pre>document.getElementById("livesearch_delete").style.border="0px";</pre>	
24	<pre>return;</pre>	
25	<pre>}</pre>	
26	<pre>let xmlhttp=new XMLHttpRequest();</pre>	
27	<pre>xmlhttp.onreadystatechange=function() {</pre>	
28	<pre>if (this.readyState==4 && this.status==200) {</pre>	
29	<pre>document.getElementById("livesearch_delete").innerHTML=this.responseText;</pre>	
30	<pre>document.getElementById("livesearch_delete").style.border="1px solid #A5ACB2";</pre>	
31	<pre>}</pre>	
32	<pre>}</pre>	
33	<pre>if (search_for == "search_first_name_delete") {</pre>	
34	<pre>xmlhttp.open("GET","requests/event_get_requests.php?flag=search_first_name_delete&prompt="+str+"&event_id=" + event_id,true);</pre>	

35	<code>xmlhttp.send();</code>	
36	<code>}</code>	
37	<code>}</code>	
38		
39	<code>showResultSearchOtherCadet()</code> <pre>function showResultSearchOtherCadet(str, event_id) {</pre>	Function for searching for what cadets are doing during an event if they are on other registers. Called onkeyup for the search other cadet search box. This function works almost identically to the add a search for adding a cadet. However, the destination of the search is different since a different query must be performed additionally the names of the divs' are different.
40	<code>if (str.length==0) {</code>	
41	<code>document.getElementById("livesearch_other_cadet").innerHTML="";</code>	
42	<code>document.getElementById("livesearch_other_cadet").style.border="0px";</code>	
43	<code>return;</code>	
44	<code>}</code>	
45	<code>let xmlhttp=new XMLHttpRequest();</code>	
46	<code>xmlhttp.onreadystatechange=function() {</code>	
47	<code>if (this.readyState==4 && this.status==200) {</code>	
48	<code>console.log(this.responseText);</code>	
49	<code>document.getElementById("livesearch_other_cadet").innerHTML=this.responseText;</code>	
50	<code>document.getElementById("livesearch_other_cadet").style.border="1px solid #A5ACB2";</code>	
51	<code>}</code>	
52	<code>}</code>	
53	<code>xmlhttp.open("GET","requests/event_get_requests.php?flag=search_first_name_other_cadet&prompt="+str+"&event_id=" + event_id,true);</code>	
54	<code>xmlhttp.send();</code>	
55		
56	<code>}</code>	
57		

58	<pre> resultHasBeenClickedAdd() function resultHasBeenClickedAdd(user_id, event_id) { </pre>	Function to add cadets to the register by posting the user_id of the cadet to be added and the event_id of the event they will be added to. Used by the search results of search for a cadet to add to the register.
59	<pre>let xmlhttp=new XMLHttpRequest();</pre>	Creating the http request variable
60	<pre> xmlhttp.open("POST", "requests/event_get_requests.php?flag=add_user_id&add_user_id =" + user_id + "&event_id=" + event_id + "") </pre>	Pointing the http request to the correct PHP file and setting the parameters of the http request to their respective values
61	<pre>xmlhttp.send();</pre>	Sending the post request
62	<pre>document.getElementById("livesearch").innerHTML="";</pre>	Clearing the search results div for the search for a cadet to add to the register
63	<pre>document.getElementById("livesearch").style.border="0px";</pre>	Setting the border of the search results div to zero so it is hidden
64	<pre>document.getElementById("search_first_name").value = "";</pre>	Clearing the contents of the search prompt box
65	<pre> setTimeout(function() { </pre>	Timeout function to reload the page after 200 ms which should be enough time for the request to complete and the cadet to appear on the register form when the page is reloaded
66	<pre>window.location.reload();</pre>	Reloading the page with the exact same URL
67	<pre>}, 200);</pre>	
68	<pre>return;</pre>	Exiting from the reload function
69	<pre>}</pre>	
70		
71	<pre> resultHasBeenClickedDelete() function resultHasBeenClickedDelete(user_id, event_id) { </pre>	This function is for removing a cadet from the event's register on click of a search result for deleting a cadet. It works almost identically to the adding a cadet function however the destination of the request is slightly different and the names of the divs to display results are slightly different.
72	<pre>let xmlhttp=new XMLHttpRequest();</pre>	
73	<pre> xmlhttp.open("POST", "requests/event_get_requests.php?flag=remove_user_id&remove_u ser_id=" + user_id + "&event_id=" + event_id + "") </pre>	
74	<pre>xmlhttp.send();</pre>	

75	<code>document.getElementById("livesearch_delete").innerHTML="";</code>	
76	<code>document.getElementById("livesearch_delete").style.border="0px";</code>	
77	<code>document.getElementById("search_first_name_delete").value = "";</code>	
78	<code>setTimeout(function() {</code>	
79	<code>window.location.reload();</code>	
80	<code>}, 200);</code>	
81	<code>return;</code>	
82	<code>}</code>	
83		
84	showResultAddEquipment() <code>function showResultAddEquipment(str, event_id) {</code>	Function to request search results for adding a piece of equipment to the events equipment log.
85	<code>if (str.length==0) {</code>	If the search prompt is an empty string
86	<code>document.getElementById("livesearch_equipment-_add").innerHTML="";</code>	Clear the div that displays the results of the search
87	<code>document.getElementById("livesearch_equipment-_add").style.border="0px";</code>	Set the border of the search results div to 0 so it is hidden from the rest of the page
88	<code>return;</code>	Exit from the function
89	<code>}</code>	
90	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialise a http request
91	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener
92	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the status is 200 (the http response is ok).
93	<code>console.log(this.responseText);</code>	
94	<code>document.getElementById("livesearch_equipment-_add").innerHTML=this.responseText;</code>	Populate the div with the results of the search.
95	<code>document.getElementById("livesearch_equipment-_add").style.border="1px solid #A5ACB2";</code>	Set the border of the search results div to 1px to differentiate it from the rest of the page
96	<code>}</code>	
97	<code>}</code>	If the search request is for the first name of the cadet, then (This allows for the function to be expanded in the future to include rank searches or last name searches)

98	<code>xmlhttp.open("GET","requests/event_get_requests.php?flag=search_equipment_add&prompt="+str+"&event_id=" + event_id,true);</code>	Point the request to the correct php page and add in the search prompt as a get request as well as the event id for conflict management purposes
99	<code>xmlhttp.send();</code>	Send the request
100	<code>}</code>	
101		
102	showResultDeleteEquipment() <pre>function showResultDeleteEquipment(str, event_id){</pre>	Function for searching for equipment to remove from the equipment request log this will search through all the requests for the event no matter the state of the equipment request. Called onkeyup for the delete an equipment request search box. This function works almost identically to the showResultsAddEquipment() function. However, the destination of the get request is different since a different query must be performed additionally the names of the divs' are different.
103	<code>if (str.length == 0) {</code>	
104	<code>document.getElementById("livesearch_delete_equipment").innerHTML = "";</code>	
105	<code>document.getElementById("livesearch_delete_equipment").style.border = "0px";</code>	
106	<code>return;</code>	
107	<code>}</code>	
108	<code>let xmlhttp = new XMLHttpRequest();</code>	
109	<code>xmlhttp.onreadystatechange = function() {</code>	
110	<code>if (this.readyState == 4 && this.status == 200) {</code>	
111	<code>document.getElementById("livesearch_delete_equipment").innerHTML = this.responseText;</code>	
112	<code>document.getElementById("livesearch_delete_equipment").style.border = "1px solid #A5ACB2";</code>	
113	<code>}</code>	
114	<code>}</code>	
115	<code>xmlhttp.open("GET", "requests/event_get_requests.php?flag=search_equipment_delete&prompt=" + str + "&event_id=" + event_id, true);</code>	
116	<code>xmlhttp.send();</code>	
117	<code>}</code>	

118		
119	showResultSearchOtherEquipment() function showResultSearchOtherEquipment(str, event_id){	Function for searching for where pieces of equipment are during the event if they are on another equipment request log that clashes with the event currently open on the event.php page. Called onkeyup for the search other equipment search box. This function works almost identically to the search for equipment to add to the log. However, the destination of the search is different since a different query must be performed additionally the names of the divs' are different.
120	if (str.length == 0) {	
121	document.getElementById("livesearch_other_equipment").innerHTML = "";	
122	document.getElementById("livesearch_other_equipment").style.border = "0px";	
123	return;	
124	}	
125	let xmlhttp = new XMLHttpRequest();	
126	xmlhttp.onreadystatechange = function() {	
127	if (this.readyState == 4 && this.status == 200) {	
128	document.getElementById("livesearch_other_equipment").innerHTML = this.responseText;	
129	document.getElementById("livesearch_other_equipment").style.border = "1px solid #A5ACB2";	
130	}	
131	}	
132	xmlhttp.open("GET", "requests/event_get_requests.php?flag=search_equipment_other&prompt=" + str + "&event_id=" + event_id, true);	
133	xmlhttp.send();	
134	}	
135		

136	<pre> resultHasBeenClickedAddEquipment() function resultHasBeenClickedAddEquipment(equipment_id, event_id){ </pre>	Function to add equipment requests to the log by posting the equipment_id of the piece of equipment to be added and the event_id of the event they will be added to. Used by the search results of search for equipment to add to the log.
137	<pre>let xmlhttp = new XMLHttpRequest();</pre>	Creating the http request variable
138	<pre>xmlhttp.open("POST", "requests/event_get_requests.php?flag=add_equipment_id&add_equip ment_id=" + equipment_id + "&event_id=" + event_id);</pre>	Pointing the http request to the correct PHP file and setting the parameters of the http request to their respective values
139	<pre>xmlhttp.send();</pre>	Sending the post request
140	<pre>document.getElementById("livesearch_equipment_add").innerHTML = "";</pre>	Clearing the search results div for the search for equipment to add to the log
141	<pre>document.getElementById("livesearch_equipment- _add").style.border = "0px";</pre>	Setting the border of the search results div to zero so it is hidden
142	<pre>document.getElementById("search_equipment_name_add").value = "";</pre>	Clearing the contents of the search prompt box
143	<pre> setTimeout(function() { </pre>	Timeout function to reload the page after 200 ms. Which should be enough time for the request to complete, and the piece of equipment to appear on the equipment request log.
144	<pre>window.location.reload();</pre>	Reloading the page with the exact same URL
145	<pre>}, 200);</pre>	
146	<pre>return;</pre>	Exiting from the reload function
147		
148	<pre>}</pre>	
149		
150	<pre> resultHasBeenClickedDeleteEquipment() function resultHasBeenClickedDeleteEquipment(equipment_id, event_id){ </pre>	This function is for removing an equipment request from the equipment request log for the event, called onclick of search results from search equipment request delete. It works almost identically to the adding a cadet function however the destination of the request is slightly different and the names of the divs to display results are slightly different.

151	<code>let xmlhttp = new XMLHttpRequest();</code>	
152	<code>xmlhttp.open("POST", "requests/event_get_requests.php?flag=remove_equipment_id&remove_equipment_id=" + equipment_id + "&event_id=" + event_id);</code>	
153	<code>xmlhttp.send();</code>	
154	<code>document.getElementById("livesearch_delete_equipment").innerHTML = "";</code>	
155	<code>document.getElementById("livesearch_delete_equipment").style.border = "0px";</code>	
156	<code>document.getElementById("search_equipment_name_delete").value = "";</code>	
157	<code>setTimeout(function() {</code>	
158	<code>window.location.reload();</code>	
159	<code>}, 200);</code>	
160	<code>return;</code>	
161	<code>}</code>	
162		
163	<code>setStateOfEventApproval()</code> <code>function setStateOfEventApproval(state){</code>	Function called onload of non-approved events.
164	<code>if(document.getElementById("final_aproval") !== "null"){</code>	If the drop-down box with element id final_aproval exists, then
165	<code>document.getElementById("final_aproval").value = state;</code>	Set the value of the drop-down box to the current approval state of the event i.e. not-approved or pending since approved events will not display the edit event details form.
166	<code>}</code>	
167	<code>}</code>	
168		
169	<code>function setStateOfEquipmentRequest(state, equipment_id){</code>	Function to be repeatedly called onload for each instance of an equipment request for the event
170	<code>if(document.getElementById(equipment_id) !== "null"){</code>	if the drop-down box with the element id equipment exists then
171	<code>document.getElementById(equipment_id).value = state;</code>	Set the value of the drop-down box to display the current state of the equipment request. (since the equipment log is the only form that must have its state set in this way using the id of the equipment their will be no ID conflicts on the html page).
172	<code>}</code>	
173	<code>}</code>	

174		
175	function checkAreAllValuesOne (obj) {	
176	for (let key in obj) {	
177	if (obj. hasOwnProperty (key) && obj[key] !== 1) {	
178	return false ;	
179	}	
180	}	
181	return true ;	
182	}	
183		
184	function returnFeedbackHTML (obj){	
185	returnHTML = ""	
186	for (let key in obj) {	
187	if (obj. hasOwnProperty (key) && obj[key] !== "") {	
188	returnHTML = returnHTML + obj[key];	
189	}	
190	}	
191	return returnHTML;	
192	}	
193		
194	REGEXCheckEvent() function REGEXCheckEvent (str, input_to_check, admin){	Function to validate the modify event details form for non-approved events on the event.php page.
195	console.log (str);	
196	if (typeof event_array === 'undefined'){	If the dictionary to track what fields are filled out validly does not exist
197	event_array = {event_type: 1, event_name: 1, event_start: 1, event_end: 1, final_aproval: 1};	Initialise it assuming all fields are populated correctly since they will be populated on the server side when the page is requested.
198	}	
199	if (typeof event_feedback === 'undefined'){	If the dictionary to store what feedback needs to be returned to the client does not exist
200	event_feedback = {event_type: "", event_name: "", event_start: "", event_end: "", final_aproval: ""};	Initialise it assuming no feedback is required
201	}	
202	if (document. getElementById ("original_aproval").value === "1") {	If the original approval hidden input field is set to 1 indicating the event was approved before the form was populated, then

203	<pre>event_feedback["final_aproval"] = "<a>this event has already been approved so can't be edited";</pre>	The form is not to be populated hence add a feedback prompt to the feedback dictionary to make the user aware of this issue. Currently this feature is not used since the form will not show up for approved events. However, if this were not the case in the future this block of code ensures there is a way for the validation feature to keep track of the events approval to allow for approved events to for instance have their names changed but not their start/finish times.
204	<pre>event_array["final_aproval"] = 0;</pre>	Set the approval state of final_aproval to 0 indicating the form is not ready for submission due to its final approval state.
205	<pre>}else {</pre>	If the event is not yet approved (it can be edited)
206	<pre>if (input_to_check === "event_type"){</pre>	If the input to validate is the event type
207	<pre>if (str.match("^.{255,}\$")){ event_feedback["event_type"] = "<a>event type is to long<a>
";</pre>	If the input for event type matches a regular expression that only accepts strings with 255 or more wildcard characters. Add a feedback prompt to the feedback dictionary to alert the user to this issue.
208	<pre>event_array["event_type"] = 0;</pre>	Set the validation state of the event type to 0 indicating the event type is not ready for submission.
209	<pre>} if(str === ""){</pre>	if the input for event type is an empty string, then
210	<pre>event_feedback["event_type"] = "<a>event type cant empty<a>
";</pre>	Add a prompt to the feedback dictionary to alert the user to this issue
211	<pre>event_array["event_type"] = 0;</pre>	Set the validation state of the event type to 0 indicating the event type is not ready for submission.
212	<pre>} else{</pre>	If the input for event type has passed the above 2 validation steps
213	<pre>event_feedback["event_type"] = "";</pre>	Clear the feedback prompts for the event type

214	<code>event_array["event_type"] = 1;</code>	Set the validation state of the event type to 1 to indicate the event type is ready for submission
215	<code>}</code>	
216	<code>} else if (input_to_check === "event_name"){</code>	If the input to validate is the event name
217	<code>if (str.match("^.{255,}\$")){ event_feedback["event_name"] = "<a>event name is to long<a>
";</code>	If the event name input doesn't match a regular expression that only accepts strings of wildcards with more than 255 characters. Add a feedback prompt for the event name to make the user aware of this issue.
218	<code>event_array["event_name"] = 0;</code>	Set the state of the event_array dictionary to reflect the fact that the event name is not ready to submit
219	<code>} else if(str === ""){ event_feedback["event_name"] = "<a>event name cant empty<a>
";</code>	If the event name is an empty string add a feedback prompt to make the user aware of this issue
220	<code>event_array["event_name"] = 0;</code>	Set the state of the event_array dictionary to reflect the fact that the event name is not ready to submit.
221	<code>} else{</code>	If the input has passed the above 2 validation conditions
222	<code>event_feedback["event_name"] = "";</code>	Clear the feedback for the event name
223	<code>event_array["event_name"] = 1</code>	Update the validation state of the event name to show the event name is ready to submit
224	<code>}</code>	
225	<code>} else if (input_to_check === "event_start"){</code>	If the input to check is the start time of the event.
226	<code>if (str === ""){</code>	If the start time is an empty string
227	<code>event_feedback["event_start"] = "<a>event start can't be empty<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue
228	<code>event_array["event_start"] = 0;</code>	Set the validation state of the event start to 0 indicating the event start is not ready for submission

229	<pre> } else if (!str.match(/^\\d{2}:\\d{2}\$/)) { event_feedback["event_start"] = "<a>event start must be in the format HH:MM<a>
"; </pre>	If the start time does not fit a regular expression of a time format. If it doesn't then add in a prompt for the user to the feedback dictionary for start time.
230	<pre> event_array["event_start"] = 0; </pre>	Change the validation state of the event start time field to 0 indicating the start time field is not yet ready to submit
231	<pre> } else { </pre>	If the start time field has passed validation
232	<pre> event_feedback["event_start"] = ""; </pre>	Empty the event start time feedback
233	<pre> event_array["event_start"] = 1; </pre>	Set the valid state of the event start time field to 1 indicating the start time field is ready to submit
234	<pre> } </pre>	
235	<pre> } else if (input_to_check === "event_end"){ </pre>	If the input to check is the event finish time. This works almost identically to the start time but with slightly different prompts and variable names.
236	<pre> if (str === ""){ </pre>	
237	<pre> event_feedback["event_end"] = "<a>event end can't be empty<a>
"; </pre>	
238	<pre> event_array["event_end"] = 0; </pre>	
239	<pre> } else if (!str.match(/^\\d{2}:\\d{2}\$/)) { event_feedback["event_end"] = "<a>event end must be in the format HH:MM<a>
"; </pre>	
240	<pre> event_array["event_end"] = 0; </pre>	
241	<pre> } else { </pre>	
242	<pre> event_feedback["event_end"] = ""; </pre>	
243	<pre> event_array["event_end"] = 1; </pre>	
244	<pre> } </pre>	
245	<pre> } else if (input_to_check === "final_aproval"){ </pre>	If the input to validate is the final approval status of the event, then
246	<pre> console.log(str); </pre>	
247	<pre> if (str === "1" && admin === "0"){ </pre>	If the input to validate is 1 meaning the event is being set to approved and the admin status of the user is 0 meaning they are not an admin (don't have the permissions to approve events) then

248	<code>event_feedback["final_aproval"] = "<a>only the admin can approve an event<a>
";</code>	add a prompt to the feedback dictionary to make the client aware of this issue.
249	<code>event_array["final_aproval"] = 0;</code>	Set the validation state of the final approval input to 0 indicating the final approval is not ready for submission
250	<code>} else {</code>	For any other case permission rules are not violated so
251	<code>event_feedback["final_aproval"] = "";</code>	Clear the feedback dictionary for the final approval field
252	<code>event_array["final_aproval"] = 1;</code>	Set the validation state of final approval to 1 indicating final approval is ready for submission
253	<code>}</code>	
254	<code>}</code>	
255	<code>}</code>	
256	<code>if (checkAreAllValuesOne(event_array) === true){</code>	If all the values in the event_array dictionary are set to 1 indicating all fields pass validation
257	<code>document.getElementById("add-event-submit").disabled = false;</code>	Enable the submit button
258	<code>document.getElementById("event-input-handeling").innerHTML = "";</code>	Clear the feedback div
259	<code>}else{</code>	If one or more fields are not yet validly populated
260	<code>document.getElementById("add-event-submit").disabled = true;</code>	Disable the submit button
261	<code>document.getElementById("event-input-handeling").innerHTML = returnFeedbackHTML(event_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
262	<code>}</code>	
263	<code>}</code>	
264		
265	<code>REGEXCheckRegister()</code> <code>function REGEXCheckRegister(str, user_id){</code>	Function to validate the inputs for any cadet on the register. Called onkeyup for any present status of a cadet on the register.
266	<code>if(str === ""){</code>	If the present state of the user is an empty string then
267	<code>register_array[user_id] = 0;</code>	Set the approval status of the user_id to 0 indicating that cadets present status is not ready for submission
268	<code>let xmlhttp = new XMLHttpRequest();</code>	Create a local variable to make a http request back to the server

269	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=user_id_info_dump&prompt="+user_id,true);</code>	Point the http request to the document for retrieving the attributes of a user with a known user_id
270	<code>xmlhttp.send();</code>	Send the http request
271	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener for the http request
272	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the http status of the request is 200 meaning the server responded ok
273	<code>let userDetails = JSON.parse(this.responseText);</code>	Convert the servers response to json and assign the object to the local variable userDetails
274	<code>register_feedback[user_id] = "<a>" + userDetails.rank + " " + userDetails.first_name + " " + userDetails.last_name + " cant have an empty register box<a>
"</code>	Add feedback to the feedback array to alert the user as to the issue and alert them to which cadet the issue applies to.
275	<code>}</code>	
276	<code>}</code>	
277	<code>}else if(str !== "1" && str !== "0"){</code>	If the present status of the cadet is neither 0 or 1. (This validation condition works very similarly to the one above however a slightly different prompt is provided to the user given)
278	<code>register_array[user_id] = 0;</code>	
279	<code>let xmlhttp = new XMLHttpRequest();</code>	
280	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=user_id_info_dump&prompt="+user_id,true);</code>	
281	<code>xmlhttp.send();</code>	
282	<code>xmlhttp.onreadystatechange=function() {</code>	
283	<code>if (this.readyState==4 && this.status==200) {</code>	
284	<code>let userDetails = JSON.parse(this.responseText);</code>	
285	<code>register_feedback[user_id] = "<a>" + userDetails.rank + " " + userDetails.first_name + " " + userDetails.last_name + " must have attendance of 1 or 0<a>
"</code>	
286	<code>}</code>	
287	<code>}</code>	
288	<code>}else{</code>	At this stage in the if condition the input is either 0 or 1 and is therefore valid
289	<code>register_array[user_id] = 1;</code>	Set the validation state of the cadet to 1 indicating they are ready for submission on the register.

290	<code>register_feedback[user_id] = "";</code>	Clear any feedback prompts regarding the cadet.
291	<code>}</code>	
292	<code>if(checkAreAllValuesOne(register_array) === true){</code>	If all the values in the register_array dictionary are set to 1 indicating all fields pass validation
293	<code>document.getElementById("register-submit").disabled = false;</code>	Enable the submit button
294	<code>document.getElementById("register-input-handeling").innerHTML = "";</code>	Clear the feedback div
295	<code>}else{</code>	If one or more fields are not yet validly populated
296	<code>document.getElementById("register-submit").disabled = true;</code>	Disable the submit button
297	<code>document.getElementById("register-input-handeling").innerHTML = returnFeedbackHTML(register_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
298	<code>}</code>	
299	<code>}</code>	
300		
301	REGEXCheckEquipment() <code>function REGEXCheckEquipment(str, equipment_id, original_aproval, G4){</code>	Function to check the validation status of the equipment requests. Called onclick of the dropdown boxes.
302	<code>console.log(str);</code>	
303	<code>console.log(original_aproval);</code>	
304	<code>if (G4 === "1"){</code>	If the G4 status of the client is set to 1 meaning they are G4 i.e. they have the permissions to approve events, then
305	<code>if(str === "0" && original_aproval !== "0"){</code>	If the approval status (str) is moved to 0 and the original approval was not 0 i.e. a request that has already had a decision made about it is moved back to requested, then
306	<code>equipment_array[equipment_id] = 0;</code>	Set the validation state of the equipment request to 0 indicating the equipment request is not ready for submission since it violates the rules of the system against reversing decisions.
307	<code>equipment_feedback[equipment_id] = "<a>can't reverse an approval decision<a>
";</code>	Add an appropriate prompt to the feedback dictionary to alert the user to this fact
308	<code>} else if(str === "1" && original_aproval === "2"){</code>	If the approval status (str) is moved to 1 (approved) and the original approval was 2 (denied) then

309	<pre>equipment_array[equipment_id] = 0;</pre>	Set the validation status of the equipment request to 0 indicating the equipment request is not ready for submission since it violates the rules of the system against switching decisions.
310	<pre>equipment_feedback[equipment_id] = "<a>can't approve an already denied request<a>
";</pre>	Add an appropriate prompt to the feedback dictionary to alert the user to this fact
311	<pre>} else if(str === "2" && original_aproval === "1"){</pre>	If the approval status (str) is moved to 2 (denied) and the original approval was 1 (approved) then
312	<pre>equipment_array[equipment_id] = 0;</pre>	Set the validation status of the equipment request to 0 indicating the equipment request is not ready for submission since it violates the rules of the system against switching decisions.
313	<pre>equipment_feedback[equipment_id] = "<a>can't deny an already aproved request<a>
";</pre>	Add an appropriate prompt to the feedback dictionary to alert the user to this fact
314	<pre>}else{</pre>	For all other cases meaning 0 -> 1 or 0 -> 2 system rules are adhered to so
315	<pre>equipment_array[equipment_id] = 1;</pre>	Set the validation state of the equipment request to 1 indicating the equipment request is ready for submission
316	<pre>equipment_feedback[equipment_id] = "";</pre>	Clear any feedback prompts for the equipment request
317	<pre>}</pre>	
318	<pre>}else{</pre>	If the client doesn't have G4 privileges
319	<pre>if(str !== original_aproval){</pre>	If the state of the equipment request is being modified i.e. it is being set to anything other than the original state permission rules of the system are being violated hence
320	<pre>equipment_array[equipment_id] = 0;</pre>	Set the validation state of the equipment request to 0 indicating the form is not ready for submission
321	<pre>equipment_feedback[equipment_id] = "<a>can't change an aproval decision you are not G4<a>
";</pre>	Add an appropriate prompt to the feedback dictionary to alert the user to this issue

322	<code>} else{</code>	if the state of the equipment request is being kept the same i.e. permission rules of the system are being adhered to then
323	<code>equipment_array[equipment_id] = 1;</code>	Set the validation state of the equipment request to 1 indicating the request is ready for submission
324	<code>equipment_feedback[equipment_id] = "";</code>	Clear any feedback prompts for the equipment request
325	<code>}</code>	
326	<code>}</code>	
327	<code>if(checkAreAllValuesOne(equipment_array) === true){</code>	If all the values in the register_array dictionary are set to 1 indicating all fields pass validation
328	<code>document.getElementById("equipment-submit").disabled = false;</code>	Enable the submit button
329	<code>document.getElementById("equipment-input-handeling").innerHTML = "";</code>	Clear the feedback div
330	<code>}else{</code>	If one or more fields are not yet validly populated
331	<code>document.getElementById("equipment-submit").disabled = true;</code>	Disable the submit button
332	<code>document.getElementById("equipment-input-handeling").innerHTML = returnFeedbackHTML(equipment_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
333	<code>}</code>	
334	<code>}</code>	

js\add.js

the attribute `duty_id` has been added to the `event_array` since this code was copied it additionally an if statement for setting `event_array("duty_id")` to 1 has been added to this function. These code additions would be around line number 244 and 249 respectively.

1	<code>showResutsSearchForUserFirstName() function showResutsSearchForUserFirstName(str) { if (str.length==0) {</code>	Function to process search prompts from the admin for users to edit. If the search prompt is empty.
2	<code>document.getElementById("livesearch_first_name").innerHTML="";</code>	Clear the search results div
3	<code>document.getElementById("livesearch_first_name").style.border="0px";</code>	set the border of the search div to 0 so it takes up no space since there are no results
4	<code>return;</code>	Exit the function
5	<code>}</code>	
6	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialise a http request
7	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener

8	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the status is 200 (the http response is ok).
9	<code>document.getElementById("livesearch_first_name").innerHTML=this.responseText;</code>	Populate the div with the results of the search.
10	<code>document.getElementById("livesearch_first_name").style.border="1px solid #A5ACB2";</code>	Set the border of the search results div to 1px to differentiate it from the rest of the page
11	<code>}</code>	
12	<code>}</code>	
13	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=search_first_name_user&prompt="+str,true);</code>	Point the http request to the correct PHP page to process the search
14	<code>xmlhttp.send();</code>	Send the request
15	<code>}</code>	
16		
17	showResultsSearchForUserLastName() <code>function showResultsSearchForUserLastName(str) { if (str.length==0) {</code>	Function to process searching for users to edit by their last name <u>This works almost identically to the first name search but makes a slightly different http request</u>
18	<code>document.getElementById("livesearch_last_name").innerHTML="";</code>	
19	<code>document.getElementById("livesearch_last_name").style.border="0px";</code>	
20	<code>return;</code>	
21	<code>}</code>	
22	<code>let xmlhttp=new XMLHttpRequest();</code>	
23	<code>xmlhttp.onreadystatechange=function() {</code>	
24	<code>if (this.readyState==4 && this.status==200) {</code>	
25	<code>document.getElementById("livesearch_last_name").innerHTML=this.responseText;</code>	
26	<code>document.getElementById("livesearch_last_name").style.border="1px solid #A5ACB2";</code>	
27	<code>}</code>	
28	<code>}</code>	
29	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=search_last_name_user&prompt="+str,true);</code>	
30	<code>xmlhttp.send();</code>	
31	<code>}</code>	
32		
33	ShowResultsSearchForParade() <code>function ShowResultsSearchForParade(str) { if (str.length==0) {</code>	Function to search for parades by their name to populate the add an event form. Called onkeyup by the search parade input box. If the search prompt is empty.

34	<code>document.getElementById("livesearch_parade_id-").innerHTML="";</code>	Clear the content of the search results box
35	<code>document.getElementById("livesearch_parade_id-").style.border="0px";</code>	Set the border of the search results box to zero.
36	<code>return;</code>	Exit from the function
37	<code>}</code>	
38	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialise a http request
39	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener
40	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready state is 4 meaning the request is complete and the status is 200 (the http response is ok).
41	<code>document.getElementById("livesearch_parade_id-").innerHTML=this.responseText;</code>	Populate the div with the results of the search.
42	<code>document.getElementById("livesearch_parade_id-").style.border="1px solid #A5ACB2";</code>	Set the border of the search results div to 1px to differentiate it from the rest of the page
43	<code>}</code>	
44	<code>}</code>	
45	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=search_parade_name&prompt="+str,true);</code>	Point the http request to the correct PHP page to process the search
46	<code>xmlhttp.send();</code>	Send the request
47	<code>}</code>	
48		
49	ResultHasBeenClickedParade() <code>function ResultHasBeenClickedParade(parade_id, parade_date, parade_name) { document.getElementById("livesearch_parade_id-").innerHTML="";</code>	Function to populate the add an event form with a parade_id called onclick of a search result from search for parade by name. Clear the div displaying the results of the search.
50	<code>document.getElementById("livesearch_parade_id-").style.border="0px";</code>	Set the border of the search results box to zero since there are no more results
51	<code>document.getElementById("parade_id_search_box").value = "";</code>	Clear the search prompt box
52	<code>document.getElementById("parade_id").value = parade_id;</code>	Set the value of the parade_id of the add an event form
53	<code>document.getElementById("display_current_parade").innerHTML = "current parade: " + parade_name + " " + parade_date;</code>	Populate the div to display the current selected parade with html displaying the parade name and parade date
54	<code>REGEXCheckEvent(parade_id, "parade_id");</code>	Call the validation function to update the state of the forms validation
55	<code>return;</code>	Exit from the function
56	<code>}</code>	

57	showResutsSearchForDuty() <code>function showResutsSearchForDuty(str){</code>	Function to show the search results when a duty cadet is being searched for
58	<code>if (str.length === 0){</code>	If the length of the search prompt is 0 characters i.e. the prompt is empty
59	<code>document.getElementById("livesearch_duty").innerHTML="";</code>	Clear the search results from the search results box
60	<code>document.getElementById("livesearch_duty").style.border="0px";</code>	Set the border of the search results box to 0px so its no longer showing
61	<code>return;</code>	Return i.e. exit from the function since there is nothing more to do
62	<code>}</code>	
63	<code>let xmlhttp=new XMLHttpRequest();</code>	Create a http request object
64	<code>xmlhttp.onreadystatechange=function() {</code>	creating a state change listener that will execute when the ready state of the request changes
65	<code>if (this.readyState===4 && this.status===200) {</code>	If the ready state is 4 i.e. the request is complete, and the http status code is 200 (the server has responded ok)
66	<code>document.getElementById("livesearch_duty").innerHTML=this.responseText;</code>	Populate the search results div with the response text from the server
67	<code>document.getElementById("livesearch_duty").style.border="1px solid #A5ACB2";</code>	Set the border of the search results div to 1px and solid to differentiate it from the rest of the page
68	<code>}</code>	
69	<code>}</code>	
70	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=search_duty&prompt="+str,true);</code>	Pointing the http request to the correct site and populating the parameters
71	<code>xmlhttp.send();</code>	Sending the http request
72	<code>}</code>	
73		
74	resultHasBeenClickedDuty() <code>function resultHasBeenClickedDuty(user_id){</code>	Function to execute when a search result for the duty cadet is clicked
75	<code>let xmlhttp=new XMLHttpRequest();</code>	Creating a http request object
76	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=user_id_info_dump&prompt="+user_id,true);</code>	Pointing the http request to the correct site and setting its parameters
77	<code>xmlhttp.send();</code>	Sending the http request
78	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener for the http request object
79	<code>if (this.readyState===4 && this.status===200) {</code>	If the ready state of the request is 4 i.e. the request is complete, and the http response conde is 200 i.e. the server has responded http ok

80	<code>let userDetails = JSON.parse(this.responseText);</code>	Convert the response from the server into a JSON object stored as userDetails
81	<code>document.getElementById("livesearch_duty").innerHTML="";</code>	Clear the contents of the search results div
82	<code>document.getElementById("livesearch_duty").style.border="0px";</code>	Set the border of the search results div to 0px so it is no longer visible
83	<code>document.getElementById("event_duty_search_box").value = "";</code>	Clear the search prompt from the search box
84	<code>document.getElementById("duty_id").value = user_id;</code>	Set the hidden filed duty_id to the user_id of the cadet that has been selected as the duty cadet
85	<code>document.getElementById("display_current_duty").innerHTML = "<a>current duty cadet: " + userDetails.rank + " " + userDetails.first_name + " " + userDetails.last_name + "<a>";</code>	Set the html in the div for displaying the duty cadet to an a tag that displays the full name of the duty cadet
86	<code>REGEXCheckEvent(user_id, "duty_id");</code>	Call the validation function for the create a new event form so it knows a duty cadet has been set so the duty cadet is ready for submission
87	<code>}</code>	
88	<code>}</code>	
89	<code>}</code>	
90		
91	<code>resultHasBeenClickedUser()</code> <code>function resultHasBeenClickedUser(user_id) {</code> <code>document.getElementById("livesearch_first_name").innerHTML="";</code>	Function to populate the edit a user form with the details of the user. Called onclick of a search result from search user by first/last name. Clearing the search results div
92	<code>document.getElementById("livesearch_first_name").style.border="0px";</code>	Setting the border of the search results div to zero since there are no results to display
93	<code>document.getElementById("input_search_first_name").value = "";</code>	Clear the first name search prompt input box
94	<code>document.getElementById("livesearch_last_name").innerHTML="";</code>	Clearing the search results div for last name
95	<code>document.getElementById("livesearch_last_name").style.border="0px";</code>	Setting the border of the search results div to zero since there are no results to display
96	<code>document.getElementById("input_search_last_name").value = "";</code>	Clearing the last name search prompt input box
97	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialising a http request variable
98	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=user_id_info_dump&prompt="+user_id,true);</code>	Point the http request to the correct PHP page to process the request
99	<code>xmlhttp.send();</code>	Send the request

100	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a ready state change listener for the http request
101	<code>if (this.readyState==4 && this.status==200) {</code>	If the ready status of the request is 4 indicating it is complete and the http code is 200 (the servers response was ok)
102	<code>let userDetails = JSON.parse(this.responseText);</code>	Convert the servers response to a JSON object
103	<code>console.log(userDetails);</code>	
104	<code>document.getElementById("modify_user_id").value = userDetails.user_id;</code>	Set the hidden field containing the user_id to the value in the response
105	<code>document.getElementById("modify_email").value = userDetails.email;</code>	Set the email to the value in the response
106	<code>document.getElementById("modify_password").value = userDetails.password;</code>	Set the password to the value in the response
107	<code>document.getElementById("modify_first_name").value = userDetails.first_name;</code>	Set the first name to the value in the response
108	<code>document.getElementById("modify_last_name").value = userDetails.last_name;</code>	Set the last name to the value in the response
109	<code>document.getElementById("modify_DOB").value = userDetails.DOB;</code>	Set the date of birth to the value in the response
110	<code>document.getElementById("modify_gender").value = userDetails.gender;</code>	Set the gender to the value in the response
111	<code>document.getElementById("modify_rank").value = userDetails.rank;</code>	Set the rank to the value in the response
112	<code>document.getElementById("modify_active").value = userDetails.active;</code>	Set the active status of the user to the value in the response
113	<code>document.getElementById("modify_admin").value = userDetails.admin;</code>	Set the admin status of the user to the value in the response
114	<code>document.getElementById("modify_G4").value = userDetails.G4;</code>	Set the G4 status of the user to the value in the response
115	<code>document.getElementById("modify-user-submit").disabled = false;</code>	Enable the form to be populated
116	<code>REGEXCheckModifyUser(userDetails.user_id, "user_id");</code>	Update the state of the forms validation by calling the validation function
117	<code>}</code>	
118	<code>}</code>	
119	<code>return;</code>	Exit from the function
120	<code>}</code>	
121		
122	<code>showResutsSearchForEquipmentByName()</code> <code>function showResutsSearchForEquipmentByName(str){ if (str.length==0) {</code>	Function to be called onkeyup for the search equipment by name input field. If the input string is empty

123	<code>document.getElementById("livesearch_equipment-_name").innerHTML="";</code>	Clear the search results div
124	<code>document.getElementById("livesearch_equipment-_name").style.border="0px";</code>	Set the border for the search results div to zero
125	<code>return;</code>	Exit from the function
126	<code>}</code>	
127	<code>let xmlhttp=new XMLHttpRequest();</code>	Creating a html request variable
128	<code>xmlhttp.onreadystatechange=function() {</code>	Block of code to be called when the state of the response changes
129	<code>if (this.readyState==4 && this.status==200) {</code>	Checking the ready state is 4 meaning completed and the status is 200 meaning the server has responded ok as the http code
130	<code>document.getElementById("livesearch_equipment-_name").innerHTML=this.responseText;</code>	Populating the div to display the results of the livesearch with the results of the http request
131	<code>document.getElementById("livesearch_equipment-_name").style.border="1px solid #A5ACB2";</code>	Setting the border of the search results div to 1px to differentiate it from the rest of the page
132	<code>}</code>	
133	<code>}</code>	
134	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=search_equipment_name&prompt="+str,true);</code>	Point the http request to the correct PHP page to process the request
135	<code>xmlhttp.send();</code>	Send the request
136	<code>}</code>	
137		
138	showResutsSearchForEquipmentByLocation() <code>function showResutsSearchForEquipmentByLocation(str){ if (str.length==0) {</code>	Function to be called onkeyup for the search equipment by location input box. Works almost exactly the same as the search equipment by name function but the request is slightly different and the names of the divs to be populated are slightly different.
139	<code>document.getElementById("livesearch_equipment-_location").innerHTML="";</code>	
140	<code>document.getElementById("livesearch_equipment-_location").style.border="0px";</code>	
141	<code>return;</code>	
142	<code>}</code>	
143	<code>let xmlhttp=new XMLHttpRequest();</code>	
144	<code>xmlhttp.onreadystatechange=function() {</code>	
145	<code>if (this.readyState==4 && this.status==200) {</code>	
146	<code>document.getElementById("livesearch_equipment-_location").innerHTML=this.responseText;</code>	

147	<code>document.getElementById("livesearch_equipment-_location").style.border="1px solid #A5ACB2";</code>	
148	<code>}</code>	
149	<code>}</code>	
150	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=search_equipment_location&prompt="+str,true);</code>	
151	<code>xmlhttp.send();</code>	
152	<code>}</code>	
153		
154	resultHasBeenClickedEquipment() <code>function resultHasBeenClickedEquipment(equipment_id){</code> <code>document.getElementById("livesearch_equipment-_name").innerHTML="";</code>	Function to be called when a search result for equipment is clicked. Equipment id is passed as a parameter so a request for the details of the equipment can later be made. Clearing the div that displays the results of the search by name.
155	<code>document.getElementById("livesearch_equipment-_name").style.border="0px";</code>	Setting the border of the div that displays the results of the search by name to zero
156	<code>document.getElementById("input_search_equipment_name").value = "";</code>	Clearing the search box for equipment by name
157	<code>document.getElementById("livesearch_equipment-_location").innerHTML="";</code>	Clearing the div that displays the results of the search by location
158	<code>document.getElementById("livesearch_equipment-_location").style.border="0px";</code>	Setting the border of the div to display results of the search by location to zero
159	<code>document.getElementById("input_search_equipment_location").value = "";</code>	Clearing the search box for equipment by location
160	<code>document.getElementById("modify-equipment-submit").disabled = false;</code>	Enabling the form submit button since the form will be validly populated
161	<code>let xmlhttp=new XMLHttpRequest();</code>	Initialising a http request
162	<code>xmlhttp.open("GET","requests/add_get_requests.php?flag=equipment_id_info_dump&prompt="+equipment_id,true);</code>	Point the http request to the correct PHP page to process the request
163	<code>xmlhttp.send();</code>	Sending the http request
164	<code>xmlhttp.onreadystatechange=function() {</code>	Creating a function to listen out for the state change of the request
165	<code>if (this.readyState==4 && this.status==200) {</code>	When the ready state is 4 meaning the request has completed and the http status is 200 meaning ok.
166	<code>let equipmentDetails = JSON.parse(this.responseText);</code>	Converting the response to JSON format
167	<code>console.log(equipmentDetails);</code>	
168	<code>document.getElementById("modify_equipment_id").value = equipmentDetails.equipment_id;</code>	Setting the equipment_id hidden field to the equipment_id.

169	<code>document.getElementById("modify_equipment_name").value = equipmentDetails.name;</code>	Setting the value of the equipment name input field
170	<code>document.getElementById("modify_equipment_location").value = equipmentDetails.location;</code>	Setting the value of the equipment location input field
171	<code>document.getElementById("modify_equipment_description").value = equipmentDetails.description;</code>	Setting the value of the equipment description input field
172	<code>REGEXCheckModifyEquipment(equipmentDetails.equipmentDetails, "equipment_id")</code>	Calling the validation function for modify equipment and updating the state of the equipment_id to show it is populated
173	<code>document.getElementById("modify-user-submit").disabled = false;</code>	Enabling the submit button since the form is now filled out validly
174	<code>}</code>	
175	<code>}</code>	
176	<code>return;</code>	
177	<code>}</code>	
178		
179	<code>checkAreAllValuesOne()</code> <code>function checkAreAllValuesOne(obj) {</code>	Function to check all the values in a dictionary are set to one indicating a form has been filled out correctly. Takes the dictionary as a parameter.
180	<code>for (let key in obj) {</code>	For loop to loop through every key in the dictionary
181	<code>if (obj.hasOwnProperty(key) && obj[key] !== 1) {</code>	If the value of the key in the dictionary is set to anything other than one then return false since this indicates the form has been filled out incorrectly
182	<code>return false;</code>	Return false since the form would not pass validation
183	<code>}</code>	
184	<code>}</code>	
185	<code>return true;</code>	The for loop has run and the function has not been killed so all values are one hence return true to reflect this
186	<code>}</code>	
187		
188	<code>returnFeedbackHTML()</code> <code>function returnFeedbackHTML(obj) {</code>	Function to return all error messages in a feedback dictionary. Takes the dictionary as a parameter.
189	<code>let returnHTML = "";</code>	Initialising the feedback string
190	<code>for (let key in obj) {</code>	For loop to loop through every key in the dictionary
191	<code>if (obj.hasOwnProperty(key) && obj[key] !== "") {</code>	If the value contains anything other than an empty string

192	<code>returnHTML = returnHTML + obj[key];</code>	Append the value to the error message string so it can be displayed in the error handling div
193	<code>}</code>	
194	<code>}</code>	
195	<code>return returnHTML;</code>	Return the error messages
196	<code>}</code>	
197		
198	REGEXCheckParade() <code>function REGEXCheckParade(str, input_to_check){ if (typeof parade_array === 'undefined'){</code>	Function to validate the add new parade form. If the parade_array dictionary does not exists
199	<code>parade_array = {date: 0, start: 0, end: 0, parade_name: 0};</code>	Initialise the parade_array dictionary and set all values to zero since the fields will be individually populated by the user and must be individually validated.
200	<code>}if (typeof parade_feedback === 'undefined'){</code>	If the parade_feedback dictionary does not exist.
201	<code>parade_feedback = {date: "", start: "", end: "", parade_name: ""};</code>	Initialise the parade_feedback dictionary and set the feedback for each field to an empty string since the required feedback is not yet known.
202	<code>}if (input_to_check === "date") {</code>	If the input to validate is the date
203	<code>if (!str.match(/^\d{4}-\d{2}-\d{2}\$/)) { parade_feedback["date"] = "<a>Invalid date format (YYYY-MM-DD)<a>
";</code>	Check the date fits a regular expression of a date format. If it doesn't then add in a prompt for the user to the feedback dictionary for date.
204	<code>parade_array["date"] = 0;</code>	Set the valid state of the date field to zero indicating it does not yet have a valid value.
205	<code>} else {</code>	If the input passes the validation
206	<code>parade_feedback["date"] = "";</code>	Empty the feedback prompts for the date field
207	<code>parade_array["date"] = 1;</code>	Set the valid state of the date field to 1 indicating the field is validly populated and can be submitted
208	<code>}</code>	
209	<code>} else if (input_to_check === "start") {</code>	If the input to validate is the start time of the parade

210	<pre>if (!str.match(/^d{2}:d{2}\$/)) { parade_feedback["start"] = "<a>Invalid time format (HH:MM)<a>
";</pre>	If the start time does not fit a regular expression of a time format. If it doesn't then add in a prompt for the user to the feedback dictionary for start time.
211	<pre>parade_array["start"] = 0;</pre>	Change the validation state of the start time field to 0 indicating the start time field is not yet ready to submit
212	<pre>} else {</pre>	If the start time field has passed validation
213	<pre>parade_feedback["start"] = "";</pre>	Empty the start time feedback
214	<pre>parade_array["start"] = 1;</pre>	Set the valid state of the start time field to 1 indicating the start time field is ready to submit
215	<pre>}</pre>	
216	<pre>} else if (input_to_check === "end") {</pre>	If the input to validate is the end time This section works almost exactly the same as start time but with different prompts and variable names but the same validation process.
217	<pre>if (!str.match(/^d{2}:d{2}\$/)) { parade_feedback["end"] = "<a>Invalid time format (HH:MM)<a>
";</pre>	
218	<pre>parade_array["end"] = 0;</pre>	
219	<pre>} else {</pre>	
220	<pre>parade_feedback["end"] = "";</pre>	
221	<pre>parade_array["end"] = 1;</pre>	
222	<pre>}</pre>	
223	<pre>} else if (input_to_check === "parade_name") {</pre>	If the input to validate is the parade name
224	<pre>if (str.match("^.{255,}\$")){ parade_feedback["parade_name"] = "<a>parade name is to long<a>
";</pre>	If the number or characters in the parade_name box is greater than 255 then add in a prompt for the user to the feedback dictionary for parade name.
225	<pre>parade_array["parade_name"] = 0;</pre>	Set the validation state of the parade name to 0 to indicate the parade name is too long to submit
226	<pre>} else if(str === ""){ parade_feedback["parade_name"] = "<a>parade name cant empty<a>
";</pre>	If the parade name is empty, then prompt the user that it can't be empty by adding the prompt to the feedback dictionary
227	<pre>parade_array["parade_name"] = 0;</pre>	Set the validation state of the parade name to 0 to indicate the parade name is not ready to submit

228	<code>} else {</code>	If the parade name has passed the above 2 validation conditions
229	<code>parade_feedback["parade_name"] = "";</code>	Empty the feedback for the parade name.
230	<code>parade_array["parade_name"] = 1;</code>	Set the validation state of the parade name to 1 indicating the parade name is ready to submit
231	<code>}</code>	
232	<code>}</code>	
233	<code>if (checkAreAllValuesOne(parade_array) === true){</code>	If all the values in the parade_array dictionary are set to 1 indicating all fields pass validation
234	<code>document.getElementById("add-parade-submit").disabled = false;</code>	Enable the submit button
235	<code>document.getElementById("parade-input-handeling").innerHTML = "";</code>	Clear the feedback div
236	<code>}else{</code>	If one or more fields are not yet validly populated
237	<code>document.getElementById("add-parade-submit").disabled = true;</code>	Disable the submit button
238	<code>document.getElementById("parade-input-handeling").innerHTML = returnFeedbackHTML(parade_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
239	<code>}</code>	
240	<code>}</code>	
241		
242	<code>REGEXCheckEvent()</code> <code>function REGEXCheckEvent(str, input_to_check){</code>	Function to check if the event form is filled out correctly. Called on change in input(onkeyup or onclick) for all fields in the event form except for the search fields
243	<code>if (typeof event_array === 'undefined'){</code>	If the event_array dictionary doesn't exist
244	<code>event_array = {event_type: 0, event_name: 0, event_start: 0, event_end: 0, parade_id: 0, owner_id: 0};</code>	Initialise it and set all values to 0 since all fields must be manually filled out by the user (except parade_id and owner_id which are set to valid by function calls when their respective results are clicked)
245	<code>}</code>	
246	<code>if (typeof event_feedback === 'undefined'){</code>	If the event_feedback dictionary doesn't exist

247	event_feedback = {event_type: "", event_name: "", event_start: "", event_end: ""};	Initialise it and set all values to empty strings since the feedback is unknown. No feedback for parade_id and owner_id since the user is prompted by the divs that show the owner and parade individually.
248	}	
249	if (input_to_check === "event_type") {	If the input to check is the event type
250	if (str.match("^.{255,\$}")){ event_feedback["event_type"] = "<a>event type is to long<a> ";	If the input for event type doesn't match a regular expression that only accepts strings longer than 255 wildcard characters. Then add a feedback prompt for event type to make the user aware of the issue.
251	event_array["event_type"] = 0;	Set the state of the event_array dictionary to reflect the fact that the event type is not yet ready to submit
252	} else if(str === ""){	If the event type is empty
253	event_feedback["event_type"] = "<a>event type cant empty<a> ";	Add a feedback prompt for event type to make the user aware of the issue
254	event_array["event_type"] = 0;	Set the state of the event_array dictionary to reflect the fact that the event type is not yet ready to submit
255	} else{	If the input has passed the validation checks above
256	event_feedback["event_type"] = "";	Empty the feedback prompts for the event type
257	event_array["event_type"] = 1;	Set the state of the event_array dictionary to reflect the fact that the event type is ready to submit
258	}	
259	} else if (input_to_check === "event_name"){	If the input to validate is the event name
260	if (str.match("^.{255,\$}")){ event_feedback["event_name"] = "<a>event name is to long<a> ";	If the event name input doesn't match a regular expression that only accepts strings of wildcards with more than 255 characters. Add a feedback prompt for the event name to make the user aware of this issue.
261	event_array["event_name"] = 0;	Set the state of the event_array dictionary to reflect the fact that the event name is not ready to submit
262	} else if(str === ""){ event_feedback["event_name"] = "<a>event name cant empty<a> ";	If the event name is an empty string add a feedback prompt to make the user aware of this issue

263	<code>event_array["event_name"] = 0;</code>	Set the state of the event_array dictionary to reflect the fact that the event name is not ready to submit.
264	<code>} else{</code>	If the input has passed the above 2 validation conditions
265	<code>event_feedback["event_name"] = "";</code>	Clear the feedback for the event name
266	<code>event_array["event_name"] = 1</code>	Update the validation state of the event name to show the event name is ready to submit
267	<code>}</code>	
268	<code>}else if (input_to_check === "event_start") {</code>	If the input to check is the start time of the event.
269	<code>if (!str.match(/^d{2}:d{2}\$/)) { event_feedback["event_start"] = "<a>Invalid time format (HH:MM)<a>
";</code>	If the start time does not fit a regular expression of a time format. If it doesn't then add in a prompt for the user to the feedback dictionary for start time.
270	<code>event_array["event_start"] = 0;</code>	Change the validation state of the event start time field to 0 indicating the start time field is not yet ready to submit
271	<code>} else {</code>	If the start time field has passed validation
272	<code>event_feedback["event_start"] = "";</code>	Empty the event start time feedback
273	<code>event_array["event_start"] = 1;</code>	Set the valid state of the event start time field to 1 indicating the start time field is ready to submit
274	<code>}</code>	
275	<code>} else if (input_to_check === "event_end") {</code>	If the input to check is the event finish time. This works almost identically to the start time but with slightly different prompts and variable names.
276	<code>if (!str.match(/^d{2}:d{2}\$/)) { event_feedback["event_end"] = "<a>Invalid time format (HH:MM)<a>
";</code>	
277	<code>event_array["event_end"] = 0;</code>	
278	<code>} else {</code>	
279	<code>event_feedback["event_end"] = "";</code>	
280	<code>event_array["event_end"] = 1;</code>	
281	<code>}</code>	
282	<code>} else if (input_to_check === "parade_id") {</code>	If the input to validate is the parade_id (this has been called by parade has been clicked function)

283	<code>event_array["parade_id"] = 1;</code>	Set the valid state of the parade id to show the parade id is ready to submit
284	<code>} else if (input_to_check === "owner_id") {</code>	If the input to validate it the owner_id (this has been called by owner has been clicked function)
285	<code>event_array["owner_id"] = 1;</code>	Set the valid state of the owner id to show the owner id is ready to submit
286	<code>}</code>	
287	<code>if (checkAreAllValuesOne(event_array) === true){</code>	If all the values in the event_array dictionary are set to 1 indicating all fields pass validation
288	<code>document.getElementById("add-event-submit").disabled = false;</code>	Enable the submit button
289	<code>document.getElementById("event-input-handeling").innerHTML = "";</code>	Clear the feedback div
290	<code>}else{</code>	If one or more fields are not yet validly populated
291	<code>document.getElementById("add-event-submit").disabled = true;</code>	Disable the submit button
292	<code>document.getElementById("event-input-handeling").innerHTML = returnFeedbackHTML(event_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
293	<code>}</code>	
294	<code>}</code>	
295		
296	REGEXCheckAddUser() <code>function REGEXCheckAddUser(str, input_to_check){ if (typeof add_user_array === 'undefined'){</code>	Function called onkeyup (or onclick) for all input fields in add a user. If the add_user_array doesn't exist
297	<code>add_user_array = {email: 0, password: 0, first_name: 0, last_name: 0, DOB: 0, gender: 0, rank: 0};</code>	Initialise it and set all values to 0 since all fields must be manually filled out by the user and individually validated
298	<code>}</code>	
299	<code>if (typeof add_user_feedback === 'undefined'){</code>	If the add_user_feedback dictionary doesn't exist
300	<code>add_user_feedback = {email: "", password: "", first_name: "", last_name: "", DOB: "", gender: "", rank: ""};</code>	Initialise it and set all values to empty strings since all fields must be manually filled out by the user and will be populated with error messages as and when needed
301	<code>}</code>	
302	<code>if (input_to_check === "email") {</code>	If the input to validate is the email

303	<pre>if (str.length > 255){ add_user_feedback["email"] = "<a>email is to long<a>
";</pre>	If the email is longer than 255 characters, then add an appropriate prompt to the feedback dictionary to alert the user to this
304	<pre>add_user_array["email"] = 0;</pre>	Change the validation state of the email field to 0 indicating the email is not ready for submission
305	<pre>} else if(str === ""){ add_user_feedback["email"] = "<a>email cant be empty<a>
";</pre>	If the email is an empty string, then add an appropriate prompt to the feedback dictionary to alert the user to this
306	<pre>add_user_array["email"] = 0;</pre>	Change the validation state of the email field to 0 to indicate the email is not ready for submission
307	<pre>} else if (!str.match("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$")) { add_user_feedback["email"] = "<a>Invalid email format<a>
";</pre>	If the email does not fit a regular expression that matches to the general format of an email. Then add an appropriate prompt to alert the user to this issue.
308	<pre>add_user_array["email"] = 0;</pre>	Change the validation state of the email field to 0 to indicate the email is not ready for submission
309	<pre>} else {</pre>	The input string to email has passed validation so
310	<pre>add_user_feedback["email"] = "";</pre>	Clear the feedback messages for email
311	<pre>add_user_array["email"] = 1;</pre>	Set the validation state of email to 1 to indicate email is ready to submit
312	<pre>}</pre>	
313	<pre>} else if (input_to_check === "password") {</pre>	If the input to validate is password
314	<pre>if (str.length < 8) {</pre>	If the password is less than 8 characters long
315	<pre>add_user_feedback["password"] = "<a>Password must be at least 8 characters long<a>
";</pre>	Add a prompt to the feedback dictionary to alert the user that the password must be 8 or more characters
316	<pre>add_user_array["password"] = 0;</pre>	Change the validation state of the password field to 0 to indicate the password is not ready for submission
317	<pre>} else if(str.length > 255){</pre>	If the password is more than 255 characters long i.e. it can't be stored
318	<pre>add_user_feedback["password"] = "<a>Password is to long long<a>
";</pre>	Add a prompt to the feedback dictionary to alert the user that the password must be less than 255 characters

319	<code>add_user_array["password"] = 0;</code>	Change the validation state of the password field to 0 to indicate the password is not ready for submission
320	<code>}else{</code>	The input string to password has passed validation so
321	<code>add_user_feedback["password"] = "";</code>	Clear the feedback messages for password
322	<code>add_user_array["password"] = 1;</code>	Set the validation state of password to 1 to indicate password is ready to submit
323	<code>}</code>	
324	<code>} else if (input_to_check === "first_name") {</code>	If the input to validate is first name
325	<code>if (str.length > 255) {</code>	If the length of the first name is greater than 255 characters
326	<code>add_user_feedback["first_name"] = "<a>First name is too long<a>
";</code>	Add a prompt to the feedback dictionary to prompt the user that the first name can't be this long
327	<code>add_user_array["first_name"] = 0;</code>	Change the validation state of the first name field to 0 to indicate the first name is not ready for submission
328	<code>} else if (str === "") {</code>	If the first name is an empty string
329	<code>add_user_feedback["first_name"] = "<a>First name is required<a>
";</code>	Add a prompt to the feedback array to alert the user that the first name can't be empty
330	<code>add_user_array["first_name"] = 0;</code>	Change the validation state of the first name field to 0 to indicate the first name is not ready for submission
331	<code>} else {</code>	The input string to first name has passed validation so
332	<code>add_user_feedback["first_name"] = "";</code>	Clear the feedback messages for first name
333	<code>add_user_array["first_name"] = 1;</code>	Set the validation state of first name to 1 to indicate first name is ready to submit
334	<code>}</code>	
335	<code>} else if (input_to_check === "last_name") {</code>	If the input to validate is last name
336	<code>if (str.length > 255) {</code>	If the last name is longer than 255 characters
337	<code>add_user_feedback["last_name"] = "<a>Last name is too long<a>
";</code>	Add a prompt to the feedback array to alert the user the last name is too long
338	<code>add_user_array["last_name"] = 0;</code>	Change the validation state of the last name field to 0 to indicate the last name is not ready for submission
339	<code>} else if (str === "") {</code>	If the last name is an empty string

340	<code>add_user_feedback["last_name"] = "<a>Last name is required<a>
";</code>	Add a prompt to the feedback array to alert the user the last name can't be empty
341	<code>add_user_array["last_name"] = 0;</code>	Change the validation state of the last name field to 0 to indicate the last name is not ready for submission
342	<code>} else {</code>	The input string to last name has passed validation so
343	<code>add_user_feedback["last_name"] = "";</code>	Clear the feedback messages for last name
344	<code>add_user_array["last_name"] = 1;</code>	Set the validation state of last name to 1 to indicate last name is ready to submit
345	<code>}</code>	
346	<code>} else if (input_to_check === "DOB") {</code>	If the input to validate is date of birth
347	<code>if (!str.match(/^d{4}-d{2}-d{2}\$/)) {</code>	Checking if the date of birth entered doesn't match a regular expression that only matches dates of birth that are general format for a date
348	<code>add_user_feedback["DOB"] = "<a>Invalid date format (YYYY-MM-DD)<a>
";</code>	Since the input's format is incorrect add a prompt to the feedback array to prompt the user appropriately
349	<code>add_user_array["DOB"] = 0;</code>	Change the validation state of the date of birth field to 0 to indicate the date of birth is not ready for submission
350	<code>} else {</code>	The input string for date of birth has passed validation so
351	<code>add_user_feedback["DOB"] = "";</code>	Clear the feedback messages for date of birth
352	<code>add_user_array["DOB"] = 1;</code>	Set the validation state of date of birth to 1 to indicate date of birth is ready to submit
353	<code>}</code>	
354	<code>} else if (input_to_check === "gender") {</code>	If the input to validate is gender
355	<code>if (str.length > 255) {</code>	If the input for gender is longer than 255 characters
356	<code>add_user_feedback["gender"] = "<a>gender is too long<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue
357	<code>add_user_array["gender"] = 0;</code>	Change the validation state of the gender field to 0 to indicate the gender is not ready for submission
358	<code>} else if (str === "") {</code>	If the input for gender is an empty string

359	<code>add_user_feedback["gender"] = "<a>gender is required<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue
360	<code>add_user_array["gender"] = 0;</code>	Change the validation state of the gender field to 0 to indicate the gender is not ready for submission
361	<code>} else {</code>	The input string to gender has passed validation so
362	<code>add_user_feedback["gender"] = "";</code>	Clear the feedback messages for gender
363	<code>add_user_array["gender"] = 1;</code>	Set the validation state of gender to 1 to indicate gender is ready to submit
364	<code>}</code>	
365	<code>} else if (input_to_check === "rank") {</code>	If the input to validate is rank
366	<code>if (str.length > 255) {</code>	If the input for rank is longer than 255 characters
367	<code>add_user_feedback["rank"] = "<a>rank is too long<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue
368	<code>add_user_array["rank"] = 0;</code>	Change the validation state of the rank field to 0 to indicate the rank is not ready for submission
369	<code>} else if (str === "") {</code>	If the input for rank is an empty string
370	<code>add_user_feedback["rank"] = "<a>rank is required<a>
";</code>	Add a prompt to the feedback dictionary to alert the user to this issue
371	<code>add_user_array["rank"] = 0;</code>	Change the validation state of the rank field to 0 to indicate the rank is not ready for submission
372	<code>} else {</code>	The input string to rank has passed validation so
373	<code>add_user_feedback["rank"] = "";</code>	Clear the feedback messages for rank
374	<code>add_user_array["rank"] = 1;</code>	Set the validation state of rank to 1 to indicate rank is ready to submit
375	<code>}</code>	
376	<code>}</code>	
377	<code>if (checkAreAllValuesOne(add_user_array) === true){</code>	If all the values in the add_user_array dictionary are set to 1 indicating all fields pass validation
378	<code>document.getElementById("add-user-submit").disabled = false;</code>	Enable the submit button
379	<code>document.getElementById("user-input-handeling").innerHTML = "";</code>	Clear the feedback div
380	<code>} else {</code>	If one or more fields are not yet validly populated

381	<code>document.getElementById("add-user-submit").disabled = true;</code>	Disable the submit button
382	<code>document.getElementById("user-input-handeling").innerHTML = returnFeedbackHTML(add_user_feedback);</code>	Print the contents of the feedback dictionary to the input handling div
383	<code>}</code>	
384	<code>}</code>	
385		
386	REGEXCheckModifyUser() <pre>function REGEXCheckModifyUser(str, input_to_check){ if (typeof modify_user_array === 'undefined'){</pre>	Function to validate the modify user form called for all input fields on the modify user form onclick (and onkeyup) This validation function works almost identically to the validation function for adding a new user. However since the 2 forms are different for clarity I have made different functions with different variable names but with the same core functionality.
387	<pre> modify_user_array = {modify_email: 1, modify_password: 1, modify_first_name: 1, modify_last_name: 1, modify_DOB: 1, modify_gender: 1, modify_rank: 1, user_id: 0};</pre>	In this function all inputs are assumed to be valid since they are populated by the resultHasBeenClickedUser() function. To prevent users from submitting this form without selecting a user_id. The modify_user_array has a key user_id which must be set to 1 by resultHasBeenClickedUser() function.
388	<code>}</code>	
389	<code>if (typeof modify_user_feedback === 'undefined'){</code>	
390	<code> modify_user_feedback = {modify_email: "", modify_password: "", modify_first_name: "", modify_last_name: "", modify_DOB: "", modify_gender: "", modify_rank: ""};</code>	
391	<code>}</code>	
392	<code>if (input_to_check === "user_id"){</code>	If the input to validate is the user_id
393	<code> modify_user_array["user_id"] = 1;</code>	Set the user_id value in the dictionary to 1 indicating the user_id has been populated
394	<code>} else if (input_to_check === "modify_email") {</code>	Email validation works exactly the same as it does in the add user validation function
395	<code> if (str.length > 255){ modify_user_feedback["modify_email"] = "<a>email is to long<a>
;</code>	
396	<code> modify_user_array["modify_email"] = 0;</code>	
397	<code> } else if(str === ""){ modify_user_feedback["modify_email"] = "<a>email cant be empty<a>
;</code>	

398	<code>modify_user_array["modify_email"] = 0;</code>	
399	<code>} else if (!str.match("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$")) { modify_user_feedback-</code> <code>["modify_email"] = "<a>Invalid email format<a>
";</code>	
400	<code>modify_user_array["modify_email"] = 0;</code>	
401	<code>} else {</code>	
402	<code>modify_user_feedback["modify_email"] = "";</code>	
403	<code>modify_user_array["modify_email"] = 1;</code>	
404	<code>}</code>	
405	<code>} else if (input_to_check === "modify_password") {</code>	password validation works differently
406	<code>if (str.length < 8 && str.length != 0) {</code>	If the password is less than 8 characters in length it is invalid. Unless it is empty since the server treats this as a case where no password change is required.
407	<code>modify_user_feedback["modify_password"] = "<a>Password must be at least 8 characters long or empty<a>
";</code>	If the password length is invalid prompt the user accordingly
408	<code>modify_user_array["modify_password"] = 0;</code>	Set the validation state of the password to 0 to indicate it is not ready for submission
409	<code>} else if(str.length > 255){</code>	If the password is more than 255 characters long i.e. it can't be stored
410	<code>modify_user_feedback["modify_password"] = "<a>Password is to long long<a>
";</code>	Add a prompt to the feedback dictionary to alert the user that the password must be less than 255 characters
411	<code>modify_user_array["modify_password"] = 0;</code>	Set the validation state of the password to 0 to indicate it is not ready for submission
412	<code>}else{</code>	If the password has passed the above submission steps
413	<code>modify_user_feedback["modify_password"] = "";</code>	Clear the feedback messages from the feedback dictionary for password
414	<code>modify_user_array["modify_password"] = 1;</code>	Set the validation state of the password to 1 indicating the password is ready for submission
415	<code>}</code>	
416	<code>} else if (input_to_check === "modify_first_name") {</code>	First name validation works exactly the same as it does in the add user validation function
417	<code>if (str.length > 255) {</code>	
418	<code>modify_user_feedback["modify_first_name"] = "<a>First name is too long<a>
";</code>	
419	<code>modify_user_array["modify_first_name"] = 0;</code>	

420	} else if (str === "") {	
421	modify_user_feedback["modify_first_name"] = "<a>First name is required<a> ";	
422	modify_user_array["modify_first_name"] = 0;	
423	} else {	
424	modify_user_feedback["modify_first_name"] = "";	
425	modify_user_array["modify_first_name"] = 1;	
426	}	
427	} else if (input_to_check === "modify_last_name") {	Last name validation works exactly the same as it does in the add user validation function
428	if (str.length > 255) {	
429	modify_user_feedback["modify_last_name"] = "<a>Last name is too long<a> ";	
430	modify_user_array["modify_last_name"] = 0;	
431	} else if (str === "") {	
432	modify_user_feedback["modify_last_name"] = "<a>Last name is required<a> ";	
433	modify_user_array["modify_last_name"] = 0;	
434	} else {	
435	modify_user_feedback["modify_last_name"] = "";	
436	modify_user_array["modify_last_name"] = 1;	
437	}	
438	} else if (input_to_check === "modify_DOB") {	Date of birth validation works exactly the same as it does in the add user validation function
439	if (!str.match(/^\d{4}-\d{2}-\d{2}\$/)) {	
440	modify_user_feedback["modify_DOB"] = "<a>Invalid date format (YYYY-MM-DD)<a> ";	
441	modify_user_array["modify_DOB"] = 0;	
442	} else {	
443	modify_user_feedback["modify_DOB"] = "";	
444	modify_user_array["modify_DOB"] = 1;	
445	}	
446	} else if (input_to_check === "modify_gender") {	gender validation works exactly the same as it does in the add user validation function
447	if (str.length > 255) {	
448	modify_user_feedback["modify_gender"] = "<a>gender is too long<a> ";	
449	modify_user_array["modify_gender"] = 0;	
450	} else if (str === "") {	
451	modify_user_feedback["modify_gender"] = "<a>gender is required<a> ";	

452	<code>modify_user_array["modify_gender"] = 0;</code>	
453	<code>} else {</code>	
454	<code>modify_user_feedback["modify_gender"] = "";</code>	
455	<code>modify_user_array["modify_gender"] = 1;</code>	
456	<code>}</code>	
457	<code>} else if (input_to_check === "modify_rank") {</code>	rank validation works exactly the same as it does in the add user validation function
458	<code>if (str.length > 255) {</code>	
459	<code>modify_user_feedback["modify_rank"] = "<a>rank is too long<a>
";</code>	
460	<code>modify_user_array["modify_rank"] = 0;</code>	
461	<code>} else if (str === "") {</code>	
462	<code>modify_user_feedback["modify_rank"] = "<a>rank is required<a>
";</code>	
463	<code>modify_user_array["modify_rank"] = 0;</code>	
464	<code>} else {</code>	
465	<code>modify_user_feedback["modify_rank"] = "";</code>	
466	<code>modify_user_array["modify_rank"] = 1;</code>	
467	<code>}</code>	
468	<code>}</code>	
469		
470	<code>if (checkAreAllValuesOne(modify_user_array) === true){</code>	This section works exactly the same as the add user validation function
471	<code>document.getElementById("modify-user-submit").disabled = false;</code>	
472	<code>document.getElementById("modify-user-input-handeling").innerHTML = "";</code>	
473	<code>} else {</code>	
474	<code>document.getElementById("modify-user-submit").disabled = true;</code>	
475	<code>document.getElementById("modify-user-input-handeling").innerHTML = returnFeedbackHTML(modify_user_feedback);</code>	
476	<code>}</code>	
477	<code>}</code>	
478		
479	REGEXCheckAddEquipment() <code>function REGEXCheckAddEquipment(str, input_to_check){ if (typeof add_equipment_array === 'undefined'){</code>	Validation function for the add equipment form. If the add_equipment_array dictionary doesn't exist.
480	<code>add_equipment_array = {name: 0, description: 0, location: 0};</code>	Initialise it and set all values to 0 to indicate the fact all input fields are not ready for submission

481	}	
482	if (typeof add_equipment_feedback === 'undefined'){	If the add_equipment_feedback dictionary doesn't exist
483	add_equipment_feedback = {name: "", description: "", location: ""};	Initialise it and set all values to empty strings since the user doesn't yet need to be prompted as to what their issues are
484	}	
485	if (input_to_check === "name") {	If the input to validate is the equipment name
486	if (str.length > 255){ add_equipment_feedback["name"] = "<a>Name is too long<a>
" ;	If the equipment name is longer than 255 characters, then add a prompt to the feedback dictionary to alert the user to this issue
487	add_equipment_array["name"] = 0 ;	Set the validation state of the equipment array to 0 to indicate the equipment name is not yet ready for submission
488	} else if (str === ""){ add_equipment_feedback["name"] = "<a>Name can't be empty<a>
" ;	If the equipment name is an empty string. Add an appropriate prompt to the feedback dictionary to alert the user to this issue.
489	add_equipment_array["name"] = 0 ;	Set the validation state of the equipment array to 0 to indicate the equipment name is not yet ready for submission
490	} else {	The name location has passed validation so
491	add_equipment_feedback["name"] = "" ;	Empty the prompts from the feedback dictionary for name
492	add_equipment_array["name"] = 1 ;	Set the validation state of the name to 1 to indicate the name is ready for submission
493	}	
494	} else if (input_to_check === "description") {	If the input to validate is the description
495	if (str.length > 65535) {	If the length of the equipment description is longer than 65,535 characters, then
496	add_equipment_feedback["description"] = "<a>Description is too long<a>
" ;	Add a prompt to the feedback dictionary to alert the user to this issue
497	add_equipment_array["description"] = 0 ;	Set the validation state of the equipment array to 0 to indicate the equipment description is not yet ready for submission

498	<code>} else {</code>	The equipment description has passed validation so
499	<code>add_equipment_feedback["description"] = "";</code>	Empty the prompts from the feedback dictionary for description
500	<code>add_equipment_array["description"] = 1;</code>	Set the validation state of the description to 1 to indicate the description is ready for submission
501	<code>}</code>	
502	<code>} else if (input_to_check === "location") {</code>	If the input to validate is location
503	<code>if (str.length > 65535) {</code>	If the input for location is longer than 65,535 characters, then
504	<code>add_equipment_feedback["location"] = "<a>Location is too long<a>
";</code>	Add a prompt to the equipment array to alert the user to this issue
505	<code>add_equipment_array["location"] = 0;</code>	Set the validation state of the equipment array to 0 to indicate the equipment location is not yet ready for submission
506	<code>} else if (str === "") {</code>	If the equipment name is an empty string
507	<code>add_equipment_feedback["location"] = "<a>Location can't be empty<a>
";</code>	Add an appropriate prompt to the feedback dictionary to alert the user to this issue
508	<code>add_equipment_array["location"] = 0;</code>	Set the validation state of the equipment array to 0 to indicate the equipment location is not yet ready for submission
509	<code>} else {</code>	The equipment location has passed validation so
510	<code>add_equipment_feedback["location"] = "";</code>	Empty the prompts from the feedback dictionary for location
511	<code>add_equipment_array["location"] = 1;</code>	Set the validation state of the location to 1 to indicate the location is ready for submission
512	<code>}</code>	
513	<code>}</code>	
514	<code>if (checkAreAllValuesOne(add_equipment_array) === true){</code>	This section works very similarly to the other validation functions however it uses different names for the dictionary and different tags for the buttons and divs it is activating/emptying
515	<code>document.getElementById("add-equipment-submit").disabled = false;</code>	
516	<code>document.getElementById("equipment-input-handeling").innerHTML = "";</code>	

517	<code>} else {</code>	
518	<code>document.getElementById("add-equipment-submit").disabled = true;</code>	
519	<code>document.getElementById("equipment-input-handeling").innerHTML = returnFeedbackHTML(add_equipment_feedback);</code>	
520	<code>}</code>	
521	<code>}</code>	
522		
523	REGEXCheckModifyEquipment() <pre>function REGEXCheckModifyEquipment(str, input_to_check){ if (typeof modify_equipment_array === 'undefined'){</pre>	Function for validating inputs for the modify equipment form. This function works almost identically to the add equipment validation function however it checks for an equipment_id, uses different variable names and accesses different html elements.
524	<pre> modify_equipment_array = {modify_equipment_name: 1, modify_equipment_description: 1, modify_equipment_location: 1, equipment_id: 0};</pre>	All inputs are assumed to be valid since they are populated by the resultHasBeenClickedEquipment() function. However, equipment_id is initialised to be zero and will only be set to one when an equipment search result is selected.
525	<code>}</code>	
526	<code>if (typeof modify_equipment_feedback === 'undefined'){</code>	If the feedback dictionary doesn't exist
527	<code> modify_equipment_feedback = {modify_equipment_name: "", modify_equipment_description: "", modify_equipment_location: ""};</code>	Initialise it and set the values to empty strings since no prompts should yet be provided to the user.
528	<code>}</code>	
529	<code>if (input_to_check === "equipment_id"){</code>	If the input to check is the equipment_id then
530	<code> modify_equipment_array["equipment_id"] = 1;</code>	Set the validation state of equipment_id to 1 indicating it is populated
531	<code>}</code>	
532	<code>if (input_to_check === "modify_equipment_name") {</code>	Validation of the name works identically to the validation function for adding equipment
533	<code> if (str.length > 255){</code>	
534	<code> modify_equipment_feedback["modify_equipment_name"] = "<a>Name is too long
";</code>	
535	<code> modify_equipment_array["modify_equipment_name"] = 0;</code>	
536	<code>} else if(str === ""){</code>	

537	modify_equipment_feedback["modify_equipment_name"] = "<a>Name can't be empty ";	
538	modify_equipment_array["modify_equipment_name"] = 0;	
539	} else {	
540	modify_equipment_feedback["modify_equipment_name"] = "";	
541	modify_equipment_array["modify_equipment_name"] = 1;	
542	}	
543	} else if (input_to_check === "modify_equipment_des- cription") {	Validation of the equipment description works identically to the validation function for adding equipment
544	if (str.length > 65535) {	
545	modify_equipment_feedback["modify_equipment_description"] = "<a>Description is too long ";	
546	modify_equipment_array["modify_equipment_description"] = 0;	
547	} else if(str === "") {	
548	modify_equipment_feedback["modify_equipment_description"] = "<a>description can't be empty "	
549	modify_equipment_array["modify_equipment_description"] = 0;	
550	} else {	
551	modify_equipment_feedback["modify_equipment_description"] = "";	
552	modify_equipment_array["modify_equipment_description"] = 1;	
553	}	
554	} else if (input_to_check === "modify_equipment_loc- ation") {	Validation of the equipment location works identically to the validation function for adding equipment
555	if (str.length > 65535) {	
556	modify_equipment_feedback["modify_equipment_location"] = "<a>Location is too long ";	
557	modify_equipment_array["modify_equipment_location"] = 0;	
558	} else if (str === "") {	
559	modify_equipment_feedback["modify_equipment_location"] = "<a>Location can't be empty ";	
560	modify_equipment_array["modify_equipment_location"] = 0;	
561	} else {	
562	modify_equipment_feedback["modify_equipment_location"] = "";	
563	modify_equipment_array["modify_equipment_location"] = 1;	
564	}	
565	}	

566	<pre>if (checkAreAllValuesOne(modify_equipment_array) === true){</pre>	This section works very similarly to the other validation functions however it uses different names for the dictionary and different tags for the buttons and divs it is activating/emptying
567	<pre>document.getElementById("modify-equipment-submit").disabled = false;</pre>	
568	<pre>document.getElementById("modify-equipment-input-handeling").innerHTML = "";</pre>	
569	<pre>} else {</pre>	
570	<pre>document.getElementById("modify-equipment-submit").disabled = true;</pre>	
571	<pre>document.getElementById("modify-equipment-input-handeling").innerHTML = returnFeedbackHTML(modify_equipment_feedback);</pre>	
572	<pre>}</pre>	
573	<pre>}</pre>	

help.php

1	<pre><head></pre>	
2	<pre><title>Help Page</title></pre>	
3	<pre><link rel="stylesheet" href="css/help-style.css"></pre>	Including the style sheet
4	<pre></head></pre>	
5	<pre><body></pre>	
6	<pre><?php</pre>	
7	<pre>session_start();</pre>	Initialising the \$_session variable to check the session information of the client
8	<pre>\$flag = false;</pre>	Setting a flag to false i.e. assuming the client hasn't logged in
9	<pre>if (isset(\$_SESSION["user_id"])){</pre>	If the user has a user_id attached to their session information i.e. they have logged in successfully
10	<pre>\$flag = true;</pre>	Set the login flag to true to indicate that the back to login button later in the document is not necessary
11	<pre>include("includes/functions.php");</pre>	Include the function check_login() since nav.php does not include this function
12	<pre>include("includes/nav.php");</pre>	Include nav.php to display the navigation bar along the top of the page. Additionally nav.php will call check_login() so if the user_id on the session information is not real then check_login() will find out and will redirect the user to the logout page where the session information will be

		unset and they will be redirected to the login page.
13	}	
14	?>	
15	<div class="container">	
16	<div class="navigation">	Navigation bar to appear on the left of the page to navigate the help/guidance page
17	Categories	
18		
19		
20	All Users Issues	
21		
22		
23	All Users Guidance	
24		
25		
26	Basic Users	
27		
28		
29	Event Owner	
30		
31		
32	Duty Cadets	
33		
34		
35	G4 (Logistics Cadets)	
36		
37		
38	Admin	
39		
40		
41	<?php	
42	if (\$flag == false){	If the login flag is set to false i.e. the navigation bar along the top will not appear then
43	echo "<button>back to the login page</button>";	Print a go back to login button in the help page navigation menu since they have no other way of going back
44	}	
45	?>	
46	</div>	
47	<div class="content">	
48	<section id="all-users-issues">	

49	All Users Issues	
50	<ul class="help-list">	
51	any issues with logins or passwords speak to the system admin they can fix these	
52	if your account has been incorrectly deactivated, speak to the system admin they can fix this	
53	any issues with permissions i.e. G4 or Admin speak to the system admin they can fix these	
54		
55	</section>	
56	<section id="all-users-guidance">	
57	All Users Guidance	
58	<ul class="help-list">	
59	for navigation at the top of all pages there is the navigation bar that will take you back to the calendar page at any time	
60	to view your user details hover over your profile photo on the right of the navigation bar	
61	to change your profile photo use the user details drop down menu to upload a photo to the system then press submit for it to be changed	
62	to view parades not in the current range on the calendar page use the: >, >>>, < or <<< buttons at the top of the calendar page	
63	since all search features on this page use regular expressions it is best to keep your searches non-descript and to see all possible results enter the characters +*	
64		
65	</section>	
66	<section id="basic-users">	
67	Basic Users Guidance	
68	<ul class="help-list">	
69	as a basic user you will be able to view the lessons you have upcoming for the next few parades on the calendar page	
70		
71	</section>	
72	<section id="event-owners">	
73	Event Owner Guidance	
74	<ul class="help-list">	
75	as an event owner you will be able to view the approval status of your event on the calendar page:	
76	<ul class="status-list">	

77	Red: Indicates the event is not-approved	
78	Amber: Indicates the event has had approval requested	
79	Green: Indicates the event is approved	
80		
81		
82	the following functionalities will be available when an event is not approved	
83		
84	to change the approval status of the event, you must click the link for your event on the calendar page which will take you to the event page where this can be done	
85	event that are approved can't change their details so be sure the details are correct before you move the approval to requested	
86	the admin will always be able to see your event in the calendar so once approval is requested, they will check the details and either approve it or set it to not-approved meaning some details must be changed	
87		
88	the following functionalities will be available once the event is approved:	
89	register	
90		
91	to add cadets to the register search their first name in the add box and click the result you want	
92	if a cadet can't make it to your event, they will not show up on this these results	
93	to see why the cadet can't make it search for them in the 'other' box to see the name of the cadet, the name of the event and the time of the event	
94	to remove cadets from the register search their first name in the remove box and click the result you want	
95	at the start of your lesson it is essential you populate the register with 1's to indicate a cadet is present and 0's if they are not then you must click the submit register button to update the register	
96		
97	equipment requests log	
98		
99	to add an equipment request to the equipment request log search for the equipment name in the add box and click the result you want	

100	if a piece of equipment is being used elsewhere and can't be used at your event, they will not show up on these results	
101	to see where the piece of equipment is search for it in the 'other' box to see the name of the equipment, the name of the event and the time of the event where it is being used	
102	to remove a piece of equipment from the register search the name of the equipment in the remove box and click the result you want	
103	the G4 cadets will then approve/deny the equipment requests as they see fit once a piece of equipment has been denied/approved its status can't be changed without deleting the request	
104		
105		
106	</section>	
107	<section id="duty-cadets">	
108	Duty Cadets Guidance	
109	<ul class="help-list">	
110	As a duty cadet you will have all the same controls over an event as the event owner. The purpose of this role is to provide you the ability to see the progress of the events you have delegated.	
111	Events that you are duty cadet for will show up on the calendar with the words duty event at the top.	
112		
113	</section>	
114	<section id="g4">	
115	G4 (Logistics Cadets) Guidance	
116	<ul class="help-list">	
117	As a G4 you will see all events on the calendar page. Additionally, they will show up as if you are the event owner hence you can see their approval status.	
118	Your role is to approve/deny equipment request as your team sees fit.	
119	If you have accidentally approved/denied an equipment request you may delete the request from the log and add the request back then set its approval status to the correct setting.	
120	It is important to note that when a request is denied the system will no longer see this request. Hence, when searches are carried out for equipment that can be added to the log these pieces of equipment will show up as available.	

121		
122	</section>	
123	<section id="admin">	
124	Admin Guidance	
125	<ul class="help-list">	
126	As the admin you have access to functions and pages other users don't:	
127	add page	
128		
129	the add page will be your primary way to create/modify/delete items on the system	
130	to add a parade to the system fill out the form and be sure to properly name the parade since they stay on the system and descriptive names will help keep the calendar organised	
131	to add an event to the calendar go to the add an event form and populate the fields. Be sure to populate the following accurately:	
132		
133	event owner: this will be the person teaching the lesson their role is to set the timings and names of the event according to the specifications set by their senior cadet (C/SGT to RSM). Once the event is approved, they must: populate the register and the equipment requests.	
134	duty cadet: this will be the person overseeing the planning of the event they are the senior (C/SGT to RSM) to the event owner. Initially you should set the event owner to this person so they may delegate the event to a lower rank for it to be organised. The duty cadet can't be changed once the event is created.	
135	parade: This will be the parade that the event will take place on. Once you have selected this be sure to check the date of the selected parade to ensure it is the correct one.	
136		
137	To add a user onto the system go to the add a user form in the top centre of the page. Populate the form here and submit it to add the user to the system.	
138	To modify the details of a user go to the modify a user form below the add a user. you must first search for the user then select the desired result. since users cant be deleted from the system you should instead set the active status of a user to non-active when they leave the cadet	

	force. this will prevent the user from logging in to the system.	
139	To add equipment to the register populate the add equipment form on the top right of the page. be sure to use descriptive locations i.e. clothing store shelf 1. This will help G4 to locate equipment when it has been requested	
140	To modify equipment go to the modify equipment form below the add equipment form. you must first search for the piece of equipment and select the desired result. here you may edit the details of the piece of equipment be sure to have the action box set to modify unless you intend to delete the piece of equipment from the system	
141		
142	calendar page	
143		
144	the calendar page for the admin will show all events for all parades withing the date range	
145	to edit an event from the calendar click the admin panel edit button for the event which will populate the admin panel edit form. From here change the necessary details and click submit to save the changes	
146	To access the event page for a lesson click the link that displays the name of the event	
147		
148	event page	
149		
150	to set the approval status of an event to approved go to the modify event details form at the centre of the page and set the approval to approved and submit the form this will update the event's status so the register may be modified and the equipment requests can be made.	
151		
152		
153	</section>	
154	</div>	
155	</div>	
156	</body>	
157	</html>	
158		

css\add-style.css

1		3	margin: 0;
2	.grid-container {	4	padding: 10px 0;
		5	background-color: #ddd;

```

6  display: grid;
7  grid-template-columns: 33% 33% 33%;
8  }
9
10
11  .grid-container .events-parades{
12  background-color: #ddd;;
13  margin-left:5px;
14  margin-right:5px;
15  }
16
17  .grid-container .users{
18  background-color: #ddd;;
19  margin-left:5px;
20  margin-right:5px;
21  }
22
23  .grid-container .equipment{
24  background-color: #ddd;;
25  margin-left:5px;
26  margin-right:5px;
27  }
28
29
30  form {
31  display: grid;
32  grid-template-columns: 50%;
33  gap: 10px;
34  padding-left: 10px;
35  padding-right:10px;
36  }
37
38
39  .livesearch {
40  grid-column: span 2;
41  font-size: 25px;
42  }
43
44  .input-error-handeling {
45  grid-column: span 2;
46  font-size: 20px;
47  background-color: #d11717;
48  }

```

```

49
50
51  h4 {
52  grid-column: span 2;
53  text-decoration: underline;
54  }
55
56  h2 {
57  grid-column: span 2;
58  text-align: center;
59  }
60
61  label {
62  font-weight: bold;
63  line-height: 2;
64  }
65
66  input, select {
67  padding: 8px;
68  font-size: 16px;
69  border: 1px solid #ccc;
70  border-radius: 4px;
71  }
72
73
74  button {
75  grid-column: span 2;
76  margin-top: 10px;
77  padding: 10px 15px;
78  font-size: 16px;
79  background-color: #6b86c0;
80  color: white
81  border: none;
82  border-radius: 5px;
83  cursor: pointer;
84  text-decoration: solid;
85  }
86
87  button:hover {
88  background-color: #4c6baf;
89  }
90
91  button:disabled {

```

```

92 color: red
93 }

```

css\calendar-style.css

```

1  .container {
2  display: grid;
3  grid-template-rows: 15% 85%;
4  }
5
6  .banner {
7  border-radius: 10px;
8  margin-bottom: 5px;
9  padding: 10px 0;
10 background-color: #ddd;
11 display: grid;
12 grid-template-columns: 10% 80% 10%;
13 text-align: center;
14 }
15
16 .calendar{
17 display: grid;
18 grid-template-columns: 20% 20% 20% 20%
19 20%;
19 text-align: center;
20 }
21
22 button {
23 margin-top: 10px;
24 margin-bottom: 10px;
25 padding: 10px 15px;
26 font-size: 16px;
27 background-color: #6b86c0;
28 color: white
29 border: none;
30 border-radius: 5px;
31 cursor: pointer;
32 }

```

css\event-display-block-style.css

```

1  .event_not_aproved{
2  text-align: center;
3  border-radius: 10px;
4  display: inline-block;

```

```

5  background-color: rgba(255, 105, 105,
6  0.582);
6  margin: 3px;
7  width: 98%;
8  }
9
10 .event_aproved{
11 text-align: center;
12 border-radius: 10px;
13 display: inline-block;
14 background-color: rgb(157, 241, 157);
15 margin: 3px;
16 width: 98%;
17 }
18
19 .event_aproval_requested{
20 text-align: center;
21 border-radius: 10px;
22 display: inline-block;
23 background-color: rgba(233, 215, 117,
24 0.952);
24 margin: 3px;
25 width: 98%;
26 }
27
28 .event{
29 text-align: center;
30 border-radius: 10px;
31 display: inline-block;
32 background-color: #ddd;
33 margin: 3px;
34 width: 98%;
35 }

```

css\event-owner-form-style.css

```
1  .modify_lesson_details{
2  display: grid;
3  grid-template-columns: 30% 70%;
4  gap: 10px;
5  padding-left: 10px;
6  padding-right:10px;
7  }
8
9  button {
10 grid-column: span 2;
11 margin-top: 10px;
12 margin-bottom: 10px;
13 padding: 10px 15px;
14 font-size: 16px;
15 background-color: #6b86c0;
16 color: white
17 border: none;
18 border-radius: 5px;
19 cursor: pointer;
20 }
21
22 button:hover {
23 background-color: #4c6baf;
24 }
25
26 button:disabled {
27 color: red
28 }
29
30 .input_handeling{
31 grid-column: span 2;
32 }
33
34 label {
35 font-weight: bold;
36 line-height: 2;
37 }
38
39 input, select {
40 padding: 8px;
41 font-size: 16px;
42 border: 1px solid #ccc;
```

```
43 border-radius: 4px;
44 }
```

css\event-style.css

```
1  .grid-container {
2  margin: 0;
3  padding: 10px 0;
4  background-color: #ddd;
5  display: grid;
6  grid-template-columns: 15% 85%;
7  }
8
9
10 .grid-container .right-side {
11 display: grid;
12 grid-template-columns: 30% 35% 35%;
13 grid-template-rows: auto auto;
14 padding-right: 2px;
15 }
16 .grid-container .right-side .register-
17 box-all {
18 display: grid;
19 grid-template-columns: 100%;
20 grid-template-rows: auto auto auto auto;
21 }
22 .grid-container .right-side .equipemnt-
23 display-register-main{
24 display: grid;
25 grid-template-columns: 100%;
26 grid-template-rows: auto auto auto auto;
27 }
28 .grid-container .right-side .search-box-
29 all {
30 padding-top: 10px;
31 }
32 .modify_lesson_details .submit_button{
33 grid-column: span 2;
34 }
35
36 .modify_lesson_details
37 .input_error_handeling{
```

```

37 grid-column: span 2;
38 }
39
40 .modify_lesson_details
  .display_current_owner{
41 grid-column: span 2;
42 }
43
44 .modify_lesson_details .livesearch{
45 grid-column: span 2;
46 }

```

css\login-style.css

```

1 body{
2 font-family: "cabril";
3 display: flex;
4 justify-content: center;
5 }
6 body .container{
7 margin-top: 200px;
8 }
9 body .container .title{
10 display: flex;
11 justify-content: center;
12 }
13 form{
14 display: grid;
15 }
16 body .container .input-heading{
17 font-size: 20px;
18 }
19 body .container .minor-container{
20 display: grid;
21 grid-template-columns: repeat(2, 1fr);
22 }
23 body .container .minor-container
  .remember-me{
24 justify-self: flex-start;
25 }
26 body .container .minor-container .forgot-
  password{
27 justify-self: flex-end;
28 }
29 body .container .login-button-container{

```

```

30 display: grid;
31 grid-template-columns: 1fr;
32 justify-self: center;
33 }
34 body .container .login-button-container
  .login-button{
35 width: 400px;
36 height: 35px;
37 }

```

css\nav-style.css

```

1 nav{
2 border-radius: 10px;
3 display: grid;
4 grid-template-columns: auto 1fr;
5 align-items: center;
6 grid-column-gap: 30px;
7 grid-template-rows: 70px;
8 box-shadow: 0 5px 5px rgba(0,0,0,0.15);
9 position: sticky;
10 top: 0;
11 background-color: white
12 padding-left: 20px;
13 padding-right: 20px;
14 margin-bottom: 10px;
15 }
16
17 nav .left{
18 margin-right: auto;
19 }
20
21 nav .right{
22 margin-left: auto;
23 }
24
25 nav a{
26 text-decoration: none;
27 display: inline-block;
28 padding: 10px 35px;
29 transition: background-color 0.3s,
  colour 0.3;
30 color: black
31 font-size: 18px;
32 }

```

```

33
34 .highlightable{
35 text-decoration: none;
36 display: inline-block;
37 padding: 10px 35px;
38 transition: background-color 0.3s,
39 colour 0.3;
40 color: #245990;
41 font-size: 18px;
42 }
43 .highlightable:hover {
44 background: #245990;
45 color: #fff;
46 border-radius: 3px;
47 }
48
49 img {
50 border-radius: 50%;
51 height: 38px;
52 width: 38px;
53 }
54
55 img:hover {
56 border-radius: 50%;
57 }
58
59 .dropdown-button {
60 text-decoration: none;
61 display: inline-block;
62 padding: 10px 35px;
63 transition: background-color 0.3s,
64 colour 0.3;
65 color: #245990;
66 font-size: 18px;
67 }
68 .dropdown-button:hover{
69 background: #245990;
70 color: #fff;
71 border-radius: 3px;
72 }
73
74 .dropdown-container {

```

```

75 position: relative;
76 display: inline-block;
77 }
78
79 .dropdown-content {
80 display: none;
81 position: absolute;
82 background-color: #f1f1f1;
83 min-width: 160px;
84 box-shadow: 0px 8px 16px 0px
85 rgba(0,0,0,0.2);
86 z-index: 1;
87 right: 0;
88 left: auto;
89 }
90 .dropdown-container:hover .dropdown-
91 content {
92 display: block;
93 }
94 .logout-button{
95 display: block;
96 text-align: center;
97 }
98
99 .logout-button:hover{
100 background: #245990;
101 color: #fff;
102 border-radius: 3px;
103 }

```

css/register-and-equipment-reuquests-style.css

```

1 .register-button {
2 margin-top: 10px;
3 margin-bottom: 10px;
4 padding: 10px 15px;
5 font-size: 16px;
6 background-color: #6b86c0;
7 color: white
8 border: none;
9 border-radius: 5px;

```

```
10 cursor: pointer;
11 }
12
13 .register-button:hover {
14 background-color: #4c6baf;
15 }
16
17 .register-button:disabled {
18 color: red
19 }
20
21 label {
22 font-weight: bold;
23 line-height: 2;
24 }
```

```
25
26 input, select {
27 padding: 8px;
28 font-size: 16px;
29 border: 1px solid #ccc;
30 border-radius: 4px;
31 width: 90%;
32 }
33
34 table {
35 border-collapse: collapse;
36 width: 100%;
37 }
```

Testing

Testing video link: <https://www.youtube.com/watch?v=YLrflXSili0>

While the system was developed it underwent iterative testing. As a result, I have chosen to not formally include features that repeatedly appear in the program, since they were tested during their development and have not changed. These features include the following:

- Scanning for malicious inputs in search prompts. I have not included this since all logic for processing search requests on the server side involve retrieving the search prompt from the get request scanning function. This is however shown in 3 different tests.
- Redirection of users after they have submitted forms (Some elements of this are included in the video).
- Testing the validation of every field in a form. This was carried out when the validation functions were written and would dramatically increase the length of the tests and the video. Instead, I have opted to show that the logic behind form validation is working by inputting invalid data into fields and showing the effect it has on the validation state of the form.

Naming conventions for test id's

Since this system is likely to expand in the future. I believe it is necessary for the naming conventions of the tests to be descriptive. This includes: what they are testing, who it is being tested for, what use case the test is for what group of features is being tested. Therefore, I have designated the test to follow a format as such:

Page being tested	User test is designated for	Use case	Test group number	Individual test identifier
Letter	Letter	Letter	Number	Number
Login page (L) Navigation banner (N) Backup systems (B) Calendar page (C) Add page (A) Event page (E) Forms (F)	Admin (A) Event owner (O) Duty cadet (D) Logistics (G) Standard user (S)	Approved event (A) Non-approved event (N)	Any integer	Any integer

By using the heading of the test tables and using the above table it should be possible to identify what the test id should look like and vice versa. It is important to note that the use case while currently less necessary is the identifier most likely to expand in the future. If the rules of the system are to be relaxed i.e. able to move the times of events when they are approved and have equipment requests will be a future use case.

Screenshots for tests not included in the video

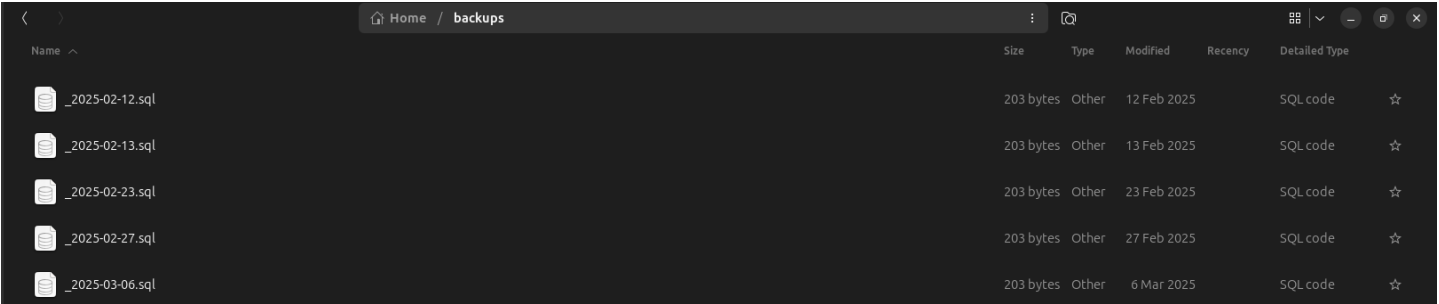


Figure 22 figure to show the routine backups of the database

login

SCC email
your-email@stcolumbascollege.org

password
your password

☒ remember me [help page](#)

Log in

Your account is inactive.

Speak to the system admin if this is an error

Figure 23 screenshot to show the results of logging in with an inactive account

127.0.0.1/NEA/add.php

my calendar help and guidance add parade/event/account/equipment

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-05-23 the next parade should be: 2025-05-30

date mm/dd/yyyy

start time --:--:--

finish time --:--:--

parade name

submit

add an event

event type

event name

event start --:--:--

event end --:--:--

event owner

current owner: none selected

duty cadet

users

add a user

email

password

first name

last name

date of birth mm/dd/yyyy

gender

rank

active active

admin non-admin

G4 non-G4

submit

modify a user

leave the password field blank to leave it unchanged

search by first name

equipment

add equipment

equipment name

equipment description

equipment location

submit

update equipment

search for equipment by name DROP

search failed

search for equipment by location

equipment name

equipment description

equipment location

action modify

submit

Figure 24 screenshot to show what happens when a malicious phrase is input into the equipment name search box

The screenshot shows a web application interface for adding a parade, event, account, or equipment. The form is divided into several sections:

- Left Sidebar:** Contains dropdowns for 'event start', 'event end', 'event owner', 'duty cadet', and 'parade'. Each dropdown has a 'submit' button below it.
- Top Navigation:** Includes links for 'my calendar', 'help and guidance', and 'add parade/event/account/equipment'.
- Main Form Area:**
 - modify a user:** A section with a 'submit' button and a message: 'leave the password field blank to leave it unchanged'.
 - search by first name:** A search box containing the text 'DROP'. Below it, a message says 'search failed'.
 - search by last name:** An empty search box.
 - Form Fields:** Fields for 'email', 'password', 'first name', 'last name', 'date of birth' (with a calendar icon), 'gender', 'rank', 'active' (dropdown), 'admin' (dropdown), and 'G4' (dropdown).
 - Buttons:** A 'submit' button at the bottom of the form.
- Right Sidebar:** Contains a section for 'equipment location' with a 'submit' button and an 'action' dropdown set to 'modify'.

Figure 25 screenshot to show what happens when a malicious phrase is input into the search user by first name to modify search box

The screenshot shows a calendar view for the period from 2025-01-24 to 2025-02-28. The calendar is organized into a grid of days, with events listed for each day. The events are color-coded and include details such as time, title, and location. The calendar also includes a sidebar for event management and a top navigation bar.

Event Type	Event Name	Event Start	Event End	Approved	Event Owner
meeting	overview of training	05:00 PM	06:00 PM	not-approved	DROP

The calendar grid shows the following events:

- 2025-01-24 (parade_0):** 16:00:00 till 16:15:00 start parade CSM Henry McMahon; 17:00:00 till 18:00:00 overview of training CSM Henry McMahon; 18:00:00 till 18:15:00 final parade CSM Henry McMahon.
- 2025-01-31 (parade_1):** 16:00:00 till 16:15:00 start parade CSM Henry McMahon; 16:05:00 till 16:30:00 potentially conflicting event CSM Henry McMahon; 16:15:00 till 17:00:00 standards and values talk CSM Henry McMahon.
- 2025-02-07 (parade_2):** 16:00:00 till 17:00:00 principles version 1 courpral gianleo stubs principles version 1; 16:00:00 till 16:15:00 start parade CSM Henry McMahon; 18:15:00 till 18:30:00 final parade CSM Henry McMahon.
- 2025-02-28 (parade_3):** 16:00:00 till 17:00:00 lesson CSM jonnie barrie; 16:00:00 till 17:00:00 lesson CSM Henry McMahon.

Figure 26 screenshot to show what happens when a malicious phrase is input into the search by first name for a different event owner

[my calendar](#)
[help and guidance](#)
[add parade/event/account/equipment](#)

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-05-23 the next parade should be: 2025-05-30

date

start time

finish time

parade name

submit

add an event

event type

event name

event start

event end

event owner

current owner: none selected

duty cadet

users

add a user

email

password

first name

last name

date of birth

gender

rank

active

admin

G4

submit

modify a user

leave the password field blank to leave it unchanged

search by first name

equipment

add equipment

equipment name

equipment description

equipment location

submit

update equipment

search for equipment by name

search for equipment by location

equipment name

equipment description

equipment location

action

submit

Figure 27 screenshot to show a 254-character input for first name of a new user

[my calendar](#)
[help and guidance](#)
[add parade/event/account/equipment](#)

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-05-23 the next parade should be: 2025-05-30

date

start time

finish time

parade name

submit

add an event

event type

event name

event start

event end

event owner

current owner: none selected

duty cadet

users

add a user

email

password

first name

last name

date of birth

gender

rank

active

admin

G4

First name is too long

submit

modify a user

leave the password field blank to leave it unchanged

search by first name

equipment

add equipment

equipment name

equipment description

equipment location

submit

update equipment

search for equipment by name

search for equipment by location

equipment name

equipment description

equipment location

action

submit

Figure 28 screenshot to show a 255-character input for first name of a new user

127.0.0.1/NEA/add.php

my calendar help and guidance add parade/event/account/equipment

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-05-23 the next parade should be: 2025-05-30

date: mm/dd/yyyy

start time: --:08 PM

finish time: --:--:--

parade name:

Invalid time format (HH:MM)

submit

add an event

event type:

event name:

event start: 02:-- PM

event end: --:--:--

event owner:

current owner: none selected

duty cadet:

current duty cadet: none selected

parade:

current parade: none selected

Invalid time format (HH:MM)

submit

users

add a user

email:

password:

first name:

last name:

date of birth: mm/dd/yyyy

gender:

rank:

active: active

admin: non-admin

G4: non-G4

submit

modify a user

leave the password field blank to leave it unchanged

search by first name:

submit

equipment

add equipment

equipment name:

equipment description:

equipment location:

submit

update equipment

search for equipment by name:

search for equipment by location:

equipment name:

equipment description:

equipment location:

action: modify

submit

Figure 31 screenshot to show validation response of form add a parade when invalid time is input

127.0.0.1/NEA/add.php

my calendar help and guidance add parade/event/account/equipment

finish time: --:--:--

parade name:

submit

add an event

event type:

event name:

event start: 02:-- PM

event end: --:--:--

event owner:

current owner: none selected

duty cadet:

current duty cadet: none selected

parade:

current parade: none selected

Invalid time format (HH:MM)

submit

last name:

date of birth: mm/dd/yyyy

gender:

rank:

active: active

admin: non-admin

G4: non-G4

submit

modify a user

leave the password field blank to leave it unchanged

search by first name:

search by last name:

email:

password:

first name:

last name:

date of birth: mm/dd/yyyy

gender:

submit

update equipment

search for equipment by name:

search for equipment by location:

equipment name:

equipment description:

equipment location:

action: modify

submit

Figure 32 screenshot to show validation response of form add an event when invalid time is input

The screenshot shows a web application interface with three main sections: 'events/parades', 'users', and 'equipment'. The 'users' section is active, displaying the 'add a user' form. The email field contains 'email@website.d', which is highlighted in red with the error message 'invalid email format'. Other fields like password, first name, last name, date of birth, gender, rank, active status, admin status, and G4 are also visible. A 'submit' button is at the bottom of the form.

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-05-23

the next parade should be: 2025-05-30

date: mm/dd/yyyy

start time: --:--:--

finish time: --:--:--

parade name:

submit

add an event

event type:

event name:

event start: --:--:--

event end: --:--:--

event owner:

current owner: none selected

duty cadet:

users

add a user

email: email@website.d

password:

first name:

last name:

date of birth: mm/dd/yyyy

gender:

rank:

active: active

admin: non-admin

G4: non-G4

invalid email format

submit

modify a user

leave the password field blank to leave it unchanged

search by first name:

equipment

add equipment

equipment name:

equipment description:

equipment location:

submit

update equipment

search for equipment by name:

search for equipment by location:

equipment name:

equipment description:

equipment location:

action: modify

submit

Figure 33 screenshot to show validation response of add a user form when an email that has an invalid format is entered

This screenshot is identical to Figure 33, showing the 'add a user' form with the email 'email@website.com' entered, which triggers the 'invalid email format' error message. The interface layout, including the 'events/parades' and 'equipment' sections, remains the same.

events/parades

add a Parade

the latest scheduled parade is currently set to: 2025-05-23

the next parade should be: 2025-05-30

date: mm/dd/yyyy

start time: --:--:--

finish time: --:--:--

parade name:

submit

add an event

event type:

event name:

event start: --:--:--

event end: --:--:--

event owner:

current owner: none selected

duty cadet:

users

add a user

email: email@website.com

password:

first name:

last name:

date of birth: mm/dd/yyyy

gender:

rank:

active: active

admin: non-admin

G4: non-G4

invalid email format

submit

modify a user

leave the password field blank to leave it unchanged

search by first name:

equipment

add equipment

equipment name:

equipment description:

equipment location:

submit

update equipment

search for equipment by name:

search for equipment by location:

equipment name:

equipment description:

equipment location:

action: modify

submit

Figure 34 screenshot to show validation response of add a user form when an email that has an invalid format is entered

The screenshot shows a web application with three main sections: 'events/parades', 'users', and 'equipment'. The 'add a user' form is the central focus, containing the following fields and controls:

- email:** A text input field containing 'email@website.com'.
- password:** A text input field.
- first name:** A text input field.
- last name:** A text input field.
- date of birth:** A date picker showing 'mm/dd/yyyy'.
- gender:** A text input field.
- rank:** A text input field.
- active:** A dropdown menu with 'active' selected.
- admin:** A dropdown menu with 'non-admin' selected.
- G4:** A dropdown menu with 'non-G4' selected.
- submit:** A blue button with the text 'submit'.

Below the 'add a user' form, there is a 'modify a user' section with a 'search by first name' input field and a 'leave the password field blank to leave it unchanged' note.

Figure 35 screenshot to show validation response of add a user form when an email that has a valid format is entered

Definitions of tests

This section is to be read in conjunction with the testing video that I have created. It is 15 minutes long and available here: <https://www.youtube.com/watch?v=YLrflXSili0>

The video follows the natural order of the system instead of the order of the tests in their table format. This is intended to make the video easier to follow. The tests are however sequential where this is logical.

Login.php

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
L.1	Login page with text input for email and password input for password	n/a	n/a	Pass	0:00
L.2	Login page with submit button that posts results to login.php	n/a	n/a	Pass	0:00
L.3	Help page accessible from login screen (before the user has logged in)	n/a	The help page appears without the nav menu and a link back to the login page is in the left navigation menu for the help page	Pass	0:20
L.4	Incorrect email and incorrect password input	Erroneous	Login fails and user redirected to login.php where an error prompt will be displayed	Pass	0:08
L.5	Correct email and incorrect password input	Erroneous	Login fails and user redirected to login.php where an error prompt will be displayed	Pass	0:12
L.6	Correct email and correct password input	Valid	Login successful and user redirected to calendar.php	Pass	0:16

L.7	Correct email and correct password input but account is inactive	Valid	Login fails and user redirected to login.php where an error prompt is displayed	Pass	Figure 23
L.8	Email and password boxes left blank then submitted	Erroneous	Login fails and user redirected to login.php where an error prompt will be displayed	Pass	0:30

Included Nav.php

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
N.1	Drop down menu displays when user hovers mouse over profile photo	n/a	n/a	Pass	0:40
N.2	Calendar.php link is displayed for all users and add.php link is displayed for admin users only	n/a	n/a	Pass	Various
N.3	New profile photo of a non-accepted format is uploaded	Erroneous	Submission is blocked and user is made aware. (for resolution of fail, see further details at end of section)	Fail message not displayed	1:10
N.4	New profile photo of an accepted file format is uploaded	Valid	Old profile photo is removed, and new profile photo is accepted	Pass	0:50

Backup.sh

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
B.1	The backup file will be run once a day in the early hours of the morning	n/a	A backup file is created at this time and the backup must be able to fully restore the database to the state it was in when the backup was created	Pass	Figure 22

Calendar.php(standard user)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
C.S.1.1	Date skip back by 1 is clicked	n/a	The displayed dates shift back by 1	Pass	1:35
C.S.1.2	Date skip back by 5 is clicked	n/a	The displayed dates shift back by 5	Pass	1:35
C.S.1.3	Date skip forward by 1 is clicked	n/a	The displayed dates shift forward by 1	Pass	1:35
C.S.1.4	Date skip forward by 5 is clicked	n/a	The displayed dates shift forward by 5	Pass	1:35
C.S.1.5	Date skip back further than then history of parades in the database	Erroneous	User redirected to the current date version of the calendar page	Pass	1:45

C.S.1.6	Date skip forward further than the parade with the biggest date in the database	Erroneous	User redirected to the current date version of the calendar page	Pass	1:45
C.S.2	Date range displayed in the top banner of the calendar	n/a	n/a	Pass	various
C.S.3	Parades display the date and name of the parade	n/a	n/a	Pass	Various
C.S.4.1	Events always display their times and name	n/a	n/a	Pass	Various
C.S.4.2	Events where the client is on the register are displayed with grey background giving no indication of their approval stage	n/a	n/a	Pass	2:10

Calendar.php(an event owner)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
C.O.1	Event where client is the owner is displayed with a colour code indicating the approval stage of the event	n/a	n/a	Pass	2:25
C.O.2	Event where client is the owner display a link that takes the client to the event.php page for the client	n/a	n/a	Pass	2:25
C.O.3	Events are only displayed if the client is on the register or if the client is the event owner	n/a	n/a	Pass	various

Calendar.php(duty cadet)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
C.D.1	When a user is listed as the duty cadet, and they load a calendar page with said event	n/a	The event displays the words duty event in its heading, everything else in the event is displayed as if they are the owner of the event. If the user is also the event owner, then the event is displayed as an event they own instead.	Pass	11:40

Calendar.php(G4)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
C.G.1.1	All events for the parade are displayed	n/a	n/a	Pass	2:43
C.G.1.2	All events show their approval status	n/a	n/a	Pass	2:43
C.G.1.3	All events have a link to their event.php page	n/a	n/a	Pass	2:43
C.G.2	A log of approved equipment requests for each parade is	n/a	n/a	Pass	2:43

	displayed with the details of the equipment request				
--	---	--	--	--	--

Calendar.php(admin)

Test ID	Criteria	Data Type	Expected Result	Pass/Fail	Figure No.
C.A.1	All events for the parade are displayed	n/a	n/a	Pass	3:12
C.A.2	All events show their approval status	n/a	n/a	Pass	3:12
C.A.3	The admin edit panel is displayed	n/a	n/a	Pass	3:12
C.A.4.1	The admin panel edit form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active. An error handling message is displayed	Pass	3:20
C.A.4.3	The admin panel edit form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	3:30
C.A.4.5	The admin panel edit form allows submission when populated by another function	Valid	Submit button text goes white; visible/hidden fields are correctly populated	Pass	3:36
C.A.4.7	The admin panel edit form moves to valid state after correcting input errors	Valid	Submit button is active, and no error messages are displayed	Pass	3:20
C.A.5.1	The delete event button on the admin panel edit form denies submission if event_id is not populated	Erroneous	Text in submit button goes red and is no longer active	Pass	3:12
C.A.5.2	The delete event button on the admin panel edit form allows submission when populated by another function	Valid	Submit button text goes white; visible/hidden fields are correctly populated	Pass	3:50
C.A.6.1	The search for an event owner box displays a response	Valid	Results appear in the results box	Pass	4:00
C.A.6.2	The search for an event owner box clears results when the prompt is cleared	Valid	Results box is emptied	Pass	4:00
C.A.6.3	The search for an event owner box will fail the search when a malicious input is detected	Malicious	The result of the search prompt is displayed as search failed	Pass	Figure 26
C.A.6.4	The search for an event owner box allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	4:00

C.A.7	The max date skips backwards button skips the date by 4 instead of 5	n/a	n/a	Pass	4:20
C.A.8	The max date skip forwards button skips the date by 4 instead of 5	n/a	n/a	Pass	4:20

Add.php(non-admin user)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
A.S.1	User attempts to load the page	Erroneous	User redirected to calendar.php since they don't have the correct permissions for add.php	pass	4:40

Add.php(admin)

Test ID	Criteria	Data Type	Expected Result	Pass/Fail	Figure No.
A.A.1	Latest scheduled parade date is correct	n/a		n/a	4:50
A.A.2	Next parade date is correct	n/a		n/a	4:50
A.A.3.1	Add a parade form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	5:00
A.A.3.2	Add a parade form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	5:00
A.A.3.3	Add a parade form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	5:00
A.A.4.1	Add an event form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	5:50
A.A.4.2	Add an event form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	5:50
A.A.4.3	Add an event form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	5:50
A.A.4.4	Add an event form denies submission if: parade_id, event owner_id or duty cadet id are not populated	Erroneous	Text in submit button goes red and is no longer active	Pass	5:50

A.A.5.1	Search for parade by name displays a response	Valid	Results appear in the results box	Pass	6:20
A.A.5.2	Search for parade by name clears results when the prompt is cleared	Valid	Results box is emptied	Pass	6:40
A.A.5.4	Search for parade by name allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	6:20
A.A.5.5	Search for parade by name calls function for new result display	Valid	Displays the parade name and the date the parade is being held on	Pass	6:40
A.A.6.1	Search for owner by name displays a response	Valid	Results appear in the results box	Pass	6:20
A.A.6.2	Search for owner by name clears results when the prompt is cleared	Valid	Results box is emptied	Pass	6:40
A.A.6.4	Search for owner by name allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	6:20
A.A.6.5	Search for owner by name calls function for new result display	Valid	Displays owner of the event	Pass	6:20
A.A.7.1	Search for duty cadet by name displays a response	Valid	Results appear in the results box	Pass	6:20
A.A.7.2	Search for duty cadet by name clears results when the prompt is cleared	Valid	Results box is emptied	Pass	6:40
A.A.7.4	Search for duty cadet by name allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	6:20
A.A.7.5	Search for duty cadet by name calls function for new result display	Valid	Displays the duty cadets full name in the display div	Pass	6:20
A.A.8.1	Add a user form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	7:10
A.A.8.2	Add a user form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	7:10
A.A.8.3	Add a user form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	7:50

A.A.8.3	The add a user form by default has admin set to non-admin, active set to active and G4 set to non-G4	n/a	n/a	Pass	7:10
A.A.9.1	Search for user by first name displays a response	Valid	Results appear in the results box	Pass	8:10
A.A.9.2	Search for user by first name clears results when the prompt is cleared	Valid	Results box is emptied	Pass	8:45
A.A.9.3	The search for a cadet by first name will fail the search when a malicious input is detected	Malicious	The result of the search prompt is displayed as search failed	Pass	Figure 25
A.A.9.4	Search for user by first name allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	8:10
A.A.9.5	Search for user by first name calls function for new result display	Valid	Displays the user details in the form	Pass	8:10
A.A.10.1	Search for user by last name displays a response	Valid	Results appear in the results box	Pass	5:50
A.A.10.2	Search for user by last name clears results when the prompt is cleared	Valid	Results box is emptied	Pass	5:50
A.A.10.4	Search for user by last name allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	8:50
A.A.10.5	Search for user by last name calls function for new result display	Valid	Displays the user details in the form	Pass	8:50
A.A.11.1	Modify a user form denies submission if user_id is not populated	Erroneous	Text in submit button goes red and is no longer active	Pass	Various
A.A.11.2	Modify a user form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	8:25
A.A.11.3	Modify a user form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	8:25
A.A.11.4	Modify a user form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	8:30
A.A.11.5	Modify a user form allows submission when populated by another function	Valid	Submit button text goes white; visible/hidden fields are correctly populated	Pass	8:25
A.A.11.6	Modify a user form carries out error handling correctly if input errors occur	Erroneous	Error handling messages appear in the error handling div	Pass	8:25
A.A.11.7	Modify a user form moves to valid state after correcting input errors	Valid	Submit button is active, and no error messages are displayed	Pass	8:30

A.A.12.1	Add equipment form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	9:20
A.A.12.2	Add equipment form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	9:20
A.A.12.3	Add equipment form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	9:20
A.A.13.1	Update equipment form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	9:50
A.A.13.2	Update equipment form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	9:50
A.A.13.3	Update equipment form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	9:50
A.A.13.4	Update equipment form denies submission if equipment_id is not populated	Erroneous	Text in submit button goes red and is no longer active	Pass	9:50
A.A.13.5	Update equipment form allows submission when populated by another function	Valid	Submit button text goes white; visible/hidden fields are correctly populated	Pass	9:50
A.A.13.6	Update equipment form carries out error handling correctly if input errors occur	Erroneous	Error handling messages appear in the error handling div	Pass	9:50
A.A.13.7	Update equipment form moves to valid state after correcting input errors	Valid	Submit button is active, and no error messages are displayed	Pass	9:50
A.A.14.1	Search for equipment by name displays a response	Valid	Results appear in the results box	Pass	10:35
A.A.14.2	Search for equipment by name clears results when the prompt is cleared	Valid	Results box is emptied	Pass	10:35
A.A.14.3	The search for equipment by name will fail the search when a malicious input is detected	Malicious	The result of the search prompt is displayed as search failed	Pass	Figure 24
A.A.14.4	Search for equipment by name allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	10:35

A.A.14.5	Search for equipment by name calls function for new result display	Valid	Displays the details of the equipment selected in the modify equipment form	Pass	10:35
A.A.15.1	Search for equipment by location displays a response	Valid	Results appear in the results box	Pass	10:35
A.A.15.2	Search for equipment by location clears results when the prompt is cleared	Valid	Results box is emptied	Pass	10:35
A.A.15.4	Search for equipment by location allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	10:35
A.A.15.5	Search for equipment by location calls function for new result display	Valid	Displays the details of the equipment selected in the modify equipment form	Pass	10:35
A.A.16	Update equipment form by default will always have the action set to modify	n/a	When form is populated, action will be set to modify	Pass	10:35

Event.php(standard user)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/ figure
E.S.1	User attempts to load the page	Erroneous	User redirected to calendar.php since they don't have the correct permissions for add.php	Pass	10:55

Event.php(event owner non-approved event)

Test ID	Criteria	Data Type	Expected Result	Pass/Fail	Figure No.
E.O.N.1	Client loads a non-approved event	n/a	The modify event details form appears populated with the details of the lesson. The update event details and delete event buttons are active	Pass	11:40
E.O.N.2	The list of events the client has permission to view for the parade is correct	n/a	Criteria is met	Pass	11:40
E.O.N.3	The drop-down box for approval of the modify event details form is correctly set	n/a	The drop-down box has the correct value set by default	Pass	11:40

E.O.N.4	If owner is non-admin and final approval is attempted at being moved to approved	Erroneous	Error prompt is displayed to the client and submit button moved to inactive	Pass	11:40
E.O.N.5	The full name of the event owner is displayed on loading the page	n/a	n/a	Pass	12:05
E.O.N.6.1	Modify event details form denies submission when invalid data is entered	Erroneous	Text in submit button goes red and is no longer active	Pass	12:10
E.O.N.6.2	Modify event details form displays error handling prompts when inputs are invalidly populated	Erroneous	Error handling messages appear in the error handling div	Pass	12:10
E.O.N.6.3	Modify event details form allows submission when correctly populated	Valid	Form is submitted and redirects user to appropriate page	Pass	12:15
E.O.N.6.5	Modify event details form moves to valid state after correcting server-side populated errors	Valid	Submit button is active, and no error messages are displayed	Pass	12:15
E.O.N.8.1	Search for new owner by first name on the modify event form displays a response	Valid	Results appear in the results box	Pass	various
E.O.N.8.2	Search for new owner by first name on the modify event form clears results when the prompt is cleared	Valid	Results box is emptied	Pass	various
E.O.N.8.4	Search for new owner by first name on the modify event form allows result selection	Valid	Results box is emptied, prompt cleared, and result is input into the form	Pass	Various
E.O.N.8.5	Search for new owner by first name on the modify event form calls function for new result display	Valid	Displays owner of the event	Pass	various

Event.php(event owner approved event)

Test ID	Criteria	Data Type	Expected Result	Pass/Fail	Video time/figure
E.O.A.1	Client loads the event.php page for the event	n/a	The modify event details form is not displayed	Pass	13:30

E.O.A.2.1	Register is correctly displayed	n/a	If cadets are on the register the form is populated correctly. If no cadets are on the register, an appropriate message is displayed	Pass	13:30
E.O.A.2.2	When client enters something other than 1 or 0 into the register	Erroneous	The form moves to invalid state and can't be submitted. The name of the cadet with the error is displayed	Pass	16:15
E.O.A.3.1	Search by first name for cadet to add to the register displays a response	Valid	Results appear in the results box	Pass	13:30
E.O.A.3.2	Search by first name for cadet to add to the register clears results when the prompt is cleared	Valid	Results box is emptied	Pass	13:30
E.O.A.4.1	Search by first name for cadet to remove from the register displays a response	Valid	Results appear in the results box	Pass	13:50
E.O.A.4.2	Search by first name for cadet to remove from the register clears results when the prompt is cleared	Valid	Results box is emptied	Pass	13:50
E.O.A.5.1	Search by first name for what another cadet is doing during the event displays a response	Valid	Results appear in the results box in the video users "rank admin admin" and "admin admin admin" are different	Pass	14:15
E.O.A.5.2	Search by first name for what another cadet is doing during the event clears results when the prompt is cleared	Valid	Results box is emptied	Pass	14:15
E.O.A.6	Equipment request log is correctly displayed	n/a	If equipment requests have been made, the log correctly shows them. If no requests have been made, an appropriate message is displayed	Pass	14:30
E.O.A.7	If the owner is non-G4 and attempts to change the status of an equipment request	Erroneous	The user is prevented from submitting the form and an appropriate message is displayed	Pass	15:51
E.O.A.8.1	Search by equipment name for equipment to add to the log displays a response	Valid	Results appear in the results box	Pass	15:00

E.O.A.8.2	Search by equipment name for equipment to add to the log clears results when the prompt is cleared	Valid	Results box is emptied	Pass	15:00
E.O.A.9.1	Search by equipment name for equipment to remove from the log displays a response	Valid	Results appear in the results box	Pass	15:30
E.O.A.9.2	Search by equipment name for equipment to remove from the log clears results when the prompt is cleared	Valid	Results box is emptied	Pass	15:30
E.O.A.10.1	Search by equipment name for where another piece of equipment is during the event displays a response	Valid	Results appear in the results box	Pass	16:00
E.O.A.10.2	Search by equipment name for where another piece of equipment is during the event clears results when the prompt is cleared	Valid	Results box is emptied	Pass	16:00

Event.php(G4)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
E.G.1	Page passes the testing criteria that applies for event.php when the event owner is visiting the page	n/a	n/a	Pass	16:40
E.G.2.1	If the user attempts to change a decision on an equipment request.	Erroneous	The equipment request log form is disabled, and a message is displayed to the user.	Pass	16:40
E.G.2.2	If the user attempts to set the decision for an equipment request	Valid	They are able to and the form is able to submit with no error handling prompts	Pass	16:40

Event.php(admin)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
E.A.1	Page passes the testing criteria that applies for event.php when the event owner is visiting the page	n/a	The criteria is met	Pass	Various
E.A.2	When the event is not approved and the user attempts to move the approval to approved	valid	No error prompt is displayed, and the modify event details for is able to be submitted.	Pass	12:55

Event.php(duty cadet)

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
E.D.1	Page passes the testing criteria that applies for event.php when the event owner is visiting the page	n/a	The criteria is met	Pass	12:30

General tests for forms

Test id	Criteria	datatype	Expected result	Pass/fail	Video time/figure
F.1	Add a user on add.php does not accept an invalid email format	erroneous	Form submit button is deactivated and user is made aware of the issue	Pass	Figures: 33, 34, 35
F.2	Add a parade on add.php does not accept an invalid time input	erroneous	Form submit button is deactivated and user is made aware of the issue	Pass	Figures: 31, 32
F.3	Inputting a string of length greater than 255 characters for a user's first name in the add a user form on add.php	erroneous	Form submit button is deactivated and user is made aware of the issue	Pass	Figures: 27, 28
F.4	Inputting a string of length greater than 255 characters for an equipment name in the add equipment form on add.php	erroneous	Form submit button is deactivated and user is made aware of the issue	Pass	Figures: 29, 30

Failed tests

The test N.4 failed due to the fact the program was told to die as soon as an incorrect file format was found to have been uploaded. I have since fixed this by getting the program to die with a message that is then displayed on the html document. This solution keeps the security benefits of the program dying and will display a message to the user.

Evaluation

Objective analysis

Overall, the objectives have been met, they are individually appraised below.

Objective Area	Specific Objectives
Security of the system	1, 2, 3, 4
User experience and guidance	5, 6
Calendar page	7, 8
Administrator permissions	9, 10, 11, 12, 13, 14
User types	15, 16, 17
Lesson planning	18, 19
Register options	20, 21, 22
Input validation	23, 24
Searching	25, 26, 27, 28
Flexibility	29
Back up of database	30

N.O.	Objective	Has the objective been met?	Evidence reference	Comment
1	One way encryption of passwords	Yes	Technical solutions login.php line 16	This was achieved through the hash sha-256 which is computationally secure.
2	Appropriate access control on accounts so that junior cadets can't access functions intended for staff/ senior cadets.	Yes	Test: A.S.1, E.S.1	Currently there are server-side restrictions on what pages accounts can access. The eventual intention of the system will be to have individual database logins for each account. This would further increase the security and prevent data breaches.
3	Users should be redirected to appropriate alternative pages in the event that they don't have the permissions required to visit a page.	Yes	Test: A.S.1, E.S.1	This is shown in the tests for event.php as a standard user and add.php as a standard user. The system checks the permissions of the client and if they do not meet the requirements they are redirected to the calendar page. Additionally, if a user attempts to access a page without login in the function check_login will redirect them to the login page

N.O.	Objective	Has the objective been met?	Evidence reference	Comment
4	A navigation bar different for users of different privileges, allowing them to select the task they wish to undertake	Yes	Test: N.2	This navigation bar is different for the different users i.e. admin/non-admin. This is evident in the testing section for the included nav.php page.
5	Easy to use by people with a low level of IT experience	Yes	See end user feedback	Unknown from testing and technical solutions. This objective is met and shown in the section for end user feedback
6	Create a help page to guide the user through the use of the system	Yes	Test: N.2	This is evident in the testing section for the included nav.php page and the login.php page.
7	Users must be able to skip forwards and backwards on the range of dates to be displayed on the calendar page	Yes	Tests: C.S.1	This is evident in the testing section for the calendar page as a standard user and as an admin.
8	Calendar must display events in different formats according to the role of the user	Yes	Tests: C.O.1, C.D.1, C.S.4.2	This is evident in the testing section for the calendar pages as a standard user, event owner, duty cadet and admin.
9	Administrator must be able to create/modify/deactivate accounts with permissions as they see fit	Yes	Tests: A.A.8	Yes, this has been achieved and is evident in my testing section for the add.php page for the admin using the modify user form.
10	Administrator must be able to create parades and name them, so they are easy to search for	Yes	Tests: A.A.3	This has been achieved evident in the testing section for add.php page for the admin using the add a parade form.
11	Administrator must be able to approve events	Yes	Test: E.A.2	Yes, this has been achieved evident in the testing section for event.php as an admin.
12	Administrator must be able to create and modify non-approved events	Yes	Tests: A.A.4, C.A.4	This has been achieved and is evident in the testing section for event.php as an admin and in calendar.php as an admin using the admin edit panel.
13	Administrator must be able to create/modify/delete pieces of equipment from the database	Yes	Tests: A.A.12, A.A.13	This has been achieved evident in the testing section for add.php for the admin using the add equipment form.
14	On the admin panel page (add.php), sensible default values must be used	Yes	Tests: A.A.8.3	This has been achieved and is evident in the test A.A.8.3 and the test A.A.16 both of these show sensible default values in drop down boxes for the forms add a user and modify equipment on the add.php page as an admin.

N.O.	Objective	Has the objective been met?	Evidence reference	Comment
15	Specialised accounts for the logistics cadets (G4) to view and approve/deny equipment requests	Yes	Tests: A.A.8, A.A.11, E.G.2, C.G.2	This has been achieved; admin accounts can give this permission evident in the testing section for add.php for the admin using the modify user form. The use of this permission for viewing all events happening on a parade is evident in the testing section for the calendar.php page for G4 users. The use of this permission to approve/deny requests is evident in the testing section for add.php for G4 users when using the equipment requests log form.
16	The system must have the position of event owner, with one assigned to every lesson to oversee event planning	Yes	Tests: A.A.6	This has been achieved when admin accounts create events, they must assign an event owner, evident in the testing section for add.php for admins using the create an event form. While the event is not-approved the event owner can change at any time using the modify event details form this is evident at various points in the testing section for event.php.
17	The system must allow duty cadets to view the progress on lessons	Yes	Tests: E.D.1	This has been achieved when admin accounts create events, they must assign an event duty cadet. Evident in the testing section for add.php for admins using the create an event form. Duty cadets can then see events on the calendar page that they have a duty to this is evident in the testing section for calendar.php page for duty cadets.
18	The system must allow the administrators to view lesson planning	Yes	Tests: E.A.1	This has been achieved within the system since admins are able to view event.php pages for all events happening on a parade on the calendar.php page. This is evident in the testing section for the calendar.php page for the admin.
19	The system must allow lesson planners to request equipment, and that piece of equipment must be reserved for the duration of the lesson when the request is approved	Yes	Tests: E.O.A.6, E.O.A.8, E.O.A.9, E.O.A.10	This has been achieved through the event.php page when the event is approved. This is evident in the testing section for event.php for event owners using the search for equipment to add function.
20	The system must allow lesson planners to fill out their register at the start of their lesson. This register state must then be stored on the database for later review and use	Yes	Tests: E.O.A.2,	This has been achieved through the event.php page when the event is approved. This is evident in the testing section for event.php for event owners using the search for a cadet to add to the register.

N.O.	Objective	Has the objective been met?	Evidence reference	Comment
21	The system must allow lesson planners to populate i.e. add/remove cadets from the register for the lesson. The register must ensure that a cadet is not in 2 lessons at the same time	Yes	Tests: E.O.A.3, E.O.A.4	Yes, this has been achieved through the event.php page when the event is approved. This is evident in the testing section for event.php for event owners using the search by first name cadet to add to the register function.
22	The register for an event must have a facility to search for a cadet to see what lesson they are in if they can't be added to the register	Yes	Tests: E.O.A.5	Yes, this has been achieved through the search cadet other box on the register section of the event page.
23	Validation rules to ensure reliable data entry and prompts to make users aware of the rules	Yes	All form validation tests	This is evident throughout the testing section. It is important to note that in the future the system should also validate data on the server side to prevent malicious requests being made through improper means from being acted upon.
24	When a user enters an invalid input into a form then corrects it they must be able to recover and submit the form.	Yes	All form validation tests	This is evident throughout the testing section. It is shown that when a form is populated invalidly and then corrected the error prompt disappears and the user is able to submit the form.
25	The system must scan all search prompts for malicious code to prevent attacks on the system.	Yes	Tests: C.A.6.3, A.A.9.3, A.A.14.3	This is evident throughout the testing section. It is shown in the tests A.A.14.3, A.A.9.3 and C.A.6.3 where a malicious input of DROP in input and the program dies as a result protecting the system.
26	The system must have a way to search for entities on the database i.e. cadets to add to a lesson register	Yes	All search tests	This is evident throughout the testing section. The example is evident in the testing section for add.php for event owners using the search for a cadet to add to the system. In the future this functionality could be improved by implementing search rules i.e. ignoring space characters at the start of searches. This would be implemented through the get requests scanning file for any get request with the word prompt as its identifier.
27	Search fields must show a dynamic output	Yes	All search tests	This is evident throughout the testing section all of the search boxes are shown to update their search results based on single key input changes. Additionally, if the input is an empty string then the results will disappear.
28	When searching for equipment or users to modify, more search options than just first name must be offered	Yes	tests: A.A.9, A.A.10	This has been achieved and is evident in the testing sections for the modify a user and modify equipment form. Which allow for searching by last name of a user and searching by location of equipment.

N.O.	Objective	Has the objective been met?	Evidence reference	Comment
29	The system must allow for as wide a variety of naming conventions as possible	Yes	See comment	This is evident throughout the system i.e. naming parades, naming equipment, naming equipment locations. In the future it would be helpful for the admin to produce a document explaining the logic behind the chosen naming conventions this document could then be displayed in the help section.
30	The system must perform a regular backup of the database in case of corruption or attacks	Yes	Test: B.1	This is evident in the definition of the tests for backup.sh. There it is demonstrated that the server has regularly preformed backups of the database that could be used to restore the DB at any time.

End user feedback

Major Jackson (admin)

I have given my admin client Major Jackson a chance to use and explore the system over a couple of days and assess its suitability for the CCF. The aim of this is to simulate an end user experience in a controlled environment and gain feedback. After the testing period I arranged an interview with him to hear his thoughts on the system and any suggestions he may have for its future.

During the interview I asked Major Jackson about a range of topics within the system. My first question was “Do you feel the system has a place in the CCF”. His response was “Yes”, which he further went on to say “The system fulfils its original purpose well and is easy to navigate. Particularly the calendar and add pages”. I then went on to ask him about his favourite feature, which was the equipment feature. He then elaborated on this explaining that the CCF has a lot of equipment it needs to keep track of and, a system where this equipment is searchable with all the relevant details of the equipment attached will be a great asset to help keep track of stock. I then asked him about his least favourite aspect of the system. He then explained that the fact events were ‘set in stone’ once approved was inconvenient. When I asked in what way was this inconvenient, he elaborated that if a mistake has been made in the details of an event and the approval decision was rushed. Then the event must be deleted and re-created for this mistake to be amended which he thought was an area for improvement.

I then went on to ask him about individual pages and their strengths and weaknesses. First was the login page which he liked especially the direct link to the help page. However, he thought a lack of a remember me function was a weakness but admitted that the automatic password saving that his browser offers is a workaround that he is used to using. Then I asked him about the calendar page which was his favourite saying it is easy to navigate and intuitive. However, he said when the admin panel edit form is submitted, “I find it frustrating that the page forgets the date range that has been chosen.”

Following this we discussed the event page which he said had the most application in the CCF. However, he admitted that it was the most difficult page to navigate initially since there is a lot going on when the event is approved and there are many items on the register/ equipment requests on the log. He also explained it may be better if the add equipment/cadets search boxes were combined to display the possible items to add and the items that couldn't be added. Finally, we discussed the add page which he found very intuitive, and he really liked how the error handling prompts were displayed in red making the easily noticeable. He thought that in the future when the system contains more users and equipment it would be beneficial to have pages dedicated to them where more details about the results of any searches would be shown instead of just the names of the items being searched.

To conclude the interview, we discussed the potential applications of the system outside the St Columba's College CCF. Presently there is an online platform for cadets called the Cadet Portal, it was designed to give cadets a more complete view of their progress through the cadet syllabus. Currently cadets at St Columba's College don't have a desire to use it, since it doesn't provide any information that Major Jackson or Captain Hewitt can't tell us. That being said he believes the system that I have built could aid cadet forces across the country. If the management team of the cadet portal agree, then young people and adults in the cadet force could benefit. From a better way of managing their activities, delegating tasks and tracking the progress of their cadets through the syllabus. This will aid cadets in their development of digital planning and preparation skills which have huge application later in life particularly the workplace in any managerial positions.

In order to ensure this system is a step in the right direction for cadets, irrespective of rank or role within the cadet force. Major Jackson recommended that I conduct similar trial periods, and interviews with cadets for their different roles within my system.

During one of my study periods, I conducted a trial period and interview with Ben. Before the trial of the system, we first discussed the process that goes into preparing all equipment requests, for Friday parade nights. He said that he receives 7 to 10 request lists a week from a variety of sources which he listed as: WhatsApp, email and convocations. These requests are not stored centrally anywhere and there is no scheduling system. Which he admitted frequently leads to conflicting requests that are not discovered until the parade night.

Following this I had Ben read the help and guidance sections for G4 and all users. He fed back that the page for help/guidance was easy to navigate, and its content was clear and concise. I then asked him to navigate to the calendar page and describe what he was seeing. He correctly identified the skip forwards/backwards buttons on the calendar as well as the approved equipment requests log. Before we continued, he fed back that having a log of equipment requests all in one place will be a great asset to G4 that will surely increase their efficiency. Unfortunately, he did not identify the meanings behind each of the approval colours for events. I realised he hadn't read the section where this is covered, so we quickly covered it there and then.

Next, I asked him how he would find requested equipment and make decisions on if it could be provided. He then navigated to the event page for the starting event on a parade with many approved events. He then worked his way through all the events and made mock decisions on if the request was able to be provided. This was exactly what I had hoped he would do, and I then asked him to tell me what equipment he had to provide for a parade. To which he navigated to the calendar page and read off the equipment request log. This marked the conclusion of the trial period.

Following this I asked him for any improvements we could make to the system. He presented the following suggestions:

1. Showing requested and approved equipment in the blocks that display events to take some of the guess work out of which event the equipment is for if the names of the events are the same.
2. Allowing for many events to request the same piece of equipment
3. When a request is approved the requests that conflict with this should be shown and automatically denied when the approval is submitted to the server.
4. Add a facility to request rooms that are being used so it can be known where the equipment request will be going to.
5. Add a facility to view requests that do not conflict but are used many times in a parade night so G4 can inform the owners of the events that they must hand the equipment over to the next event owner

In light of improvement 4 we discussed adding lesson locations to the database of equipment, he said was a valid and useful solution to the potential problem. We then went on to discuss the benefits of using this system. He highlighted that the system was easy to navigate, and someone could likely learn it with just a short period of time without reading the help/guidance.

We then discussed the centralised nature of the system. Which would be extremely useful for controlling what was coming in/out of the equipment stores, allowing for centralised prior preparation of equipment request. Additionally, since the system handles all the scheduling automatically, he believed G4 would likely see an efficiency boost that would be sustained. In order to realise this efficiency boost, the system would need to be populated with the equipment available for request. He admitted that this would be an initial input task, however it would be worth it since it will only need to be performed once.

Finally, I brought up the question of how he would decide what equipment requests to approve/deny. Ben responded by saying it would most likely come down to what he knows of the lesson. As a result, I presented the idea of making the lesson plan accessible through the system to which he said this would help inform approval decisions. In conclusion this interview was very positive and it is clear that the logistics team will significantly benefit from the systems implementation. He could really see how parades and requests could be managed effectively. He really liked the system.

During a lunch at school, I took the opportunity to introduce Tom to my system, for potential feedback. I began by logging him into an account that was event owner of many events. I then navigated him to the help/guidance page and asked him to read the sections on all users and event owners.

Following this we began the interview. I asked him to navigate to the calendar page and explain to me what he was seeing. He correctly identified the approval status of the events and after being prompted, he elaborated explaining what the different approval states of events meant for the functionality on offer to the owner. Additionally, he was able to identify events that he was just a cadet on the register for (events appear Gray). After this I asked him to show me how to move forward and backward in time on the calendar which he was able to do without prompt.

Next, I asked him how he would show that one of the not-approved events on the calendar is ready for review by the Admin. He then navigated to the event page for the event and was able to change the approval status to requested and submit the form. Following this I asked him how he would hand ownership of the event back to his duty cadet which he was able to successfully do by searching for the duty cadet's first name and selecting him.

To keep the trial realistic, I asked him how he would add cadets to the start parade that he was owner of. He then navigated to the parade from the calendar and was able to add a list of 5 cadets to the register. Finally, I asked him how he would add 3 equipment requests for rifles to the start parade. He was able to successfully perform the equipment requests that were not conflicting with other events. However, on the last request the item did not show up which he believed was because it was not on the system. However, I explained this could only be the case if the item doesn't show up when searched for in the other box, this then made perfect sense to him. Finally, I asked him how he would logout of the system which he did without prompt. This marked the end of the structured section of the trial period and interview.

Following this we discussed what he had thought so far. First, I asked him about the help page and whether it was clear what he as a user was able to do. He replied stating the help page was clear and descriptive, however it could benefit from some photos of the interface. Staying on the topic of user experience I asked him how intuitive the system was to navigate. His response was positive stating that the user interface was very easy to understand and, similar to software's that provide a similar service i.e. online calendar. I then asked him about potential improvements I could make in the user experience. He quickly identified that combining the equipment request add and other boxes would make the interface simpler and easier to understand/use.

The next portion of the interview focused on the utilisation of this software in the St Calumbas College CCF. His first question to me was if there was an ability to request rooms for teaching the lessons. I responded that while there is no purpose-built feature for this, it is possible through the equipment requests feature. He responded positively to this elaborating that it is a common issue that cadets face and, by having rooms booked for events this issue would crop up less frequently. To conclude the current system review, Tom thought the system had a place in the CCF elaborating that it solves many of the inefficiencies in the planning, logistics and delegation processes.

Following this I asked him about potential improvements the system could have, to which he listed the following:

- Production of standard lists of cadets on registers and a facility to use these i.e. lists of cadets all in the same year group
- Production of standard lists of equipment requests for the equipment request log to be used for standard lessons i.e. navigation lessons will have a standard request for: 10 compasses, 10 maps and 10 route cards
- Facility for the duty cadet of an event to approve an event so the workload for admins is reduced speeding up the planning process
- Messaging facilities for events so it can be made clear without the use of emails what changes must be made to the plan of an event to be moved to approved

Student 3 (standard user)

Due to the simplistic nature of the standard user accounts, I conducted the trial period and the interview with Student 3 in one sitting during a lunch time. We began the trial period by me logging him on to the system and then taking him to the help/guidance page. From here he read the sections for all users and standard users. I then asked him to go to the calendar page and show me what he could see and what it meant. He correctly identified the different parades and what they would mean i.e. Fridays after school from 4:00 pm till 6:30 pm. He also identified the lessons that he would be in and what the lesson topics would be. I then asked him to look at parades in the future which he correctly did by pressing the > button, he was also able to go back on the dates for the calendar. Following this, I asked him to find out his rank on the system which he correctly did by hovering over the profile photo and viewing his name. finally, I asked him to identify how to logout of the system which he also successfully managed to do. From this trial period and structured interview, I have concluded the system is intuitive and useable for standard users.

Following this we then had time to discuss the utility of the system for him and his peers. He said that the system would be helpful for knowing what you're doing in the near future. However, he felt that more information could be accessible through the calendar. He elaborated on this by giving the example of knowing you are in a lesson and knowing what equipment you would need to bring i.e. notebook and pencil or cold weather clothing. I then presented the idea of having sections for what clothes you should have for each lesson and a section for equipment you should have. He liked these ideas and thought it would further contribute to the utility of the software.

End user feedback analysis

In conclusion of the feedback received I have learned the following:

- The user interface is intuitive to use and easy to understand. The help/guidance page provides adequate advice on the use of the software.
- All groups of users identified see this system as a step in the right direction for cadets/ adults in the CCF.
- All groups of users see a practical application of the system in routine tasks, and they agree that this system will save time and increase the productivity of cadets/ adults during planning phases.
- All groups particularly G4, will benefit from equipment requests being centralised and equipment stores being properly catalogued.
- The system has been highlighted as an opportunity for cadets to improve their digital planning and preparation skills, which will have application later in life.

Additionally I have learned the following features will be a welcome addition to the system:

- Reduced structure of rules surrounding event approval to make the system more dynamic
- Increased visibility for junior cadets of their upcoming lessons i.e. facility to see what they need to bring, what they should wear and where they will be
- Enhanced search functionality that is more in line with industry norms i.e. combining the add and other boxes for the registers and the equipment requests

I have also been presented with the following ideas, of how to integrate the system in a fully practical setting:

- When cataloguing the equipment in the stores, the admin should have conventions to serialising the names of equipment and serialising the locations equipment will be. This will make working through equipment requests a less complex and more structured job.
- The system could be used in conjunction with Microsoft teams and when using a file system that is properly organised. Concurrent access to planning documents could easily be utilised in the planning process.

Overall, I am pleased with the feedback given. It is clear that this system is a welcome addition to the arsenal of digital tools that cadets of today have at their disposal. Although improvements have been suggested it is clear that the system as it currently operates is functional and caters for the day-to-day needs of the majority of cadets in the CCF. It brings me satisfaction that I have built a system that will improve the operations of the CCF beyond what we had when I first joined in year 8.

Future technical improvements

As the developer of this system, I believe the ‘elephant in the room’ is the usage of the HTTP protocol for client server communication in all aspects. This is problematic for a multitude of reasons primarily its plaintext nature that would allow anybody snooping a network to view password hashes and any information sent across the network. This could in future could be updated to https by registering for an SSL certificate. My second major improvement of the system would be setting up individual MySQL accounts for each user. This would allow for better permission management and improved concurrent access to the database i.e. cadets from day one will no longer have access to: insert, update and delete SQL commands. Additionally, rules could be set up on the system so users can only view records they have a connection to i.e. events where they are on the register. Finally in the event that an attacker was able to disrupt the database, identifying the attacker when they have a unique MySQL account will be a lot easier.

Outside of these security concerns. I would make the following technical enhancements to the system:

1. Re-write the logic behind moving forward and backward on the calendar, so the parade_id is used instead of the parade date. This will allow many parades to have the same date as originally intended.
2. Add a parade_id attribute to the user_data dictionary that would indicate the left most parade on the calendar page, so when the admin does submit the admin panel edit form, they will be re-directed to the calendar page they had originally.
3. The parade starts and end times could be used when validating the times for an event. Since as it currently stands the start and end times of a parade are not used anywhere in the system.
4. When validating data on the client side, I would like to use switch statements instead of many else if statements since this will increase the efficiency and readability of my code.
5. A area for improvement would be ensuring the spelling of variables and attributes in the codebase are correct throughout. I found occasionally some spellings would go initially un-noticed in the earlier stage of development, these would then become embedded in the system and difficult to change. This would have to be corrected as part of the next iteration of the system.
6. Change many of the output sections of my search features, so they use JSON instead of HTML for communicating search results. This would be beneficial since it would decrease the overall load on my server.
7. When searching for cadets/equipment requests to remove, I would like to perform these actions on the client side. since dictionaries of the entities in these forms already exist on the client side this will further reduce the load on the server.
8. When a search result is clicked it would be useful to have a way to update the registers/equipment requests when a result is clicked without reloading the page since the majority of it is unchanged. Therefore, this request is majority un-necessary.
9. Re-write many of my SQL queries so they use table joins more frequently since this will decrease the time and space complexity of many of my queries. Which will further reduce the load on the server and decrease search response times.
10. Add in a level of permission above the admin and call it super admin. This user will be able to restore the database in the event it is deleted. They will be able to do this through a graphical user interface.
11. Re-design the rules of the system so events that have been approved can have attributes changed so events don't have to be deleted then re-created.
12. Store pre-made lists of cadets/equipment requests (similar to how email groups work) so standard groups of cadets can be added to an event.

Future functionality and user experience improvements

Following the trial periods and interviews with my end users I would like to make the following changes to the system in the future.

1. Facility to display more details about lessons for all users. These details would be equipment lists and dress lists(clothing). These will be editable on the event page for the event, and they will be beneficial to all people hopefully leading to cadets being more prepared.
2. In general, I would like to utilise the profile photos of users more often. I think this will be particularly helpful when searching for cadets and when completing the register, the profile photos of each cadet could be displayed.
3. When a user is visting an event.php page of an event multiple people are also visiting. There should be an ability to see what other users are visiting.
4. For event pages there should be a capacity for all users who have access, to post notices specifically regarding the event. These notices should be permanently stored on the system, and they will be viewable to all users who have access to the event at any future stage.

Summary of the effectiveness of my solution

Overall, I would say my system is an effective solution to the problem of planning, management, delegation and logistics at the St Columba's College CCF. As seen in the objective analysis section I have achieved all my original objectives for the system and my client Major Jackson is pleased with the end product. The logistics cadets have indicated they will really welcome the system fully seeing its benefits. The event owners see the systems practical application in planning lessons and managing the registers/equipment requests and also welcome the change. The cadets who are in the lessons also commented on the enhanced visibility of lessons seeing its benefits in preparing for lesson.

However, as with any solution to a problem, there will always be room for improvement.